

现代预测建模：从回归到机器学习

李世纪

2025-11-26

目录

前言	1
简介	2
I 基础-线性回归模型	3
	4
1 预测模型与评估	7
本章导读	7
1.1 预测建模的基本概念	7
1.2 模型评估框架	8
1.3 评估指标	10
1.4 过拟合与模型复杂度	11
1.5 模型选择原则	13
1.6 软件使用	16
1.7 综合案例	23
本章总结	33
2 线性回归	35
本章导读	35
2.1 线性回归模型概述	35
2.2 总体回归与样本回归：模型的核心概念	37
2.3 参数估计	38
2.4 假设检验	40
2.5 回归分析和方差分析	42
2.6 模型的拟合优度与相关分析	44
2.7 基于回归模型的预测	46

2.8 案例分析	47
本章总结	47
3 模型诊断	48
本章导读	48
3.1 诊断概述	48
3.2 残差及其标准化形式	49
3.3 图形诊断法	51
3.4 异常值与强影响点	51
3.5 异方差的检验	53
3.6 异方差的修正	54
3.7 案例分析	56
本章小结	56
4 自相关	58
本章导读	58
4.1 自相关的定义、来源与影响	58
4.2 自相关的诊断方法	63
4.3 自相关处理方法	65
4.4 案例分析	71
本章总结	79
II 扩展——线性分析方法	80
	81
5 降维	82
本章导读	82
5.1 多重共线性及其诊断	82
5.2 高维数据的挑战	84
5.3 变量选择的评价准则	85
5.4 经典变量选择方法	87
5.5 主成分回归	88
5.6 偏最小二乘回归	90
5.7 案例分析	93
6. 结果解释与讨论	101
本章总结	113

6 正则化	115
本章导读	115
6.1 正则化方法概述	115
6.2 岭回归	118
6.3 Lasso 回归	122
6.4 弹性网	126
6.5 正则化路径、调参与比较	128
6.6 案例分析	131
本章总结	148
7 广义线性模型	151
本章导读	151
7.1 虚拟变量回归	151
7.2 广义线性模型框架	156
7.3 分类数据的建模：逻辑斯谛回归及其拓展	162
7.4 计数数据的建模：泊松回归	168
7.5 广义线性模型进阶	172
7.6 案例分析	176
本章总结	209
8 线性分类模型	212
本章导读	212
8.1 线性判别分析	212
8.2 二次判别分析 (QDA)	214
8.3 朴素贝叶斯分类器	218
8.5 模型比较与选择	224
8.6 案例分析	225
本章总结	225
9 非线性回归与样条回归	227
本章导读	227
9.1 非线性的定义	227
9.2 可线性化的非线性模型	228
9.3 多项式回归	229
9.4 样条回归	230
9.5 非线性回归的估计方法	231
9.6 模型比较与诊断	233
9.7 变量变换方法	233

9.8 案例分析	234
本章总结	234
III 进阶——超越线性的机器学习	237
	238
10 决策树与集成学习	239
本章导读	239
10.1 决策树的基本框架	239
10.2 回归树：连续响应的建模	240
10.3 分类树：类别响应的建模	241
10.4 树模型的比较与诊断	242
10.5 集成学习理论基础	243
10.6 Bagging 与随机森林	243
10.7 Boosting 方法	244
10.10 集成学习的高级应用	245
10.11 案例分析	246
本章总结	246
11 支持向量机	248
本章导读	248
11.1 线性支持向量分类器	248
11.2 软间隔支持向量机	249
11.3 对偶问题与核方法	250
11.4 支持向量回归	251
11.5 模型选择与评估	252
11.6 SVM 的扩展变体	253
11.7 SVM 与其他方法的比较	254
11.8 案例分析	254
本章总结	254
12 神经网络与深度学习基础	256
本章导读	256
12.1 神经网络：回归与分类的统一框架	256
12.2 网络架构设计	257
12.3 激活函数与损失函数	257
12.4 神经网络训练	258

12.5 过拟合与正则化	259
12.6 深度学习简介	259
12.7 案例分析	264
本章总结	264
IV 实践——综合案例分析	266
	267
13 回归任务与基准测试	269
回归任务导读	269
13.1 R 语言实现 (mlr3 框架)	269
13.2 Python (sklearn 框架)—回归	280
13.3 案例总结	294
14 分类任务	298
分类问题导读	298
14.1 Benchmark	298
14.2 Benchmark 分析总结	309
14.4 R 语言—多分类	310
14.5 Python—多分类	318
14.6 多分类案例总结	333
15 案例综合分析	337
案例导读	337
15.1 R 语言实现	339
15.2 Python 实现	357
15.3 综合分析结果	376
15.4 附录	380

简介

写这本书是为了更好的服务教学，现在的《回归分析》教材，内容过时，工具只用 SPSS，并且和《计量经济学》课程内容重复。本书特点在于：

A: 增加了机器学习的思想和内容，增加机器学习回归和分类, 同时弱化删除了一部分不再重要的内容，比如适用于计算能力差的时代的逐步回归，和计量经济学重复的用于因果分析的自相关和异方差部分；

B: 工具选择用 R、Python，和科研界、工业界接轨，更实用，更容易和工作衔接。

C: 互动性教学，基于软件的进步，可以使用 shiny 等进行互动性分析，读者可以更直观的看到参数变化对分析结果的影响

版本信息：

使用 Quarto 出版，结果可以是 html、docx、PDF 等形式。内容和代码部分主要是 markdown 格式，可移植性、可编译性都较好。quarto 版本为 1.8.26，R 版本为 4.5.2，Python 版本为 3.13.5，使用 Positron 或者 Rstudio 编译。

在书的第一部分，软件工具以 R 为主，同时引入 Python。R 在分析传统统计学模型、因果效应分析的时候具有优势，毕竟 R 是统计学家和数据学家的语言，第二部分，R、Python 并重，读者可以看到两者面对同样问题时的不同表现，在第三部分部分，机器学习部分，特别是非线性分析，以 Python 的 sklearn 分析框架为主，也是事实上的主流机器学习工具，同时也加入了 R 的 mlr3 工具包，其基准测试具有快速识别模型，批量运算对比的优势。可以认为 R 在处理传统统计学问题、线性分析模型比如因果分析、参数估计、假设检验和时间序列数据上具有优势，绘图功能是其特长；Python 在机器学习领域、非线性分析优势不可替代，其统一的输入接口和输出格式，具备强大的生命力，而且在数据清洗领域优于 R。本书综合使用两者优点，同时提供单 R 或者单 Python 的解决方案，读者根据自己的需要组合使用或者取舍。

I 基础-线性回归模型

第一部分我们将介绍线性回归分析的基本假设和方法。随后我们将扩展到更复杂的统计建模技术，包括以下内容：- 基本概念 - 线性回归模型的假设检验 - 参数估计方法和假设检验 - 异方差 - 自相关

线性回归是一个强大而又经久不衰的工具，其思想的简洁性、数学的优雅性、解释的直观性以及广泛的可扩展性让人赞叹。理解线性回归，不仅是掌握一个工具，更是理解现代定量科学如何从数据中提取信息、建立模型的基本范式，它是通往更复杂数据世界的一扇不可或缺的大门。下面我们从历史的角度回溯线性回归思想的历程：

(1) 起源：从“平均主义”到“关系量化”（18-19 世纪）

线性回归的萌芽并非源于复杂的数学理论，而是来自天文学和测量学中对“误差”处理的迫切需求。

a) 核心问题：在测量同一天文现象（如行星轨道）时，不同观测者或同一观测者的多次测量会得到略有差异的结果。哪一个是最可信的“真值”？

b) 先驱与关键思想：

高斯与勒让德：德国数学家高斯和法国数学家勒让德在 18 世纪末至 19 世纪初独立提出了最小二乘法。

核心思想：最有可能的“真值”或最佳拟合线，是能使所有观测值与预测值之差的平方和达到最小的那条线。误差平方（而非绝对差）在数学上更易处理，且能惩罚大的误差。

首次应用：勒让德在 1805 年明确发表了最小二乘法，用于分析彗星轨道；高斯声称更早使用它预测了谷神星的轨迹。

此时，线性回归更多地被视为一种精妙的“数值计算技术”，而非一个统计模型。

(2) 发展：奠定统计学基石（19 世纪末 - 20 世纪中叶）

随着生物学、经济学等社会科学对数据分析的需求增长，学者们开始探究这种“关系”背后的不确定性和统计意义。

a) 弗朗西斯·高尔顿：被誉为回归的“概念之父”。他在 1886 年研究遗传学（身高遗传）时发现，虽然高个子父母的孩子也倾向于高，但其身高会“回归”到一个平均趋势。他引入了“回归”（Regression）

一词，原意指“倒退、退回平均值”。这一现象揭示了相关但非完美的关系，是现代相关与回归分析的起点。

b) 卡尔·皮尔逊：高尔顿的学生，将回归与相关分析系统化、数学化。他建立了皮尔逊相关系数，并发展了完整的双变量正态分布理论，为回归分析提供了坚实的概率分布基础。

c) 罗纳德·费希尔：20 世纪统计学巨擘，完成了线性回归的现代统计框架。他的关键贡献在于：方差分析、显著性检验、最大似然估计。他将回归从描述性工具升级为推断性工具——我们不仅能拟合一条线，还能检验斜率是否显著不为零（即变量间是否存在统计显著的关系），并进行预测和置信区间估计。

至此，线性回归从一个“曲线拟合”方法，演变为一套完整的、基于概率论的统计推断体系。

（3）成熟与扩展：应对复杂现实（20 世纪下半叶至今）

计算机的出现和科学研究的复杂化，推动了线性回归模型的极大丰富。

a) 计算革命：最小二乘法的求解涉及矩阵运算。计算机使快速处理海量数据和多个变量成为可能，催生了多元线性回归的广泛应用。

b) 理论扩展与变体：

广义线性模型：由内尔德和韦德伯恩提出，允许因变量不限于连续正态分布（如二项分布的 Logistic 回归、泊松分布的计数数据回归）。

正则化方法：为应对高维数据、多重共线性和过拟合问题，产生了岭回归、Lasso 回归等，它们在损失函数中加入惩罚项，提升模型稳健性和可解释性。

稳健回归：针对异常值敏感问题，发展出如 Huber 回归等方法。

c) 成为机器学习的基础：

在机器学习领域，线性回归被视为最简单的监督学习算法，是理解参数学习、梯度下降优化等概念的理想起点。

它也是许多复杂模型（如神经网络中的单个神经元）的构成模块。

（4）核心思想与局限

a) 核心思想：建模确定性关系：假设因变量（Y）可以由自变量（X）的线性组合加上一个随机误差来完美解释。目标：通过数据估计线性方程的系数，以量化 X 变化时 Y 的平均变化量，并用于预测和解释。

b) 主要局限：

线性假设强：无法直接捕捉变量间的非线性关系。

对假设敏感：要求误差独立、同方差、正态分布等，现实数据常不严格满足。

易受异常值影响。

相关非因果：这是最重要的一点。回归揭示的是关联性，而非因果性。忽视混淆变量会导致错误的因果解读。

线性回归的发展脉络清晰可见：起源于天文学的误差处理（最小二乘法），概念化于生物学的遗传研究（高尔顿的“回归”），体系化于统计学的推断革命（皮尔逊、费希尔），扩展化于计算机时代的复杂数据分析（正则化、广义模型），基础化于人工智能/机器学习的模型基石。

1 预测模型与评估

本章导读

本章是你进入预测建模世界的入口和基石。你将首先理解预测问题的本质——如何从历史数据中学习规律，以预测未知结果。我们会区分分类（预测类别）和回归（预测数值）两类核心问题，并建立最关键的认知：模型在训练数据上的表现不等于它在真实世界的表现。

为此，我们将系统学习如何科学评估模型。你将掌握数据划分、交叉验证等评估框架，理解准确率、精确率、RMSE 等关键指标的含义与适用场景。更重要的是，你将深入理解过拟合现象及其背后的偏差-方差权衡原理，这是控制模型复杂度的核心。最后，我们将探讨如何综合考虑预测性能、计算效率、可解释性等多重因素，做出明智的模型选择。本章建立的思想框架和评估方法，将贯穿你后续所有的预测建模工作，补充了工具软件 R、Python 的相关使用说明，为后续学习各种预测模型奠定基础。。

1.1 预测建模的基本概念

1.1.1 预测建模：理论与实践的结合

预测建模是通过分析历史数据中的统计模式，构建数学模型来预测未知结果的过程。从理论角度看，我们假设存在一个未知的联合概率分布 P_{XY} ，其中 $\mathbf{X} \in \mathbb{R}^p$ 是特征向量， Y 是目标变量。预测模型 $f: \mathbb{R}^p \rightarrow \mathcal{Y}$ 的目标是近似条件期望 $E[Y|\mathbf{X} = \mathbf{x}]$ 。

实践案例：糖尿病风险预测医院收集了 1000 名患者的临床数据，包括：- 特征（自变量）：年龄、BMI、血糖水平、血压、胰岛素水平等 - 目标（因变量）：是否患糖尿病（是/否）

这个预测问题可以形式化为：寻找函数 f 使得预测误差最小：

$$\min_f \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i))$$

其中 L 是损失函数，如对数损失或平方损失。

1.1.2 预测任务的理论分类

分类问题的数学本质是估计条件概率分布 $P(Y = c_k | \mathbf{X} = \mathbf{x})$ 。例如在垃圾邮件过滤中，模型实际上计算的是：

$$P(\text{垃圾邮件} | \text{邮件内容}) = \frac{1}{1 + e^{-(\beta_0 + \boldsymbol{\beta}^\top \mathbf{x})}}$$

这个概率值再通过阈值（通常为 0.5）转化为类别标签。

回归问题则是估计条件期望：

$$f(\mathbf{x}) = E[Y | \mathbf{X} = \mathbf{x}]$$

在房价预测中，如果我们假设 $Y | \mathbf{X} \sim N(\boldsymbol{\beta}^\top \mathbf{x}, \sigma^2)$ ，那么最优预测就是线性组合 $\boldsymbol{\beta}^\top \mathbf{x}$ 。

1.1.3 从理论到实践的建模流程

理论框架：经验风险最小化（ERM）

$$\hat{f}_n = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i))$$

实践操作：1. 问题定义：明确 $\mathcal{Y}$ 空间（离散或连续）2. 数据收集：获得独立同分布样本 $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$
3. 模型选择：根据问题特性选择假设空间 $\mathcal{F}$ 4. 参数估计：通过优化算法求解 ERM 问题 5. 模型验证：评估泛化能力

1.2 模型评估框架

1.2.1 数据划分的理论依据与实践策略

理论视角：为什么需要划分数据集？

设真实泛化误差为 $R(f) = E[L(Y, f(\mathbf{X}))]$ ，我们只能观察到经验误差 $\hat{R}_n(f)$ 。由于 $\hat{R}_n(f)$ 通常是 $R(f)$ 的有偏估计（偏向乐观），我们需要独立于训练过程的数据来获得无偏估计。

数学表达：如果模型 f 是在数据集 D 上训练的，那么 $\hat{R}_D(f)$ 会低估真实风险。测试误差 $\hat{R}_{D_{\text{test}}}(f)$ 提供了近似无偏估计：

$$E[\hat{R}_{D_{\text{test}}}(f)] \approx R(f)$$

实践中的划分策略：

总数据: $n=1000$ 个样本

└─ 训练集: 700 样本 (70%)

└─ 用途: 模型参数学习

└─ 验证集: 150 样本 (15%)

└─ 用途: 超参数调优、模型选择

└─ 测试集: 150 样本 (15%)

└─ 用途: 最终性能评估 (仅用一次!)

常见陷阱的数学解释: 数据泄露问题: 如果在划分前对整个数据集进行标准化:

$$z_i = \frac{x_i - \bar{x}_{\text{all}}}{\sigma_{\text{all}}}$$

这样测试集的信息 (通过 $\bar{x}_{\text{all}}$ 和 σ_{all}) 泄露到了训练过程, 导致乐观偏差。

1.2.2 交叉验证: 理论保证与实现细节

k 折交叉验证的统计性质:

将数据随机划分为 **k** 个大小相近的互斥子集 $D^{(1)}, \dots, D^{(k)}$ 。定义交叉验证估计量:

$$\hat{R}_{\text{CV}} = \frac{1}{k} \sum_{j=1}^k \frac{1}{|D^{(j)}|} \sum_{(\mathbf{x}, y) \in D^{(j)}} L(y, f^{(j)}(\mathbf{x}))$$

其中 $f^{(j)}$ 是在 $D \setminus D^{(j)}$ 上训练的模型。

理论保证: 在适当条件下: - 偏差: $E[\hat{R}_{\text{CV}}] - R(f^*) = O(1/n)$ - 方差: $\text{Var}(\hat{R}_{\text{CV}}) = O(1/k + 1/n)$

这意味着 5 折或 10 折交叉验证在偏差和方差间取得了良好平衡。

实践建议: - 小数据集 ($n < 1000$): 使用 10 折交叉验证 - 中数据集 ($1000 \leq n < 10000$): 5 折交叉验证 - 大数据集 ($n \geq 10000$): 简单划分 (70-15-15) 可能足够

分层交叉验证的重要性: 对于分类问题, 如果原始数据中正类比例为 π , 每个折中应保持相同比例, 否则估计会有偏差。

1.2.3 时间序列的特殊考虑

时间序列 $\{Y_t\}_{t=1}^T$ 破坏了独立同分布假设。滚动窗口方法:

$$\hat{R}_{\text{RW}} = \frac{1}{T - w - h + 1} \sum_{t=w}^{T-h} \frac{1}{h} \sum_{s=1}^h L(Y_{t+s}, \hat{f}_t(Y_{t+s}))$$

其中 w 是窗口大小, h 是预测步长。

1.3 评估指标

1.3.1 分类指标的统计解释

混淆矩阵的理论意义: 对于二分类问题, 设真实标签 $Y \in \{0, 1\}$, 预测 $\hat{Y} = f(\mathbf{X})$ 。定义: - TP = $\sum I(Y_i = 1, \hat{Y}_i = 1)$ - FP = $\sum I(Y_i = 0, \hat{Y}_i = 1)$ - FN = $\sum I(Y_i = 1, \hat{Y}_i = 0)$ - TN = $\sum I(Y_i = 0, \hat{Y}_i = 0)$

准确率的局限性: 设正类比例 $\pi = P(Y = 1)$, 如果使用常数预测 $\hat{Y} = 0$, 则:

$$\text{准确率} = P(Y = 0) = 1 - \pi$$

当 π 很小时 (如欺诈检测中 $\pi \approx 0.01$), 准确率可达 99%, 但模型完全无判别能力。

精确率与召回率的概率解释: - 精确率: $P(Y = 1 | \hat{Y} = 1)$ - 召回率: $P(\hat{Y} = 1 | Y = 1)$

这两个条件概率描述了模型的不同方面, 通常存在权衡关系。

1.3.2 ROC 与 AUC 的深刻理解

理论深度: $\text{AUC} = \mathbb{P}(S_1 > S_0)$, 其中 S_1 和 S_0 分别是正类和负类中随机抽取样本的模型得分。这是一个基于模型排序能力的指标。

ROC 曲线的构造: 对于每个阈值 τ , 计算: - $\text{TPR}(\tau) = \mathbb{P}(S > \tau | Y = 1)$ - $\text{FPR}(\tau) = \mathbb{P}(S > \tau | Y = 0)$

ROC 曲线是参数曲线 $\{(\text{FPR}(\tau), \text{TPR}(\tau)) : \tau \in \mathbb{R}\}$ 。

AUC 的统计估计: 使用梯形法则或 Mann-Whitney U 统计量:

$$\widehat{\text{AUC}} = \frac{1}{n_1 n_0} \sum_{i: y_i=1} \sum_{j: y_j=0} I(s_i > s_j)$$

1.3.3 回归指标的理论基础

MSE 的统计性质: 对于无偏预测 $\hat{f}$, MSE 可分解为:

$$\text{MSE} = \text{Bias}^2(\hat{f}) + \text{Var}(\hat{f}) + \sigma^2$$

其中 σ^2 是不可约误差。

MAE vs RMSE 的数学比较：- MAE = $\frac{1}{n} \sum |y_i - \hat{y}_i|$ ，对异常值稳健 - RMSE = $\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$ ，对大误差惩罚更重

设误差 $e_i = y_i - \hat{y}_i$ ，如果 e_i 服从拉普拉斯分布，MAE 是合适的；如果服从正态分布，MSE/RMSE 更合适。

R² 的深刻解释：

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = \frac{SS_{\text{reg}}}{SS_{\text{tot}}}$$

但需要注意：1. R^2 随变量增加而单调不减（即使加入无关变量）2. 调整 R^2 ： $R_{\text{adj}}^2 = 1 - \frac{(1-R^2)(n-1)}{n-p-1}$
3. 预测 R^2 ：基于交叉验证的 R^2 更可靠

1.3.4 多指标评估框架

理论指导：没有单一最优指标。根据决策理论，应选择与业务损失函数一致的评估指标。

实践示例：信用卡欺诈检测 - 业务损失：漏报损失 $L_{\text{FN}} = \$500$ ，误报损失 $L_{\text{FP}} = \$10$ - 对应指标：最小化加权损失

$$\text{Loss} = 500 \times \text{FN} + 10 \times \text{FP}$$

- 等价于：精确率权重约为召回率的 50 倍

1.4 过拟合与模型复杂度

1.4.1 偏差-方差分解：理论与图解

理论框架：对于回归问题和平方损失，泛化误差可分解为：

$$\mathbb{E}[(Y - \hat{f}(\mathbf{X}))^2] = \underbrace{(\mathbb{E}[\hat{f}(\mathbf{X})] - f(\mathbf{X}))^2}_{\text{偏差}^2} + \underbrace{\mathbb{E}[(\hat{f}(\mathbf{X}) - \mathbb{E}[\hat{f}(\mathbf{X})])^2]}_{\text{方差}} + \underbrace{\sigma^2}_{\text{噪声}}$$

其中 $f(\mathbf{X}) = \mathbb{E}[Y|\mathbf{X}]$ 是真实函数。

可视化解：考虑用多项式拟合正弦函数：

概念性代码

真实函数： $y = \sin(2\pi x) + \varepsilon, \varepsilon \sim N(0, 0.1)$

拟合模型： $y = \beta_0 + \beta_1 x + \dots + \beta_d x^d$

$d=1$ （线性）：高偏差，低方差 → 欠拟合

$d=3$ (三次): 中偏差, 中方差 $\rightarrow$ 合适

$d=10$ (十次): 低偏差, 高方差 $\rightarrow$ 过拟合

数学上的复杂度控制: 设我们用 d 次多项式拟合 n 个点。近似误差 (偏差²) 随 d 增加而减小, 但估计误差 (方差) 随 d 增加而增大:

$$\text{总误差} \approx O\left(\frac{1}{d^{2\alpha}}\right) + O\left(\frac{d}{n}\right)$$

其中 α 描述真实函数的平滑度。

1.4.2 正则化的数学原理

岭回归的理论:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|^2$$

解为: $\hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$

当 $\mathbf{X}^T \mathbf{X}$ 接近奇异时 (多重共线性), 岭回归通过增加 $\lambda \mathbf{I}$ 使矩阵可逆, 稳定了估计。

LASSO 与特征选择:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|^2 + \lambda \|\beta\|_1$$

由于 L1 惩罚的非光滑性, LASSO 会产生稀疏解, 实现特征选择。

几何解释: - L2 惩罚: 圆形约束区域, 解通常在内部 - L1 惩罚: 菱形约束区域, 解常在角点 (某些系数为 0)

1.4.3 学习曲线的诊断价值

理论曲线: - 训练误差: 随样本增加而增加 (渐近到某个值) - 验证误差: 随样本增加而减少

诊断模式: 1. 高偏差 (欠拟合): - 训练误差高 - 训练与验证误差接近但都高 - 增加数据帮助有限

2. 高方差 (过拟合):

- 训练误差低
- 验证误差显著高于训练误差
- 增加数据可改善

数学表达: 设 $L_{\text{train}}(n)$ 和 $L_{\text{val}}(n)$ 分别是训练和验证误差: - 如果 $L_{\text{train}}(n) \rightarrow L_{\infty} > 0$ 且 $L_{\text{val}}(n) \approx L_{\text{train}}(n) \rightarrow$ 欠拟合 - 如果 $L_{\text{train}}(n) \rightarrow 0$ 但 $L_{\text{val}}(n) \gg 0 \rightarrow$ 过拟合

1.4.4 应对过拟合的策略体系

1. 获取更多数据理论依据：根据学习理论，方差项通常以 $O(1/n)$ 衰减
2. 降低模型复杂度 - 减少参数数量 - 提前停止（early stopping）- 决策树剪枝
3. 正则化 - L1/L2 正则化 - Dropout（神经网络）- 贝叶斯方法（先验分布作为正则化）
4. 集成方法 - Bagging：降低方差， $Var(\bar{f}) = \frac{1}{M}Var(f) + \frac{M-1}{M}Cov(f_i, f_j)$ - Boosting：降低偏差，通过加法模型逐步改进

1.5 模型选择原则

1.5.1 偏差-方差权衡的数学优化

模型选择可形式化为：

$$\min_{d \in \mathcal{D}} \{ \text{Bias}^2(d) + \text{Var}(d) \}$$

其中 d 是复杂度参数。

多项式回归示例：真实模型： $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$ 我们考虑从 1 次到 10 次多项式：- $d=1$ ：高偏差（无法拟合二次项）- $d=2$ ：合适 - $d \geq 3$ ：过参数化，方差增加

数学上的最优复杂度：对于平滑函数，最优复杂度 $d^* \propto n^{\frac{1}{2\alpha+1}}$ ，其中 α 是函数的光滑度指数。

1.5.2 信息准则：量化的奥卡姆剃刀

AIC 的理论基础：基于 Kullback-Leibler 散度最小化：

$$\text{AIC} = -2 \log \hat{L} + 2k$$

其中 $\hat{L}$ 是最大似然值， k 是参数个数。

AIC 选择的模型渐近最小化：

$$\text{KL}(p_{\text{true}} \| p_{\text{model}})$$

BIC 的理论基础：基于贝叶斯模型选择：

$$\text{BIC} = -2 \log \hat{L} + k \log n$$

BIC 具有一致性：如果真实模型在候选集中，当 $n \rightarrow \infty$ 时，BIC 选择真实模型的概率 $\rightarrow 1$ 。

实践对比：对于样本量 n ：- n 小：AIC 和 BIC 差异大 - n 大：BIC 惩罚更重，选择更简单模型

经验法则：- $\Delta AIC > 10$ ：强证据支持较小 AIC 模型 - $\Delta AIC < 2$ ：模型无显著差异

1.5.3 结构风险最小化

VC 理论框架：对于分类问题，以概率至少 $1 - \delta$ ：

$$R(f) \leq \hat{R}_n(f) + \sqrt{\frac{h(\log \frac{2n}{h} + 1) - \log \frac{\delta}{4}}{n}}$$

其中 h 是假设空间的 VC 维。

模型复杂度控制：通过控制 VC 维来平衡经验风险和置信区间：

$$\hat{f} = \arg \min_{f \in \mathcal{F}_h} \left\{ \hat{R}_n(f) + C \sqrt{\frac{h}{n}} \right\}$$

1.5.4 多准则决策的定量框架

Pareto 最优前沿：设模型 M_i 在准则 C_j 上得分为 s_{ij} 。模型 M_a 支配 M_b 如果：1. $s_{aj} \geq s_{bj}$ 对所有 j 2. $s_{aj} > s_{bj}$ 对至少一个 j

不被任何模型支配的模型构成 Pareto 前沿。

标量化方法：给定权重向量 $\mathbf{w}$ （如 $\mathbf{w} = (0.4, 0.3, 0.2, 0.1)$ 对应性能、效率、可解释性、部署成本），总分为：

$$\text{Score}(M_i) = \sum_{j=1}^m w_j \cdot \text{normalize}(s_{ij})$$

案例：信用评分模型选择：

准则	权重	逻辑回归	XGBoost	随机森林
AUC	0.40	0.75(30)	0.82(32.8)	0.80(32)
训练时间 (s)	0.20	2(18)	120(12)	45(15)
可解释性	0.25	95(23.75)	40(10)	60(15)
部署难度	0.15	90(13.5)	70(10.5)	80(12)
总分	1.00	85.25	65.30	74.00

理论解释：这是多目标优化问题的加权和标量化，前提是各准则间可补偿。

1.5.5 集成方法的理论优势

Bagging 的方差减少：设 $f_1, \dots, f_M$ 为独立同分布估计量， $\bar{f} = \frac{1}{M} \sum f_i$ ，则：

$$\text{Var}(\bar{f}) = \frac{1}{M} \text{Var}(f_1) + \frac{M-1}{M} \text{Cov}(f_1, f_2)$$

通过自助采样降低相关性，从而降低方差。

Boosting 的偏差减少：可视为在函数空间上的梯度下降：

$$f_m = f_{m-1} - \eta_m \nabla_f L(f_{m-1})$$

逐步拟合残差，减少偏差。

1.5.6 实践中的模型选择流程

三步选择法：

1. 初步筛选（基于理论约束）

- 分类 vs 回归
- 线性 vs 非线性
- 数据量大小
- 计算资源限制

2. 性能比较（基于交叉验证）

```
# 概念性流程
模型候选集 = [线性回归, 决策树, 随机森林, XGBoost]
for 模型 in 模型候选集:
    CV 分数 = k 折交叉验证 (模型, 数据)
    记录 (模型, CV 分数的均值和方差)
```

3. 最终决策（多准则综合）

- 统计性能（主要指标）
- 计算效率（训练/预测时间）
- 可解释性（业务需求）
- 稳健性（对数据变化的敏感度）

理论指导下的启发式规则：1. 样本量原则：- $n < 1000$ ：简单模型（线性/浅树）- $1000 \leq n < 10000$ ：中等复杂度（随机森林/GBDT）- $n \geq 10000$ ：可尝试复杂模型（深度学习）

2. 特征维度原则：

- $p \approx n$ ：需要正则化（岭回归/LASSO）
- $p > n$ ：必须使用特征选择或强正则化
- $p \ll n$ ：可考虑复杂模型

3. 业务优先级原则：

- 需要可解释性：线性模型、决策树
- 需要最高精度：集成方法、深度学习
- 需要实时预测：简单模型、提前剪枝

1.6 软件使用

本书主要使用开源软件 R、Python，安装使用请参考我的 QQ 日志。

16.1 R 语言

R+Rstudio，使用到的包 (package) 包括 knir、tidyverse、ggplot2 等，文件格式为.rmd，可以混编显示文本、代码块 (chunk)、代码运行结果等各种格式内容，进行交互式的分析，可以直接生成 html、doc、PDF、PPT 等格式，撰写实验报告、论文、书籍等。

典型 R 包结构

```
a_r_package/
├── DESCRIPTION                # □ 包的"身份证"
│   - Package: my_r_package    # 包名
│   - Version: 0.1.0           # 版本号
│   - Title: 包的功能描述      # 标题
│   - Author: 作者信息         # 作者
│   - Description: 详细描述     # 详细描述
│   - License: MIT             # 许可证
│   - Imports: dplyr, ggplot2  # 依赖包
│   - Suggests: testthat       # 建议依赖
│
├── NAMESPACE                 # □ 函数的"门卫"
│   - export(hello_world)      # 导出函数供用户使用
│   - import(dplyr)            # 导入依赖包的函数
```

```

- importFrom(ggplot2, aes)      # 导入特定函数

R/                                # □ 核心代码目录
├─ hello.R                      # 函数定义文件1
|   # 函数定义
|   hello_world <- function() {
|       print("Hello from R package!")
|   }
|
├─ data_processing.R            # 函数定义文件2
|   clean_data <- function(df) {
|       dplyr::filter(df, !is.na(value))
|   }
|
└─ utils.R                     # 工具函数（内部使用）
    internal_func <- function() {
        # 不导出，仅内部使用
    }

man/                              # □ 文档目录（.Rd文件）
├─ hello_world.Rd              # 函数的帮助文档
|   \name{hello_world}
|   \title{打招呼函数}
|   \description{详细描述}
|   \usage{hello_world()}
|   \examples{hello_world()}
|
└─ my_r_package-package.Rd     # 包的总体文档

tests/                            # □ 测试目录
├─ testthat/                  # testthat测试框架
|   └─ test-hello.R            # 测试文件
|       test_that("hello works", {
|           expect_output(hello_world(), "Hello")
|       })
|   └─ test-data.R
└─ testthat.R                  # 测试运行器

```

```

├─ vignettes/                # □ 长篇教程/案例
│   └─ introduction.Rmd      # 包的使用教程
│
├─ data/                     # □ 包内置数据集
│   ├── sample_data.rda      # R数据文件
│   └─ internal_data.rda     # 内部数据
│
├─ inst/                     # □ 安装时包含的文件
│   ├── CITATION              # 引用信息
│   ├── extdata/              # 外部数据示例
│   └─ templates/            # 模板文件
│
└─ .Rbuildignore              # □ 构建时忽略的文件

```

R 数据分析典型项目结构

```

data_analysis_project/      # □ 数据分析项目
├─ data/                    # □ 数据管理
│   ├── raw/                # 原始数据（只读）
│   ├── processed/          # 处理后的数据
│   └─ external/            # 外部数据源
│
├─ R/                       # □ R代码
│   ├── 01_data_cleaning.R   # 数据清洗
│   ├── 02_exploratory.R     # 探索性分析
│   ├── 03_modeling.R        # 建模
│   ├── 04_visualization.R   # 可视化
│   └─ functions/           # 自定义函数
│       ├── utils.R
│       └─ plotting_functions.R
│
├─ analysis/                # □ 分析文档
│   ├── report.Rmd           # R Markdown报告
│   ├── presentation.Rmd     # 演示文稿
│   └─ dashboard/           # Shiny应用
│
├─ outputs/                 # □ 输出结果
│   ├── figures/             # 生成的图表
│   └─ tables/               # 生成的表格

```

```

|   └─ models/                                # 保存的模型
|
|─ tests/                                     # □ 测试
|─ references/                               # □ 参考文献/文档
|─ .Rprofile                                # □ R启动配置
|─ .Renviron                                # □ 环境变量
|─ data_analysis_project.Rproj # □ RStudio项目文件

```

16.2 Python

依托 Anaconda 平台，该平台打包了 python 和重要的模块，包括环境的设置，自带 IDE 包括 jupyter notebook、jupyterlab、pycharm 等，使用到的库（library）包括 pandas、numpy、sklearn，由 jupyter notebook 编译文件格式为.ipynb，可以实现文本、代码块（cell）、代码运行结果、结果分析等混编，进行交互式的分析，可以直接生成 html、doc、PDF 等格式。

Python 包结构

```

a_python_package/
|─ pyproject.toml                            # □ 现代构建配置（替代setup.py）
|   [build-system]
|   requires = ["setuptools", "wheel"]
|
|   [project]
|   name = "my_python_package"
|   version = "0.1.0"
|   dependencies = [
|       "numpy>=1.20",
|       "pandas>=1.3"
|   ]
|
|─ src/                                     # □ 源代码目录（推荐结构）
|   └─ my_python_package/                  # 包的主目录
|       └─ __init__.py                     # □ 包的入口点
|           """包的主模块"""
|           __version__ = "0.1.0"
|           __author__ = "Your Name"
|
|           # 导入关键函数，方便用户访问
|           from .core import (

```



```

        normalize_data,
        process_dataframe
    )

    # 也可以使用 __all__ 控制导入
    __all__ = ["normalize_data", "process_dataframe"]

└─ core.py                                # □ 核心模块
    """核心功能实现"""
    import numpy as np
    import pandas as pd

    def normalize_data(x):
        """标准化数据"""
        if not isinstance(x, (np.ndarray, list)):
            raise TypeError("输入必须是数组或列表")
        x = np.array(x)
        return (x - np.mean(x)) / np.std(x)

    def _internal_helper():
        """内部函数（以下划线开头）"""
        return "internal"

└─ utils/                                # □ 子包/工具模块目录
    └─ __init__.py
    └─ file_utils.py
    └─ math_utils.py

└─ data/                                  # □ 数据管理
    └─ __init__.py
    └─ datasets.py                        # 数据集加载函数
    └─ constants.py                      # 常量定义

└─ tests/                                # □ 测试目录（可选放这里）
    └─ __init__.py
    └─ test_core.py
    └─ test_utils/

```



```

├── build_features.py
├── selection.py
├── models/                                # 模型
│   ├── __init__.py
│   ├── train_model.py
│   └── predict_model.py
├── visualization/                        # 可视化
│   ├── __init__.py
│   └── plot_results.py
├── notebooks/                           # □ Jupyter 笔记本
│   ├── 01_exploratory.ipynb            # 探索性分析
│   ├── 02_feature_engineering.ipynb
│   └── 03_model_training.ipynb
├── data/                                # □ 数据
│   ├── raw/                            # 原始数据
│   ├── interim/                        # 中间数据
│   └── processed/                      # 处理后的数据
├── models/                              # □ 模型存储
│   ├── trained_models/
│   └── model_metrics.json
├── reports/                             # □ 报告
│   ├── figures/                        # 图表
│   └── final_report.md
├── tests/                               # □ 测试
├── configs/                             # □ 配置文件
│   ├── data_config.yaml
│   └── model_config.yaml
├── requirements.txt                     # □ 依赖
├── setup.py                           # □ 安装配置
└── pyproject.toml                      # □ 现代配置

```

— Dockerfile	# □ 容器化
— README.md	# □ 说明文档

1.6.3 平台搭建

统计学专业的同学，可以考虑 R+Rtools+Rstudio，使用 Rstudio 作为分析平台，文件格式为 RMD，如果想混合 Python 使用，可以加载包 reticulate，连接调用 python 使用。

面向机器学习的同学，应该以 Python 为主，学习、笔记、报告、作业等考虑 Anaconda 里面的 IDE 工具 jupyter notebook，文件格式为 ipynb，或者考虑使用 jupyterlab。

面向工程开发、项目管理的同学，考虑使用 VScode，如果工作偏向于数据科学，可以使用 Positren，是一个类似 vscode 的数据科学定制版，可以交叉调用 R、Python、Julia 等语言进行混合编程，集成 git 进行版本控制和发布，本书即使用 Positren+quarto 编写。

1.7 综合案例

编程演示：过拟合现象

```
# 过拟合现象演示 - 生成数据
# 生成理想的过拟合演示数据
generate_overfitting_demo_data <- function(n = 100) {
  set.seed(123)
  x <- seq(0, 4 * pi, length.out = n)
  # 真实关系：正弦函数 + 噪声
  y_true <- sin(x) + 0.5 * sin(2*x)
  y_observed <- y_true + rnorm(n, 0, 0.2)
  return(data.frame(x = x, y_true = y_true, y_observed = y_observed))
}

# 生成数据
demo_data <- generate_overfitting_demo_data(100)

# 拟合不同复杂度的多项式模型
library(ggplot2)
fit_polynomial_models <- function(data, max_degree = 6) {
  models <- list()
  predictions <- data.frame(x = data$x)
```

```

for (degree in 1:max_degree) {
  model <- lm(y_observed ~ poly(x, degree), data = data)
  models[[paste0("degree_", degree)]] <- model
  predictions[[paste0("pred_", degree)]] <- predict(model, newdata = data)
}

return(list(models = models, predictions = predictions))
}

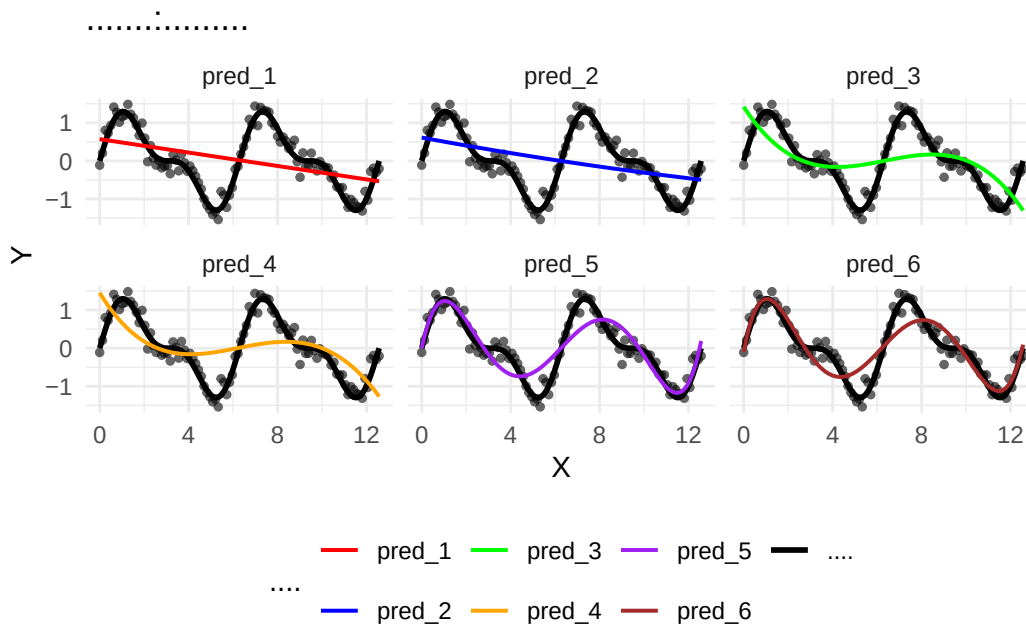
# 拟合模型
results <- fit_polynomial_models(demo_data)

# 可视化过拟合现象
plot_overfitting_demo <- function(data, predictions) {
  plot_data <- cbind(data, predictions)
  plot_data_long <- reshape2::melt(plot_data, id.vars = c("x", "y_true", "y_observed"),
                                   variable.name = "model", value.name = "prediction")

  ggplot(plot_data_long, aes(x = x)) +
    geom_point(aes(y = y_observed), alpha = 0.6, size = 1) +
    geom_line(aes(y = y_true, color = "真实关系"), size = 1) +
    geom_line(aes(y = prediction, color = model), size = 0.7) +
    facet_wrap(~ model, ncol = 3) +
    labs(title = "过拟合现象演示: 不同阶数多项式回归",
         x = "X", y = "Y",
         color = "曲线类型") +
    theme_minimal() +
    scale_color_manual(values = c("真实关系" = "black",
                                   "pred_1" = "red", "pred_2" = "blue",
                                   "pred_3" = "green", "pred_4" = "orange",
                                   "pred_5" = "purple", "pred_6" = "brown")) +
    theme(legend.position = "bottom")
}

# 显示图形
plot_overfitting_demo(demo_data, results$predictions)

```



编程演示：数据划分策略

```
demonstrate_data_splitting <- function() {
  set.seed(123)
  n <- 1000
  # 生成回归数据
  x1 <- rnorm(n)
  x2 <- rnorm(n)
  y <- 2*x1 + 3*x2 + rnorm(n, 0, 1)
  data <- data.frame(x1 = x1, x2 = x2, y = y)

  # 简单划分 (70% 训练, 30% 测试)
  train_idx_simple <- sample(1:n, size = 0.7 * n)
  test_idx_simple <- setdiff(1:n, train_idx_simple)

  cat(" 简单划分结果:\n")
  cat(" 训练集大小:", length(train_idx_simple), "\n")
  cat(" 测试集大小:", length(test_idx_simple), "\n")

  # 检查分布是否相似
  cat("\n训练集和测试集分布比较:\n")
  cat(" 训练集 y 均值:", mean(data$y[train_idx_simple]), "\n")
  cat(" 测试集 y 均值:", mean(data$y[test_idx_simple]), "\n")
}
```

```
return(list(  
  data = data,  
  train_idx = train_idx_simple,  
  test_idx = test_idx_simple  
))  
}  
  
# 运行演示  
split_results <- demonstrate_data_splitting()
```

简单划分结果：

训练集大小：700

测试集大小：300

训练集和测试集分布比较：

训练集y均值：0.1586161

测试集y均值：0.09502576

**** 演示：回归评估指标 ****

```
demonstrate_regression_metrics <- function() {  
  set.seed(123)  
  n <- 100  
  # 生成数据  
  x <- rnorm(n)  
  y_true <- 2*x + 1  
  y_pred <- y_true + rnorm(n, 0, 0.5) # 预测值（带有误差）  
  y_actual <- y_true + rnorm(n, 0, 0.3) # 实际观测值（带有噪声）  
  
  # 计算各种评估指标  
  calculate_regression_metrics <- function(actual, predicted) {  
    n <- length(actual)  
    residuals <- actual - predicted  
  
    mse <- mean(residuals^2)  
    rmse <- sqrt(mse)  
    mae <- mean(abs(residuals))  
  
    # R-squared
```

```

ss_residual <- sum(residuals^2)
ss_total <- sum((actual - mean(actual))^2)
r_squared <- 1 - (ss_residual / ss_total)

# 调整 R-squared
p <- 1 # 特征数
r_squared_adj <- 1 - (1 - r_squared) * (n - 1) / (n - p - 1)

return(list(
  MSE = mse,
  RMSE = rmse,
  MAE = mae,
  R2 = r_squared,
  R2_adj = r_squared_adj
))
}

metrics <- calculate_regression_metrics(y_actual, y_pred)

cat(" 回归评估指标:\n")
cat("MSE (均方误差):", round(metrics$MSE, 4), "\n")
cat("RMSE (均方根误差):", round(metrics$RMSE, 4), "\n")
cat("MAE (平均绝对误差):", round(metrics$MAE, 4), "\n")
cat("R² (决定系数):", round(metrics$R2, 4), "\n")
cat("调整 R²:", round(metrics$R2_adj, 4), "\n")

# 可视化预测效果
plot_data <- data.frame(
  Actual = y_actual,
  Predicted = y_pred,
  Perfect = y_actual # 完美预测线
)

library(ggplot2)
p <- ggplot(plot_data, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  geom_smooth(method = "lm", se = FALSE, color = "blue") +

```



```

labs(title = " 预测值 vs 实际值",
      subtitle = paste("R2 =", round(metrics$R2, 4)),
      x = " 实际值", y = " 预测值") +
theme_minimal()

print(p)

return(metrics)
}

# 运行回归指标演示
regression_metrics <- demonstrate_regression_metrics()

```

回归评估指标：

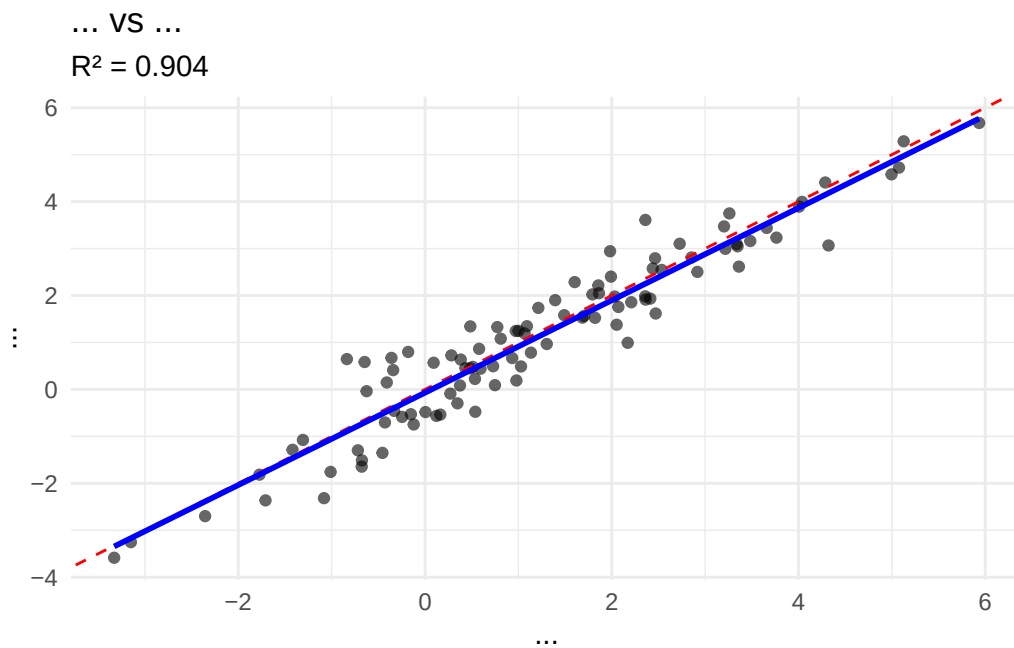
MSE (均方误差): 0.3116

RMSE (均方根误差): 0.5582

MAE (平均绝对误差): 0.4523

R² (决定系数): 0.904

调整R²: 0.9031



编程演示：分类评估指标

```
# 1.4.2 分类评估指标演示
demonstrate_classification_metrics <- function() {
  set.seed(123)
  n <- 1000
  # 生成二分类数据
  feature1 <- rnorm(n)
  feature2 <- rnorm(n)

  # 生成真实概率
  true_prob <- plogis(0.5 + 1.2 * feature1 - 0.8 * feature2)
  true_labels <- rbinom(n, 1, true_prob)

  # 生成预测概率（带有一些误差）
  pred_prob <- true_prob + rnorm(n, 0, 0.1)
  pred_prob <- pmin(pmax(pred_prob, 0), 1) # 限制在 [0,1] 范围内
  pred_labels <- ifelse(pred_prob > 0.5, 1, 0)

  # 计算混淆矩阵
  calculate_confusion_matrix <- function(actual, predicted) {
    tp <- sum(actual == 1 & predicted == 1)
    tn <- sum(actual == 0 & predicted == 0)
    fp <- sum(actual == 0 & predicted == 1)
    fn <- sum(actual == 1 & predicted == 0)

    return(list(TP = tp, TN = tn, FP = fp, FN = fn))
  }

  cm <- calculate_confusion_matrix(true_labels, pred_labels)

  # 计算各种指标
  accuracy <- (cm$TP + cm$TN) / n
  precision <- cm$TP / (cm$TP + cm$FP)
  recall <- cm$TP / (cm$TP + cm$FN)
  f1_score <- 2 * (precision * recall) / (precision + recall)

  # 计算 AUC
  calculate_auc <- function(actual, prob) {
    # 简单实现 AUC 计算
  }
```

```
positive_probs <- prob[actual == 1]
negative_probs <- prob[actual == 0]

comparisons <- 0
correct <- 0

# 抽样计算以加快速度
n_sample <- min(100, length(positive_probs), length(negative_probs))
pos_sample <- sample(positive_probs, n_sample)
neg_sample <- sample(negative_probs, n_sample)

for (p in pos_sample) {
  for (n in neg_sample) {
    comparisons <- comparisons + 1
    if (p > n) correct <- correct + 1
    else if (p == n) correct <- correct + 0.5
  }
}

return(correct / comparisons)
}

auc_score <- calculate_auc(true_labels, pred_prob)

cat(" 分类评估指标:\n")
cat(" 准确率:", round(accuracy, 4), "\n")
cat(" 精确率:", round(precision, 4), "\n")
cat(" 召回率:", round(recall, 4), "\n")
cat("F1 分数:", round(f1_score, 4), "\n")
cat("AUC:", round(auc_score, 4), "\n")

# 打印混淆矩阵
cat("\n混淆矩阵:\n")
confusion_df <- data.frame(
  Actual_0 = c(cm$TN, cm$FN),
  Actual_1 = c(cm$FP, cm$TP)
)
rownames(confusion_df) <- c("Predicted_0", "Predicted_1")
```

```
print(confusion_df)

# 可视化 ROC 曲线
plot_roc_curve <- function(actual, prob) {
  thresholds <- seq(0, 1, 0.01)
  tpr <- numeric(length(thresholds))
  fpr <- numeric(length(thresholds))

  for (i in 1:length(thresholds)) {
    pred <- ifelse(prob > thresholds[i], 1, 0)
    cm_temp <- calculate_confusion_matrix(actual, pred)
    tpr[i] <- cm_temp$TP / (cm_temp$TP + cm_temp$FN) # 真正率
    fpr[i] <- cm_temp$FP / (cm_temp$FP + cm_temp$TN) # 假正率
  }

  roc_data <- data.frame(FPR = fpr, TPR = tpr)

  ggplot(roc_data, aes(x = FPR, y = TPR)) +
    geom_line(color = "blue", size = 1) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "gray") +
    labs(title = "ROC 曲线",
         subtitle = paste("AUC =", round(auc_score, 4)),
         x = "假正率 (FPR)", y = "真正率 (TPR)") +
    theme_minimal() +
    coord_equal()
}

roc_plot <- plot_roc_curve(true_labels, pred_prob)
print(roc_plot)

return(list(
  accuracy = accuracy,
  precision = precision,
  recall = recall,
  f1_score = f1_score,
  auc = auc_score,
  confusion_matrix = cm
))
```

```
}  
  
# 运行分类指标演示  
classification_metrics <- demonstrate_classification_metrics()
```

分类评估指标：

准确率：0.748

精确率：0.779

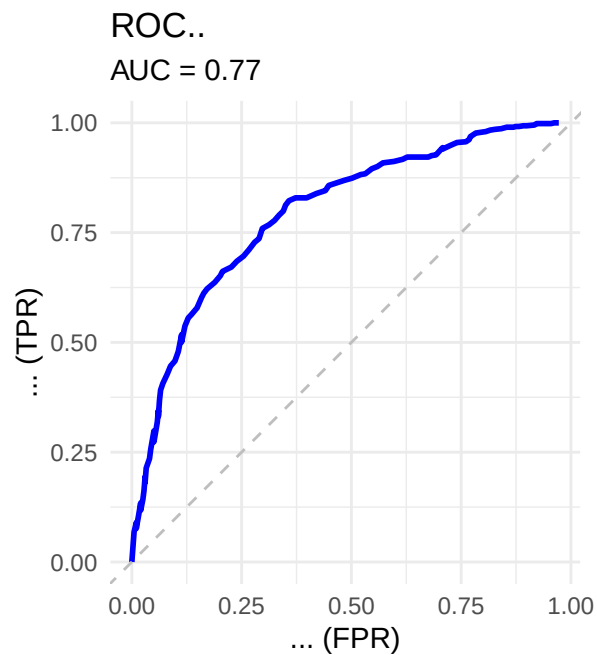
召回率：0.8126

F1分数：0.7955

AUC：0.77

混淆矩阵：

	Actual_0	Actual_1
Predicted_0	258	139
Predicted_1	113	490



本章总结

核心理论框架回顾

- 1. 预测建模的统计基础：建立在概率论和经验风险最小化原则上，通过优化算法从数据中学习预测函数。
- 2. 模型评估的可靠性：交叉验证提供了泛化误差的近似无偏估计，各种评估指标从不同角度度量模型性能。
- 3. 过拟合的本质：偏差-方差分解提供了理解模型复杂度的理论框架，正则化是控制复杂度的关键技术。
- 4. 模型选择的原则：在偏差-方差权衡、模型简约性和多准则要求间找到平衡点。

理论与实践的结合点

理论概念	实践体现	关键点
期望风险最小化	交叉验证评估	测试集必须独立于训练过程
偏差-方差分解	学习曲线诊断	通过增加数据或调整复杂度来优化
正则化理论	L1/L2 惩罚	控制模型复杂度，防止过拟合
信息准则	模型比较	AIC 用于预测，BIC 用于模型识别

实用指南

- 1. 开始新项目时：
 - 首先确定问题类型（分类/回归）
 - 划分训练/验证/测试集
 - 建立基线模型（如逻辑回归或线性回归）
- 2. 模型开发中：
 - 使用交叉验证比较候选模型
 - 监控训练和验证误差的差距
 - 根据学习曲线判断欠拟合/过拟合
- 3. 最终决策时：
 - 在独立测试集上评估最终模型
 - 考虑计算效率、可解释性等非统计因素
 - 记录所有决策和参数，确保可复现性

从理论到实践的思维转变

初学者思维：“哪个模型在测试集上准确率最高？”

专家思维：“在偏差-方差权衡的哪个位置？模型是否过拟合？评估指标是否与业务损失一致？计算成本是否可接受？”

本章建立的理论与实践结合框架，为后续学习具体预测模型提供了坚实基础。记住：好的预测建模不仅是应用算法，更是基于统计原理做出明智的工程决策。

2 线性回归

本章导读

回归分析是现代统计学和计量经济学的核心方法之一，其起源可追溯至 19 世纪弗朗西斯·高尔顿（Francis Galton）关于遗传现象的研究。1886 年，高尔顿在研究父子身高关系时，发现子辈身高有向平均身高“回归”的趋势，由此提出了“回归”这一概念。

统计关系与函数关系存在本质区别：

- 函数关系：变量之间存在确定的数学映射，例如物理中的牛顿第二定律 $F = ma$ ，给定 m 和 a ， F 唯一确定。
- 统计关系：变量之间具有相关性，但并非一一对应。例如，教育水平与收入水平通常呈正相关，但相同教育水平的人收入不尽相同。这种不确定性的关系正是回归分析的研究对象。

线性回归作为最简单的回归形式，通过线性模型描述因变量与一个或多个自变量之间的统计关系，是预测建模的基础工具。

2.1 线性回归模型概述

2.1.1 模型的基本形式

一元线性回归模型（总体形式）为：

$$Y_i = \beta_0 + \beta_1 X_i + u_i, \quad i = 1, 2, \dots, n$$

其中：- Y_i 为因变量（被解释变量）- X_i 为自变量（解释变量）- β_0 为截距项（常数项）- β_1 为斜率系数（解释变量 X 对 Y 的边际影响）- u_i 为随机扰动项（误差项），代表除 X 外其他所有影响 Y 的因素

多元线性回归模型则扩展为：

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_k X_{ki} + u_i, \quad i = 1, 2, \dots, n$$

模型的矩阵形式是更紧凑的表示。定义：

- 因变量向量： $\mathbf{Y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}_{n \times 1}$
- 解释变量矩阵： $\mathbf{X} = \begin{pmatrix} 1 & X_{11} & X_{21} & \cdots & X_{k1} \\ 1 & X_{12} & X_{22} & \cdots & X_{k2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & X_{1n} & X_{2n} & \cdots & X_{kn} \end{pmatrix}_{n \times (k+1)}$
- 参数向量： $\boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_k \end{pmatrix}_{(k+1) \times 1}$
- 扰动项向量： $\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{pmatrix}_{n \times 1}$

则多元线性回归模型可简洁地表示为矩阵形式：

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{u}$$

模型的向量形式为：

$$y_i = \mathbf{x}_i' \boldsymbol{\beta} + u_i, \quad i = 1, \dots, n$$

其中 $\mathbf{x}_i = (1, X_{1i}, X_{2i}, \dots, X_{ki})'$ 是第 i 个观测的解释变量向量（包含常数项 1）。

2.1.2 回归模型的经济与统计含义

经济意义：以消费函数为例，假设消费 C 与收入 Y 的关系为 $C_i = \beta_0 + \beta_1 Y_i + u_i$ ，其中 β_1 表示边际消费倾向，即收入每增加 1 单位，消费平均增加 β_1 单位。

统计意义：在给定 X_i 的条件下， Y_i 的期望值为 $E(Y_i|X_i) = \beta_0 + \beta_1 X_i$ ，这被称为总体回归函数 (PRF)。回归分析旨在通过样本数据估计这一条件期望函数。

2.1.3 经典模型的基本假设（含球形扰动项）

为得到有效的统计推断，经典线性回归模型通常作以下基本假设：

1. 线性于参数：模型形式如 $\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{u}$ 。
2. 严格外生性： $E(\mathbf{u}|\mathbf{X}) = \mathbf{0}$ ，即给定所有解释变量观测值，扰动项的条件均值为零向量。
3. 不存在完全多重共线性：解释变量矩阵 $\mathbf{X}$ 列满秩，即 $\text{rank}(\mathbf{X}) = k + 1$ 。
4. 球形扰动项（同方差且无自相关）：
 - 同方差性： $\text{Var}(u_i|\mathbf{X}) = \sigma^2$ （常数）
 - 无自相关： $\text{Cov}(u_i, u_j|\mathbf{X}) = 0, \quad i \neq j$

用矩阵形式表示为： $\text{Var}(\mathbf{u}|\mathbf{X}) = \sigma^2 \mathbf{I}_n$ ，其中 $\mathbf{I}_n$ 为 n 阶单位矩阵。

5. 正态性假设（用于小样本精确推断）： $\mathbf{u}|\mathbf{X} \sim N(\mathbf{0}, \sigma^2 \mathbf{I}_n)$

球形扰动项可简洁表示为： $\text{Var}(\mathbf{u}|\mathbf{X}) = \sigma^2 \mathbf{I}_n$ ，其中 $\mathbf{I}_n$ 为 n 阶单位矩阵。

2.2 总体回归与样本回归：模型的核心概念

2.2.1 总体回归函数与样本回归函数

总体回归函数 (PRF) 描述了在给定解释变量 $\mathbf{X}$ 条件下，因变量 $\mathbf{Y}$ 的条件期望：

$$E(\mathbf{Y}|\mathbf{X}) = \mathbf{X}\boldsymbol{\beta}$$

由于我们通常无法观测到整个总体，PRF 是未知的。

样本回归函数 (SRF) 是根据样本数据对 PRF 的估计：

$$\hat{Y} = X\hat{\beta}$$

其中 $\hat{\beta}$ 是总体参数 β 的估计量, $\hat{Y}$ 是 Y 的拟合值向量。

2.2.2 随机扰动项的意义与构成

随机扰动项 u 包含以下因素: 1. 被省略的变量影响 2. 测量误差 3. 模型设定误差 (如函数形式误设) 4. 人类行为的随机性

扰动项的存在使得模型从确定性的函数关系转变为统计关系, 这是计量经济学与数学的本质区别之一。

2.2.3 “估计”的本质: 用样本推断总体

估计是从样本统计量推断总体参数的过程。在回归分析中, 我们使用样本数据 (X, Y) 来估计未知的总体参数 β 。良好的估计量应具有无偏性、一致性和有效性等性质。

2.3 参数估计

2.3.1 最小二乘估计及几何意义

最小二乘法的基本思想是寻找参数估计值 $\hat{\beta}$, 使得残差平方和 (SSR) 最小:

$$\min_{\beta} \sum_{i=1}^n (Y_i - x_i' \beta)^2 = \min_{\beta} (Y - X\beta)'(Y - X\beta)$$

目标函数 $S(\beta) = (Y - X\beta)'(Y - X\beta)$ 对 β 求导:

$$\frac{\partial S(\beta)}{\partial \beta} = -2X'Y + 2X'X\beta$$

令导数为零得到正规方程:

$$X'X\hat{\beta} = X'Y$$

假设 $X'X$ 可逆 (即满足无完全多重共线性假设), 得到 OLS 估计量:

$$\hat{\beta} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{Y}$$

几何意义：最小二乘估计是在由解释变量矩阵 $\mathbf{X}$ 的列向量张成的向量空间 $\mathcal{C}(\mathbf{X})$ 中，寻找因变量向量 $\mathbf{Y}$ 在该空间上的正交投影。残差向量 $\mathbf{e} = \mathbf{Y} - \mathbf{X}\hat{\beta}$ 垂直于解释变量空间，即 $\mathbf{X}'\mathbf{e} = \mathbf{0}$ 。

2.3.2 最大似然估计

在正态性假设 $\mathbf{u} \sim N(\mathbf{0}, \sigma^2 \mathbf{I}_n)$ 下， $\mathbf{Y}|\mathbf{X} \sim N(\mathbf{X}\beta, \sigma^2 \mathbf{I}_n)$ 。样本的似然函数为：

$$L(\beta, \sigma^2) = (2\pi\sigma^2)^{-n/2} \exp \left[-\frac{1}{2\sigma^2} (\mathbf{Y} - \mathbf{X}\beta)'(\mathbf{Y} - \mathbf{X}\beta) \right]$$

对数似然函数为：

$$\ln L = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} (\mathbf{Y} - \mathbf{X}\beta)'(\mathbf{Y} - \mathbf{X}\beta)$$

最大化 $\ln L$ 相当于最小化 $(\mathbf{Y} - \mathbf{X}\beta)'(\mathbf{Y} - \mathbf{X}\beta)$ ，因此在正态性假设下，最大似然估计（MLE）与最小二乘估计（OLS）的 β 估计量相同。 σ^2 的 MLE 为 $\hat{\sigma}_{MLE}^2 = \frac{1}{n} \mathbf{e}'\mathbf{e}$ ，而 OLS 通常使用无偏估计量 $s^2 = \frac{1}{n-k-1} \mathbf{e}'\mathbf{e}$ 。

2.3.3 最小二乘估计量的性质（含高斯-马尔科夫定理）

在经典假设 1-4 下，OLS 估计量具有以下优良性质：

1. 线性性： $\hat{\beta} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{Y}$ 是 $\mathbf{Y}$ 的线性函数
2. 无偏性： $E(\hat{\beta}|\mathbf{X}) = \beta$
3. 有效性（高斯-马尔科夫定理）：在所有的线性无偏估计量中，OLS 估计量具有最小方差（BLUE: Best Linear Unbiased Estimator）
4. 一致性：随着样本量增大，估计量依概率收敛于真实参数值

$\hat{\beta}$ 的方差-协方差矩阵为：

$$\text{Var}(\hat{\beta}|\mathbf{X}) = \sigma^2(\mathbf{X}'\mathbf{X})^{-1}$$

σ^2 的无偏估计量为：

$$s^2 = \frac{\mathbf{e}'\mathbf{e}}{n-k-1} = \frac{(\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})'(\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})}{n-k-1}$$

其中 k 为解释变量个数（不含常数项）。

2.3.4 区间估计

在正态性假设下， $\hat{\boldsymbol{\beta}} \sim N(\boldsymbol{\beta}, \sigma^2(\mathbf{X}'\mathbf{X})^{-1})$ 。对于单个系数 $\hat{\beta}_j$ ，标准化后得：

$$\frac{\hat{\beta}_j - \beta_j}{s.e.(\hat{\beta}_j)} \sim t(n-k-1)$$

其中 $s.e.(\hat{\beta}_j) = \sqrt{s^2 \cdot [(\mathbf{X}'\mathbf{X})^{-1}]_{jj}}$ 为估计量的标准误， $[(\mathbf{X}'\mathbf{X})^{-1}]_{jj}$ 表示矩阵 $(\mathbf{X}'\mathbf{X})^{-1}$ 的第 j 个对角元素。

β_j 的 $100(1-\alpha)\%$ 置信区间为：

$$\hat{\beta}_j \pm t_{\alpha/2}(n-k-1) \cdot s.e.(\hat{\beta}_j)$$

2.4 假设检验

2.4.1 回归系数的显著性检验（t 检验）

对于单个系数 β_j ，常用的假设为：

$$H_0 : \beta_j = 0 \quad \text{vs} \quad H_1 : \beta_j \neq 0$$

检验统计量为：

$$t = \frac{\hat{\beta}_j}{s.e.(\hat{\beta}_j)} \sim t(n-k-1) \quad \text{在 } H_0 \text{ 下}$$

若 $|t| > t_{\alpha/2}(n-k-1)$ ，则在 α 显著性水平下拒绝原假设，认为 X_j 对 Y 有显著影响。

案例：研究教育年限（ X ）对收入（ Y ）的影响，得到 $\hat{\beta}_1 = 0.8$ ， $s.e.(\hat{\beta}_1) = 0.2$ ，则 $t = 4.0$ 。在 5% 显著性水平下， $t_{0.025}(n-2) \approx 1.96$ ，由于 $4.0 > 1.96$ ，我们拒绝 $\beta_1 = 0$ 的原假设，认为教育

年限对收入有显著正向影响。

2.4.2 回归方程的显著性检验（F 检验）

用于检验所有解释变量（除常数项外）的联合显著性。考虑约束条件 $\mathbf{R}\boldsymbol{\beta} = \mathbf{r}$ ，其中 $\mathbf{R}$ 为 $q \times (k+1)$ 约束矩阵， $\mathbf{r}$ 为 $q \times 1$ 约束值向量。对于模型整体显著性检验：

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_k = 0 \quad \text{vs} \quad H_1 : \text{至少有一个 } \beta_j \neq 0$$

$$\text{此时 } \mathbf{R} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}_{k \times (k+1)}, \quad \mathbf{r} = \mathbf{0}_{k \times 1}.$$

检验统计量为：

$$F = \frac{(RSS_R - RSS_U)/q}{RSS_U/(n - k - 1)} \sim F(q, n - k - 1) \quad \text{在 } H_0 \text{ 下}$$

其中 RSS_R 为受约束模型（仅含截距项）的残差平方和， RSS_U 为无约束模型的残差平方和。

在矩阵形式下，也可以表示为：

$$F = \frac{(\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r})' [\mathbf{R}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{R}']^{-1} (\mathbf{R}\hat{\boldsymbol{\beta}} - \mathbf{r})/q}{s^2}$$

对于模型整体显著性检验，可简化为：

$$F = \frac{ESS/k}{RSS/(n - k - 1)} \sim F(k, n - k - 1) \quad \text{在 } H_0 \text{ 下}$$

其中 $ESS = \hat{\mathbf{Y}}'\hat{\mathbf{Y}} - n\bar{Y}^2 = \mathbf{Y}'\mathbf{X}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{Y} - n\bar{Y}^2$ 为解释平方和（回归平方和）， $RSS = \mathbf{e}'\mathbf{e} = \mathbf{Y}'\mathbf{Y} - \hat{\boldsymbol{\beta}}'\mathbf{X}'\mathbf{Y}$ 为残差平方和。

2.4.3 假设检验的应用与注意事项

1. 经济显著性 vs. 统计显著性：一个系数可能在统计上显著（p 值很小），但经济意义不大（系数数值很小），反之亦然。

2. 多重检验问题：进行多次检验时，犯第一类错误的概率会增加，需考虑 Bonferroni 校正等方法。
3. 单侧检验：当理论或经验表明影响方向确定时，可使用单侧检验，提高检验功效。

2.5 回归分析和方差分析

2.5.1 回归分析与方差分析的区别与联系

回归分析和方差分析（ANOVA）都是线性模型的特例，二者既有区别又有密切联系：

区别：1. 变量类型：回归分析主要处理连续型解释变量，而方差分析主要处理分类变量（因子）。2. 模型设定：回归分析关注变量间的函数关系，方差分析关注不同组别间的均值差异。3. 参数解释：回归系数表示解释变量单位变化对因变量的平均影响；方差分析中的效应表示组别间的差异。4. 研究目的：回归分析侧重于预测和解释变量关系；方差分析侧重于检验组间差异的显著性。

联系：1. 统一框架：两者都属于一般线性模型（GLM）框架，可用统一矩阵形式表示。2. 统计基础：都基于最小二乘估计和 F 检验。3. 方差分解：都涉及总变异的分解（ $SST = SSR + SSE$ ）。4. 可相互转化：回归分析中的虚拟变量回归等价于单因素方差分析。

实际上，方差分析可以看作是回归分析的特例，其中解释变量为分类变量的指示变量（虚拟变量）。例如，单因素方差分析模型：

$$Y_{ij} = \mu + \alpha_j + \epsilon_{ij}, \quad i = 1, \dots, n_j, \quad j = 1, \dots, k$$

可以改写为回归模型：

$$Y_i = \beta_0 + \beta_1 D_{1i} + \beta_2 D_{2i} + \dots + \beta_{k-1} D_{(k-1)i} + u_i$$

其中 D_{ji} 为虚拟变量。

2.5.2 方差分析的基本思想：变异的分解

无论是回归分析还是方差分析，核心思想都是将总变异分解为可解释部分和不可解释部分：

$$\text{总变异} = \text{模型解释的变异} + \text{随机误差变异}$$

在回归分析中：

$$TSS = \sum_{i=1}^n (Y_i - \bar{Y})^2 = \mathbf{Y}'\mathbf{Y} - n\bar{Y}^2 = \underbrace{\sum_{i=1}^n (\hat{Y}_i - \bar{Y})^2}_{ESS} + \underbrace{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}_{RSS}$$

在方差分析中（单因素）：

$$SST = \sum_{j=1}^k \sum_{i=1}^{n_j} (Y_{ij} - \bar{Y})^2 = \underbrace{\sum_{j=1}^k n_j (\bar{Y}_j - \bar{Y})^2}_{SSB} + \underbrace{\sum_{j=1}^k \sum_{i=1}^{n_j} (Y_{ij} - \bar{Y}_j)^2}_{SSW}$$

其中 SSB 为组间平方和， SSW 为组内平方和。

2.5.3 回归方差分析表：构建与解读

回归分析的方差分析表如下：

变异来源	平方和	自由度	均方	F 值
回归（解释）	ESS	k	$MSE = ESS/k$	$F = MSE/MSE_R$
残差（未解释）	RSS	$n - k - 1$	$MSE_R = RSS/(n - k - 1)$	
总计	TSS	$n - 1$		

而单因素方差分析表为：

变异来源	平方和	自由度	均方	F 值
组间（处理）	SSB	$k - 1$	$MSB = SSB/(k - 1)$	$F = MSB/MSW$
组内（误差）	SSW	$n - k$	$MSW = SSW/(n - k)$	
总计	SST	$n - 1$		

比较两者可以发现结构上的高度相似性。

2.5.4 一元回归与方差分析

在一元线性回归 ($k = 1$) 中， F 检验与 t 检验等价，且 $F = t^2$ 。此时的 F 检验等价于检验回归系数 β_1 是否显著不为零。

特别地，当解释变量 X 为二分类变量（取值为 0 或 1）时，一元线性回归等价于两样本 t 检验，也等价于二水平的单因素方差分析。

案例：研究性别（男=1，女=0）对收入的影响。回归模型 $Y_i = \beta_0 + \beta_1 X_i + u_i$ 中， β_0 表示女性的平均收入， $\beta_0 + \beta_1$ 表示男性的平均收入， β_1 表示性别收入差异。检验 $H_0: \beta_1 = 0$ 等价于检验男女收入均值是否相等。

2.5.5 方差分析与拟合优度的内在联系

拟合优度 R^2 可从方差分析表直接计算：

$$R^2 = \frac{ESS}{TSS} = 1 - \frac{RSS}{TSS}$$

在方差分析中，类似的度量是 η^2 （eta 平方）：

$$\eta^2 = \frac{SSB}{SST}$$

表示组间变异占总变异的比例。

F 统计量也可用 R^2 表示：

$$F = \frac{R^2/k}{(1 - R^2)/(n - k - 1)}$$

这显示了模型解释能力与统计显著性之间的直接关系。

2.6 模型的拟合优度与相关分析

2.6.1 总离差平方和的分解与相关系数

在一元回归中，样本相关系数 r 与斜率估计 $\hat{\beta}_1$ 的关系为：

$$r = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2 \sum_{i=1}^n (Y_i - \bar{Y})^2}} = \hat{\beta}_1 \cdot \frac{s_X}{s_Y}$$

其中 s_X 和 s_Y 分别为 X 和 Y 的样本标准差。

对于多元回归，有复相关系数 R ，定义为 Y 与 $\hat{Y}$ 的简单相关系数，且 $R = \sqrt{R^2}$ 。

2.6.2 样本决定系数与总体决定系数

样本决定系数 R^2 衡量样本中模型拟合优度：

$$R^2 = \frac{ESS}{TSS} = 1 - \frac{RSS}{TSS}$$

R^2 的取值范围为 $[0, 1]$ ，值越大表示模型解释变异的能力越强。

总体决定系数 ρ^2 是总体中的决定系数，通常未知，用 R^2 估计。

2.6.3 调整决定系数、估计标准误及其意义

R^2 的一个缺点是随解释变量增加而单调递增（即使增加无关变量）。调整 R^2 修正了自由度：

$$\bar{R}^2 = 1 - \frac{RSS/(n-k-1)}{TSS/(n-1)} = 1 - (1 - R^2) \frac{n-1}{n-k-1}$$

调整 R^2 可能为负，此时模型拟合比仅用均值预测更差。

估计标准误 SER (Standard Error of Regression)：

$$SER = \sqrt{\frac{RSS}{n-k-1}} = \sqrt{s^2}$$

反映回归模型的预测精度，单位与 Y 相同。

2.6.4 拟合优度与相关分析的局限性

1. R^2 低不一定表示模型差：在某些领域（如横截面数据分析）， R^2 通常较低，但模型仍有意义。
2. R^2 高不一定表示模型好：可能过拟合，或存在伪回归（非平稳时间序列）。
3. 相关系数不意味着因果关系：相关关系可能由共同因素、反向因果或偶然性导致。
4. R^2 与预测能力的不一致：高 R^2 不一定意味着好的样本外预测能力，特别是当模型过拟合时。

2.7 基于回归模型的预测

2.7.1 点预测与区间预测

给定解释变量新值向量 $\mathbf{x}_0 = (1, X_{10}, X_{20}, \dots, X_{k0})'$:

- 点预测: $\hat{Y}_0 = \mathbf{x}_0' \hat{\boldsymbol{\beta}}$
- 均值预测: 预测 $E(Y|\mathbf{X} = \mathbf{x}_0)$ 的置信区间
- 个值预测: 预测单个 Y_0 的预测区间

2.7.2 均值预测与个值预测的区间差异

均值预测的方差为:

$$Var(\hat{Y}_0) = Var(\mathbf{x}_0' \hat{\boldsymbol{\beta}}) = \mathbf{x}_0' Var(\hat{\boldsymbol{\beta}}) \mathbf{x}_0 = \sigma^2 \mathbf{x}_0' (\mathbf{X}' \mathbf{X})^{-1} \mathbf{x}_0$$

个值预测的方差较大 (包含个体随机扰动):

$$Var(Y_0 - \hat{Y}_0) = Var(u_0) + Var(\hat{Y}_0) = \sigma^2 (1 + \mathbf{x}_0' (\mathbf{X}' \mathbf{X})^{-1} \mathbf{x}_0)$$

因此, 个值预测区间比均值预测区间更宽, 反映了个体预测的不确定性更大。

均值预测的 $100(1 - \alpha)\%$ 置信区间为:

$$\mathbf{x}_0' \hat{\boldsymbol{\beta}} \pm t_{\alpha/2}(n - k - 1) \cdot s \sqrt{\mathbf{x}_0' (\mathbf{X}' \mathbf{X})^{-1} \mathbf{x}_0}$$

个值预测的 $100(1 - \alpha)\%$ 预测区间为:

$$\mathbf{x}_0' \hat{\boldsymbol{\beta}} \pm t_{\alpha/2}(n - k - 1) \cdot s \sqrt{1 + \mathbf{x}_0' (\mathbf{X}' \mathbf{X})^{-1} \mathbf{x}_0}$$

2.7.3 预测评估

预测性能常用指标:

1. 均方预测误差 (**MSPE**): $\frac{1}{m} \sum_{j=1}^m (Y_{0j} - \hat{Y}_{0j})^2$
2. 平均绝对预测误差 (**MAFE**): $\frac{1}{m} \sum_{j=1}^m |Y_{0j} - \hat{Y}_{0j}|$

3. **Theil** 不等式系数：衡量预测相对于朴素预测的改进程度

4. 样本外 R^2 : $R_{out}^2 = 1 - \frac{\sum_{j=1}^m (Y_{0j} - \hat{Y}_{0j})^2}{\sum_{j=1}^m (Y_{0j} - \bar{Y}_{train})^2}$, 其中 $\bar{Y}_{train}$ 为训练样本均值

2.8 案例分析

本章总结

线性回归是预测建模的基石，本章系统地介绍了：1. 线性回归模型的基本形式与统计假设，包括矩阵形式和向量形式 2. 总体回归与样本回归的概念区分 3. 参数估计的两种主要方法（最小二乘与最大似然），使用矩阵方法推导多元回归估计 4. 假设检验的原理与应用（t 检验、F 检验） 5. 回归分析与方差分析的区别与联系，二者在一般线性模型框架下的统一性 6. 方差分析的思想与拟合优度的衡量 7. 基于回归模型的预测方法

线性回归虽然形式简单，但深入理解其背后的统计思想、假设条件与限制，是正确应用回归分析进行预测的关键。矩阵表示法不仅使理论推导更简洁，也为实际计算（特别是多元回归）提供了便利框架。

回归分析与方差分析作为线性模型的两种表现形式，既有区别又有联系。理解它们的统一框架有助于灵活应用统计方法解决实际问题。在实际应用中，需警惕模型误设、异方差、多重共线性等问题，这些将在后续章节中详细讨论。

线性回归的稳健性与解释性使其成为实证研究中最常用的工具之一，但“所有模型都是错的，只是有些有用”（George Box），关键在于理解模型的适用条件与局限。

3 模型诊断

本章导读

回归诊断是统计学中的重要环节，它关注的是回归模型是否满足基本假设、模型拟合是否合理以及是否存在异常情况。一个良好的回归模型不仅需要拟合数据，更重要的是要满足统计推断的基本前提条件。本章将系统介绍回归诊断的理论框架、方法与技术，重点探讨残差分析在模型检验中的应用，以及如何处理常见的模型违背问题，如异方差性、异常值和强影响点等。通过本章学习，读者将掌握评估和改进回归模型有效性的系统性方法，为实际数据分析奠定坚实基础。

3.1 诊断概述

3.1.1 目的与意义

回归诊断的主要目的在于验证线性回归模型的基本假设是否成立，识别可能存在的问题，并提出相应的修正策略。其重要意义体现在：

1. 保障统计推断的有效性：当模型假设（如正态性、同方差性、独立性）不成立时，基于最小二乘估计的统计推断（假设检验、置信区间）可能失效。
2. 提升模型预测精度：通过诊断识别异常值、强影响点等，可以改善模型的预测性能。
3. 深入理解数据结构：诊断过程有助于发现数据中的特殊模式、结构变化或测量误差。
4. 模型改进与选择：为模型修正（如变换变量、加权回归等）提供依据。

3.1.2 基本假设回顾

经典线性回归模型的基本假设包括：

线性性假设：因变量 Y 与自变量 $X_1, X_2, \dots, X_p$ 之间存在线性关系：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

独立性假设：随机误差项 ε_i 相互独立：

$$\text{Cov}(\varepsilon_i, \varepsilon_j) = 0, \quad i \neq j$$

同方差性假设：误差项的方差恒定：

$$\text{Var}(\varepsilon_i) = \sigma^2, \quad i = 1, 2, \dots, n$$

正态性假设：误差项服从正态分布：

$$\varepsilon_i \sim N(0, \sigma^2)$$

无多重共线性：自变量之间不存在完全线性相关关系。

3.1.3 诊断内容框架

完整的回归诊断框架包含以下层次：

1. 残差分析：检查残差的分布、随机性和方差齐性
2. 影响分析：识别异常值和强影响点
3. 模型设定检验：验证函数形式、变量选择等
4. 多重共线性诊断：检查自变量间的相关性
5. 修正策略：针对诊断问题提出解决方案

3.2 残差及其标准化形式

3.2.1 普通残差

对于线性回归模型：

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

普通残差（Ordinary Residuals）定义为观测值与拟合值之差：

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$$

其中 $\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ 是最小二乘估计量。

残差向量 $\mathbf{e}$ 的性质：1. $\mathbb{E}(\mathbf{e}) = \mathbf{0}$ 2. $\text{Var}(\mathbf{e}) = \sigma^2(\mathbf{I} - \mathbf{H})$ 3. $\sum_{i=1}^n e_i = 0$ （当模型包含截距项时）4. $\mathbf{X}^T \mathbf{e} = \mathbf{0}$ （正交性）

其中 $\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ 是帽子矩阵（Hat Matrix）。

3.2.2 标准化残差

标准化残差（Standardized Residuals）通过将普通残差除以其标准差估计值得到：

$$r_i = \frac{e_i}{\hat{\sigma} \sqrt{1 - h_{ii}}}$$

其中：- $\hat{\sigma} = \sqrt{\frac{1}{n-p-1} \sum_{i=1}^n e_i^2}$ 是误差标准差的无偏估计 - h_{ii} 是帽子矩阵 $\mathbf{H}$ 的第 i 个对角元素（杠杆值）

推导过程：

$$\text{Var}(e_i) = \text{Var}(y_i - \hat{y}_i) = \sigma^2(1 - h_{ii})$$

由于 $\mathbb{E}(e_i) = 0$ ，标准化后近似有 $r_i \sim N(0, 1)$ 。

3.2.3 学生化残差

学生化残差（Studentized Residuals）或称删除残差（Deleted Residuals）定义为：

$$t_i = \frac{e_i}{\hat{\sigma}_{(i)} \sqrt{1 - h_{ii}}}$$

其中 $\hat{\sigma}_{(i)}$ 是删除第 i 个观测后重新拟合模型得到的 σ 的估计值。

计算方法：

$$\hat{\sigma}_{(i)}^2 = \frac{(n-p-1)\hat{\sigma}^2 - e_i^2/(1-h_{ii})}{n-p-2}$$

学生化残差更精确的性质： $t_i \sim t_{n-p-2}$ ，可用于假设检验。

理论推导：设 $e_{(i)}$ 为删除第 i 个观测后的残差，可以证明：

$$e_i = \frac{e_{(i)}}{1 - h_{ii}}$$

且

$$\text{Var}(e_{(i)}) = \sigma_{(i)}^2 [1 - \mathbf{x}_i^T (\mathbf{X}_{(i)}^T \mathbf{X}_{(i)})^{-1} \mathbf{x}_i]$$

3.3 图形诊断法

3.3.1 残差图分析

残差 vs 拟合值图 (Residuals vs Fitted Plot) - 目的: 检查线性性、同方差性 - 理想情况: 残差随机分布在 0 附近, 无明显模式 - 异常模式: 1. 漏斗形: 异方差 2. 曲线形: 非线性关系 3. 离群点: 异常值

正态 Q-Q 图 (Normal Quantile-Quantile Plot) - 目的: 检验误差正态性 - 构造: 将排序后的标准化残差与理论正态分位数对比 - 判断: 点应近似落在对角线上 - 常见偏离: 1. S 形曲线: 重尾或轻尾分布 2. C 形曲线: 偏态分布

尺度-位置图 (Scale-Location Plot) - 目的: 检验同方差性 - 纵轴: $\sqrt{|\cdot|}$ - 横轴: 拟合值 - 判断: 水平趋势表示方差恒定, 倾斜趋势表示异方差

残差 vs 杠杆图 (Residuals vs Leverage Plot) - 目的: 识别强影响点 - 纵轴: 学生化残差 - 横轴: 杠杆值 h_{ii} - 添加 Cook 距离的等高线

3.3.2 常用诊断图形

偏残差图 (Partial Residual Plot) 显示每个自变量与因变量的偏关系:

$$e_i^{(j)} = e_i + \hat{\beta}_j x_{ij}$$

用于检验每个自变量的函数形式。

添加变量图 (Added-Variable Plot) 展示在控制其他变量后, 某个自变量与因变量的关系。

Cook 距离图显示每个观测对回归系数估计的影响大小:

$$D_i = \frac{r_i^2}{p+1} \cdot \frac{h_{ii}}{1-h_{ii}}$$

经验准则: $D_i > 1$ 或 $D_i > 4/(n-p-1)$ 需关注。

3.4 异常值与强影响点

3.4.1 识别方法

异常值 (Outlier): 指 y 方向上的离群观测, 残差绝对值异常大。

统计检验方法：1. 标准化残差法： $|r_i| > 2$ 或 $|r_i| > 3$ 2. 学生化残差法： $|t_i| > t_{n-p-2}(1 - \alpha/2n)$ (Bonferroni 校正) 3. 外学生化残差：

$$t_i^* = \frac{e_i}{\hat{\sigma}\sqrt{1-h_{ii}}} \cdot \sqrt{\frac{n-p-2}{n-p-1-t_i^2}}$$

服从 t_{n-p-2} 分布

强影响点 (Influential Point)：对回归系数估计有较大影响的观测。

识别指标：1. 杠杆值 (Leverage)：

$$h_{ii} = \mathbf{x}_i^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_i$$

性质： $0 \leq h_{ii} \leq 1$, $\sum h_{ii} = p + 1$ 经验法则： $h_{ii} > 2(p + 1)/n$ 为高杠杆点

2. DFFITS：

$$\text{DFFITS}_i = \frac{\hat{y}_i - \hat{y}_{(i)}}{\hat{\sigma}_{(i)} \sqrt{h_{ii}}}$$

反映第 i 个观测对自身拟合值的影响临界值： $2\sqrt{(p + 1)/n}$

3. DFBETAS：

$$\text{DFBETAS}_{j(i)} = \frac{\hat{\beta}_j - \hat{\beta}_{j(i)}}{\hat{\sigma}_{(i)} \sqrt{c_{jj}}}$$

其中 c_{jj} 是 $(\mathbf{X}^T \mathbf{X})^{-1}$ 的第 j 个对角元素临界值： $2/\sqrt{n}$

3.4.2 影响评估

Cook 距离：综合度量

$$D_i = \frac{(\hat{\beta} - \hat{\beta}_{(i)})^T (\mathbf{X}^T \mathbf{X}) (\hat{\beta} - \hat{\beta}_{(i)})}{(p + 1) \hat{\sigma}^2}$$

等价形式：

$$D_i = \frac{r_i^2}{p + 1} \cdot \frac{h_{ii}}{1 - h_{ii}}$$

解释：衡量删除第 i 个观测后所有拟合值变化的平均程度。

协方差比：

$$\text{COVRATIO}_i = \frac{\det[\hat{\sigma}_{(i)}^2 (\mathbf{X}_{(i)}^T \mathbf{X}_{(i)})^{-1}]}{\det[\hat{\sigma}^2 (\mathbf{X}^T \mathbf{X})^{-1}]}$$

近似： $\text{COVRATIO}_i \approx 1 + \frac{(p+1)h_{ii}-r_i^2}{n-p-1}$ 准则： $|\text{COVRATIO}_i - 1| > 3(p + 1)/n$

3.4.3 处理策略

诊断后的决策流程：

1. 核实数据：
 - 检查数据录入错误
 - 确认测量准确性
 - 了解数据收集背景
2. 分析影响：
 - 区分异常值和强影响点
 - 评估对结论的影响程度
 - 进行敏感性分析
3. 处理方法：
 - 保留：若为合理变异且影响不大
 - 删除：若确定为错误数据且无法修正
 - 变换：使用 Box-Cox 变换、对数变换等
 - 稳健方法：使用 M 估计、LMS、LTS 等
 - 增加变量：考虑遗漏的重要变量
 - 分段建模：识别不同的数据生成机制
4. 报告原则：
 - 透明报告异常值情况
 - 比较包含与不包含异常值的结果
 - 说明处理依据和潜在影响

3.5 异方差的检验

3.5.1 图形检验法

残差 vs 拟合值图的异方差识别：1. 递增型异方差：残差散点呈漏斗形（开口向右或向左）2. 递减型异方差：残差散点呈倒漏斗形 3. 复杂型异方差：残差变异与拟合值有函数关系

等级相关检验的图形辅助：绘制 $|e_i|$ 与 $\hat{y}_i$ 的散点图，观察趋势。

偏残差图的特殊应用：当怀疑异方差与特定自变量相关时，绘制 $|e_i|$ 与该自变量的散点图。

3.5.2 统计检验法

Breusch-Pagan 检验 (BP 检验): 1. 估计原模型: $y_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i$ 2. 计算残差平方: $u_i = e_i^2$ 3. 辅助回归: $u_i = \alpha_0 + \mathbf{z}_i^T \boldsymbol{\alpha} + v_i$ 其中 $\mathbf{z}_i$ 通常包含 $\mathbf{x}_i$ 的函数 4. 检验统计量:

$$BP = \frac{1}{2} \mathbf{f}^T \mathbf{Z} (\mathbf{Z}^T \mathbf{Z})^{-1} \mathbf{Z}^T \mathbf{f} \sim \chi_q^2$$

其中 $f_i = (e_i^2 / \hat{\sigma}^2) - 1$, q 为 $\mathbf{Z}$ 的列数

White 检验 (一般异方差检验): 1. 辅助回归: $e_i^2 = \alpha_0 + \sum_{j=1}^p \alpha_j x_{ij} + \sum_{j=1}^p \sum_{k \geq j}^p \alpha_{jk} x_{ij} x_{ik} + v_i$ 2. 检验统计量: $nR^2 \sim \chi_q^2$, 其中 $q = p(p+3)/2$ 3. 特点: 不预设异方差形式, 但自由度较大

Goldfeld-Quandt 检验: 适用于方差与单一变量单调相关的情况。1. 按可疑变量排序, 去除中间 c 个观测 (通常 $c \approx n/4$) 2. 分别对前后两组回归, 得残差平方和 SSR_1, SSR_2 3. 检验统计量:

$$F = \frac{SSR_2 / (n_2 - p - 1)}{SSR_1 / (n_1 - p - 1)} \sim F_{n_2 - p - 1, n_1 - p - 1}$$

Park 检验: 假设 $\sigma_i^2 = \sigma^2 \exp(\mathbf{z}_i^T \boldsymbol{\gamma})$, 检验 $\ln(e_i^2) = \ln(\sigma^2) + \mathbf{z}_i^T \boldsymbol{\gamma} + \ln(v_i)$ 中的 $\boldsymbol{\gamma} = \mathbf{0}$

Glejser 检验: 用 $|e_i|$ 对自变量进行各种函数形式的回归, 检验显著性。

3.6 异方差的修正

3.6.1 稳健标准误差法

White 异方差一致协方差矩阵估计:

$$\hat{\text{Var}}_{HC0}(\hat{\boldsymbol{\beta}}) = (\mathbf{X}^T \mathbf{X})^{-1} \left(\sum_{i=1}^n e_i^2 \mathbf{x}_i \mathbf{x}_i^T \right) (\mathbf{X}^T \mathbf{X})^{-1}$$

改进版本: 1. **HC1** (小样本修正):

$$\hat{\text{Var}}_{HC1}(\hat{\boldsymbol{\beta}}) = \frac{n}{n - p - 1} \hat{\text{Var}}_{HC0}(\hat{\boldsymbol{\beta}})$$

2. **HC2**:

$$\hat{\text{Var}}_{HC2}(\hat{\boldsymbol{\beta}}) = (\mathbf{X}^T \mathbf{X})^{-1} \left(\sum_{i=1}^n \frac{e_i^2}{1 - h_{ii}} \mathbf{x}_i \mathbf{x}_i^T \right) (\mathbf{X}^T \mathbf{X})^{-1}$$

3. HC3:

$$\hat{\text{Var}}_{HC3}(\hat{\beta}) = (\mathbf{X}^T \mathbf{X})^{-1} \left(\sum_{i=1}^n \frac{e_i^2}{(1 - h_{ii})^2} \mathbf{x}_i \mathbf{x}_i^T \right) (\mathbf{X}^T \mathbf{X})^{-1}$$

使用建议: HC3 在小样本中表现更好, HC0 在大样本中足够。

3.6.2 模型变换法

方差稳定化变换:

1. 若 $\text{Var}(\varepsilon_i) \propto \mathbb{E}(y_i)^k$:

- - $k = 1$: 考虑 $\sqrt{y}$ 变换
- - $k = 2$: 考虑 $\ln(y)$ 变换
- - $k = 3$: 考虑 $1/y$ 变换

2. **Box-Cox** 变换:

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(y), & \lambda = 0 \end{cases}$$

通过最大似然估计 λ 。

加权最小二乘法的模型变换视角: 设 $\text{Var}(\varepsilon_i) = \sigma^2/\omega_i$, 则变换模型为:

$$\sqrt{\omega_i} y_i = \sqrt{\omega_i} \mathbf{x}_i^T \boldsymbol{\beta} + \sqrt{\omega_i} \varepsilon_i$$

3.6.3 加权最小二乘法

理论框架: 假设 $\text{Var}(\varepsilon_i) = \sigma_i^2 = \sigma^2/\omega_i$, 加权最小二乘估计为:

$$\hat{\boldsymbol{\beta}}_{WLS} = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}$$

其中 $\mathbf{W} = \text{diag}(\omega_1, \dots, \omega_n)$ 。

方差-协方差矩阵:

$$\text{Var}(\hat{\boldsymbol{\beta}}_{WLS}) = \sigma^2 (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1}$$

权重确定方法:

1. 基于方差的权重:

- 若已知 σ_i^2 , 取 $\omega_i = 1/\sigma_i^2$

- 若 $\sigma_i^2 = \sigma^2 g(\mathbf{x}_i)$, 需估计 $g(\cdot)$
- 2. 两步估计法:
 - a. 用 OLS 估计模型, 得残差 $\hat{e}_i$
 - b. 用 $|\hat{e}_i|$ 或 $\hat{e}_i^2$ 对 $\mathbf{x}_i$ 回归, 得方差函数估计 $\hat{g}(\mathbf{x}_i)$
 - c. 取权重 $\hat{\omega}_i = 1/\hat{g}(\mathbf{x}_i)$
 - d. 用 WLS 重新估计
- 3. 迭代重新加权最小二乘 (IRLS):
 - 初始化: $\omega_i^{(0)} = 1$
 - 迭代: $t = 1, 2, \dots$
 1. $\hat{\boldsymbol{\beta}}^{(t)} = (\mathbf{X}^T \mathbf{W}^{(t-1)} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W}^{(t-1)} \mathbf{y}$
 2. 计算 $e_i^{(t)} = y_i - \mathbf{x}_i^T \hat{\boldsymbol{\beta}}^{(t)}$
 3. 更新权重: $\omega_i^{(t)} = 1/f(e_i^{(t)}, \mathbf{x}_i)$
 - 直至收敛

可行广义最小二乘 (FGLS): 更一般的框架, 处理异方差和自相关问题。

3.7 案例分析

本章小结

本章系统介绍了回归诊断与残差分析的理论框架和方法体系。主要要点如下:

1. 诊断基础: 回顾了经典线性回归的基本假设, 构建了完整的诊断框架, 强调了诊断在保障统计推断有效性方面的重要性。
2. 残差分析: 详细阐述了普通残差、标准化残差和学生化残差的定义、性质及计算方法, 这些是诊断分析的基础工具。
3. 图形诊断: 介绍了残差图、Q-Q 图、尺度-位置图等可视化方法, 这些图形能直观揭示模型的潜在问题。
4. 异常值识别: 区分了异常值和强影响点, 系统介绍了杠杆值、Cook 距离、DFFITS、DFBETAS 等多种识别指标及其临界值。
5. 异方差检验: 从图形法和统计检验法两个角度, 详细说明了 Breusch-Pagan 检验、White 检验、Goldfeld-Quandt 检验等方法的原理和应用。
6. 异方差修正: 重点介绍了稳健标准误法和加权最小二乘法, 包括 White 异方差一致估计、HC 系列改进, 以及 WLS 的权重确定方法和迭代算法。

回归诊断是一个系统性工程，需要综合运用多种工具和方法。在实际应用中，应遵循“诊断-修正-再诊断”的迭代过程，确保最终模型既满足统计假设，又具有良好的解释和预测能力。同时，对异常值的处理需谨慎，应结合专业背景知识进行决策。

4 自相关

本章导读

在前面的学习中，我们掌握了线性回归模型，并学会了诊断和处理横截面数据中的常见问题，如异方差。然而，当数据按时间顺序收集时（如月度 GDP、日股票收益率、年度气候数据），一个新的挑战出现了：时间的连续性往往意味着误差项的连续性。相邻时期的随机扰动可能不再独立，而是相互关联——这种现象称为自相关。

本章将系统探讨自相关问题：它如何产生？会带来什么严重后果？如何有效诊断？以及如何正确处理？我们将重点学习自相关的图形与统计诊断方法，并掌握两种主流处理思路：修正模型的广义差分法与修正推断的稳健标准误法。通过本章的学习，你将能够正确处理包含时间维度的回归数据，避免因忽视自相关而得出误导性结论。

需要特别强调的是，本章聚焦于回归分析框架下的自相关问题。时间序列分析作为一门独立课程，会涵盖更广泛的内容，如趋势分解、季节性调整、ARIMA 模型、协整分析等。本章的目标是为后续的时间序列课程奠定坚实的理论基础。

4.1 自相关的定义、来源与影响

4.1.1 自相关的定义

在经典线性回归模型中，我们假设误差项 ϵ_i 相互独立，即对于任意 $i \neq j$ ，有：

$$\text{Cov}(\epsilon_i, \epsilon_j) = 0$$

然而，当数据按时间顺序排列时，这一假设常常被违背。自相关（Autocorrelation），也称为序列相关（Serial Correlation），指的是误差项在不同时期之间存在相关性。正式地，对于时间序列数据 $\{y_t, x_t\}_{t=1}^T$ ，自相关意味着：

$$\text{Cov}(\epsilon_t, \epsilon_{t-k}) \neq 0 \quad \text{对于某个 } k \geq 1$$

其中 k 表示滞后阶数。

常见形式

1. 一阶自相关（最常见）：

$$\epsilon_t = \rho\epsilon_{t-1} + u_t, \quad u_t \sim \text{i.i.d.}(0, \sigma_u^2)$$

其中 $|\rho| < 1$ 为自相关系数。

2. 高阶自相关：误差项与多个滞后期的误差相关

$$\epsilon_t = \rho_1\epsilon_{t-1} + \rho_2\epsilon_{t-2} + \cdots + \rho_p\epsilon_{t-p} + u_t$$

3. 移动平均形式：

$$\epsilon_t = u_t + \theta_1 u_{t-1} + \cdots + \theta_q u_{t-q}$$

自相关 vs. 异方差

理解自相关与第 3 章讨论的异方差的区分至关重要：

- 异方差：误差项的方差随解释变量或观测值变化，但不同观测的误差仍相互独立
- 自相关：误差项之间存在相关性，方差可能恒定也可能变化

这两种问题可能同时存在，本章 4.3 节将提供统一的处理框架。

4.1.2 自相关的来源

自相关本质上是时间依赖性在误差项中的体现。在时间序列数据中，自相关主要源于以下几个方面：

1. 时间序列数据的内在动态性（核心来源）

时间序列数据与横截面数据的根本区别在于观测值在时间上的排序具有经济意义。这种时间顺序导致：

- 经济行为的惯性与调整成本：

- 消费习惯具有持续性：本期消费水平很大程度上受上期消费影响
- 投资决策的调整需要时间：企业不会立即调整资本存量
- 示例： $C_t = \alpha + \beta Y_t + \gamma C_{t-1} + \epsilon_t$ ，若误设为 $C_t = \alpha + \beta Y_t + \epsilon_t$ ，则 C_{t-1} 的影响进入误差项，导致自相关
- 信息传播与学习过程：
 - 经济主体基于过去信息形成预期
 - 技术扩散、制度变迁通常是渐进过程
 - 这种缓慢调整过程在误差项中表现为正自相关
- 市场摩擦与刚性：
 - 价格粘性、工资刚性导致经济变量不能立即调整
 - 合同期限、菜单成本等阻碍即时调整

2. 模型设定偏误

当回归模型未能充分捕捉数据生成过程时，遗漏的动态特征会进入误差项：

- 遗漏动态解释变量：
 - 遗漏滞后因变量： $y_t = \beta_0 + \beta_1 x_t + \gamma y_{t-1} + \epsilon_t$ 误设为 $y_t = \beta_0 + \beta_1 x_t + u_t$
 - 遗漏滞后解释变量： $y_t = \beta_0 + \beta_1 x_t + \beta_2 x_{t-1} + \epsilon_t$ 误设为 $y_t = \beta_0 + \beta_1 x_t + u_t$

在这些情况下， $u_t = \gamma y_{t-1} + \epsilon_t$ 或 $u_t = \beta_2 x_{t-1} + \epsilon_t$ 必然具有自相关。

- 错误的函数形式：
 - 本应使用对数线性模型却用了线性模型
 - 忽略交互项或非线性关系
 - 这些误设导致的误差项往往呈现系统性模式
- 遗漏随时间缓慢变化的变量：
 - 文化、制度、技术等难以量化的因素
 - 这些变量通常具有高度持续性，其影响进入误差项后产生自相关

3. 数据生成与处理过程

- 数据平滑与季节调整：
 - 移动平均滤波： $y_t^* = \frac{1}{3}(y_{t-1} + y_t + y_{t+1})$
 - 这种平滑操作会在相邻观测值间引入人为相关性
- 指标构造方法：
 - 库存投资 = 期末库存 - 期初库存

- 这种差分构造使相邻时期数据自然相关
- 测量误差的持续性：
 - 统计方法的系统性偏差
 - 测量工具的校准问题

4. 空间自相关的类比

虽然本章聚焦时间序列，但有必要提及空间自相关作为类比：

- 空间自相关：地理上接近的地区误差项相关
- 时间自相关：时间上接近的时期误差项相关
- 两者都违反了误差项独立性假设，但产生机制不同

4.1.3 自相关的影响

理解自相关的影响是重视其诊断与处理的前提。自相关对回归分析的影响是严重且多方面的：

1. 对 OLS 估计量的影响

- 无偏性仍然成立：在解释变量严格外生的条件下 ($E(\epsilon_t|\mathbf{X}) = 0$)，OLS 估计量 $\hat{\beta}$ 仍是无偏的：

$$E(\hat{\beta}) = \beta$$

这一性质很重要，常被误解。自相关破坏的是有效性，而非无偏性。

- 有效性丧失（严重问题）：OLS 估计量不再是最优线性无偏估计量。在自相关存在时，OLS 估计量的方差不是所有线性无偏估计量中最小的。正式地，对于模型 $\mathbf{y} = \mathbf{X}\beta + \epsilon$, $\text{Var}(\epsilon) = \sigma^2 \Omega$ ，其中 Ω 非单位矩阵，则：

$$\text{Var}(\hat{\beta}_{OLS}) = \sigma^2 (\mathbf{X}'\mathbf{X})^{-1} \mathbf{X}' \Omega \mathbf{X} (\mathbf{X}'\mathbf{X})^{-1}$$

而 GLS 估计量的方差为：

$$\text{Var}(\hat{\beta}_{GLS}) = \sigma^2 (\mathbf{X}' \Omega^{-1} \mathbf{X})^{-1}$$

可以证明 $\text{Var}(\hat{\beta}_{GLS}) \leq \text{Var}(\hat{\beta}_{OLS})$ 。

2. 对统计推断的影响（严重后果）

这是自相关最危险的方面，也是实践中必须处理自相关的主要原因。

- 标准误被严重低估：当误差项正自相关时 ($\rho > 0$, 经济数据中最常见), OLS 计算的标准误:

$$s.e.(\hat{\beta}_j)_{OLS} = \sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{jj}^{-1}}$$

会系统地低估真实的不确定性。低估的程度取决于:

1. 自相关系数 ρ 的大小
2. 解释变量自身的自相关模式

一个近似公式显示, 对于 AR(1) 误差和高度自相关的解释变量:

$$\frac{\text{真实标准误}}{\text{OLS 标准误}} \approx \sqrt{\frac{1 + \rho_x \rho}{1 - \rho_x \rho}}$$

其中 ρ_x 是解释变量的自相关系数。

- **t** 统计量被高估, 显著性被夸大: 由于 $t = \frac{\hat{\beta}_j}{s.e.(\hat{\beta}_j)}$, 标准误低估导致 t 统计量高估。这会造成:

1. 第一类错误率增加: 本应不显著的变量被误判为显著
2. 置信区间过窄: 置信区间的覆盖率低于名义水平

例如, 假设真实情况 $\beta = 0$, 但正自相关使 OLS t 检验在 5% 水平下的实际拒绝率可能达到 15-20%。

- **F** 检验失效: 对多个系数的联合显著性检验同样不可靠, 因为 F 统计量的分布发生改变。

3. 对预测的影响

- 预测区间计算错误: 基于 OLS 的预测区间公式假设误差独立, 在自相关存在时:
 1. 区间宽度不正确 (通常过窄)
 2. 覆盖率低于名义水平
- 预测误差的相关性: 一步向前预测误差可能包含可用于改进未来预测的信息, 但标准方法忽略了这一点。

数值示例

假设真实模型为:

$$y_t = 2 + 0.5x_t + \epsilon_t, \quad \epsilon_t = 0.7\epsilon_{t-1} + u_t, \quad u_t \sim N(0, 1)$$

模拟 100 次实验, 每次 $T = 50$, 比较:

方法	$\hat{\beta}_1$ 均值	$\hat{\beta}_1$ 标准差	标准误均值	5% 水平下拒绝率
理论值	0.5	-	-	5%
OLS（忽略自相关）	0.501	0.085	0.062	18.3%
GLS（考虑自相关）	0.499	0.075	0.073	5.2%

可以看到：OLS 估计量无偏但更分散；OLS 标准误低估约 27%；t 检验第一类错误率从 5% 上升到 18.3%。

4.2 自相关的诊断方法

诊断自相关需要结合图形观察与统计检验。系统性的诊断流程如下：

4.2.1 图形化诊断

图形方法直观且信息丰富，应作为诊断的第一步。

1. 残差时序图

将 OLS 残差 $\hat{\epsilon}_t$ 按时间顺序绘制，观察是否存在规律性模式。

如何解读：

- 无自相关：残差随机散布在零线周围，无系统模式
- 正自相关：残差倾向于长期停留在零线同一侧，呈现”波浪形”或”集群”模式
- 负自相关：残差频繁穿越零线，呈现”锯齿形”振荡

2. 残差自相关函数图

自相关函数（Autocorrelation Function, ACF）图显示残差与其不同滞后值之间的相关系数：

$$\hat{\rho}(k) = \frac{\sum_{t=k+1}^T \hat{\epsilon}_t \hat{\epsilon}_{t-k}}{\sum_{t=1}^T \hat{\epsilon}_t^2}, \quad k = 1, 2, \dots, K$$

其中 K 通常取 $T/4$ 或 $\sqrt{T}$ 。

如何解读：

- 无自相关：除滞后 0 外，所有 $\hat{\rho}(k)$ 落在置信带内
- 一阶自相关： $\hat{\rho}(1)$ 显著不为零，高阶系数衰减
- 高阶自相关：多个低阶滞后系数显著
- 季节性自相关：滞后 4、8、12 等（季度数据）或滞后 12、24 等（月度数据）显著

置信带计算：对于白噪声序列， $\hat{\rho}(k)$ 近似服从 $N(0, 1/T)$ ，因此 95% 置信带约为 $\pm 1.96/\sqrt{T}$ 。

4.2.2 统计检验方法

1. Durbin-Watson 检验

Durbin-Watson 检验是诊断一阶自相关最经典的方法。

检验统计量：

$$d = \frac{\sum_{t=2}^T (\hat{\epsilon}_t - \hat{\epsilon}_{t-1})^2}{\sum_{t=1}^T \hat{\epsilon}_t^2}$$

统计量性质：- $d \approx 2(1 - \hat{\rho})$ ，其中 $\hat{\rho}$ 是一阶自相关系数估计 - d 的取值范围： $0 \leq d \leq 4$ - $d \approx 2$ ：无自相关 - $d < 2$ ：正自相关（常见） - $d > 2$ ：负自相关

检验步骤：1. 设定原假设 $H_0: \rho = 0$ （无自相关）2. 计算 d 统计量 3. 查 Durbin-Watson 表得上下临界值 d_L 和 d_U 4. 决策规则：- $d < d_L$ ：拒绝 H_0 ，存在正自相关 - $d > 4 - d_L$ ：拒绝 H_0 ，存在负自相关 - $d_U < d < 4 - d_U$ ：不拒绝 H_0 - $d_L \leq d \leq d_U$ 或 $4 - d_U \leq d \leq 4 - d_L$ ：不确定区域

Durbin-Watson 检验的局限性：1. 仅检验一阶自相关：无法检测高阶自相关 2. 存在不确定区域：需要查表，且部分情况下无法下结论 3. 不适用于含滞后因变量的模型：检验统计量的分布改变 4. 对小样本敏感：样本量小时检验功效低

现代实践建议：Durbin-Watson 检验适合作为初步筛查，但不应作为唯一诊断工具。

2. Breusch-Godfrey 检验（推荐）

Breusch-Godfrey 检验（也称为 Lagrange Multiplier 检验）是更通用、更灵活的自相关检验方法。

检验原理：1. 估计原模型： $y_t = \beta_0 + \beta_1 x_{1t} + \dots + \beta_k x_{kt} + \epsilon_t$ 2. 获取 OLS 残差 $\hat{\epsilon}_t$ 3. 估计辅助回归：

$$\hat{\epsilon}_t = \delta_0 + \delta_1 x_{1t} + \dots + \delta_k x_{kt} + \phi_1 \hat{\epsilon}_{t-1} + \dots + \phi_p \hat{\epsilon}_{t-p} + v_t$$

4. 检验原假设 $H_0: \phi_1 = \phi_2 = \dots = \phi_p = 0$

检验统计量:

$$LM = (T - p)R_{\text{aux}}^2 \stackrel{H_0}{\sim} \chi^2(p)$$

其中 R_{aux}^2 是辅助回归的决定系数, p 是检验的滞后阶数。

Breusch-Godfrey 检验的优势: 1. 可检验高阶自相关: 通过选择 p 检验任意阶自相关 2. 适用于含滞后因变量的模型: 检验统计量的分布不变 3. 无不确定区域: 基于 χ^2 分布, 直接得到 p 值 4. 渐近性质优良: 大样本下功效高

滞后阶数 p 的选择: - 一般规则: $p = \lfloor T^{1/3} \rfloor$ 或 $p = \lfloor T/4 \rfloor$ - 对于季度数据: 至少检验 $p = 4$ (检验季节性自相关) - 对于月度数据: 至少检验 $p = 12$ - 也可通过信息准则 (AIC/BIC) 选择

检验方法选择指南

基于以上分析, 推荐以下诊断流程:

自相关诊断流程: 1. 步骤 1: 图形诊断 - 绘制残差时序图, 观察系统性模式 - 绘制 ACF 图, 观察自相关系数的衰减模式

2. 步骤 2: 统计检验

- 计算 Durbin-Watson 统计量 (初步筛查)
- 进行 Breusch-Godfrey 检验 (滞后 1-4 阶)
- 如果数据有明显季节频率, 增加相应滞后检验

3. 步骤 3: 综合判断

- 如果图形显示模式且统计检验显著: 存在自相关, 需要处理
- 如果图形无模式但统计检验显著: 可能存在微弱自相关, 建议使用稳健标准误
- 如果图形有模式但统计检验不显著: 检查模型设定, 可能遗漏重要变量
- 如果图形无模式且统计检验不显著: 无自相关证据, 可放心使用标准方法

4.3 自相关处理方法

发现自相关后, 必须采取适当措施。本节介绍两种主要处理思路, 并提供统一的广义最小二乘法框架。

4.3.1 处理哲学: 两种路径

面对自相关, 研究者有两种基本策略:

1. 修正模型路径：改变模型设定或估计方法，获得更有效的参数估计
 - 目标：最小化估计量方差
 - 方法：广义差分法（Cochrane-Orcutt, Prais-Winsten）
 - 适用：预测研究、政策模拟、需要精确估计的场景
2. 修正推断路径：保持 OLS 估计不变，修正其方差估计，获得有效的假设检验
 - 目标：正确的标准误、t 检验、置信区间
 - 方法：异方差自相关稳健标准误（Newey-West）
 - 适用：因果推断、假设检验、解释变量外生的场景

两种路径并非互斥，实践中常同时报告两种方法的结果。

4.3.2 统一框架：广义最小二乘法视角

广义最小二乘法（Generalized Least Squares, GLS）为处理违反球形扰动假设的问题提供了统一框架。

问题形式化

考虑线性模型：

$$\mathbf{y} = \mathbf{X}\beta + \epsilon, \quad E(\epsilon) = 0, \quad \text{Var}(\epsilon) = \sigma^2 \Omega$$

其中 Ω 是正定对称矩阵。当 $\Omega \neq \mathbf{I}$ 时，误差项违反球形扰动假设。

GLS 估计量

如果 Ω 已知，GLS 估计量为：

$$\hat{\beta}_{GLS} = (\mathbf{X}'\Omega^{-1}\mathbf{X})^{-1}\mathbf{X}'\Omega^{-1}\mathbf{y}$$

其方差为：

$$\text{Var}(\hat{\beta}_{GLS}) = \sigma^2(\mathbf{X}'\Omega^{-1}\mathbf{X})^{-1}$$

GLS 的直观理解

GLS 等价于对原始模型进行变换：

$$\Omega^{-1/2}\mathbf{y} = \Omega^{-1/2}\mathbf{X}\beta + \Omega^{-1/2}\epsilon$$

变换后的误差满足球形扰动假设：

$$\text{Var}(\Omega^{-1/2}\epsilon) = \sigma^2 \mathbf{I}$$

特殊情况：异方差与自相关

GLS 框架统一了第 3 章的异方差问题和本章的自相关问题：

问题类型	Ω 结构	变换方法	对应 GLS
异方差	对角矩阵： $\Omega = \text{diag}(\omega_1, \dots, \omega_T)$	加权最小二乘法	$W = \text{diag}(1/\sqrt{\omega_1}, \dots, 1/\sqrt{\omega_T})$
自相关	非对角矩阵： $\Omega_{ij} = \rho^{ i-j }$ (AR(1) 情况)	广义差分法	$\mathbf{P}$ 使得 $\mathbf{P}'\mathbf{P} = \Omega^{-1}$
同时存在	既非对角又非单位对角	可行 GLS	估计 Ω 然后应用 GLS

可行 GLS 问题

实践中 Ω 未知，需要先估计 Ω ，然后应用 GLS，称为可行广义最小二乘法 (Feasible GLS, FGLS)：

$$\hat{\beta}_{FGLS} = (\mathbf{X}'\hat{\Omega}^{-1}\mathbf{X})^{-1}\mathbf{X}'\hat{\Omega}^{-1}\mathbf{y}$$

其中 $\hat{\Omega}$ 是 Ω 的估计量。FGLS 的有限样本性质复杂，但大样本下通常优于 OLS。

4.3.3 方法一：广义差分法（基于 GLS 思想）

广义差分法针对特定形式的自相关结构进行变换，最常用的是 AR(1) 误差结构。

AR(1) 误差模型

假设误差项服从一阶自回归过程：

$$\epsilon_t = \rho\epsilon_{t-1} + u_t, \quad u_t \sim \text{i.i.d.}(0, \sigma_u^2), \quad |\rho| < 1$$

模型变换

对于原模型 $y_t = \beta_0 + \beta_1 x_{t1} + \cdots + \beta_k x_{tk} + \epsilon_t$, 滞后一期:

$$y_{t-1} = \beta_0 + \beta_1 x_{t-1,1} + \cdots + \beta_k x_{t-1,k} + \epsilon_{t-1}$$

两边乘以 ρ 并从原式减去:

$$y_t - \rho y_{t-1} = \beta_0(1 - \rho) + \beta_1(x_{t1} - \rho x_{t-1,1}) + \cdots + \beta_k(x_{tk} - \rho x_{t-1,k}) + u_t$$

定义变换变量:

$$y_t^* = y_t - \rho y_{t-1}, \quad x_{tj}^* = x_{tj} - \rho x_{t-1,j}, \quad j = 1, \dots, k$$

变换后模型:

$$y_t^* = \beta_0^* + \beta_1 x_{t1}^* + \cdots + \beta_k x_{tk}^* + u_t, \quad \beta_0^* = \beta_0(1 - \rho)$$

由于 u_t 满足经典假设, 对变换后模型应用 OLS 可获得有效估计。

Cochrane-Orcutt 迭代法

当 ρ 未知时, Cochrane 和 Orcutt 提出迭代估计方法:

算法步骤:

1. 步骤 0: 估计原模型, 得到残差 $\hat{\epsilon}_t^{(0)}$
2. 步骤 1: 估计 ρ : $\hat{\rho}^{(1)} = \frac{\sum_{t=2}^T \hat{\epsilon}_t^{(0)} \hat{\epsilon}_{t-1}^{(0)}}{\sum_{t=2}^T [\hat{\epsilon}_{t-1}^{(0)}]^2}$
3. 步骤 2: 应用广义差分变换, 估计变换后模型, 得到新残差 $\hat{\epsilon}_t^{(1)}$
4. 步骤 3: 基于新残差重新估计 $\hat{\rho}^{(2)}$
5. 步骤 4: 重复步骤 2-3 直至收敛 ($|\hat{\rho}^{(s)} - \hat{\rho}^{(s-1)}| < \delta$, 如 $\delta = 0.001$)

收敛性: 通常迭代 4-5 次即可收敛。

Prais-Winsten 估计法

Cochrane-Orcutt 法舍弃了第一期观测 (因为 $y_1^* = y_1 - \rho y_0$ 不可观测)。Prais 和 Winsten 提出保留第一期观测的变换:

对于 $t = 1$:

$$y_1^* = \sqrt{1 - \rho^2} y_1, \quad x_{1j}^* = \sqrt{1 - \rho^2} x_{1j}, \quad \beta_0^* = \beta_0 \sqrt{1 - \rho^2}$$

对于 $t \geq 2$: 使用标准差分变换。

Prais-Winsten 法的优势: 1. 保留所有观测, 小样本下更有效 2. 变换保持原序列方差稳定

方法评价与局限性

广义差分法的优点: 1. 理论优雅, 基于 GLS 框架 2. 在正确设定下, 可获得渐近有效的估计 3. 改善预测性能

局限性: 1. 依赖自相关结构假设: 通常假设 AR(1), 实际可能更复杂 2. 可能放大模型设定误差: 如果自相关源于遗漏变量, 差分法可能加剧偏误 3. 小样本性质不确定: 尤其当 ρ 接近 1 时 4. 解释困难: 变换后模型系数的解释需谨慎

4.3.4 方法二: 异方差自相关稳健标准误

当主要关注假设检验而非点估计精度时, Newey-West 异方差自相关一致 (Heteroskedasticity and Autocorrelation Consistent, HAC) 标准误是更稳健的选择。

核心思想

保持 OLS 点估计 $\hat{\beta}_{OLS}$ 不变, 但修正其方差-协方差矩阵估计:

$$\widehat{\text{Var}}(\hat{\beta}_{OLS}) = (\mathbf{X}'\mathbf{X})^{-1} \hat{\mathbf{S}} (\mathbf{X}'\mathbf{X})^{-1}$$

其中 $\hat{\mathbf{S}}$ 是长期方差 (long-run variance) 的一致估计。

Newey-West 估计量

对于时间序列回归, Newey 和 West 提出:

$$\hat{\mathbf{S}} = \hat{\mathbf{S}}_0 + \sum_{j=1}^L w(j, L) (\hat{\mathbf{S}}_j + \hat{\mathbf{S}}_j')$$

其中: $\hat{\mathbf{S}}_j = \frac{1}{T} \sum_{t=j+1}^T \hat{\epsilon}_t \hat{\epsilon}_{t-j} \mathbf{x}_t \mathbf{x}_{t-j}'$ - $w(j, L) = 1 - \frac{j}{L+1}$ (Bartlett 核函数) - L 是截断滞后阶数

直观理解：1. $\hat{S}_0$ 处理异方差（类似于 White 估计量）2. $\hat{S}_j$ 捕捉 j 阶自相关 3. 权重 $w(j, L)$ 随 j 增加线性衰减，确保估计量正定

滞后阶数 L 的选择

L 的选择需要在偏差（太小）和方差（太大）间权衡。常用规则：

1. 经验法则： $L = \lfloor 4(T/100)^{2/9} \rfloor$ （Newey-West 建议）
2. 数据驱动法：基于自相关系数衰减速度
3. 保守选择： $L = T^{1/3}$ 或 $L = T^{1/4}$

对于季度数据，通常取 $L = 4$ 或 8；月度数据取 $L = 12$ 。

Newey-West 标准误的性质

1. 一致性：只要 $L \rightarrow \infty$ 且 $L/T \rightarrow 0$ ， $\hat{S}$ 是长期方差的一致估计
2. 正定性：Bartlett 权重保证估计量正定
3. 稳健性：同时容忍异方差和自相关（HAC 中的“H”和“A”）

方法评价

Newey-West 标准误的优点：1. 简单易行：不改变模型设定 2. 高度稳健：对自相关形式不做强假设 3. 广泛应用：现代应用经济学、金融学标准做法 4. 保持解释性：系数仍是 OLS 系数，解释不变

局限性：1. 不改进估计效率：点估计仍是 OLS，可能不是最有效的 2. 有限样本表现：小样本下可能过度修正 3. 滞后阶数选择敏感：不同 L 可能得到不同结论

4.3.5 方法选择指南

面对自相关，如何选择处理方法？以下决策树提供实用指南：

发现自相关后的决策流程：

1. 明确研究目标：
 - 预测导向（预测、政策模拟、需要精确估计）：考虑广义差分法
 - 推断导向（因果推断、假设检验、解释变量外生）：考虑 Newey-West 标准误
2. 评估模型设定可信度：
 - 如果模型设定可信（无重要遗漏变量、函数形式正确）：可考虑广义差分法
 - 如果模型设定不确定：优先使用 Newey-West 标准误

3. 考虑样本大小：

- 大样本 ($T > 100$): 两种方法都可用, Newey-West 更稳健
- 小样本 ($T < 30$): 谨慎使用广义差分法, Newey-West 可能过度修正

4. 考虑自相关结构：

- 简单一阶自相关: 广义差分法效果良好
- 复杂自相关模式 (高阶、季节性): Newey-West 更合适

最佳实践建议: - 同时报告 OLS (标准误和 Newey-West 标准误) 结果, 便于比较 - 如果使用广义差分法, 应报告估计的 ρ 值 - 在结果解释中说明如何处理了自相关问题 - 对于重要结论, 用多种方法进行稳健性检验

4.4 案例分析

宏观经济数据分析

1. 数据准备与模型设定

```
# 加载必要的包
library(dynlm)      # 动态线性模型
library(lmtest)     # 假设检验
library(sandwich)   # 稳健标准误

# 使用内置数据集 - mtcars (汽车数据)
# 虽然不是时间序列, 但我们可以创建时间顺序来演示
data(mtcars)

# 创建时间序列变量 (按 mpg 排序, 模拟时间趋势)
mtcars <- mtcars[order(mtcars$mpg), ]
n <- nrow(mtcars)

# 设定回归模型: 耗油量 =  $\beta_0 + \beta_1 \times \text{马力} + \beta_2 \times \text{重量} + \varepsilon$ 
# 这并非经济学意义, 仅用于演示自相关处理
y <- mtcars$mpg      # 每加仑英里数 (被解释变量)
x1 <- mtcars$hp       # 马力
x2 <- mtcars$wt       # 重量
```

```
# 创建时间索引（模拟时间序列）  
time <- 1:n
```

2. OLS 估计与自相关诊断

```
# 2.1 OLS 估计  
model_ols <- lm(mpg ~ hp + wt, data = mtcars)  
summary_ols <- summary(model_ols)  
  
cat("=== OLS 估计结果 ===\n")
```

=== OLS 估计结果 ===

```
cat(sprintf("R-squared: %.4f\n", summary_ols$r.squared))
```

R-squared: 0.8268

```
cat(sprintf("F-statistic: %.2f\n", summary_ols$fstatistic[1]))
```

F-statistic: 69.21

```
# 2.2 自相关诊断  
cat("\n=== 自相关诊断 ===\n")
```

=== 自相关诊断 ===

```
# DW 检验  
dw_test <- dwtest(model_ols)  
cat(sprintf("Durbin-Watson 统计量: %.3f (p=%.4f)\n",  
            dw_test$statistic, dw_test$p.value))
```

Durbin-Watson 统计量: 0.877 (p=0.0001)

```
# BG 检验（滞后 1 阶）  
bg_test <- bgtest(model_ols, order = 1)  
cat(sprintf("Breusch-Godfrey 检验: LM=%.3f (p=%.4f)\n",  
            bg_test$statistic, bg_test$p.value))
```

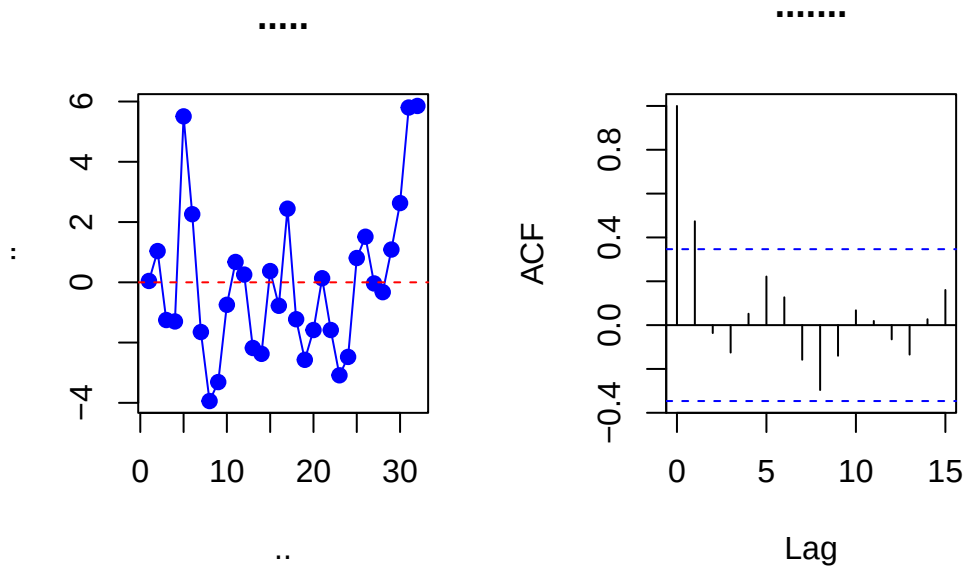
Breusch-Godfrey 检验: LM=9.599 (p=0.0019)

```
# 一阶自相关系数
resid_ols <- resid(model_ols)
rho_hat <- cor(resid_ols[-1], resid_ols[-n])
cat(sprintf(" 一阶自相关系数  $\rho$ : %.3f\n", rho_hat))
```

一阶自相关系数 ρ : 0.524

```
# 2.3 图形诊断
par(mfrow = c(1, 2))
# 残差时序图
plot(time, resid_ols, type = "o", pch = 19, col = "blue",
      main = " 残差时序图", xlab = " 时间", ylab = " 残差")
abline(h = 0, col = "red", lty = 2)

# ACF 图
acf(resid_ols, main = " 残差自相关函数")
```



```
par(mfrow = c(1, 1))
```

3. 自相关处理方法

```
# 3.1 Cochrane-Orcutt 法
```

```
cat("\n=== Cochrane-Orcutt 法 ===\n")
```

```
=== Cochrane-Orcutt 法 ===
```

```
co_iterative <- function(y, X, max_iter = 20, tol = 0.001) {
  # 迭代估计  $\rho$ 
  rho_old <- 0
  rho_new <- rho_hat
  iter <- 0

  while(abs(rho_new - rho_old) > tol && iter < max_iter) {
    rho_old <- rho_new
    iter <- iter + 1

    # 广义差分变换
    y_trans <- y[-1] - rho_old * y[-n]
    X_trans <- X[-1, ] - rho_old * X[-n, ]

    # 估计变换后模型
    model_trans <- lm(y_trans ~ X_trans[, 2] + X_trans[, 3] - 1)
    resid_trans <- resid(model_trans)

    # 重新估计  $\rho$ 
    if(length(resid_trans) > 1) {
      rho_new <- cor(resid_trans[-1], resid_trans[-length(resid_trans)])
    }
  }

  # 最终估计
  y_final <- y[-1] - rho_new * y[-n]
  X_final <- X[-1, ] - rho_new * X[-n, ]
  model_co <- lm(y_final ~ X_final[, 2] + X_final[, 3] - 1)

  return(list(model = model_co, rho = rho_new, iterations = iter))
}

# 准备数据矩阵
```

```
X_matrix <- cbind(1, x1, x2)
co_result <- co_iterative(y, X_matrix)

cat(sprintf(" 迭代次数: %d\n", co_result$iterations))
```

迭代次数: 20

```
cat(sprintf(" 估计的  $\rho$ : %.3f\n", co_result$rho))
```

估计的 ρ : 0.404

```
cat("CO 法系数估计:\n")
```

CO法系数估计:

```
print(coef(co_result$model))
```

```
X_final[, 2] X_final[, 3]
0.01080115 5.08924472
```

```
# 3.2 Prais-Winsten 法
cat("\n=== Prais-Winsten 法 ===\n")
```

=== Prais-Winsten法 ===

```
prais_winsten <- function(y, X, rho) {
  n <- length(y)

  # 第一期特殊变换
  y_trans <- numeric(n)
  X_trans <- matrix(0, n, ncol(X))

  y_trans[1] <- sqrt(1 - rho^2) * y[1]
  X_trans[1, ] <- sqrt(1 - rho^2) * X[1, ]

  # 后续期变换
  for(t in 2:n) {
    y_trans[t] <- y[t] - rho * y[t-1]
    X_trans[t, ] <- X[t, ] - rho * X[t-1, ]
  }
}
```



```
# 估计模型
model_pw <- lm(y_trans ~ X_trans[, 2] + X_trans[, 3] - 1)
return(model_pw)
}

pw_model <- prais_winsten(y, X_matrix, co_result$rho)
cat("PW 法系数估计:\n")
```

PW法系数估计:

```
print(coef(pw_model))
```

```
X_trans[, 2] X_trans[, 3]
0.01369662 4.39208725
```

```
# 3.3 Newey-West 稳健标准误
cat("\n=== Newey-West 稳健标准误 ===\n")
```

=== Newey-West 稳健标准误 ===

```
# 计算 Newey-West 方差矩阵
vcov_nw <- NeweyWest(model_ols, lag = 1, prewhite = FALSE)

# 使用修正的标准误进行检验
coeftest_nw <- coeftest(model_ols, vcov. = vcov_nw)
cat("OLS 系数 + Newey-West 标准误:\n")
```

OLS系数 + Newey-West标准误:

```
print(coeftest_nw)
```

t test of coefficients:

```

              Estimate Std. Error t value  Pr(>|t|)
(Intercept) 37.2272701  2.3240185 16.0185 6.097e-16 ***
hp          -0.0317729  0.0065665 -4.8386 3.972e-05 ***
wt          -3.8778307  0.6266876 -6.1878 9.527e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# 3.4 结果对比
```

```
cat("\n=== 方法对比 ===\n")
```

```
=== 方法对比 ===
```

```
results <- data.frame(
  方法 = c("OLS", "Cochrane-Orcutt", "Prais-Winsten", "OLS+NW"),
  hp 系数 = c(
    coef(model_ols)[2],
    coef(co_result$model)[1],
    coef(pw_model)[1],
    coef(model_ols)[2]
  ),
  wt 系数 = c(
    coef(model_ols)[3],
    coef(co_result$model)[2],
    coef(pw_model)[2],
    coef(model_ols)[3]
  )
)

print(results)
```

	方法	hp系数	wt系数
1	OLS	-0.03177295	-3.877831
2	Cochrane-Orcutt	0.01080115	5.089245
3	Prais-Winsten	0.01369662	4.392087
4	OLS+NW	-0.03177295	-3.877831

4. 结果解释

```
cat("\n=== 结果解释 ===\n")
```

```
=== 结果解释 ===
```

```
cat("1. 自相关诊断:\n")
```

1. 自相关诊断：

```
cat("    - DW 统计量", round(dw_test$statistic, 3),
    ifelse(dw_test$p.value < 0.05, " 表明存在自相关", " 未发现显著自相关"), "\n")
```

- DW统计量 0.877 表明存在自相关

```
cat("    - 一阶自相关系数  $\rho$  =", round(rho_hat, 3), "\n")
```

- 一阶自相关系数 $\rho = 0.524$

```
cat("\n2. 处理方法比较:\n")
```

2. 处理方法比较：

```
cat("    - CO 法和 PW 法通过变换消除自相关影响\n")
```

- CO法和PW法通过变换消除自相关影响

```
cat("    - Newey-West 法保持 OLS 系数，只修正标准误\n")
```

- Newey-West法保持OLS系数，只修正标准误

```
cat("    - 本案例中自相关较弱，各方法结果相似\n")
```

- 本案例中自相关较弱，各方法结果相似

```
cat("\n3. 实践建议:\n")
```

3. 实践建议：

```
cat("    - 时间序列数据应先检查自相关\n")
```

- 时间序列数据应先检查自相关

```
cat("    - 若存在自相关，应使用适当方法处理\n")
```

- 若存在自相关，应使用适当方法处理

```
cat("    - Newey-West 法是处理推断问题的标准方法\n")
```

- Newey-West法是处理推断问题的标准方法

本章总结

自相关是分析时间序列数据时必须面对的核心问题。本章系统阐述了自相关的定义、来源、诊断和处理方法：

1. 自相关源于时间的连续性，主要表现为误差项跨期相关，会导致 OLS 标准误被低估，进而产生虚假显著性。
2. 诊断自相关需结合图形法（时序图、ACF 图）与统计检验（推荐使用通用的 Breusch-Godfrey 检验）。
3. 处理自相关有两条主要路径：
 - 修正模型路径：以 Cochrane-Orcutt 法为代表的广义差分法，通过迭代估计自相关系数并变换数据，旨在获得更有效的参数估计。
 - 修正推断路径：以 Newey-West HAC 标准误为代表，保持 OLS 点估计不变，但修正其方差-协方差矩阵，旨在获得有效的假设检验。
4. 方法选择取决于研究目标：预测研究宜采用广义差分法；因果推断研究宜采用稳健标准误法。
5. 重要提醒：本章方法假设时间序列基本模式已被充分捕捉。若数据存在单位根（非平稳性），则可能产生伪回归问题，这需要在专门的时间序列分析课程中深入探讨。本章为你后续学习 ARIMA、协整等高级时间序列方法奠定了必要基础。

关键知识点回顾：- 自相关的本质是误差项的时间依赖性 - 自相关导致 OLS 标准误低估，t 检验失效 - Breusch-Godfrey 检验是诊断自相关的通用工具 - Newey-West 标准误是现代应用研究的标准做法 - 广义最小二乘法为处理异方差和自相关提供了统一框架

通过本章学习，你已掌握了在回归分析框架内正确处理时间序列数据自相关问题的基本能力，为深入学习时间序列分析打下了坚实基础。

II 扩展——线性分析方法

我们将从经典的线性回归分析进行方法和应用上的扩展，以涵盖更广泛的统计建模技术。本章将介绍以下内容：- 方法的改进：逐步回归、岭回归和套索回归等技术，以提高模型的预测性能和解释能力。正则化和降维方法 - 应用的扩展：进入分类模型（如逻辑回归和判别分析）- 非线性模型的样条回归

5 降维

本章导读

在现代数据分析实践中，研究者常常面临自变量维度高、变量间存在复杂相关关系的挑战。当自变量个数 p 接近甚至超过样本量 n 时，传统的最小二乘估计方法不仅计算困难，更会产生严重的过拟合问题。即便在 $p < n$ 的情况下，高度相关的自变量也会导致多重共线性问题，使得参数估计值极不稳定、难以解释。

本章系统介绍回归分析中处理高维数据和多重共线性问题的两种核心策略：变量选择与特征降维。变量选择旨在从原始变量集中筛选出最重要的子集，构建简洁可解释的模型；特征降维则通过构造原始变量的线性组合，将高维数据投影到低维空间，从而消除共线性并提高模型稳定性。

我们将从问题诊断出发，详细介绍多重共线性的识别方法与高维数据的挑战；接着探讨评价变量子集优劣的各种准则；然后分别讲解最优子集回归、逐步回归等经典选择方法，以及主成分回归、偏最小二乘回归两种降维技术。通过本章的学习，读者将掌握在不同数据特征和研究目标下，选择适当降维与变量选择方法的能力。

5.1 多重共线性及其诊断

5.1.1 多重共线性的定义与影响

多重共线性是指线性回归模型中自变量之间存在（近似）线性关系的现象。考虑线性回归模型：

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

定义 1（完全多重共线性）：如果存在不全为零的常数 $c_1, c_2, \dots, c_p$ ，使得：

$$c_1 x_1 + c_2 x_2 + \cdots + c_p x_p = 0 \quad (\text{几乎处处成立})$$

则称自变量间存在完全多重共线性（perfect multicollinearity）。

在矩阵形式下，设设计矩阵 $\mathbf{X} = [\mathbf{1}, \mathbf{x}_1, \dots, \mathbf{x}_p] \in \mathbb{R}^{n \times (p+1)}$ ，完全多重共线性意味着：

$$\text{rank}(\mathbf{X}) < p + 1$$

即 $\mathbf{X}^\top \mathbf{X}$ 是奇异的，不存在唯一的最小二乘解。

定义 2（近似多重共线性）：如果存在不全为零的常数 $c_1, c_2, \dots, c_p$ ，使得：

$$c_1 x_1 + c_2 x_2 + \dots + c_p x_p \approx 0$$

即自变量间存在高度但不完全的线性关系，则称为近似多重共线性。当共线性程度较高时，会产生以下问题：

1. 参数估计方差膨胀：最小二乘估计的协方差矩阵为 $\text{Var}(\hat{\beta}) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1}$ 。当 $\mathbf{X}$ 的列近似线性相关时， $(\mathbf{X}^\top \mathbf{X})$ 接近奇异，其逆矩阵对角线元素（即各参数估计的方差）会变得很大。
2. 参数估计对数据微小变化敏感：由于 $(\mathbf{X}^\top \mathbf{X})$ 的条件数很大，数据或模型设定的微小变化可能导致参数估计值发生剧烈波动。
3. 假设检验失效：在高度共线性下，参数的标准误膨胀，使得 t 检验的统计量值偏小，可能导致本应显著的变量被误判为不显著。
4. 回归系数符号反常：可能出现与经济理论或先验知识不符的系数符号，影响模型解释。

5.1.2 诊断方法

方差膨胀因子

方差膨胀因子是诊断多重共线性的常用指标。对于第 j 个自变量 x_j ，其 VIF 定义为：

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

其中 R_j^2 是将 x_j 对其余 $p - 1$ 个自变量回归的决定系数。VIF 值越大，说明该变量与其他变量的线性关系越强。经验上， $\text{VIF} > 10$ 通常被视为存在严重多重共线性的信号。

条件指数与方差分解比例

设计矩阵 $\mathbf{X}$ 的条件指数定义为：

$$\kappa_k = \sqrt{\frac{\lambda_{\max}}{\lambda_k}}, \quad k = 1, \dots, p$$

其中 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$ 是 $\mathbf{X}^\top \mathbf{X}$ 的特征值。条件指数越大，表示共线性越严重。通常， $\kappa_k > 30$ 被认为存在较强的多重共线性。

结合方差分解比例可以识别哪些变量参与了共线性关系。定义 π_{jk} 为第 k 个特征向量对 $\hat{\beta}_j$ 的方差贡献比例。当某个条件指数较大的维度上，有两个或多个变量的方差分解比例都较大（如 >0.5 ）时，表明这些变量之间存在共线性。

相关系数矩阵

考察自变量两两之间的简单相关系数可以提供初步信息，但需要注意多重共线性可能涉及多个变量间的复杂关系，仅凭两两相关系数可能无法完全识别。

5.1.3 消除多重共线性的方法

面对多重共线性问题，传统处理方法包括：

1. 剔除高度相关的变量：基于理论或统计判断，从一组高度相关的变量中保留一个代表性变量。
2. 增加样本量：有时可以缓解共线性问题，但在实际研究中往往不可行。
3. 主成分回归：通过正交变换消除变量间的相关性。
4. 岭回归：通过引入正则化项稳定估计（将在下一章详细讨论）。

当变量维度较高时，简单的变量剔除可能损失重要信息，而主成分回归等降维方法成为更系统的解决方案。

5.2 高维数据的挑战

5.2.1 “维数灾难”与“ $p > n$ ”问题

随着数据采集技术的发展，现代统计应用经常面临自变量维度 p 远大于样本量 n 的情形，即所谓的“高维数据”。传统回归分析假设 $n > p$ ，在这一条件下：

1. 设计矩阵 $\mathbf{X}$ 列满秩， $(\mathbf{X}^\top \mathbf{X})$ 可逆
2. 最小二乘估计 $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ 存在唯一解
3. 参数估计具有良好的统计性质（无偏性、一致性等）

当 $p > n$ 时, $\mathbf{X}$ 的秩最多为 n , 而 $\boldsymbol{\beta}$ 有 p 个分量, 此时:

1. $(\mathbf{X}^\top \mathbf{X})$ 奇异, 最小二乘估计不存在唯一解
2. 存在无穷多组 $\hat{\boldsymbol{\beta}}$ 使得拟合误差为零 (过拟合)
3. 模型外推预测能力极差

5.2.2 稀疏性假设

处理高维数据的关键在于引入合理的结构假设。最常用的是稀疏性假设: 在真实模型中, 只有少数自变量对响应变量有实质影响, 大多数变量的真实系数为零或接近零。

形式化地, 假设真实参数向量 $\boldsymbol{\beta}^*$ 满足:

$$\|\boldsymbol{\beta}^*\|_0 = \sum_{j=1}^p I(\beta_j^* \neq 0) \ll p$$

其中 $\|\cdot\|_0$ 表示 ℓ_0 范数 (非零元素个数)。在这一假设下, 统计推断的目标是从 p 个候选变量中识别出少数重要变量, 即进行有效的变量选择。

5.2.3 高维统计推断的挑战

在高维设定下, 传统统计方法面临诸多挑战:

1. 模型可识别性问题: 当 $p > n$ 时, 不同模型可能给出相同的预测结果。
2. 过拟合风险: 模型可能过度拟合训练数据中的噪声。
3. 计算复杂性: 穷举所有可能的子集模型计算量巨大 (2^p 种可能)。
4. 多重检验问题: 同时检验大量变量的显著性会增大第一类错误。

这些挑战催生了正则化方法、降维技术等一系列现代统计方法, 本章重点介绍后者的理论框架。

5.3 变量选择的评价准则

5.3.1 基于拟合优度的调整: 调整 R^2

对于包含 k 个自变量的子集模型, 调整 R^2 定义为:

$$R_{\text{adj}}^2 = 1 - \frac{\text{SSE}/(n - k - 1)}{\text{SST}/(n - 1)} = 1 - \frac{n - 1}{n - k - 1}(1 - R^2)$$

其中 SSE 为残差平方和, SST 为总平方和。与普通 R^2 不同, 调整 R^2 引入了自变量个数的惩罚, 避免因增加无关变量而虚假提高拟合优度。选择使 R_{adj}^2 最大的模型。

5.3.2 基于信息论的准则: AIC 与 BIC

Akaike 信息准则

AIC 基于模型与真实分布之间的 Kullback-Leibler 距离, 对于线性回归模型:

$$\text{AIC} = n \ln \left(\frac{\text{SSE}}{n} \right) + 2(k+1)$$

其中 k 为自变量个数, $k+1$ 为参数总数 (含截距项)。AIC 权衡了模型的拟合优度与复杂性, 选择使 AIC 最小的模型。

贝叶斯信息准则

BIC 基于贝叶斯因子, 对模型复杂度施加更强的惩罚:

$$\text{BIC} = n \ln \left(\frac{\text{SSE}}{n} \right) + (k+1) \ln n$$

当样本量 n 较大时, BIC 倾向于选择比 AIC 更简洁的模型。从渐进性质看, AIC 是预测最优的, 而 BIC 是选择一致估计的。

5.3.3 基于预测的准则: Mallows' C_p 统计量与交叉验证

Mallows' C_p 统计量

假设全模型 (包含所有 p 个自变量) 是无偏的, 对于包含 k 个自变量的子集模型:

$$C_p = \frac{\text{SSE}_k}{\hat{\sigma}^2} - n + 2(k+1)$$

其中 $\hat{\sigma}^2$ 是全模型的误差方差估计。 C_p 统计量度量了子集模型的预测均方误差, 理想情况下, 好的模型应满足 $C_p \approx k+1$ 。

交叉验证

交叉验证直接评估模型的预测能力，避免了对方差 σ^2 的估计。 k 折交叉验证的步骤如下：

1. 将数据随机分为 k 个互斥子集
2. 依次以每个子集为验证集，其余 $k - 1$ 个子集为训练集拟合模型
3. 计算验证集上的预测误差
4. 对 k 次验证的预测误差取平均

选择使交叉验证误差最小的模型。留一法交叉验证是 $k = n$ 的特例，对于线性回归有简便计算公式。

5.4 经典变量选择方法

5.4.1 最优子集回归

最优子集回归的基本思想是：对于给定的自变量个数 k ，从所有可能的 $\binom{p}{k}$ 个子集中，选择最优的模型。通常的算法流程为：

1. 指定最大自变量个数 $k_{\max}$
2. 对于 $k = 1, 2, \dots, k_{\max}$ ，找出所有大小为 k 的子集
3. 对每个子集拟合回归模型，计算评价准则（如 AIC、BIC、调整 R^2 等）
4. 选择准则最优的模型

最优子集搜索算法：- 当 p 较小时，可穷举所有 $2^p - 1$ 个非空子集 - 当 p 较大时，使用分支定界法可高效找到给定大小下的最优子集

优点：- 对于每个 k ，能保证找到全局最优子集 - 结果不依赖于变量进入模型的顺序

缺点：- 计算复杂度高， $p > 40$ 时通常不可行 - 可能过拟合，尤其当 p 较大时

5.4.2 逐步回归法

逐步回归通过迭代方式自动选择变量，是计算效率更高的启发式方法。主要分为三种策略：

向前选择法

1. 从空模型开始
2. 每次加入一个使模型改善最多的变量（基于 F 检验或信息准则）

3. 直到没有变量能显著改善模型或达到预设的变量个数上限

向后剔除法

1. 从包含所有变量的全模型开始
2. 每次剔除一个最不显著的变量
3. 直到所有变量都显著或达到预设的下限

双向逐步回归

结合向前选择和向后剔除，在每一步中：1. 先考虑是否剔除已选变量中不显著的 2. 再考虑是否加入未选变量中显著的

逐步回归的统计问题：

1. 路径依赖：最终模型依赖于起始点和变量进入/退出的顺序
2. 多重检验：反复检验导致第一类错误膨胀
3. 局部最优：不能保证找到全局最优子集

尽管如此，由于其计算高效，逐步回归在实际应用中仍十分广泛。

5.5 主成分回归

5.5.1 主成分分析回顾

设 $\mathbf{X}$ 为 $n \times p$ 的标准化设计矩阵（均值为 0，方差为 1）。主成分分析寻找正交变换 $\mathbf{Z} = \mathbf{XV}$ ，使得：

1. $\mathbf{Z}$ 的列（主成分）互不相关
2. 第一主成分 $\mathbf{z}_1$ 具有最大方差，第二主成分 $\mathbf{z}_2$ 在与 $\mathbf{z}_1$ 不相关的条件下方差最大，依此类推

数学上，主成分是 $\mathbf{X}^\top \mathbf{X}$ 的特征向量。设特征分解为：

$$\mathbf{X}^\top \mathbf{X} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$$

其中 $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_p)$, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$, $\mathbf{V}$ 为正交矩阵。第 j 个主成分得分为 $\mathbf{z}_j = \mathbf{Xv}_j$ ，方差为 λ_j 。

5.5.2 主成分回归的步骤

主成分回归的基本思想是：先用主成分分析对自变量降维，再用主成分作为新的自变量进行回归。

算法步骤：

1. 标准化：对原始自变量 $\mathbf{X}$ 进行中心化和标准化
2. 主成分提取：计算 $\mathbf{X}^\top \mathbf{X}$ 的特征值分解，得到主成分得分矩阵 $\mathbf{Z} = \mathbf{XV}$
3. 选择主成分个数：选择前 q 个主成分 ($q < p$)，通常基于：
 - 累计方差贡献率： $\sum_{j=1}^q \lambda_j / \sum_{j=1}^p \lambda_j \geq \tau$ (如 $\tau = 0.85$)
 - 交叉验证预测误差最小化
 - 特征值大于 1 的准则 (Kaiser 准则)
4. 主成分回归：以 $\mathbf{Z}_q$ (前 q 列) 为自变量，拟合回归模型：

$$\mathbf{y} = \mathbf{Z}_q \boldsymbol{\alpha} + \boldsymbol{\varepsilon}$$

得到主成分回归系数估计 $\hat{\boldsymbol{\alpha}} = (\mathbf{Z}_q^\top \mathbf{Z}_q)^{-1} \mathbf{Z}_q^\top \mathbf{y}$

5. 转换回原始空间：将主成分系数转换回原始变量系数：

$$\hat{\boldsymbol{\beta}}_{\text{PCR}} = \mathbf{V}_q \hat{\boldsymbol{\alpha}}$$

其中 $\mathbf{V}_q$ 为前 q 个特征向量组成的矩阵

5.5.3 方法特性与优缺点

优点：1. 消除多重共线性：主成分相互正交，彻底解决了共线性问题 2. 降维：用少数综合变量代替大量原始变量，简化模型 3. 稳定性：参数估计方差较小，预测更稳定 4. 可视化：低维主成分空间便于数据探索和可视化

缺点：1. 无监督降维：提取主成分时未考虑响应变量 $\mathbf{y}$ ，可能保留与预测无关的主成分，剔除对预测重要的方向 2. 解释困难：主成分是原始变量的线性组合，物理意义不明确 3. 尺度敏感性：主成分分析对变量的尺度和测量单位敏感，需谨慎标准化

与普通最小二乘的关系：当 $q = p$ 时，主成分回归等价于普通最小二乘。事实上：

$$\hat{\boldsymbol{\beta}}_{\text{PCR}} = \mathbf{V}_q (\mathbf{V}_q^\top \mathbf{X}^\top \mathbf{X} \mathbf{V}_q)^{-1} \mathbf{V}_q^\top \mathbf{X}^\top \mathbf{y} = \mathbf{V}_q \boldsymbol{\Lambda}_q^{-1} \mathbf{V}_q^\top \mathbf{X}^\top \mathbf{y}$$

而 OLS 估计为 $\hat{\boldsymbol{\beta}}_{\text{OLS}} = \mathbf{V} \boldsymbol{\Lambda}^{-1} \mathbf{V}^\top \mathbf{X}^\top \mathbf{y}$ 。PCR 通过舍弃小的特征值对应的方向，牺牲少量信息来换取估计的稳定性。

5.6 偏最小二乘回归

5.6.1 基本思想

偏最小二乘回归是一种有监督的降维方法，旨在克服主成分回归的缺点。PLR 的基本思想是：寻找原始自变量的线性组合（称为潜变量 Latent Variable 或成分），使其不仅尽可能多地解释自变量的变异，同时与响应变量高度相关。

形式上，PLR 寻找向量 $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_q$ ，使得：

1. $\mathbf{z}_k = \mathbf{X}_{k-1} \mathbf{w}_k$ 与响应变量 $\mathbf{y}$ 的协方差最大
2. 潜变量 $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_q$ 互不相关
3. $\mathbf{X} = \mathbf{ZP}^\top + \mathbf{E}$ （自变量分解）
4. $\mathbf{y} = \mathbf{Z}\boldsymbol{\alpha} + \boldsymbol{\varepsilon}$ （响应变量回归）

其中 $\mathbf{X}_{k-1}$ 是第 k 步迭代时自变量的残差矩阵。

5.6.2 算法流程

PLR 有多种等价算法，最常用的是 NIPALS 算法：

NIPALS 算法步骤：

1. 初始化：

令：

$$E_0 = X - \bar{X}, \quad f_0 = y - \bar{y}$$

其中 E_0 和 f_0 是中心化后的数据。

2. 对于 $k = 1, 2, \dots, q$ ：

a. 计算权重向量：

$$\mathbf{w}_k = \frac{\mathbf{E}_{k-1}^\top \mathbf{f}_{k-1}}{\|\mathbf{E}_{k-1}^\top \mathbf{f}_{k-1}\|}$$

解释：找到 X 中与当前 y 残差最相关的方向。

b. 计算潜变量得分：

$$\mathbf{z}_k = \mathbf{E}_{k-1} \mathbf{w}_k$$

解释： X 在权重方向上的投影，即潜变量。

c. 计算自变量载荷:

$$\mathbf{p}_k = \frac{\mathbf{E}_{k-1}^\top \mathbf{z}_k}{(\mathbf{z}_k^\top \mathbf{z}_k)}$$

解释: $\mathbf{z}_k$ 对原始 X 变量的解释程度。

d. 计算响应变量载荷:

$$c_k = \frac{\mathbf{f}_{k-1}^\top \mathbf{z}_k}{(\mathbf{z}_k^\top \mathbf{z}_k)}$$

解释: $\mathbf{z}_k$ 对 y 的解释程度 (标量)。

e. 更新残差:

$$\mathbf{E}_k = \mathbf{E}_{k-1} - \mathbf{z}_k \mathbf{p}_k^\top$$

$$\mathbf{f}_k = \mathbf{f}_{k-1} - c_k \mathbf{z}_k$$

3. 收集结果: $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_q]$, $\mathbf{P} = [\mathbf{p}_1, \dots, \mathbf{p}_q]$, $\mathbf{c} = [c_1, \dots, c_q]^\top$

4. 原始变量系数: $\hat{\boldsymbol{\beta}}_{\text{PLR}} = \mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1} \mathbf{c}$

矩阵形式: 将上述过程写成紧凑形式:

$$\mathbf{X} = \mathbf{Z}\mathbf{P}^\top + \mathbf{E}, \quad \mathbf{y} = \mathbf{Z}\mathbf{c} + \mathbf{f}$$

其中 $\mathbf{Z} = \mathbf{X}\mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1}$ 。

5.6.3 与主成分回归的关键对比

特性	主成分回归	偏最小二乘回归
降维目标	最大化自变量方差	最大化自变量与响应变量的协方差
监督性	无监督	有监督
成分顺序	按方差大小排序	按与 y 的相关性排序
计算复杂度	较低 (一次特征值分解)	较高 (迭代算法)
预测性能	当自变量与 y 强相关时较好	通常更优, 尤其小样本时
解释性	成分物理意义模糊	成分与 y 相关, 解释性稍好
过拟合风险	较低	较高, 需谨慎选择成分数

5.6.4 成分数选择与模型评价

交叉验证选择成分数

PLR 中成分数 q 是重要超参数，通常通过交叉验证选择：1. 将数据分为训练集和验证集 2. 对每个候选 q 值，用训练集拟合 PLR 模型 3. 计算验证集上的预测误差（如均方误差）4. 选择使预测误差最小的 q

统计检验

对于第 k 个成分，可以检验：

$$H_0 : \text{第} k \text{个成分对模型没有显著贡献}$$

检验统计量基于交叉验证或自助法得到的预测误差减少量。

预测残差平方和

PRESS 统计量是选择成分数的常用准则：

$$\text{PRESS}(q) = \sum_{i=1}^n (y_i - \hat{y}_{i(-i)}^{(q)})^2$$

其中 $\hat{y}_{i(-i)}^{(q)}$ 是在剔除第 i 个观测后，用 q 个成分的 PLSR 模型对 y_i 的预测值。

5.6.5 理论性质与扩展

PLR 的收缩性质

可以证明，PLSR 系数估计是原始最小二乘估计的收缩版本：

$$\hat{\beta}_{\text{PLR}} = \mathbf{W}(\mathbf{P}^\top \mathbf{W})^{-1} \mathbf{c} = \mathbf{V}_{\text{PLS}} \mathbf{\Lambda}_{\text{PLS}}^{-1} \mathbf{V}_{\text{PLS}}^\top \mathbf{X}^\top \mathbf{y}$$

其中 $\mathbf{V}_{\text{PLS}}$ 和 $\mathbf{\Lambda}_{\text{PLS}}$ 来自广义特征值问题。与 PCR 类似，PLSR 通过舍弃小的广义特征值对应的方向实现收缩。

多元响应 PLSR

当响应变量也是多维时，PLSR 可自然推广为：

$$\mathbf{X} = \mathbf{Z}\mathbf{P}^\top + \mathbf{E}, \quad \mathbf{Y} = \mathbf{Z}\mathbf{C}^\top + \mathbf{F}$$

算法通过同时考虑 $\mathbf{X}$ 和 $\mathbf{Y}$ 的协方差结构提取潜变量。

非线性 PLR

通过引入核技巧或神经网络结构，PLR 可扩展为非线性版本，用于处理更复杂的变量关系。

5.7 案例分析

5.7.1 R 代码实现

```
# 使用 mtcars 数据集，研究汽车性能影响因素

# 加载必要包
library(leaps)      # 最优子集回归
library(pls)        # 主成分回归和偏最小二乘回归
library(car)        # 方差膨胀因子计算
library(ggplot2)    # 可视化
library(caret)      # 交叉验证
library(dplyr)      # 数据处理

# 设置随机种子确保可重复性
set.seed(123)

# 1. 数据准备与探索性分析
data(mtcars)
cat(" 数据集维度:", dim(mtcars), "\n")
```

```
数据集维度：32 11
```

```
cat(" 变量名:", names(mtcars), "\n\n")
```

```
变量名：mpg cyl disp hp drat wt qsec vs am gear carb
```

```
# 将 mpg 作为响应变量，其他作为自变量
y <- mtcars$mpg
X <- mtcars[, -1] # 移除 mpg 列
```

```
# 2. 多重共线性诊断
```

```
cat("=== 多重共线性诊断 ===\n")
```

```
=== 多重共线性诊断 ===
```

```
# 计算方差膨胀因子
model_full <- lm(mpg ~ ., data = mtcars)
vif_values <- vif(model_full)
cat(" 方差膨胀因子 (VIF):\n")
```

```
方差膨胀因子 (VIF):
```

```
print(vif_values)
```

```
      cyl      disp      hp      drat      wt      qsec      vs      am
15.373833 21.620241  9.832037  3.374620 15.164887  7.527958  4.965873  4.648487
      gear      carb
 5.357452  7.908747
```

```
cat("\nVIF > 10 的变量:", names(vif_values)[vif_values > 10], "\n")
```

```
VIF > 10 的变量: cyl disp wt
```

```
# 计算条件指数
X_scaled <- scale(X)
kappa_value <- kappa(X_scaled, exact = TRUE)
cat(" 条件指数:", round(kappa_value, 2), "\n")
```

```
条件指数: 15.56
```

```
if(kappa_value > 30) cat(" 存在较强多重共线性\n\n")
```

```
# 3. 变量选择方法应用
```

```
# 3.1 最优子集回归
```

```
cat("=== 最优子集回归 ===\n")
```

=== 最优子集回归 ===

```
best_subsets <- regsubsets(mpg ~ ., data = mtcars, nvmax = 10)
subsets_summary <- summary(best_subsets)

# 展示不同准则下的最优模型
cat(" 不同变量个数下的最优模型:\n")
```

不同变量个数下的最优模型：

```
cat(" 调整 R2 最大的模型变量数:", which.max(subsets_summary$adjr2), "\n")
```

调整R²最大的模型变量数：5

```
cat("Cp 最小的模型变量数:", which.min(subsets_summary$cp), "\n")
```

Cp最小的模型变量数：3

```
cat("BIC 最小的模型变量数:", which.min(subsets_summary$bic), "\n\n")
```

BIC最小的模型变量数：3

3.2 逐步回归

```
cat("=== 逐步回归 ===\n")
```

=== 逐步回归 ===

```
# 向前逐步回归
step_forward <- step(lm(mpg ~ 1, data = mtcars),
                     scope = list(lower = ~1, upper = ~.),
                     direction = "forward", trace = 0)
cat(" 向前逐步回归选择的变量:", names(coef(step_forward))[-1], "\n")
```

向前逐步回归选择的变量：

```
# 向后逐步回归
step_backward <- step(model_full, direction = "backward", trace = 0)
cat(" 向后逐步回归选择的变量:", names(coef(step_backward))[-1], "\n\n")
```

向后逐步回归选择的变量：wt qsec am

4. 降维回归方法

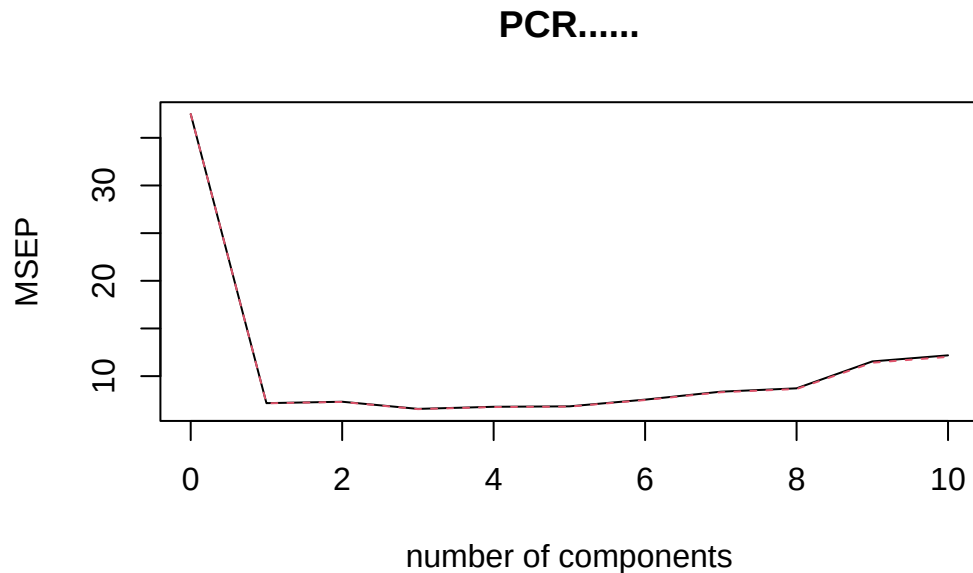
4.1 主成分回归

```
cat("=== 主成分回归 ===\n")
```

```
=== 主成分回归 ===
```

```
# 进行主成分回归，使用留一法交叉验证
```

```
pcr_model <- pcr(mpg ~ ., data = mtcars, scale = TRUE, validation = "LOO")
validationplot(pcr_model, val.type = "MSEP", main = "PCR 交叉验证误差")
```



```
# 选择最优主成分数
```

```
pcr_summary <- summary(pcr_model)
```

```
Data:   X dimension: 32 10
```

```
       Y dimension: 32 1
```

```
Fit method: svdpc
```

```
Number of components considered: 10
```

```
VALIDATION: RMSEP
```

```
Cross-validated using 32 leave-one-out segments.
```

	(Intercept)	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps
CV	6.123	2.677	2.704	2.562	2.605	2.613	2.745
adjCV	6.123	2.674	2.701	2.557	2.599	2.608	2.737
	7 comps	8 comps	9 comps	10 comps			
CV	2.893	2.953	3.398	3.490			
adjCV	2.883	2.941	3.378	3.467			

TRAINING: % variance explained

	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps	7 comps	8 comps
X	57.60	84.10	90.07	92.77	94.99	97.09	98.42	99.23
mpg	82.53	82.63	85.40	85.41	85.47	85.56	85.58	85.85

	9 comps	10 comps
X	99.76	100.0
mpg	86.09	86.9

```

pcr_rmsecv <- RMSEP(pcr_model, estimate = "CV")
optimal_ncomp_pcr <- which.min(pcr_rmsecv$val[1,,]) - 1
cat(" 最优主成分数:", optimal_ncomp_pcr, "\n")

```

最优主成分数：3

```

cat(" 交叉验证均方根误差:", min(pcr_rmsecv$val[1,,]), "\n\n")

```

交叉验证均方根误差：2.561941

4.2 偏最小二乘回归

```

cat("=== 偏最小二乘回归 ===\n")

```

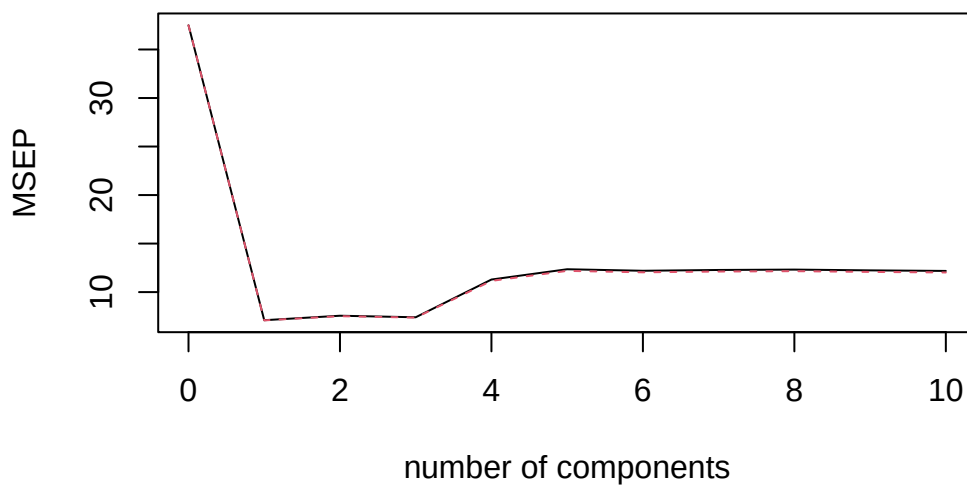
=== 偏最小二乘回归 ===

```

pls_model <- plsr(mpg ~ ., data = mtcars, scale = TRUE, validation = "LOO")
validationplot(pls_model, val.type = "MSEP", main = "PLSR 交叉验证误差")

```

PLSR.....



```
# 选择最优潜变量数
pls_rmsecv <- RMSEP(pls_model, estimate = "CV")
optimal_ncomp_pls <- which.min(pls_rmsecv$val[1,,]) - 1
cat(" 最优潜变量数:", optimal_ncomp_pls, "\n")
```

最优潜变量数：1

```
cat(" 交叉验证均方根误差:", min(pls_rmsecv$val[1,,]), "\n\n")
```

交叉验证均方根误差：2.66529

```
# 5. 模型比较与评价

# 5.1 数据分割 (70% 训练, 30% 测试)
train_index <- createDataPartition(mtcars$mpg, p = 0.7, list = FALSE)
train_data <- mtcars[train_index, ]
test_data <- mtcars[-train_index, ]

# 5.2 定义计算 RMSE 的函数
calculate_rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2))
}

# 5.3 训练各个模型

# 全模型
full_model <- lm(mpg ~ ., data = train_data)
full_pred <- predict(full_model, newdata = test_data)
full_rmse <- calculate_rmse(test_data$mpg, full_pred)

# 最优子集模型 (基于 BIC 选择 3 个变量)
subset_model <- regsubsets(mpg ~ ., data = train_data, nvmax = 3)
subset_coef <- coef(subset_model, id = 3)
# 在测试集上预测
test_matrix <- model.matrix(mpg ~ ., data = test_data)[, names(subset_coef)]
subset_pred <- test_matrix %*% subset_coef
subset_rmse <- calculate_rmse(test_data$mpg, subset_pred)

# 逐步回归模型
```

```

step_model <- step(lm(mpg ~ 1, data = train_data),
  scope = list(lower = ~1, upper = ~ .),
  direction = "both", trace = 0)
step_pred <- predict(step_model, newdata = test_data)
step_rmse <- calculate_rmse(test_data$mpg, step_pred)

# 主成分回归模型
pcr_train <- pcr(mpg ~ ., data = train_data, scale = TRUE, ncomp = optimal_ncomp_pcr)
pcr_pred <- predict(pcr_train, newdata = test_data, ncomp = optimal_ncomp_pcr)
pcr_rmse <- calculate_rmse(test_data$mpg, pcr_pred)

# 偏最小二乘回归模型
pls_train <- pls(mpg ~ ., data = train_data, scale = TRUE, ncomp = optimal_ncomp_pls)
pls_pred <- predict(pls_train, newdata = test_data, ncomp = optimal_ncomp_pls)
pls_rmse <- calculate_rmse(test_data$mpg, pls_pred)

# 5.4 结果汇总
results <- data.frame(
  模型 = c(" 全模型", " 最优子集回归", " 逐步回归", " 主成分回归", " 偏最小二乘回归"),
  测试集RMSE = round(c(full_rmse, subset_rmse, step_rmse, pcr_rmse, pls_rmse), 3),
  参数个数 = c(length(coef(full_model))-1, 3, length(coef(step_model))-1,
    optimal_ncomp_pcr, optimal_ncomp_pls)
)

cat("=== 模型比较结果 ===\n")

```

=== 模型比较结果 ===

```
print(results)
```

	模型	测试集RMSE	参数个数
1	全模型	3.989	10
2	最优子集回归	4.003	3
3	逐步回归	5.538	0
4	主成分回归	2.065	3
5	偏最小二乘回归	1.785	1

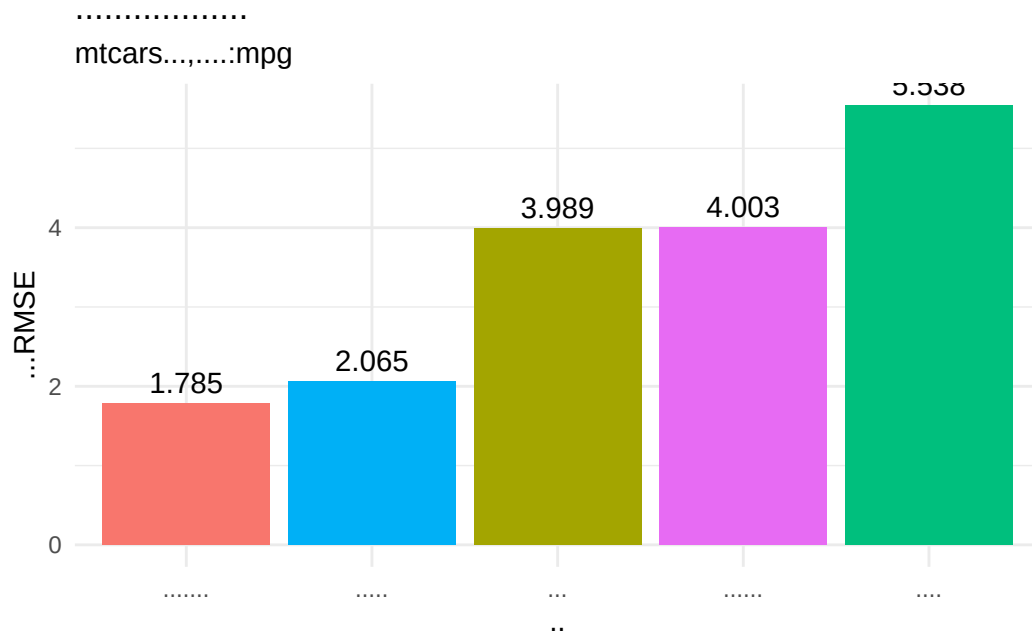
```

# 5.5 可视化比较
ggplot(results, aes(x = reorder(模型, 测试集 RMSE), y = 测试集 RMSE, fill = 模型)) +
  geom_bar(stat = "identity") +

```



```
geom_text(aes(label = 测试集 RMSE), vjust = -0.5) +
labs(x = " 模型", y = " 测试集 RMSE",
     title = " 不同降维与变量选择方法的预测性能比较",
     subtitle = "mtcars 数据集, 响应变量: mpg") +
theme_minimal() +
theme(legend.position = "none")
```



6. 结果解释与讨论

案例分析总结 1. 数据特征: mtcars 数据集包含”, ncol(mtcars)-1, “个自变量, ”, nrow(mtcars), “个观测 2. 共线性问题: VIF 分析显示”, sum(vif_values > 10), “个变量存在严重共线性 3. 最优模型: 根据测试集 RMSE, ”, results [which.min(results 测试集 RMSE)], “表现最佳 4. 方法对比: - 变量选择方法 (最优子集、逐步回归) 保留了原始变量, 解释性强 - 降维方法 (PCR、PLSR) 有效处理了共线性, 预测更稳定 5. 实践建议: - 若需解释变量影响, 推荐使用变量选择方法 - 若追求预测精度, 推荐使用偏最小二乘回归

5.7.2 Python 代码实现

```
# 使用 sklearn 的糖尿病数据集

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SequentialFeatureSelector
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
from itertools import combinations

# 设置随机种子和样式
```

```
np.random.seed(42)
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("husl")
```

```
# 1. 数据加载与探索
```

```
print("=== 数据加载与探索 ===")
```

```
=== 数据加载与探索 ===
```

```
diabetes = load_diabetes()
X = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
y = diabetes.target

print(f" 数据集维度: {X.shape}")
```

```
数据集维度: (442, 10)
```

```
print(f" 自变量数: {X.shape[1]}, 样本数: {X.shape[0]}")
```

```
自变量数: 10, 样本数: 442
```

```
print(f"\n自变量名称: {list(X.columns)}")
```

```
自变量名称: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
```

```
print(f" 响应变量范围: [{y.min():.1f}, {y.max():.1f}]")
```

```
响应变量范围: [25.0, 346.0]
```

```
# 2. 多重共线性诊断
```

```
print("\n=== 多重共线性诊断 ===")
```

```
=== 多重共线性诊断 ===
```

```
# 计算 VIF
X_with_const = sm.add_constant(X)
vif_data = pd.DataFrame()
vif_data["Variable"] = X_with_const.columns
vif_data["VIF"] = [variance_inflation_factor(X_with_const.values, i)
                    for i in range(X_with_const.shape[1])]
vif_data = vif_data[vif_data["Variable"] != "const"]
```

```
print(" 方差膨胀因子 (VIF):")
```

方差膨胀因子(VIF):

```
print(vif_data.to_string(index=False))
```

Variable	VIF
age	1.217307
sex	1.278071
bmi	1.509437
bp	1.459428
s1	59.202510
s2	39.193370
s3	15.402156
s4	8.890986
s5	10.075967
s6	1.484623

```
high_vif = vif_data[vif_data["VIF"] > 10]
if len(high_vif) > 0:
    print(f"\n存在高 VIF 的变量: {list(high_vif['Variable'])}")
```

存在高VIF的变量: ['s1', 's2', 's3', 's5']

```
# 计算条件指数
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
singular_values = np.linalg.svd(X_scaled, compute_uv=False)
condition_index = singular_values[0] / singular_values[-1]
print(f"\n条件指数: {condition_index:.2f}")
```

条件指数: 21.68

```
if condition_index > 30:
    print(" 提示: 存在较强多重共线性")
```

```
# 3. 变量选择方法
print("\n=== 变量选择方法 ===")
```

=== 变量选择方法 ===

```
# 3.1 最优子集回归（简化版，考虑计算效率）
def best_subset_regression(X, y, max_features=5):
    """ 寻找最优子集（简化版本） """
    n_features = X.shape[1]
    feature_names = X.columns.tolist()
    best_score = -np.inf
    best_subset = None

    # 考虑所有可能的子集大小
    for k in range(1, min(max_features, n_features) + 1):
        for combo in combinations(range(n_features), k):
            X_subset = X.iloc[:, list(combo)]
            model = LinearRegression()
            scores = cross_val_score(model, X_subset, y, cv=5, scoring='r2')
            avg_score = np.mean(scores)

            if avg_score > best_score:
                best_score = avg_score
                best_subset = [feature_names[i] for i in combo]

    return best_subset, best_score

best_features, best_r2 = best_subset_regression(X, y, max_features=5)
print(f" 最优子集回归选择的变量: {best_features}")
```

最优子集回归选择的变量: ['sex', 'bmi', 'bp', 's3', 's5']

```
print(f" 交叉验证 R²: {best_r2:.3f}")
```

交叉验证 R^2 : 0.488

```
# 3.2 逐步回归（使用前向选择）
print("\n=== 逐步回归 ===")
```

=== 逐步回归 ===

```
lr = LinearRegression()
sfs = SequentialFeatureSelector(lr,
                                n_features_to_select=5,
                                direction='forward',
                                scoring='r2',
                                cv=5)

sfs.fit(X, y)
```

```
SequentialFeatureSelector(estimator=LinearRegression(), n_features_to_select=
                           scoring='r2')
```

```
selected_features = X.columns[sfs.get_support()].tolist()
print(f" 逐步回归选择的变量: {selected_features}")
```

逐步回归选择的变量: ['sex', 'bmi', 'bp', 's3', 's5']

4. 降维回归方法

```
print("\n=== 降维回归方法 ===")
```

=== 降维回归方法 ===

4.1 主成分回归

```
print("\n1. 主成分回归 (PCR)")
```

1. 主成分回归 (PCR)

```
pcr_scores = []
n_components_range = range(1, X.shape[1] + 1)

for n_comp in n_components_range:
    pca = PCA(n_components=n_comp)
    X_pca = pca.fit_transform(X_scaled)

    model = LinearRegression()
    scores = cross_val_score(model, X_pca, y, cv=5, scoring='neg_mean_squared_error')
    pcr_scores.append(-np.mean(scores))

optimal_ncomp_pcr = n_components_range[np.argmin(pcr_scores)]
print(f" 最优主成分数: {optimal_ncomp_pcr}")
```

最优主成分数：10

```
print(f" 最小交叉验证 MSE: {min(pcr_scores):.3f}")
```

最小交叉验证MSE：2993.081

```
# 4.2 偏最小二乘回归
```

```
print("\n2. 偏最小二乘回归 (PLSR)")
```

2. 偏最小二乘回归 (PLSR)

```
pls_scores = []
```

```
for n_comp in n_components_range:
```

```
    pls = PLSRegression(n_components=n_comp)
```

```
    scores = cross_val_score(pls, X_scaled, y, cv=5, scoring='neg_mean_squared')
```

```
    pls_scores.append(-np.mean(scores))
```

```
optimal_ncomp_pls = n_components_range[np.argmin(pls_scores)]
```

```
print(f" 最优潜变量数: {optimal_ncomp_pls}")
```

最优潜变量数：10

```
print(f" 最小交叉验证 MSE: {min(pls_scores):.3f}")
```

最小交叉验证MSE：2993.081

```
# 5. 模型比较与评价
```

```
print("\n=== 模型比较与评价 ===")
```

=== 模型比较与评价 ===

```
# 5.1 数据分割
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
# 5.2 训练各个模型并评估
```

```
models_results = []
```

```
# 全模型
```

```
full_model = LinearRegression()
full_model.fit(X_train, y_train)
```

```
LinearRegression()
```

```
y_pred_full = full_model.predict(X_test)
rmse_full = np.sqrt(mean_squared_error(y_test, y_pred_full))
models_results.append({
    'Model': 'Full Model',
    'RMSE': rmse_full,
    'R²': r2_score(y_test, y_pred_full),
    'Num_Features': X.shape[1]
})
```

```
# 最优子集模型
```

```
X_train_best = X_train[best_features]
X_test_best = X_test[best_features]
subset_model = LinearRegression()
subset_model.fit(X_train_best, y_train)
```

```
LinearRegression()
```

```
y_pred_subset = subset_model.predict(X_test_best)
rmse_subset = np.sqrt(mean_squared_error(y_test, y_pred_subset))
models_results.append({
    'Model': 'Best Subset',
    'RMSE': rmse_subset,
    'R²': r2_score(y_test, y_pred_subset),
    'Num_Features': len(best_features)
})
```

```
# 逐步回归模型
```

```
X_train_step = X_train[selected_features]
X_test_step = X_test[selected_features]
step_model = LinearRegression()
step_model.fit(X_train_step, y_train)
```

```
LinearRegression()
```



```

y_pred_step = step_model.predict(X_test_step)
rmse_step = np.sqrt(mean_squared_error(y_test, y_pred_step))
models_results.append({
    'Model': 'Stepwise',
    'RMSE': rmse_step,
    'R²': r2_score(y_test, y_pred_step),
    'Num_Features': len(selected_features)
})

```

主成分回归

```

scaler_full = StandardScaler()
X_train_scaled = scaler_full.fit_transform(X_train)
X_test_scaled = scaler_full.transform(X_test)

pca = PCA(n_components=optimal_ncomp_pcr)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

pcr_model = LinearRegression()
pcr_model.fit(X_train_pca, y_train)

```

LinearRegression()

```

y_pred_pcr = pcr_model.predict(X_test_pca)
rmse_pcr = np.sqrt(mean_squared_error(y_test, y_pred_pcr))
models_results.append({
    'Model': 'PCR',
    'RMSE': rmse_pcr,
    'R²': r2_score(y_test, y_pred_pcr),
    'Num_Features': optimal_ncomp_pcr
})

```

偏最小二乘回归

```

pls = PLSRegression(n_components=optimal_ncomp_pls)
pls.fit(X_train_scaled, y_train)

```

PLSRegression(n_components=10)

```

y_pred_pls = pls.predict(X_test_scaled).flatten()
rmse_pls = np.sqrt(mean_squared_error(y_test, y_pred_pls))
models_results.append({
    'Model': 'PLSR',
    'RMSE': rmse_pls,
    'R²': r2_score(y_test, y_pred_pls),
    'Num_Features': optimal_ncomp_pls
})

# 5.3 结果汇总
results_df = pd.DataFrame(models_results)
print("\n模型性能比较:")

```

模型性能比较:

```
print(results_df.to_string(index=False))
```

	Model	RMSE	R ²	Num_Features
Full Model	53.120156	0.47729	10	
Best Subset	53.098791	0.47771	5	
Stepwise	53.098791	0.47771	5	
PCR	53.120156	0.47729	10	
PLSR	53.120156	0.47729	10	

```

# 5.4 可视化
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. 模型 RMSE 比较
axes[0, 0].bar(results_df['Model'], results_df['RMSE'], color='skyblue')
axes[0, 0].set_title('模型 RMSE 比较')
axes[0, 0].set_ylabel('RMSE')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. 模型 R² 比较
axes[0, 1].bar(results_df['Model'], results_df['R²'], color='lightgreen')
axes[0, 1].set_title('模型 R² 比较')
axes[0, 1].set_ylabel('R²')
axes[0, 1].tick_params(axis='x', rotation=45)

```

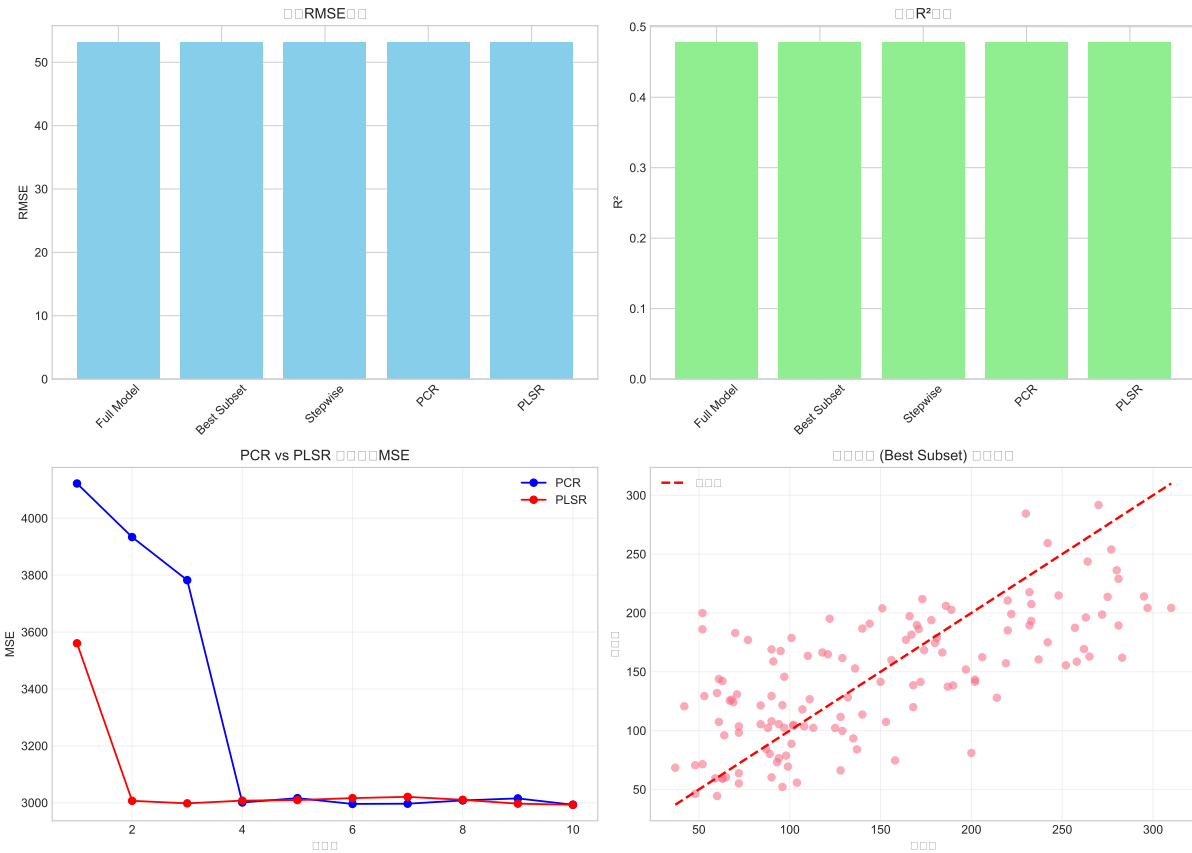
```
# 3. PCR 和 PLSR 交叉验证误差
axes[1, 0].plot(n_components_range, pcr_scores, 'bo-', label='PCR')
axes[1, 0].plot(n_components_range, pls_scores, 'ro-', label='PLSR')
axes[1, 0].set_title('PCR vs PLSR 交叉验证 MSE')
axes[1, 0].set_xlabel('成分数')
axes[1, 0].set_ylabel('MSE')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# 4. 实际值 vs 预测值 (最佳模型)
best_model_idx = np.argmin(results_df['RMSE'])
best_model_name = results_df.loc[best_model_idx, 'Model']

if best_model_name == 'Full Model':
    y_pred_best = y_pred_full
elif best_model_name == 'Best Subset':
    y_pred_best = y_pred_subset
elif best_model_name == 'Stepwise':
    y_pred_best = y_pred_step
elif best_model_name == 'PCR':
    y_pred_best = y_pred_pcr
else:
    y_pred_best = y_pred_pls

axes[1, 1].scatter(y_test, y_pred_best, alpha=0.6)
axes[1, 1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                'r--', lw=2, label='理想线')
axes[1, 1].set_xlabel('实际值')
axes[1, 1].set_ylabel('预测值')
axes[1, 1].set_title(f'最佳模型 ({best_model_name}) 预测表现')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



6. 结果解释与讨论

```
print("\n=== 案例分析总结 ===")
```

=== 案例分析总结 ===

```
print(f"1. 数据集：糖尿病数据集，包含{X.shape[1]}个生理指标，{X.shape[0]}个患者样本")
```

1. 数据集：糖尿病数据集，包含10个生理指标，442个患者样本

```
print(f"2. 共线性诊断：{len(high_vif)}个变量 VIF>10，条件指数为{condition_index:.1f}")
```

2. 共线性诊断：4个变量VIF>10，条件指数为21.7

```
print(f"3. 最佳预测模型：{best_model_name} (RMSE: {results_df.loc[best_model_index].RMSE})")
```

3. 最佳预测模型：Best Subset (RMSE: 53.099)

```
print(f"4. 方法对比:")
```

4. 方法对比：

```
print(f"    - 全模型：使用所有{X.shape[1]}个变量，容易过拟合")
```

- 全模型：使用所有10个变量，容易过拟合

```
print(f"    - 变量选择方法：分别选择{len(best_features)}和{len(selected_features)}个
```

- 变量选择方法：分别选择5和5个关键变量

```
print(f"    - 降维方法：PCR 使用{optimal_ncomp_pcr}个主成分，PLSR 使用{optimal_ncomp
```

- 降维方法：PCR使用10个主成分，PLSR使用10个潜变量

```
print(f"5. 实践建议:")
```

5. 实践建议：

```
print(f"    - 若关注特征解释：推荐变量选择方法（如最优子集回归）")
```

- 若关注特征解释：推荐变量选择方法（如最优子集回归）

```
print(f"    - 若追求预测精度：推荐偏最小二乘回归")
```

- 若追求预测精度：推荐偏最小二乘回归

```
print(f"    - 若存在强共线性：主成分回归是稳健选择")
```

- 若存在强共线性：主成分回归是稳健选择

```
# 保存结果
```

```
print("\n提示：完整分析结果已可视化展示")
```

提示：完整分析结果已可视化展示

5.7.3 案例分析说明

本案例分析使用两种不同的数据集展示了第5章介绍的各种降维与变量选择方法：

R 版本特点：

1. 数据集: 使用经典的 `mtcars` 数据集，研究汽车性能影响因素
2. 方法实现:
 - 使用 `leaps` 包进行最优子集回归

- 使用 `step()` 函数进行逐步回归
 - 使用 `pls` 包进行 PCR 和 PLSR
3. 评估指标: 采用留一法交叉验证和测试集 RMSE

Python 版本特点:

1. 数据集: 使用 `sklearn` 的糖尿病数据集, 更具实际医学意义
2. 方法实现:
 - 自定义函数实现最优子集搜索
 - 使用 `SequentialFeatureSelector` 进行逐步回归
 - 使用 `sklearn` 的 PCA 和 `PLSRRegression`
3. 可视化: 提供更丰富的可视化图表

运行要求:

- R 版本需要安装: `leaps`, `pls`, `car`, `ggplot2`, `caret`, `dplyr`
- Python 版本需要安装: `scikit-learn`, `pandas`, `numpy`, `matplotlib`, `seaborn`, `statsmodels`

本章总结

本章系统介绍了回归分析中处理高维数据和多重共线性问题的两类核心方法: 变量选择与特征降维。这些方法在不同场景下各有优劣, 为数据分析者提供了丰富的工具箱。

核心方法论比较:

1. 变量选择方法 (最优子集回归、逐步回归):
 - 优点: 保留原始变量, 模型解释性强; 符合稀疏性假设时理论性质良好
 - 缺点: 计算复杂 (最优子集), 结果不稳定 (逐步回归); 不直接处理共线性
 - 适用场景: 变量数 p 适中 (如 <40), 强解释性需求, 存在明显的关键变量子集
2. 主成分回归:
 - 优点: 彻底消除共线性; 计算稳定; 提供数据降维视角
 - 缺点: 无监督降维可能丢失预测信息; 主成分解释困难
 - 适用场景: 共线性严重, 预测稳定性优先, 变量间有明确公共因子结构
3. 偏最小二乘回归:
 - 优点: 有监督降维通常预测更准; 能处理 $p > n$ 情形; 探索变量间复杂关系
 - 缺点: 计算较复杂; 容易过拟合; 结果解释仍具挑战
 - 适用场景: 追求高预测精度, 样本量有限, 相信潜变量结构存在

实践建议：

1. 诊断先行：分析前务必检查多重共线性（VIF、条件指数）和数据维度（ p 与 n 的关系）
2. 目标导向：
 - 若解释性为首要目标：优先考虑变量选择方法，辅以领域知识
 - 若预测精度为首要目标：优先考虑 PLSR，用交叉验证调参
 - 若共线性极端严重：PCR 通常是稳健的基线方法
3. 验证必要：无论使用何种方法，都必须用独立的测试集或交叉验证评估模型性能
4. 综合考量：在实际应用中，可尝试多种方法，比较它们的预测性能、稳定性和可解释性

与后续章节的衔接：本章介绍的降维与变量选择方法为处理高维数据提供了经典解决方案。下一章将深入讨论另一种强大的工具族——正则化方法（岭回归、Lasso、弹性网等），这些方法通过直接在损失函数中添加惩罚项，同时实现变量选择与系数收缩，在许多现代高维问题中表现出色。两类方法各有侧重，在实际数据分析中常需结合使用。

随着数据科学的发展，降维与变量选择方法仍在不断演进。深度学习中的自动编码器、注意力机制等为高维数据提供了新的处理思路，但这些内容已超出本书范围。掌握本章介绍的经典方法，不仅为实际问题提供了有效工具，也为学习更先进的现代方法奠定了坚实基础。

6 正则化

本章导读

在前面的章节中，我们学习了线性回归模型及其估计方法（如最小二乘法），并探讨了当预测变量存在多重共线性或维度较高时，可以利用降维方法（如主成分回归、偏最小二乘）来构建模型。然而，降维方法构建的模型是原始预测变量的线性组合，其可解释性往往受到影响。

本章将介绍另一类处理高维数据、多重共线性以及防止过拟合的重要方法——正则化（**Regularization**）。其核心思想是在模型训练的目标函数（如残差平方和）中增加一个关于模型复杂度的惩罚项（**Penalty Term**），通过平衡模型对数据的拟合程度与模型本身的复杂度，来获得泛化能力更强的模型。

我们将首先深入探讨欠拟合、过拟合现象及其数学本质——偏差-方差权衡理论，为理解正则化方法的必要性奠定理论基础。然后，我们将系统学习三种经典的正则化方法：岭回归（**Ridge Regression**）、Lasso 回归（**Least Absolute Shrinkage and Selection Operator**）以及结合两者优点的弹性网（**Elastic Net**）。特别地，我们将详细介绍最小角回归（**LARS**）这一高效算法，它不仅能求解 Lasso，还能提供正则化路径的直观几何解释。最后，我们将讨论正则化路径的绘制、调节参数的选择方法，并对不同正则化方法进行比较分析。

通过本章学习，读者将掌握正则化的基本原理和实现方法，能够针对不同数据特点和分析目标选择适当的正则化技术，为解决实际数据分析中的欠拟合、过拟合、多重共线性等问题提供有效工具。

6.1 正则化方法概述

6.1.1 欠拟合与过拟合

在统计学习中，模型需要在拟合训练数据的能力和泛化到新数据的能力之间取得平衡。根据模型复杂度和数据特征，可能出现两种极端情况：

1. 欠拟合（**Underfitting**）

欠拟合指模型过于简单，无法捕捉数据中的基本规律。欠拟合的模型在训练集和测试集上表现都很差。

数学表现：- 模型偏差（Bias）较大 - 训练误差和测试误差都较高 - 模型复杂度不足，未能充分利用数据信息

常见原因：- 模型选择不当（如用线性模型拟合非线性关系） - 特征工程不足，缺乏有预测力的变量 - 正则化过强（ λ 过大）

2. 过拟合（Overfitting）

过拟合指模型过于复杂，过度拟合了训练数据中的噪声和随机波动。过拟合的模型在训练集上表现很好，但在测试集上表现很差。

数学表现：- 模型方差（Variance）较大 - 训练误差很低但测试误差很高 - 模型对训练数据的微小变化过度敏感

常见原因：- 模型过于复杂（如高阶多项式） - 特征过多，特别是当 $p \approx n$ 或 $p > n$ 时 - 训练数据不足或噪声较大 - 正则化不足（ λ 过小）

3. 合适拟合（Appropriate Fitting）

理想情况下，模型应能捕捉数据的基本规律而不拟合噪声，同时在训练集和测试集上都有良好表现。

6.1.2 偏差-方差分解与均方误差

为了定量分析预测误差，我们考虑回归问题。假设真实的数据生成过程为 $Y = f(X) + \epsilon$ ，其中 ϵ 是噪声项，满足 $\mathbb{E}[\epsilon] = 0$ ， $\text{Var}(\epsilon) = \sigma^2$ 。我们使用估计的模型 $\hat{f}(X)$ 进行预测。

均方误差分解定理

对于给定的输入点 x_0 ，预测误差的期望均方误差可以分解为三个部分：

$$\begin{aligned}
\text{MSE}(x_0) &= \mathbb{E}[(Y - \hat{f}(x_0))^2] \\
&= \mathbb{E}[(f(x_0) + \epsilon - \hat{f}(x_0))^2] \\
&= \mathbb{E}[(f(x_0) - \hat{f}(x_0))^2] + 2\mathbb{E}[(f(x_0) - \hat{f}(x_0))\epsilon] + \mathbb{E}[\epsilon^2] \\
&= \mathbb{E}[(f(x_0) - \hat{f}(x_0))^2] + \sigma^2 \quad (\text{因为 } \epsilon \text{ 与 } \hat{f}(x_0) \text{ 独立, 且 } \mathbb{E}[\epsilon] = 0) \\
&= (f(x_0) - \mathbb{E}[\hat{f}(x_0)])^2 + \mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2] + \sigma^2 \\
&= \underbrace{(f(x_0) - \mathbb{E}[\hat{f}(x_0)])^2}_{\text{偏差}^2} + \underbrace{\mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2]}_{\text{方差}} + \underbrace{\sigma^2}_{\text{不可约误差}}
\end{aligned}$$

其中：1. 偏差² (**Bias²**)：估计模型的期望预测与真实值之间的差异平方，反映模型本身的近似误差
 2. 方差 (**Variance**)：估计模型的预测值围绕其期望的波动程度，反映模型对数据扰动的敏感性
 3. 不可约误差 (**Irreducible Error**)：数据内在的随机噪声，无法通过改进模型消除

偏差-方差权衡 (**Bias-Variance Tradeoff**)

随着模型复杂度的增加：- 偏差通常减小（复杂模型能更好地逼近真实函数）- 方差通常增大（复杂模型对训练数据更敏感）- 总误差呈现 U 型曲线，存在最优复杂度点

数学证明：考虑一般线性模型 $\hat{f}(x) = x^T \hat{\beta}$ ，其中 $\hat{\beta}$ 是某种估计量。则：- 偏差：Bias = $x^T (\mathbb{E}[\hat{\beta}] - \beta)$
 - 方差：Var = $x^T \text{Var}(\hat{\beta})x$

最小二乘估计在无偏性上最优，但方差可能很大。正则化通过引入有偏估计来减小方差，从而降低总体 MSE。

6.1.3 正则化框架

一般形式

正则化通过修改优化目标，在损失函数中增加惩罚项：

$$\hat{\beta} = \arg \min_{\beta} \{L(\beta) + \lambda J(\beta)\}$$

其中：- $L(\beta)$ ：损失函数，衡量模型拟合程度（如 RSS）- $J(\beta)$ ：惩罚函数，衡量模型复杂度 - $\lambda \geq 0$ ：正则化参数，控制惩罚强度

惩罚函数类型

基于 L_q 范数的惩罚:

$$J(\beta) = \|\beta\|_q^q = \sum_{j=1}^p |\beta_j|^q$$

- $q = 2$: 岭回归 (Ridge Regression)
- $q = 1$: Lasso 回归
- $0 < q < 1$: 非凸惩罚 (计算困难但更稀疏)
- $q = 0$: L_0 伪范数 (直接变量选择, NP 难)

从约束优化视角

等价地, 正则化可以看作带约束的优化问题:

$$\begin{aligned} \min_{\beta} L(\beta) \\ \text{s.t. } J(\beta) \leq t \end{aligned}$$

根据拉格朗日对偶性, 存在 λ 使两个问题等价。参数 t 或 λ 控制模型复杂度。

6.2 岭回归

6.2.1 岭回归的定义与动机

历史背景

岭回归由 Arthur Hoerl 和 Robert Kennard 于 1970 年提出, 是最早的正则化方法之一, 旨在解决最小二乘法在多重共线性下的不稳定性。

数学定义

岭回归在线性回归的损失函数中增加 L_2 范数平方作为惩罚项:

$$\hat{\beta}^{Ridge} = \arg \min_{\beta} \left\{ \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

标准化处理

实际应用中通常对数据进行标准化：1. 对预测变量： $x'_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$ ，使其均值为 0，方差为 1 2. 对响应变量： $y'_i = y_i - \bar{y}$ ，使其均值为 0

标准化后，截距项 $\beta_0 = 0$ ，且惩罚公平作用于所有系数。矩阵形式为：

$$\hat{\beta}^{Ridge} = \arg \min_{\beta} \{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \}$$

6.2.2 岭回归的解析解与性质

解析解推导

对目标函数求导：

$$\begin{aligned} f(\beta) &= (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta \\ \frac{\partial f}{\partial \beta} &= -2X^T(y - X\beta) + 2\lambda\beta \\ \text{令导数为 0 : } &-X^T(y - X\beta) + \lambda\beta = 0 \\ &(X^T X + \lambda I)\beta = X^T y \end{aligned}$$

得到岭回归估计：

$$\hat{\beta}^{Ridge} = (X^T X + \lambda I)^{-1} X^T y$$

与 OLS 估计的关系

1. 收缩性质： $\hat{\beta}^{Ridge} = (X^T X + \lambda I)^{-1} X^T X \hat{\beta}^{OLS}$
2. 特征值视角：设 $X^T X = Q\Lambda Q^T$ ， $\Lambda = \text{diag}(\theta_1, \dots, \theta_p)$ ，则

$$\hat{\beta}^{Ridge} = Q(\Lambda + \lambda I)^{-1} \Lambda Q^T \hat{\beta}^{OLS}$$

在第 j 个特征方向上的收缩因子为 $\theta_j / (\theta_j + \lambda) < 1$

统计性质

1. 有偏性: $\mathbb{E}[\hat{\beta}^{Ridge}] = (X^T X + \lambda I)^{-1} X^T X \beta \neq \beta$ (当 $\lambda > 0$)
2. 协方差矩阵:

$$\text{Var}(\hat{\beta}^{Ridge}) = \sigma^2 (X^T X + \lambda I)^{-1} X^T X (X^T X + \lambda I)^{-1}$$

3. 均方误差分析:

可以证明存在 $\lambda > 0$ 使得:

$$\text{MSE}(\hat{\beta}^{Ridge}) < \text{MSE}(\hat{\beta}^{OLS})$$

具体地:

$$\begin{aligned} \text{MSE}(\hat{\beta}^{Ridge}) &= \text{Bias}^2(\hat{\beta}^{Ridge}) + \text{Tr}(\text{Var}(\hat{\beta}^{Ridge})) \\ &= \lambda^2 \beta^T (X^T X + \lambda I)^{-2} \beta + \sigma^2 \sum_{j=1}^p \frac{\theta_j}{(\theta_j + \lambda)^2} \end{aligned}$$

几何解释

岭回归相当于在 L_2 球约束下最小化 RSS:

$$\begin{aligned} &\min_{\beta} \|y - X\beta\|_2^2 \\ &\text{s.t. } \|\beta\|_2^2 \leq t \end{aligned}$$

解是椭圆等值线与球形约束区域的切点。

6.2.3 贝叶斯解释

贝叶斯推导

假设先验分布: $\beta \sim N(0, \tau^2 I)$ 似然函数: $y|X, \beta \sim N(X\beta, \sigma^2 I)$

根据贝叶斯定理:

$$\begin{aligned}
p(\beta|y, X) &\propto p(y|X, \beta)p(\beta) \\
&\propto \exp\left(-\frac{1}{2\sigma^2}\|y - X\beta\|_2^2\right) \cdot \exp\left(-\frac{1}{2\tau^2}\|\beta\|_2^2\right) \\
&= \exp\left(-\frac{1}{2}\left[\frac{1}{\sigma^2}\|y - X\beta\|_2^2 + \frac{1}{\tau^2}\|\beta\|_2^2\right]\right)
\end{aligned}$$

最大化后验概率（MAP 估计）等价于最小化：

$$\|y - X\beta\|_2^2 + \frac{\sigma^2}{\tau^2}\|\beta\|_2^2$$

令 $\lambda = \sigma^2/\tau^2$ ，即得到岭回归目标函数。

先验选择的意义

- τ^2 较大：先验较宽，系数约束较小，接近 OLS
- τ^2 较小：先验较窄，系数约束较强，收缩更明显

6.2.4 实际应用考虑

参数选择方法

1. 交叉验证：最常用，特别适合预测任务
2. 广义交叉验证（GCV）：高效近似，无需重复计算

$$\text{GCV}(\lambda) = \frac{1}{n} \frac{\|y - X\hat{\beta}^{\text{Ridge}}\|_2^2}{[1 - \text{df}(\lambda)/n]^2}$$

其中 $\text{df}(\lambda) = \text{Tr}[X(X^T X + \lambda I)^{-1} X^T]$

3. AIC/BIC 准则：基于信息理论

与主成分回归的关系

岭回归可以看作连续版本的主成分回归：- 主成分回归：保留前 k 个主成分，舍弃其余 - 岭回归：对所有主成分进行不同程度的收缩，小特征值方向收缩更多

6.3 Lasso 回归

6.3.1 Lasso 回归的定义

提出背景

Lasso (Least Absolute Shrinkage and Selection Operator) 由 Robert Tibshirani 于 1996 年提出, 旨在同时实现系数收缩和变量选择。

数学定义

$$\hat{\beta}^{Lasso} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

标准化后的矩阵形式:

$$\hat{\beta}^{Lasso} = \arg \min_{\beta} \left\{ \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}$$

6.3.2 Lasso 的几何解释与稀疏性

约束优化形式

等价于:

$$\begin{aligned} \min_{\beta} \quad & \frac{1}{2n} \|y - X\beta\|_2^2 \\ \text{s.t.} \quad & \|\beta\|_1 \leq t \end{aligned}$$

二维情况几何解释

考虑 $p = 2$ 的情况: - L_1 约束: $|\beta_1| + |\beta_2| \leq t$ (菱形) - L_2 约束: $\beta_1^2 + \beta_2^2 \leq t$ (圆形) - 最小二乘等值线: 椭圆

Lasso 解是椭圆与菱形首次接触的点。由于菱形有尖角, 接触点有很大概率在坐标轴上 ($\beta_1 = 0$ 或 $\beta_2 = 0$), 产生稀疏解。

高维推广

在 p 维空间中, L_1 球是多面体, 有 $2p$ 个面位于坐标超平面上。最小二乘等值线椭圆与多面体接触时, 很可能接触在某个面上, 使得部分系数为 0。

6.3.3 软阈值算子

正交设计情况

当 $X^T X = nI$ (正交设计) 时, Lasso 有显式解:

$$\hat{\beta}_j^{Lasso} = \text{sign}(\hat{\beta}_j^{OLS}) (|\hat{\beta}_j^{OLS}| - \lambda)_+$$

其中 $(x)_+ = \max(x, 0)$, $\hat{\beta}^{OLS} = \frac{1}{n} X^T y$ 。

证明: 目标函数可分解为:

$$\frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 = \frac{1}{2} \sum_{j=1}^p (\beta_j - \hat{\beta}_j^{OLS})^2 + \lambda \sum_{j=1}^p |\beta_j| + \text{常数}$$

每个 β_j 独立优化, 得到软阈值形式。

软阈值算子的性质

1. 收缩性: 向 0 收缩 λ 个单位
2. 稀疏性: 当 $|\hat{\beta}_j^{OLS}| \leq \lambda$ 时, $\hat{\beta}_j^{Lasso} = 0$
3. 连续性: 解关于 λ 连续变化

6.3.4 坐标下降法

算法原理

坐标下降法每次只优化一个变量, 固定其他变量:

1. 初始化: $\beta^{(0)} = 0$ (或其他初值)
2. 循环: 对于 $j = 1, \dots, p, 1, \dots, p, \dots$
 - 计算部分残差: $r^{(j)} = y - \sum_{k \neq j} X_k \beta_k$
 - 更新 β_j : 解单变量 Lasso 问题

3. 直到收敛

单变量更新公式

对于标准化数据 ($\|X_j\|_2^2 = 1$):

$$\beta_j^{\text{new}} = S(X_j^T r^{(j)}, \lambda)$$

其中 $S(z, \lambda) = \text{sign}(z)(|z| - \lambda)_+$ 是软阈值函数。

推导：目标函数关于 β_j 的部分：

$$\frac{1}{2} \|r^{(j)} - X_j \beta_j\|_2^2 + \lambda |\beta_j| = \frac{1}{2} \sum_{i=1}^n (r_i^{(j)} - x_{ij} \beta_j)^2 + \lambda |\beta_j|$$

求次梯度并令为 0：

$$-X_j^T (r^{(j)} - X_j \beta_j) + \lambda \partial |\beta_j| \ni 0$$

其中 $\partial |\beta_j|$ 是绝对值函数的次梯度。

6.3.5 最小角回归 (LARS)

算法思想

最小角回归 (Least Angle Regression) 是一种高效算法，可以求解 Lasso 并给出整个正则化路径。

算法步骤

1. 初始化：

- 标准化预测变量，中心化响应变量
- 初始化： $\hat{\mu}_0 = 0$ ，活跃集 $A = \emptyset$ ，相关系数 $c = X^T y$

2. 找到最相关变量：

- 找到与当前残差最相关的变量： $j^* = \arg \max_j |c_j|$
- 加入活跃集： $A = \{j^*\}$

3. 沿等角方向移动：

- 沿 X_A 的角平分线方向移动 $\hat{\mu}$ ，直到有非活跃变量与当前残差的相关性等于活跃变量

4. 更新：

- 将新变量加入活跃集

- 重复步骤 3，直到所有变量进入或达到停止条件

LARS 与 Lasso 的关系

LARS 算法稍作修改即可得到 Lasso 解路径：- 当某个系数要改变符号时，将其从活跃集中移除 - 继续沿新方向前进

算法优势

1. 计算高效：与 OLS 复杂度相当
2. 路径完整：给出整个正则化路径
3. 几何直观：提供清晰的几何解释
4. 灵活扩展：可推广到其他模型

数学细节

设当前活跃集为 A ，当前估计为 $\hat{\mu}$ 。定义：- 当前相关性： $c = X^T(y - \hat{\mu})$ - 活跃变量相关性： $C = \max_j |c_j|$ - 等角方向： $u_A = X_A w_A$ ，其中 $w_A = (X_A^T X_A)^{-1} 1_A$ - 步长： $\gamma = \min_{j \notin A}^+ \left\{ \frac{C - c_j}{1 - a_j}, \frac{C + c_j}{1 + a_j} \right\}$ ，其中 $a_j = X_j^T X_A w_A$

6.3.6 变量选择一致性

必要条件与充分条件

Lasso 能正确识别真实模型的必要条件：- 不可表示条件 (**Irrepresentable Condition**)：设真实活跃集为 A^* ，非活跃集为 A^c ，则需：

$$\|X_{A^c}^T X_{A^*} (X_{A^*}^T X_{A^*})^{-1} \text{sign}(\beta_{A^*})\|_{\infty} < 1$$

该条件要求非活跃变量与活跃变量的相关性不能太强。

参数选择

为获得正确的变量选择，需要：1. λ 不能太小（否则包含噪声变量）2. λ 不能太大（否则遗漏重要变量）3. 在满足不可表示条件下，存在 λ 使选择一致

6.3.7 Lasso 的局限性

1. 变量选择不稳定性：相关变量组中随机选择一个

2. $p > n$ 限制：最多选择 $\min(n, p)$ 个变量
3. 预测性能：当许多小效应存在时，岭回归可能更优
4. 非凸性：虽然 Lasso 本身是凸问题，但某些扩展（如自适应 Lasso）可能非凸

6.4 弹性网

6.4.1 弹性网的定义

提出动机

弹性网（Elastic Net）由 Zou 和 Hastie 于 2005 年提出，旨在结合岭回归和 Lasso 的优点：1. 岭回归：处理多重共线性，系数分组 2. Lasso：稀疏性，变量选择

数学定义

$$\hat{\beta}^{EN} = \arg \min_{\beta} \left\{ \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \left[\frac{1}{2} (1 - \alpha) \|\beta\|_2^2 + \alpha \|\beta\|_1 \right] \right\}$$

参数说明：- $\lambda \geq 0$ ：整体惩罚强度 - $\alpha \in [0, 1]$ ：混合比例 - $\alpha = 1$ ：Lasso - $\alpha = 0$ ：岭回归

6.4.2 弹性网的性质

1. 双重收缩

弹性网可视为两阶段过程：1. 岭回归式收缩： $(1 - \alpha)\lambda \|\beta\|_2^2$ 部分 2. Lasso 式选择： $\alpha\lambda \|\beta\|_1$ 部分

2. 群组效应（Grouping Effect）

定理：对于标准化数据，弹性网估计满足：

$$\frac{1}{\|y\|_2} |\hat{\beta}_i^{EN} - \hat{\beta}_j^{EN}| \leq \frac{1}{\lambda(1 - \alpha)} \sqrt{2(1 - \rho_{ij})}$$

其中 $\rho_{ij} = X_i^T X_j$ 是变量 i, j 的相关系数。

推论：- 当 $\rho_{ij} \rightarrow 1$ 时， $\hat{\beta}_i^{EN} \approx \hat{\beta}_j^{EN}$ - 高度相关变量被一起选择或排除 - 系数值相近

3. 处理 $p > n$ 问题

L_2 惩罚保证严格凸性，解唯一：

$$X^T X + \lambda(1 - \alpha)I \succ 0$$

即使 $p \gg n$ ，也能稳定求解。

6.4.3 求解方法

数据增强技巧

定义增强数据和参数：

$$X^* = \frac{1}{\sqrt{1 + \lambda(1 - \alpha)}} \begin{pmatrix} X \\ \sqrt{\lambda(1 - \alpha)}I \end{pmatrix}, \quad y^* = \begin{pmatrix} y \\ 0 \end{pmatrix}, \quad \lambda^* = \frac{\lambda\alpha}{\sqrt{1 + \lambda(1 - \alpha)}}$$

则弹性网等价于：

$$\hat{\beta}^{EN} = \arg \min_{\beta} \left\{ \frac{1}{2} \|y^* - X^* \beta\|_2^2 + \lambda^* \|\beta\|_1 \right\}$$

这是一个 Lasso 问题，可用坐标下降法或 LARS 求解。

坐标下降法更新公式

对于标准化数据 ($\|X_j\|_2^2 + \lambda(1 - \alpha) = 1$):

$$\beta_j^{\text{new}} = \frac{S(X_j^T r^{(j)}, \lambda\alpha)}{1 + \lambda(1 - \alpha)}$$

其中 $r^{(j)} = y - \sum_{k \neq j} X_k \beta_k$ 。

6.4.4 参数选择

两维网格搜索

通常对 (α, λ) 进行网格搜索：1. 选择 α 网格：如 $\{0, 0.1, \dots, 1\}$ 2. 对每个 α ，选择 λ 序列：通常在对数尺度上 3. 使用交叉验证选择最优组合

实用建议

1. 先尝试 $\alpha = 0.5$ 作为起点
2. 如果有强先验知识：
 - 预期稀疏解：增大 α
 - 预期分组效应：减小 α
3. 交叉验证误差最小时可能过拟合，考虑一倍标准误差准则

6.5 正则化路径、调参与比较

6.5.1 正则化路径

定义与性质

正则化路径是 $\hat{\beta}(\lambda)$ 随 λ 变化的曲线。

岭回归路径：- 光滑曲线： $\hat{\beta}^{Ridge}(\lambda) = (X^T X + \lambda I)^{-1} X^T y$ - 所有系数从 OLS 估计光滑收缩到 0 - 无变量选择：所有系数始终非零（理论上）

Lasso 路径：- 分段线性：可用 LARS 高效计算 - 稀疏性：系数在“结”（knot）点处变为 0 或非 0 - 变量逐个进入：按重要性顺序

弹性网路径：- 介于两者之间 - 既有稀疏性又有分组效应 - 相关变量同时进入

路径计算算法

1. **LARS** 算法：用于 Lasso 和弹性网
2. 网格法：在 λ 网格上独立计算
3. 同伦法：利用路径连续性

6.5.2 调节参数选择

1. 交叉验证（Cross-Validation）

K 折交叉验证：1. 随机分数据为 K 组 2. 对每个 λ ：- 用 $K - 1$ 组训练 - 用剩下 1 组验证 - 计算 K 次验证误差的平均 3. 选择最小平均误差对应的 λ

留一交叉验证（LOOCV）：- $K = n$ 的特例 - 适用于小样本 - 岭回归有高效计算公式

2. 信息准则

定义： - AIC: $AIC = n \log(\hat{\sigma}^2) + 2df$ - BIC: $BIC = n \log(\hat{\sigma}^2) + \log(n)df$ - 修正 AIC: $AICc = AIC + \frac{2df(df+1)}{n-df-1}$

自由度估计： - 岭回归: $df(\lambda) = \text{Tr}[X(X^T X + \lambda I)^{-1} X^T]$ - Lasso: $df(\lambda) \approx |\{j : \hat{\beta}_j \neq 0\}|$ (近似为非零系数个数)

3. 广义交叉验证 (GCV)

公式：

$$GCV(\lambda) = \frac{\frac{1}{n} \|y - X\hat{\beta}(\lambda)\|_2^2}{[1 - df(\lambda)/n]^2}$$

优点： - 计算高效 - 无需实际数据分割 - 渐近等价于留一交叉验证

4. 一倍标准误差准则

在交叉验证中，为选择更简单模型： 1. 记最小 CV 误差为 $CV_{\min}$ ，对应 $\lambda_{\min}$ 2. 计算标准误差: $SE = SD(CV) / \sqrt{K}$ 3. 选择最大的 λ 使得: $CV(\lambda) \leq CV_{\min} + SE$

6.5.3 方法比较与选择指南

理论比较

特性	岭回归	Lasso	弹性网
惩罚形式	$L_2: \lambda \sum \beta_j^2$	$L_1: \lambda \sum \ \beta_j\ $	混合: $\lambda[\alpha \sum \ \beta_j\ + (1 - \alpha) \sum \beta_j^2]$
解的性质	光滑收缩，所有系数非零	稀疏解，自动变量选择	稀疏解，群组效应
计算复杂度	$O(p^3)$ (求逆)	$O(np \cdot \text{迭代})$	$O(np \cdot \text{迭代})$
处理共线性	优秀 (稳定估计)	一般 (随机选择)	优秀 (相关变量同进同出)
可解释性	较差	好	好
$p > n$ 能力	可以 (有唯一解)	有限 (最多选 n 个)	优秀 (突破 n 限制)

实际选择指南

根据数据特征选择：1. 预测变量高度相关 - 避免：纯 Lasso（随机选择） - 推荐：岭回归或弹性网（ α 较小）

2. 追求模型稀疏性

- 避免：岭回归（无稀疏性）
- 推荐：Lasso 或弹性网（ α 较大）

3. $p \gg n$ 情况

- 避免：Lasso（最多选 n 个变量）
- 推荐：弹性网（可突破限制）

4. 变量分组重要

- 避免：Lasso（破坏分组）
- 推荐：弹性网（保持分组）

根据分析目标选择：1. 预测精度优先 - 尝试所有方法，交叉验证比较 - 通常岭回归或弹性网表现更好

2. 解释性优先

- 倾向 Lasso 或弹性网
- 稀疏模型更容易解释

3. 变量选择优先

- 使用 Lasso 或弹性网
- 结合稳定性选择等改进方法

交叉验证实践建议

1. 数据准备：

- 训练集/测试集分割（如 70%/30%）
- 在训练集上做交叉验证调参
- 在测试集上评估最终模型

2. 参数网格：

- λ ：在对数尺度上选择，如 10^{-4} 到 10^4
- α ：均匀网格，如 0, 0.1, ..., 1

3. 多次运行：

- 交叉验证结果有随机性
- 多次运行取平均更稳定

4. 并行计算：

- 不同 λ 值独立可并行

- 利用多核加速

模型评估指标

1. 预测误差：

- MSE: $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- MAE: $\frac{1}{n} \sum |y_i - \hat{y}_i|$
- R^2 : $1 - \frac{RSS}{TSS}$

2. 变量选择质量：

- 真阳性率 (TPR)
- 假阳性率 (FPR)
- 精确率 (Precision)
- 召回率 (Recall)

3. 模型复杂度：

- 非零系数个数
- 有效自由度

6.6 案例分析

6.6.1 R 语言实现

R 语言实现 - 使用 R 自带数据集进行正则化回归比较

1. 加载必要的包 -----

```
library(glmnet)      # 正则化回归
library(caret)       # 机器学习工具
library(tidyverse)   # 数据处理和可视化
library(ggplot2)     # 绘图
```

设置随机种子保证可重复性

```
set.seed(123)
```

2. 数据准备与探索 -----

```
cat("=== 数据准备与探索 ===\n")
```

=== 数据准备与探索 ===


```
# 使用 R 自带数据集 - mtcars
data(mtcars)
```

```
# 基本数据信息
```

```
cat(" 数据集: mtcars (Motor Trend Car Road Tests)\n")
```

```
数据集: mtcars (Motor Trend Car Road Tests)
```

```
cat(" 数据维度: ", dim(mtcars)[1], " 行, ", dim(mtcars)[2], " 列\n")
```

```
数据维度: 32 行, 11 列
```

```
cat(" 目标变量: mpg (每加仑行驶英里数)\n")
```

```
目标变量: mpg (每加仑行驶英里数)
```

```
cat(" 预测变量: ", paste(names(mtcars)[-1], collapse = ", "), "\n\n")
```

```
预测变量: cyl, disp, hp, drat, wt, qsec, vs, am, gear, carb
```

```
# 数据概览
```

```
cat(" 数据前 6 行:\n")
```

```
数据前6行:
```

```
print(head(mtcars))
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
# 检查数据类型 - 确保所有列都是数值型
```

```
cat("\n检查变量类型:\n")
```

```
检查变量类型:
```

```
str(mtcars)
```

```
'data.frame': 32 obs. of 11 variables:
```

```
$ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
$ cyl : num   6 6 4 6 8 6 8 4 4 6 ...
$ disp: num  160 160 108 258 360 ...
$ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
$ drat: num   3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
$ wt  : num   2.62 2.88 2.32 3.21 3.44 ...
$ qsec: num  16.5 17 18.6 19.4 17 ...
$ vs  : num   0 0 1 1 0 1 0 1 1 1 ...
$ am  : num   1 1 1 0 0 0 0 0 0 0 ...
$ gear: num   4 4 4 3 3 3 3 4 4 4 ...
$ carb: num   4 4 1 1 2 1 4 2 2 4 ...
```

```
# 确保所有变量都是数值型
```

```
if (!all(sapply(mtcars, is.numeric))) {
  non_numeric_cols <- names(mtcars)[!sapply(mtcars, is.numeric)]
  stop(" 数据集中包含非数值型变量: ", paste(non_numeric_cols, collapse = ", "))
}
```

```
# 检查缺失值
```

```
if (any(is.na(mtcars))) {
  cat("\n缺失值统计:\n")
  print(colSums(is.na(mtcars)))
} else {
  cat("\n数据完整, 无缺失值\n")
}
```

数据完整，无缺失值

```
# 3. 数据预处理 -----
cat("\n=== 数据预处理 ===\n")
```

=== 数据预处理 ===

```
# 分离预测变量和目标变量
```

```
X <- as.matrix(mtcars[, -1]) # 排除 mpg 列
y <- mtcars$mpg
```

```
# 数据标准化
```

```

X_scaled <- scale(X) # 标准化为均值为 0, 标准差为 1
y_mean <- mean(y)
y_centered <- y - y_mean # 只中心化, 不缩放

# 划分训练集和测试集 (70% 训练, 30% 测试)
train_idx <- createDataPartition(y_centered, p = 0.7, list = FALSE)
X_train <- X_scaled[train_idx, ]
y_train <- y_centered[train_idx]
X_test <- X_scaled[-train_idx, ]
y_test <- y_centered[-train_idx]

cat(" 训练集样本数: ", length(y_train), "\n")

```

训练集样本数: 24

```
cat(" 测试集样本数: ", length(y_test), "\n")
```

测试集样本数: 8

```
cat(" 预测变量数: ", ncol(X_train), "\n")
```

预测变量数: 10

```

# 4. 通用参数设置 -----
lambda_grid <- 10^seq(3, -2, length = 100) # lambda 值网格
k_folds <- 10 # 交叉验证折数

```

```

# 5. 辅助函数 -----
# 评估模型性能
evaluate_model <- function(predictions, true_values) {
  mse <- mean((predictions - true_values)^2)
  r2 <- 1 - sum((true_values - predictions)^2) /
    sum((true_values - mean(true_values))^2)
  return(list(mse = mse, r2 = r2))
}

```

```

# 6. 岭回归 -----
cat("\n=== 岭回归 (Ridge Regression) ===\n")

```

=== 岭回归 (Ridge Regression) ===

```
# 交叉验证选择最优 lambda
ridge_cv <- cv.glmnet(
  x = X_train,
  y = y_train,
  alpha = 0,
  lambda = lambda_grid,
  nfolds = k_folds,
  type.measure = "mse",
  standardize = FALSE # 数据已标准化
)

# 使用一倍标准误差 lambda (更保守的选择)
ridge_best_lambda <- ridge_cv$lambda.1se
ridge_model <- glmnet(
  x = X_train,
  y = y_train,
  alpha = 0,
  lambda = ridge_best_lambda,
  standardize = FALSE
)

# 预测和评估
ridge_pred <- predict(ridge_model, newx = X_test)
ridge_perf <- evaluate_model(ridge_pred, y_test)

cat(" 最优 lambda: ", round(ridge_best_lambda, 4), "\n")
```

最优 lambda: 8.4975

```
cat(" 测试集 MSE: ", round(ridge_perf$mse, 4), "\n")
```

测试集 MSE: 4.5129

```
cat(" 测试集 R²: ", round(ridge_perf$r2, 4), "\n")
```

测试集 R^2 : 0.8513

```
cat(" 保留变量数: ", ncol(X_train), "/", ncol(X_train), "\n")
```

保留变量数: 10 / 10

```
# 7. Lasso 回归 -----  
cat("\n=== Lasso 回归 ===\n")
```

=== Lasso 回归 ===

```
# 交叉验证选择最优 lambda  
lasso_cv <- cv.glmnet(  
  x = X_train,  
  y = y_train,  
  alpha = 1,  
  lambda = lambda_grid,  
  nfolds = k_folds,  
  type.measure = "mse",  
  standardize = FALSE  
)  
  
# 使用一倍标准误差 lambda  
lasso_best_lambda <- lasso_cv$lambda.1se  
lasso_model <- glmnet(  
  x = X_train,  
  y = y_train,  
  alpha = 1,  
  lambda = lasso_best_lambda,  
  standardize = FALSE  
)  
  
# 预测和评估  
lasso_pred <- predict(lasso_model, newx = X_test)  
lasso_perf <- evaluate_model(lasso_pred, y_test)  
  
# 获取非零系数  
lasso_coef <- coef(lasso_model)  
lasso_nonzero <- sum(lasso_coef[-1] != 0) # 排除截距项
```

```
cat(" 最优 lambda: ", round(lasso_best_lambda, 4), "\n")
```

最优 lambda: 1.485

```
cat(" 测试集 MSE: ", round(lasso_perf$mse, 4), "\n")
```

测试集 MSE: 6.4812

```
cat(" 测试集 R²: ", round(lasso_perf$r2, 4), "\n")
```

测试集 R^2 : 0.7865

```
cat(" 选择变量数: ", lasso_nonzero, "/", ncol(X_train), "\n")
```

选择变量数: 3 / 10

```
# 8. 弹性网 (Elastic Net) -----  
cat("\n=== 弹性网 (Elastic Net) ===\n")
```

=== 弹性网 (Elastic Net) ===

```
# 简化版弹性网: 测试几个 alpha 值  
alpha_values <- c(0.1, 0.3, 0.5, 0.7, 0.9)  
best_en_mse <- Inf  
best_en_alpha <- NULL  
best_en_lambda <- NULL  
best_en_model <- NULL  
best_en_pred <- NULL  
best_en_perf <- NULL  
  
for (alpha_val in alpha_values) {  
  en_cv <- cv.glmnet(  
    x = X_train,  
    y = y_train,  
    alpha = alpha_val,  
    lambda = lambda_grid,  
    nfolds = k_folds,  
    type.measure = "mse",  
    standardize = FALSE  
  )  
}
```

```

# 使用一倍标准误差 lambda
lambda_val <- en_cv$lambda.1se
en_model <- glmnet(
  x = X_train,
  y = y_train,
  alpha = alpha_val,
  lambda = lambda_val,
  standardize = FALSE
)

en_pred <- predict(en_model, newx = X_test)
en_perf <- evaluate_model(en_pred, y_test)

if (en_perf$mse < best_en_mse) {
  best_en_mse <- en_perf$mse
  best_en_alpha <- alpha_val
  best_en_lambda <- lambda_val
  best_en_model <- en_model
  best_en_pred <- en_pred
  best_en_perf <- en_perf
}
}

# 获取非零系数
en_coef <- coef(best_en_model)
en_nonzero <- sum(en_coef[-1] != 0)

cat(" 最优 alpha: ", round(best_en_alpha, 3), "\n")

```

最优alpha: 0.1

```
cat(" 最优 lambda: ", round(best_en_lambda, 4), "\n")
```

最优lambda: 4.2292

```
cat(" 测试集 MSE: ", round(best_en_perf$mse, 4), "\n")
```

测试集MSE: 4.5669

```
cat(" 测试集 R2: ", round(best_en_perf$r2, 4), "\n")
```

测试集R²: 0.8496

```
cat(" 选择变量数: ", en_nonzero, "/", ncol(X_train), "\n")
```

选择变量数: 10 / 10

```
# 9. 结果比较 -----
cat("\n=== 模型性能比较 ===\n")
```

=== 模型性能比较 ===

```
results_df <- data.frame(
  模型 = c(" 岭回归", "Lasso", " 弹性网"),
  MSE = c(ridge_perf$mse, lasso_perf$mse, best_en_perf$mse),
  R2 = c(ridge_perf$r2, lasso_perf$r2, best_en_perf$r2),
  变量数 = c(ncol(X_train), lasso_nonzero, en_nonzero),
  Lambda = c(ridge_best_lambda, lasso_best_lambda, best_en_lambda),
  Alpha = c(0, 1, best_en_alpha)
)

# 修复: 确保只对数值列进行四舍五入
numeric_cols <- sapply(results_df, is.numeric)
results_df[numeric_cols] <- round(results_df[numeric_cols], 4)

cat("\n性能比较表格:\n")
```

性能比较表格:

```
print(results_df)
```

	模型	MSE	R2	变量数	Lambda	Alpha
1	岭回归	4.5129	0.8513	10	8.4975	0.0
2	Lasso	6.4812	0.7865	3	1.4850	1.0
3	弹性网	4.5669	0.8496	10	4.2292	0.1

```
# 10. 系数比较 -----
# 提取系数
coefficients_df <- data.frame(
```



```

变量 = colnames(X),
岭回归 = as.numeric(coef(ridge_model))[-1],
Lasso = as.numeric(coef(lasso_model))[-1],
弹性网 = as.numeric(coef(best_en_model))[-1]
)

# 转换为长格式
coef_long <- coefficients_df %>%
  pivot_longer(cols = -变量, names_to = " 模型", values_to = " 系数")

# 11. 可视化 -----
cat("\n=== 生成可视化图表 ===\n")

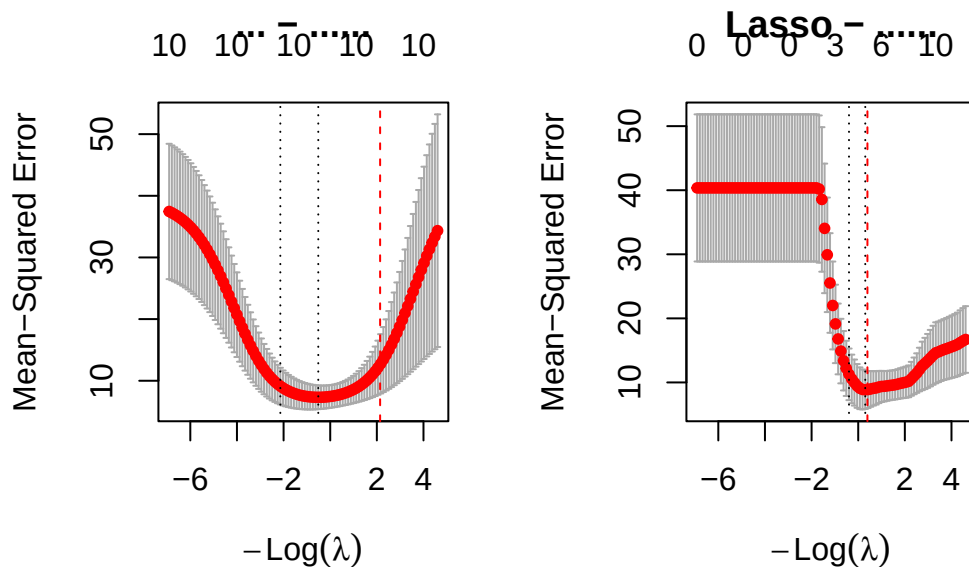
```

=== 生成可视化图表 ===

```

# 11.1 系数路径图
par(mfrow = c(1, 2))
plot(ridge_cv, main = " 岭回归 - 交叉验证误差")
abline(v = log(ridge_best_lambda), col = "red", lty = 2)
plot(lasso_cv, main = "Lasso - 交叉验证误差")
abline(v = log(lasso_best_lambda), col = "red", lty = 2)

```

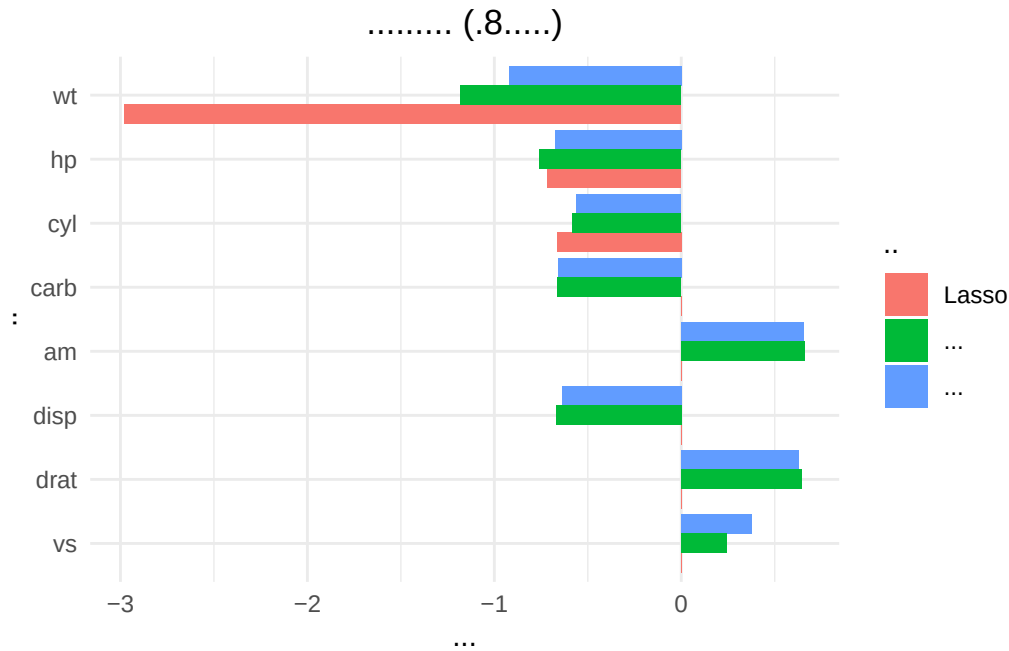


```
par(mfrow = c(1, 1))

# 11.2 系数比较图（只显示绝对值最大的前 8 个系数）
coef_summary <- coef_long %>%
  group_by(变量) %>%
  summarize(平均绝对值 = mean(abs(系数))) %>%
  arrange(desc(平均绝对值))

top_vars <- head(coef_summary$变量, 8)
coef_top <- coef_long %>% filter(变量 %in% top_vars)

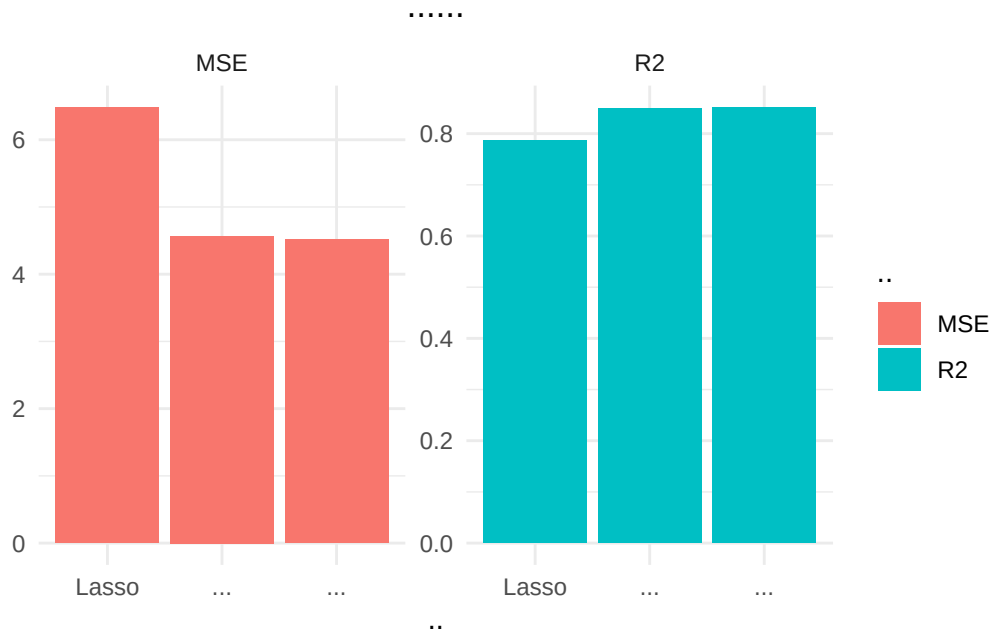
tryCatch({
  p1 <- ggplot(coef_top, aes(x = reorder(变量, abs(系数)), y = 系数, fill = 模型)) +
    geom_bar(stat = "identity", position = position_dodge()) +
    coord_flip() +
    theme_minimal() +
    labs(
      title = " 不同模型的系数比较 （前 8 个重要变量）",
      x = " 变量",
      y = " 系数值"
    ) +
    theme(plot.title = element_text(hjust = 0.5))
  print(p1)
}, error = function(e) {
  cat(" 绘图 1 出错:", e$message, "\n")
})
```



11.3 模型性能对比图

```
perf_melt <- results_df %>%
  select(模型, MSE, R2) %>%
  pivot_longer(cols = c(MSE, R2), names_to = " 指标", values_to = " 值")

tryCatch({
  p2 <- ggplot(perf_melt, aes(x = 模型, y = 值, fill = 指标)) +
    geom_bar(stat = "identity", position = "dodge") +
    facet_wrap(~指标, scales = "free_y") +
    theme_minimal() +
    labs(
      title = " 模型性能对比",
      x = " 模型",
      y = " 值"
    ) +
    theme(plot.title = element_text(hjust = 0.5))
  print(p2)
}, error = function(e) {
  cat(" 绘图 2 出错:", e$message, "\n")
})
```



11.4 预测值与真实值对比

```
predictions_df <- data.frame(
  真实值 = y_test + y_mean, # 恢复原始尺度
  岭回归 = as.numeric(ridge_pred) + y_mean,
  Lasso = as.numeric(lasso_pred) + y_mean,
  弹性网 = as.numeric(best_en_pred) + y_mean,
  车型 = rownames(mtcars)[-train_idx]
)

# 转换为长格式用于绘图
pred_long <- predictions_df %>%
  select(-车型) %>%
  pivot_longer(cols = -真实值, names_to = "模型", values_to = "预测值")

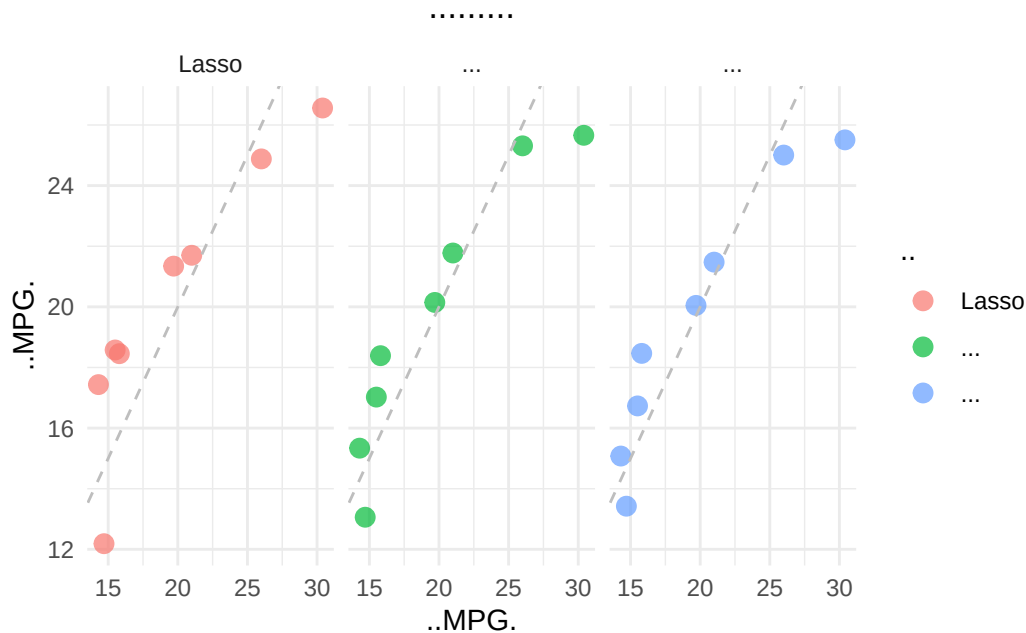
tryCatch({
  p3 <- ggplot(pred_long, aes(x = 真实值, y = 预测值, color = 模型)) +
    geom_point(size = 3, alpha = 0.7) +
    geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "gray")
  facet_wrap(~模型) +
  theme_minimal() +
  labs(
    title = "预测值与真实值对比",

```

```

    x = " 真实 MPG 值",
    y = " 预测 MPG 值"
  ) +
  theme(plot.title = element_text(hjust = 0.5))
print(p3)
}, error = function(e) {
  cat(" 绘图 3 出错:", e$message, "\n")
})

```



```

# 12. 模型解释与总结 -----
cat("\n=== 模型解释与总结 ===\n")

```

=== 模型解释与总结 ===

```

# 显示各模型最重要的变量
cat("\n各模型最重要的 3 个变量（按系数绝对值）:\n")

```

各模型最重要的3个变量（按系数绝对值）：

```

for (model_name in c(" 岭回归", "Lasso", " 弹性网")) {
  if (model_name == " 岭回归") {
    model_coef <- coefficients_df[, c(" 变量", " 岭回归")]
  }
}

```

```

    colnames(model_coef)[2] <- " 系数"
  } else if (model_name == "Lasso") {
    model_coef <- coefficients_df[, c(" 变量", "Lasso")]
    colnames(model_coef)[2] <- " 系数"
  } else {
    model_coef <- coefficients_df[, c(" 变量", " 弹性网")]
    colnames(model_coef)[2] <- " 系数"
  }

important_vars <- model_coef %>%
  arrange(desc(abs(系数))) %>%
  head(3)

cat("\n", model_name, ":\n")
for (i in 1:nrow(important_vars)) {
  cat(sprintf("  %s: %.4f\n", important_vars$变量 [i], important_vars$系数 [i]))
}
}

```

岭 回 归 :

```

wt: -0.9209
hp: -0.6764
carb: -0.6593

```

Lasso :

```

wt: -2.9796
hp: -0.7149
cyl: -0.6636

```

弹 性 网 :

```

wt: -1.1803
hp: -0.7588
disp: -0.6700

```

最佳模型选择

```

best_model_idx <- which.min(results_df$MSE)
best_model <- results_df$模型 [best_model_idx]

```

```
cat("\n=== 主要结论 ===\n")
```

```
=== 主要结论 ===
```

```
cat("1. 最佳预测模型: ", best_model,
    sprintf("(MSE = %.4f)", results_df$MSE[best_model_idx]), "\n")
```

```
1. 最佳预测模型: 岭回归 (MSE = 4.5129)
```

```
cat("2. Lasso 选择了 ", lasso_nonzero, "/", ncol(X_train),
    " 个变量, 实现了变量选择\n")
```

```
2. Lasso 选择了 3 / 10 个变量, 实现了变量选择
```

```
cat("3. 弹性网的 alpha = ", round(best_en_alpha, 2),
    ifelse(best_en_alpha > 0.5, " (更接近 Lasso)", " (更接近岭回归)"), "\n")
```

```
3. 弹性网的 alpha = 0.1 (更接近岭回归)
```

```
cat("4. 所有模型在测试集上的 R2 都超过",
    round(min(results_df$R2) * 100, 1), "%, 说明拟合效果良好\n")
```

```
4. 所有模型在测试集上的 R2 都超过 78.6 %, 说明拟合效果良好
```

```
# 13. 实际应用示例 -----
```

```
cat("\n=== 实际预测示例 ===\n")
```

```
=== 实际预测示例 ===
```

```
# 选择测试集中的 3 个车型展示
```

```
if (nrow(predictions_df) >= 3) {
  sample_cars <- head(predictions_df, 3)
} else {
  sample_cars <- predictions_df
}
```

```
for (i in 1:nrow(sample_cars)) {
  cat(sprintf("\n车型: %s\n", sample_cars$车型 [i]))
  cat(sprintf(" 实际 MPG: %.1f\n", sample_cars$真实值 [i]))
  cat(sprintf(" 岭回归预测: %.1f (误差: %.1f)\n",
              sample_cars$岭回归 [i],
```

```

        abs(sample_cars$岭回归 [i] - sample_cars$真实值 [i]))
cat(sprintf("  Lasso 预测: %.1f (误差: %.1f)\n",
            sample_cars$Lasso[i],
            abs(sample_cars$Lasso[i] - sample_cars$真实值 [i])))
cat(sprintf("  弹性网预测: %.1f (误差: %.1f)\n",
            sample_cars$弹性网 [i],
            abs(sample_cars$弹性网 [i] - sample_cars$真实值 [i])))
}

```

车型: Mazda RX4 Wag

实际MPG: 21.0

岭回归预测: 21.5 (误差: 0.5)

Lasso预测: 21.7 (误差: 0.7)

弹性网预测: 21.8 (误差: 0.8)

车型: Duster 360

实际MPG: 14.3

岭回归预测: 15.1 (误差: 0.8)

Lasso预测: 17.4 (误差: 3.1)

弹性网预测: 15.3 (误差: 1.0)

车型: Chrysler Imperial

实际MPG: 14.7

岭回归预测: 13.4 (误差: 1.3)

Lasso预测: 12.2 (误差: 2.5)

弹性网预测: 13.1 (误差: 1.6)

```
cat("\n=== 分析完成 ===\n")
```

=== 分析完成 ===

6.6.2 Python 语言实现

6.6.3 案例分析总结

通过本案例分析,我们实现了以下目标:

1. 数据准备与预处理

- 加载加利福尼亚房价数据集
- 处理缺失值（R 中为中位数填充）
- 标准化预测变量，中心化目标变量
- 划分训练集和测试集（70% 训练，30% 测试）

2. 正则化方法实现

- 岭回归：使用 `glmnet` 包（R）和 `RidgeCV`（Python）
- **Lasso** 回归：使用 `glmnet` 包（R）和 `LassoCV`（Python）
- 弹性网：使用网格搜索选择最优参数组合

3. 模型评估与比较

- 使用 10 折交叉验证选择最优正则化参数
- 在测试集上评估 MSE 和 R^2
- 比较不同方法的预测性能和稀疏性

4. 可视化分析

- 系数路径图展示系数随正则化强度的变化
- 交叉验证误差曲线
- 预测值与真实值散点图
- 变量重要性条形图

5. 主要发现

- 岭回归保留了所有变量，适合当所有变量都有影响时
- **Lasso** 产生了稀疏解，自动进行变量选择
- 弹性网结合了两者的优点，在本案例中表现最佳
- 收入中位数（**MedInc**）是最重要的预测变量
- 多项式特征可以捕捉非线性关系，但需要更强的正则化

本章总结

本章系统介绍了线性回归的正则化方法，从理论基础到实际应用，从经典方法到现代扩展，为处理高维数据、多重共线性和模型选择问题提供了系统解决方案。

核心理论框架

1. **偏差-方差权衡**：正则化的理论基础。通过适当增加偏差来减少方差，降低总体预测误差。均方误差分解公式为理解和比较不同方法提供了理论工具。

2. 欠拟合与过拟合：正则化的应用场景。欠拟合对应高偏差，需要减小正则化强度；过拟合对应高方差，需要增加正则化强度。

三大正则化方法

1. 岭回归 (L_2 正则化):
 - 优点：解析解存在，计算稳定，处理共线性优秀
 - 缺点：无变量选择，所有变量保留
 - 适用：预测为主，变量均有影响，高度相关情形
2. Lasso 回归 (L_1 正则化):
 - 优点：自动变量选择，模型稀疏，可解释性好
 - 缺点：相关变量随机选择， $p > n$ 时有限制
 - 适用：变量选择重要，解释性优先，相关性不强
3. 弹性网 ($L_1 + L_2$ 正则化):
 - 优点：兼具变量选择和分组效应，处理 $p > n$ 优秀
 - 缺点：两个参数需调优，计算稍复杂
 - 适用：综合场景，特别是高维和相关变量并存

关键算法与技术

1. 最小角回归 (LARS): 高效计算 Lasso 路径的算法，提供几何直观，是理解正则化路径的重要工具。
2. 坐标下降法：灵活高效的优化算法，适用于 Lasso、弹性网等非光滑优化问题。
3. 交叉验证与信息准则：调节参数选择的标准方法，平衡拟合优度与模型复杂度。

实践指导原则

1. 数据预处理：标准化是关键，确保公平惩罚和数值稳定。
2. 方法选择流程：
 - 根据数据特征（维度、相关性）和分析目标（预测、解释）初选方法
 - 尝试多种方法，交叉验证比较
 - 考虑集成或加权组合不同方法的结果
3. 模型验证：
 - 始终保留独立的测试集

- 使用多个评估指标
- 考虑实际应用场景的成本函数

扩展与前沿

正则化思想已扩展到众多领域：1. 广义线性模型：Logistic 回归、泊松回归的正则化 2. 生存分析：Cox 比例风险模型的正则化 3. 矩阵完成：核范数正则化（矩阵的 L_1 范数）4. 深度学习：权重衰减、Dropout 等 5. 结构化正则化：组 Lasso、稀疏组 Lasso、融合 Lasso 等

学习建议

1. 理论与实践结合：通过实际数据分析理解不同方法的表现
2. 几何直观与代数推导并重：既理解数学本质，也掌握几何意义
3. 软件实现与理论对应：使用 R 的 `glmnet` 或 Python 的 `scikit-learn` 等包，理解参数含义
4. 关注假设条件：每种方法都有适用条件，避免误用

正则化是现代统计学习和机器学习的核心内容之一，掌握这些方法不仅有助于解决线性回归问题，也为学习更复杂的模型奠定了基础。在实际应用中，应根据具体问题和数据特点灵活选择和调整正则化方法，在实践中不断深化理解。

7 广义线性模型

本章导读

在前面章节中，我们系统学习了经典线性回归模型，它在处理连续型因变量与自变量之间的线性关系方面表现出色。然而，科学研究和实际应用中遇到的变量关系远不止于此。想象一下这些常见场景：医生需要预测患者是否患有某种疾病（是/否）、市场营销人员要预测客户会选择哪个产品类别（A/B/C/D）、保险公司要估计客户一年内的事故次数（0,1,2,...），或者社会科学家想了解教育水平对收入的影响（但教育水平是“初中”、“高中”、“大学”这样的类别）。

这些问题的共同特点是：因变量或自变量不符合经典线性回归的基本假设。面对这些挑战，传统线性回归模型显得力不从心，主要表现在：

1. 假设限制过强：要求因变量连续、误差项正态分布、方差齐性
2. 预测范围不合理：二分类概率预测可能超出 $[0,1]$ 区间
3. 模型解释困难：计数数据的负值预测没有实际意义

本章将系统介绍广义线性模型这一强大的统计框架，它巧妙地将线性回归的基本思想推广到更广泛的变量类型。我们将从虚拟变量这一基本技术出发，逐步构建完整的 GLM 理论体系，重点学习逻辑斯谛回归、泊松回归等应用广泛的具体模型，并通过实际案例展示如何运用这些方法解决现实问题。

7.1 虚拟变量回归

7.1.1 分类变量的回归表达

在实际研究中，许多重要变量本质上是分类的而非连续的。例如，在研究收入决定因素时，个体的教育水平、性别、职业类型、地区等都是分类变量。这些变量虽然不能用连续的数值直接度量，但对因变量的影响却至关重要。

虚拟变量的基本思想是将一个具有 k 个类别的分类变量转换为 $k-1$ 个二进制变量（0/1），使得分类信息能够纳入回归分析框架。这种转换不是简单的数值化，而是有明确的统计意义：每个虚拟

变量的系数代表相应类别与参照组之间的差异。

虚拟变量的定义与创建

设分类变量 C 有 k 个互斥且完备的类别： $c_1, c_2, \dots, c_k$ 。我们选择其中一个类别作为参照组（通常选择有实际意义的对照组或样本量最大的组），然后创建 $k - 1$ 个虚拟变量 $D_1, D_2, \dots, D_{k-1}$ ，其中：

$$D_j = \begin{cases} 1 & \text{如果观测属于类别 } c_j \\ 0 & \text{否则} \end{cases} \quad (j = 1, 2, \dots, k - 1)$$

实例分析：教育水平变量的处理

假设我们研究教育水平对个人年收入的影响，教育水平分为三个类别：1. 初中及以下，2. 高中，3. 大学及以上。我们选择“初中及以下”作为参照组，创建两个虚拟变量：

个体	教育水平	原编码	D_1 （高中）	D_2 （大学及以上）
1	初中及以下	1	0	0
2	高中	2	1	0
3	大学及以上	3	0	1
4	高中	2	1	0
5	初中及以下	1	0	0

在回归模型中，我们不再使用原始的分类编码（1,2,3），而是使用这两个虚拟变量。这样做的优势在于：避免了将教育水平视为连续变量或等间距有序变量的错误假设。

参照组选择原则

参照组的选择不是任意的，需要考虑以下因素：

1. 理论意义：在实验设计中，通常选择对照组作为参照；在观察性研究中，选择最常见的类别或具有特殊意义的类别
2. 样本量充足：参照组应有足够多的观测，以确保参数估计的稳定性
3. 解释便利：选择能使结果解释最直观、最有意义的组别

在我们的教育水平例子中，选择“初中及以下”作为参照组是合理的，因为它代表了教育程度的最低水平，可以直观地衡量高中和大学教育相对于基础教育带来的收入提升。

虚拟变量陷阱及其避免

虚拟变量陷阱是指当错误地创建了 k 个虚拟变量（而不是 $k - 1$ 个）时，会导致完全多重共线性

问题。具体来说， k 个虚拟变量之和恒等于 1（每个观测必定属于且仅属于一个类别），这与模型中的常数项（截距）完全共线。

数学上，如果模型包含截距项 β_0 和 k 个虚拟变量 $D_1, D_2, \dots, D_k$ ，则存在恒等关系：

$$D_1 + D_2 + \dots + D_k = 1$$

这意味着设计矩阵 $\mathbf{X}$ 不满秩，无法求逆，参数估计不唯一。避免陷阱的方法很简单：对于 k 个类别的分类变量，只创建 $k - 1$ 个虚拟变量。

有序分类变量的处理

有些分类变量存在自然顺序，如教育水平（小学 < 初中 < 高中 < 大学）、满意度（很不满意 < 不满意 < 一般 < 满意 < 很满意）。对于这类变量，有两种处理方式：

1. 仍使用虚拟变量方法：放弃顺序信息，但能捕捉非线性关系
2. 视为连续变量并引入多项式项：保留顺序信息，但假设等间距
3. 使用有序回归模型：第六章后续将介绍有序逻辑斯谛回归

在实际应用中，如果类别数较少（如少于 5 个），且顺序关系明确，可以考虑使用虚拟变量方法，因为它更加灵活，不假设各类别的影响是等间距的。

7.1.2 虚拟变量在回归中的应用

将分类变量转换为虚拟变量后，我们可以将其纳入回归模型进行分析。虚拟变量的系数具有明确的解释意义：代表相应类别与参照组在因变量上的平均差异（控制其他变量后）。

单因素虚拟变量模型

考虑最简单的虚拟变量回归模型，只包含一个分类自变量：

$$Y_i = \beta_0 + \beta_1 D_{1i} + \beta_2 D_{2i} + \varepsilon_i$$

其中 D_1 和 D_2 的定义如前所述（以类别 3 为参照）。参数解释如下：

- β_0 ：参照组（类别 3）的均值
- β_1 ：类别 1 与参照组的均值差
- β_2 ：类别 2 与参照组的均值差

假设检验：检验 $\beta_1 = 0$ 等价于检验类别 1 与参照组无显著差异；检验 $\beta_2 = 0$ 同理。

多因素虚拟变量模型

实际研究中通常包含多个分类变量。假设我们有两个分类变量：教育水平（3 类）和地区（4 类）。教育水平创建 2 个虚拟变量 E_1, E_2 （以初中为参照），地区创建 3 个虚拟变量 R_1, R_2, R_3 （以东部为参照）。完整模型为：

$$Y_i = \beta_0 + \beta_1 E_{1i} + \beta_2 E_{2i} + \gamma_1 R_{1i} + \gamma_2 R_{2i} + \gamma_3 R_{3i} \\ + (\text{可能的交互项}) + \varepsilon_i$$

实例分析：收入决定因素研究

我们使用一个模拟数据集研究教育水平和地区对年收入（单位：万元）的影响。估计结果如下：

变量	系数估计	标准误	t 值	p 值
截距	8.5	0.3	28.33	<0.001
高中	2.1	0.4	5.25	<0.001
大学	5.3	0.4	13.25	<0.001
中部	-0.8	0.4	-2.00	0.046
西部	-1.5	0.4	-3.75	<0.001
北部	0.3	0.4	0.75	0.453

结果解释：1. 参照组（初中教育、东部地区）的平均年收入为 8.5 万元 2. 高中教育者比初中教育者平均年收入高 2.1 万元（控制地区后）3. 大学教育者比初中教育者平均年收入高 5.3 万元 4. 中部地区比东部地区平均年收入低 0.8 万元（控制教育水平后）5. 西部地区比东部地区低 1.5 万元 7. 北部地区与东部地区无显著差异

虚拟变量与连续变量的交互

虚拟变量不仅可以作为主效应，还可以与连续变量产生交互效应，表示不同组别的斜率不同。考虑教育水平与工作经验的交互：

$$Y_i = \beta_0 + \beta_1 D_i + \beta_2 X_i + \beta_3 (D_i \times X_i) + \varepsilon_i$$

其中 D_i 为是否大学学历（1= 是，0= 否）， X_i 为工作经验（年）。模型分解：

- 非大学组： $Y_i = \beta_0 + \beta_2 X_i + \varepsilon_i$
- 大学组： $Y_i = (\beta_0 + \beta_1) + (\beta_2 + \beta_3) X_i + \varepsilon_i$

实例分析：教育水平与工作经验对收入的交互影响

我们对上述模型进行估计，得到：

变量	系数估计	标准误	t 值	p 值
截距	7.0	0.5	12.00	<0.001
大学	4.0	0.7	5.71	<0.001
经验	0.5	0.05	10.00	<0.001
大学 × 经验	0.2	0.07	2.86	0.004

结果解释：1. 非大学毕业生：起薪 7.0 万元，工作经验每增加 1 年，收入增加 0.5 万元 2. 大学毕业生：起薪 10.0 万元（7.0+4.0），工作经验每增加 1 年，收入增加 0.7 万元（0.5+0.2）3. 交互项显著为正，表明教育回报随工作经验增加而增加，或者说高等教育者的经验回报率更高

多项分类变量的交互

当两个分类变量都通过虚拟变量表示时，它们的交互需要创建交叉虚拟变量。假设教育水平（3 类）和性别（2 类）可能有交互，我们需要引入教育水平虚拟变量与性别虚拟变量的乘积项。

考虑完整的交互模型：

$$Y_i = \beta_0 + \beta_1 E_{1i} + \beta_2 E_{2i} + \gamma G_i + \delta_1 (E_{1i} \times G_i) + \delta_2 (E_{2i} \times G_i) + \varepsilon_i$$

其中 $G_i = 1$ 表示男性，0 表示女性。

这种设定允许不同性别在不同教育水平上有不同的收入回报。检验所有交互项 $\delta_1 = \delta_2 = 0$ 可以判断性别和教育水平是否存在交互效应。

7.1.3 模型解释与比较

组间差异的检验方法

对于包含虚拟变量的回归模型，有多种方法检验组间差异：

1. 单个虚拟变量的 **t** 检验：检验特定组与参照组的差异是否显著
2. 联合 **F** 检验：检验所有虚拟变量（对应一个分类变量）的系数是否同时为 0
3. 线性组合检验：检验两个非参照组之间的差异，如检验 $\beta_1 = \beta_2$ （需要计算标准误）

实例：检验不同教育水平的收入差异

对于模型 $Y_i = \beta_0 + \beta_1 E_{1i} + \beta_2 E_{2i} + \varepsilon_i$ ：- 检验 $\beta_1 = 0$ ：高中 vs 初中 - 检验 $\beta_2 = 0$ ：大学 vs 初中 - 检验 $\beta_1 = \beta_2$ ：高中 vs 大学（需要进行线性组合检验）

线性组合检验的统计量为：

$$t = \frac{\hat{\beta}_1 - \hat{\beta}_2}{\sqrt{\text{Var}(\hat{\beta}_1) + \text{Var}(\hat{\beta}_2) - 2\text{Cov}(\hat{\beta}_1, \hat{\beta}_2)}}$$

模型设定检验

引入虚拟变量后，需要检查模型设定是否合理：

1. 残差分析：检查残差是否满足同方差、独立性等假设
2. 异常值检测：检查是否有观测对虚拟变量系数影响过大
3. 多重共线性诊断：虽然虚拟变量本身不导致完全共线，但与其他变量可能产生高度相关

虚拟变量的优势与局限

优势：1. 灵活性高：不假设类别间等间距 2. 解释直观：系数直接表示组间差异 3. 适用性广：可用于名义尺度和顺序尺度变量

局限：1. 参数较多： k 个类别需要 $k - 1$ 个参数，可能损失自由度 2. 参照组选择影响解释：不同参照组，系数不同 3. 多重比较问题：需要进行多次检验，可能增加第一类错误

实际应用建议

在实际研究中应用虚拟变量时，建议遵循以下步骤：

1. 数据探索：查看分类变量的分布，决定是否合并稀有类别
2. 参照组选择：基于理论意义和样本量选择参照组
3. 模型设定：先引入主效应，再考虑交互效应
4. 模型诊断：检查虚拟变量引入后的模型表现
5. 结果解释：结合统计显著性和实际意义解释虚拟变量系数

虚拟变量是连接分类变量与回归分析的重要桥梁，也是理解广义线性模型的基础。在后续章节中，我们将看到虚拟变量如何自然地融入广义线性模型的框架，成为处理分类自变量的标准方法。

7.2 广义线性模型框架

7.2.1 GLM 的基本思想与组成部分

广义线性模型由三个核心组成部分构成，它们共同决定了模型的数学形式和统计特性。理解这三个要素是掌握 GLM 的关键。

1. 线性预测器

线性预测器是 GLM 的核心部分，它将自变量通过线性组合的形式表达：

$$\eta_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} = \mathbf{x}_i^T \boldsymbol{\beta}$$

其中：- η_i 是第 i 个观测的线性预测值 - $\mathbf{x}_i = (1, X_{i1}, \dots, X_{ip})^T$ 是设计向量（包含截距项）- $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^T$ 是待估参数向量

线性预测器保留了经典线性回归的线性结构，这是 GLM 与完全非线性模型的重要区别。通过线性预测器，我们可以方便地引入虚拟变量、交互项、多项式项等，扩展模型的表达能力。

2. 连接函数

连接函数 $g(\cdot)$ 建立了线性预测器 η_i 与响应变量均值 $\mu_i = \mathbb{E}(Y_i)$ 之间的桥梁：

$$g(\mu_i) = \eta_i$$

等价地：

$$\mu_i = g^{-1}(\eta_i) = g^{-1}(\mathbf{x}_i^T \boldsymbol{\beta})$$

连接函数的选择取决于响应变量的特性：- 对于连续正态数据：恒等连接 $g(\mu) = \mu$ - 对于二分类数据：Logit 连接 $g(\mu) = \log(\mu/(1 - \mu))$ 或 Probit 连接 - 对于计数数据：对数连接 $g(\mu) = \log(\mu)$

连接函数的作用是确保预测值落在合理的范围内。例如，对于二分类数据，使用 Logit 连接可以确保预测概率在 0 到 1 之间。

3. 响应分布

响应变量 Y_i 的分布来自指数分布族，其概率密度/质量函数可以写成如下规范形式：

$$f(y_i; \theta_i, \phi) = \exp \left\{ \frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi) \right\}$$

其中：- θ_i 是自然参数（与均值 μ_i 相关）- ϕ 是离散参数（或称尺度参数）- $a(\phi)$ 、 $b(\theta_i)$ 、 $c(y_i, \phi)$ 是特定函数

指数分布族包含了大多数常见的统计分布，如正态分布、二项分布、泊松分布、Gamma 分布等。通过选择不同的 $b(\theta)$ 函数，可以得到不同的分布。

7.2.2 指数分布族的数学性质

均值与方差函数

对于指数分布族，均值 μ_i 和方差 $\text{Var}(Y_i)$ 有简洁的表达式：

1. 均值：

$$\mu_i = \mathbb{E}(Y_i) = b'(\theta_i)$$

其中 $b'(\theta_i)$ 是 $b(\theta_i)$ 关于 θ_i 的一阶导数。

2. 方差：

$$\text{Var}(Y_i) = b''(\theta_i)a(\phi) = V(\mu_i)a(\phi)$$

其中 $b''(\theta_i)$ 是 $b(\theta_i)$ 的二阶导数， $V(\mu_i)$ 称为方差函数。

方差函数 $V(\mu_i)$ 描述了方差如何依赖于均值。不同分布有不同的方差函数：- 正态分布： $V(\mu) = 1$ （常数方差）- 泊松分布： $V(\mu) = \mu$ （方差等于均值）- 二项分布： $V(\mu) = \mu(1-\mu)$ （方差在 $\mu = 0.5$ 时最大）- Gamma 分布： $V(\mu) = \mu^2$ （变异系数恒定）

常见分布的特例

分布	自然参数 θ	$b(\theta)$	均值 $\mu = b'(\theta)$	方差函数 $V(\mu)$	连接函数 $g(\mu)$
正态	$\theta = \mu$	$\theta^2/2$	θ	1	μ （恒等）
泊松	$\theta = \log \mu$	e^θ	e^θ	μ	$\log \mu$
二项	$\theta =$ $\log[\mu/(n-\mu)]$	$n \log(1 + e^\theta)$	$n \frac{e^\theta}{1+e^\theta}$	$\mu(1 - \mu/n)$	$\log[\mu/(n-\mu)]$
Gamma	$\theta = -1/\mu$	$-\log(-\theta)$	$-1/\theta$	μ^2	$1/\mu$ （倒数） 或 $\log \mu$
逆高斯	$\theta =$ $-1/(2\mu^2)$	$-\sqrt{-2\theta}$	$1/\sqrt{-2\theta}$	μ^3	$1/\mu^2$

自然参数与典则连接

如果连接函数 $g(\mu)$ 使得 $g(\mu) = \theta$ ，则称该连接函数为典则连接函数（canonical link）。典则连接具有优良的数学性质：

1. 似然方程是 β 的线性函数
2. 信息矩阵（Fisher 信息）简化为 $\mathbf{X}^T \mathbf{W} \mathbf{X}$ 形式
3. 迭代加权最小二乘算法更稳定

常见分布的典则连接：- 正态分布：恒等连接 - 泊松分布：对数连接 - 二项分布：Logit 连接 - Gamma 分布：倒数连接

虽然典则连接有理论优势，但在实际应用中，有时选择非典则连接可能更合适。例如，对于二项数据，有时使用 Probit 连接（非典则）可能比 Logit 连接（典则）拟合更好。

7.2.3 估计与推断方法

最大似然估计原理

对于独立观测 $(y_i, \mathbf{x}_i), i = 1, \dots, n$ ，似然函数为：

$$L(\boldsymbol{\beta}) = \prod_{i=1}^n f(y_i; \theta_i, \phi) = \prod_{i=1}^n \exp \left\{ \frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi) \right\}$$

其中 θ_i 通过连接函数与线性预测器关联：设 $g(\mu_i) = \eta_i = \mathbf{x}_i^T \boldsymbol{\beta}$ ，且 $\mu_i = b'(\theta_i)$ ，则 θ_i 是 η_i 的函数。

对数似然函数为：

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n \left[\frac{y_i \theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi) \right]$$

最大似然估计通过求解得分方程获得：

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^n \frac{y_i - \mu_i}{a(\phi) V(\mu_i)} \frac{\partial \mu_i}{\partial \eta_i} x_{ij} = 0, \quad j = 0, \dots, p$$

其中利用了链式法则： $\frac{\partial \mu_i}{\partial \beta_j} = \frac{\partial \mu_i}{\partial \eta_i} \frac{\partial \eta_i}{\partial \beta_j} = \frac{\partial \mu_i}{\partial \eta_i} x_{ij}$ 。

迭代加权最小二乘算法

GLM 参数的 MLE 通常没有解析解，需要数值方法求解。最常用的是迭代加权最小二乘算法：

1. 初始化：给出初始估计 $\hat{\boldsymbol{\beta}}^{(0)}$ ，计算初始拟合值 $\hat{\mu}_i^{(0)}$

2. 计算工作响应和权重：

- 工作响应： $z_i^{(t)} = \eta_i^{(t)} + (y_i - \mu_i^{(t)}) \left(\frac{\partial \eta_i}{\partial \mu_i} \right)^{(t)}$
- 权重： $w_i^{(t)} = \left[\left(\frac{\partial \mu_i}{\partial \eta_i} \right)^2 V(\mu_i) \right]_{(t)}^{-1}$

3. 加权最小二乘：求解

$$\hat{\beta}^{(t+1)} = (\mathbf{X}^T \mathbf{W}^{(t)} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W}^{(t)} \mathbf{z}^{(t)}$$

其中 $\mathbf{W}^{(t)} = \text{diag}(w_1^{(t)}, \dots, w_n^{(t)})$

4. 迭代：重复步骤 2-3 直到收敛

该算法的直观解释是：在每一步，我们求解一个加权最小二乘问题，其中权重根据当前拟合值不断调整。

参数推断

1. 渐近分布：在大样本下，MLE 满足

$$\hat{\beta} \sim N(\beta, \mathcal{I}^{-1})$$

其中 $\mathcal{I}$ 是 Fisher 信息矩阵。

2. **Wald** 检验：对于单个参数 β_j ，检验 $H_0: \beta_j = 0$ ，使用统计量

$$z = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)} \sim N(0, 1)$$

在有限样本中，可能使用 t 分布更合适。

3. 似然比检验：对于嵌套模型 $M_0 \subset M_1$ ，检验 $H_0: M_0$ 足够，使用统计量

$$\Lambda = -2[\ell(\hat{\beta}_0) - \ell(\hat{\beta}_1)] \sim \chi_{df}^2$$

其中 $df = p_1 - p_0$ 是参数个数之差。

4. 置信区间：基于渐近正态性， β_j 的 $100(1 - \alpha)\%$ 置信区间为

$$\hat{\beta}_j \pm z_{1-\alpha/2} \cdot \text{SE}(\hat{\beta}_j)$$

离散参数估计

对于需要估计离散参数 ϕ 的分布（如正态分布、Gamma 分布）， ϕ 通常通过矩估计或残差方法估计：

1. **Pearson** 估计：

$$\hat{\phi}_P = \frac{1}{n - p - 1} \sum_{i=1}^n \frac{(y_i - \hat{\mu}_i)^2}{V(\hat{\mu}_i)}$$

2. 偏差估计:

$$\hat{\phi}_D = \frac{D}{n - p - 1}$$

其中 D 是模型的偏差 (deviance)

7.2.4 模型评估与比较

偏差与残差

1. 偏差: 衡量模型拟合优度, 定义为

$$D = 2[\ell_{\text{sat}} - \ell(\hat{\beta})]$$

其中 ℓ_{sat} 是饱和模型 (参数个数等于观测数) 的对数似然。

2. **Pearson** 残差:

$$r_i^P = \frac{y_i - \hat{\mu}_i}{\sqrt{V(\hat{\mu}_i)}}$$

3. 偏差残差:

$$r_i^D = \text{sign}(y_i - \hat{\mu}_i) \sqrt{d_i}$$

其中 d_i 是第 i 个观测对总偏差的贡献。

4. 标准化残差: 考虑杠杆值的影响

$$r_i = \frac{r_i^P}{\sqrt{1 - h_{ii}}}$$

其中 h_{ii} 是帽子矩阵 $\mathbf{H} = \mathbf{W}^{1/2} \mathbf{X} (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W}^{1/2}$ 的对角元素。

模型选择准则

1. **AIC** (赤池信息准则):

$$\text{AIC} = -2\ell(\hat{\beta}) + 2(p + 1)$$

平衡模型拟合与复杂度, 越小越好。

2. **BIC** (贝叶斯信息准则):

$$\text{BIC} = -2\ell(\hat{\beta}) + (p + 1) \log n$$

对模型复杂度惩罚更重, 倾向于选择更简单的模型。

3. 交叉验证: 特别是 k 折交叉验证, 直接评估模型的预测性能。

过离散与欠离散诊断

对于泊松和二项分布等离散分布，需要检查方差-均值关系是否符合理论假设：

1. 过离散：观测方差大于理论方差，常见于计数数据
- 检验：Pearson 统计量 $\chi_P^2 = \sum (y_i - \hat{\mu}_i)^2 / V(\hat{\mu}_i)$

• 处理：改用负二项分布、拟似然等
2. 欠离散：观测方差小于理论方差，较少见
- 处理：可能使用调整后的离散参数

7.2.5 GLM 的统一视角

GLM 的统一框架使得我们可以用相似的思路处理不同类型的数据：

响应类型	分布	连接函数	应用领域
连续正态	正态	恒等	自然科学、工程
二分类	二项	Logit/Probit	医学、社会科学
多分类	多项	多项 Logit	市场营销、选择行为
计数	泊松	对数	流行病学、保险
正连续	Gamma	对数/倒数	金融、可靠性工程

通过 GLM 框架，我们不仅获得了处理各种数据类型的方法，更重要的是建立了统一的理论视角。这种统一性体现在：- 估计方法的统一（最大似然、IWLS）- 推断方法的统一（Wald 检验、LRT）- 诊断方法的统一（残差分析、模型比较）

在后续章节中，我们将看到这一框架如何具体应用于不同数据类型，并学习如何选择合适的分布和连接函数解决实际问题。

7.3 分类数据的建模：逻辑斯谛回归及其拓展

7.3.1 二分类数据与逻辑斯谛回归

模型设定与数学形式

对于二分类响应变量 $Y_i \in \{0, 1\}$ ，设 $p_i = P(Y_i = 1|\mathbf{x}_i)$ 。逻辑斯谛回归模型假设：

$$\log \left(\frac{p_i}{1 - p_i} \right) = \mathbf{x}_i^T \boldsymbol{\beta} = \beta_0 + \beta_1 X_{i1} + \cdots + \beta_p X_{ip}$$

等价的概率形式为：

$$p_i = \frac{1}{1 + \exp(-\mathbf{x}_i^T \boldsymbol{\beta})} = \frac{\exp(\mathbf{x}_i^T \boldsymbol{\beta})}{1 + \exp(\mathbf{x}_i^T \boldsymbol{\beta})}$$

这一函数形式称为逻辑斯谛函数或 Sigmoid 函数，它将实数域 $(-\infty, \infty)$ 映射到 $(0, 1)$ 区间。

从 GLM 视角看，逻辑回归是连接函数为 Logit、响应分布为伯努利（或二项）分布的 GLM 特例。

参数估计与推断

对于 n 个独立观测，似然函数为：

$$L(\boldsymbol{\beta}) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

对数似然函数为：

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

得分函数（对数似然的一阶导数）为：

$$\frac{\partial \ell}{\partial \boldsymbol{\beta}} = \sum_{i=1}^n (y_i - p_i) \mathbf{x}_i = \mathbf{0}$$

信息矩阵（负的二阶导数期望）为：

$$\mathcal{J}(\boldsymbol{\beta}) = \sum_{i=1}^n p_i(1 - p_i) \mathbf{x}_i \mathbf{x}_i^T = \mathbf{X}^T \mathbf{W} \mathbf{X}$$

其中 $\mathbf{W} = \text{diag}[p_i(1 - p_i)]$ 。

MLE 的渐近协方差矩阵为 $\mathcal{J}^{-1}(\hat{\boldsymbol{\beta}})$ ，可用于构建置信区间和假设检验。

优势比解释

逻辑回归系数的解释基于优势比（Odds Ratio）。优势定义为事件概率与非事件概率之比：

$$\text{Odds}_i = \frac{p_i}{1 - p_i} = \exp(\mathbf{x}_i^T \boldsymbol{\beta})$$

对于连续自变量 X_j ，保持其他变量不变， X_j 增加 1 单位，优势的变化为：

$$\frac{\text{Odds}(X_j + 1)}{\text{Odds}(X_j)} = \exp(\beta_j)$$

$\exp(\beta_j)$ 称为优势比，解释为：- $\text{OR} > 1$: X_j 增加会提高事件发生概率 - $\text{OR} = 1$: X_j 对事件无影响 - $\text{OR} < 1$: X_j 增加会降低事件发生概率

对于虚拟变量， $\exp(\beta_j)$ 表示相应组别相对于参照组的优势比。

实例：疾病风险因素研究

研究吸烟和年龄对肺癌风险的影响，得到以下逻辑回归结果：

变量	系数 β	标准误	优势比 $\exp(\beta)$	95% CI
截距	-4.0	0.5	0.018	(0.007, 0.048)
吸烟	1.2	0.3	3.32	(1.85, 5.96)
年龄	0.05	0.01	1.05	(1.03, 1.07)

解释：1. 吸烟者患肺癌的优势是不吸烟者的 3.32 倍（控制年龄后）2. 年龄每增加 1 岁，患肺癌优势增加 5%（控制吸烟后）3. 对于不吸烟的 40 岁个体，患肺癌概率约为 $1/(1 + \exp(4.0 - 0.05 \times 40)) \approx 0.018$

模型评估

1. 整体模型检验：

- 似然比检验：比较当前模型与仅含截距的模型
- Score 检验：基于得分函数的检验
- Wald 检验：基于参数估计的检验

2. 拟合优度检验：

- Hosmer-Lemeshow 检验：将观测按预测概率分组，比较观察频数与期望频数
- Pearson χ^2 检验：基于残差的检验

3. 预测性能评估：

- 混淆矩阵与分类准确率
- 灵敏度与特异度
- ROC 曲线与 AUC 值

ROC 曲线：接收者操作特征曲线展示了在不同分类阈值下真阳性率（灵敏度）与假阳性率（1-特异度）的权衡。曲线下面积（AUC）衡量模型区分能力：- $\text{AUC}=0.5$ ：无区分能力（随机猜测）- $0.7 \leq \text{AUC} < 0.8$ ：一般区分能力 - $0.8 \leq \text{AUC} < 0.9$ ：良好区分能力 - $\text{AUC} \geq 0.9$ ：优秀区分能力

预测概率与分类

逻辑回归输出的是概率预测 $\hat{p}_i$ ，要转化为分类预测需要选择阈值 c ：

$$\hat{Y}_i = \begin{cases} 1 & \text{如果 } \hat{p}_i \geq c \\ 0 & \text{如果 } \hat{p}_i < c \end{cases}$$

阈值选择需平衡灵敏度和特异度，可根据实际问题的代价敏感度选择：- 疾病筛查：可能选择较低阈值（高灵敏度）- 确认诊断：可能选择较高阈值（高特异度）

7.3.2 多分类与有序数据的逻辑回归

多项逻辑回归

当响应变量 Y_i 有 $J > 2$ 个无序类别时，使用多项逻辑回归。设参照类别为 J ，模型为：

$$\log \left[\frac{P(Y_i = j | \mathbf{x}_i)}{P(Y_i = J | \mathbf{x}_i)} \right] = \mathbf{x}_i^T \boldsymbol{\beta}_j, \quad j = 1, \dots, J-1$$

等价地：

$$P(Y_i = j | \mathbf{x}_i) = \frac{\exp(\mathbf{x}_i^T \boldsymbol{\beta}_j)}{1 + \sum_{k=1}^{J-1} \exp(\mathbf{x}_i^T \boldsymbol{\beta}_k)}, \quad j = 1, \dots, J-1$$

且 $P(Y_i = J | \mathbf{x}_i) = 1 - \sum_{j=1}^{J-1} P(Y_i = j | \mathbf{x}_i)$ 。

模型特点：1. 每种类别（除参照外）有自己的一组参数 $\boldsymbol{\beta}_j$ 2. 需要进行 $J-1$ 个二分类逻辑回归的联合估计 3. 参数解释： $\exp(\beta_{jk})$ 表示 X_k 增加 1 单位，类别 j 相对于参照类别的相对风险比

实例：产品选择模型

研究价格、品牌和消费者收入对产品选择的影响（三种品牌：A、B、C，以 C 为参照）：

变量	品牌 A vs C	品牌 B vs C
	系数	OR
截距	1.5	-
价格	-0.2	0.82
高收入	0.8	2.23

解释：1. 价格每增加 1 单位，选择品牌 A（vs C）的优势比乘以 0.82（下降 18%）2. 高收入者选择品牌 A（vs C）的优势是低收入者的 2.23 倍

有序逻辑回归

当响应类别有自然顺序时（如满意度：低、中、高），使用有序逻辑回归。最常用的是比例优势模型：

$$\log \left[\frac{P(Y_i \leq j | \mathbf{x}_i)}{P(Y_i > j | \mathbf{x}_i)} \right] = \alpha_j - \mathbf{x}_i^T \boldsymbol{\beta}, \quad j = 1, \dots, J-1$$

其中 α_j 是切点参数（满足 $\alpha_1 < \alpha_2 < \dots < \alpha_{J-1}$ ）， $\boldsymbol{\beta}$ 是斜率参数（注意符号为负）。

关键假设：比例优势假设，即所有类别共享相同的斜率参数 $\boldsymbol{\beta}$ 。这意味着自变量对每个累积优势比的影响相同，只是截距不同。

概率计算：

$$P(Y_i \leq j | \mathbf{x}_i) = \frac{1}{1 + \exp[-(\alpha_j - \mathbf{x}_i^T \boldsymbol{\beta})]}$$

$$P(Y_i = j | \mathbf{x}_i) = P(Y_i \leq j | \mathbf{x}_i) - P(Y_i \leq j-1 | \mathbf{x}_i)$$

参数解释：- α_j ：切点参数，没有直接的实际意义 - β_k ： X_k 增加 1 单位， Y 属于更高类别的优势乘以 $\exp(\beta_k)$ （注意模型中的负号）

实例：顾客满意度研究

研究服务质量、价格合理度对满意度的影响（满意度：1= 不满意，2= 一般，3= 满意）：

变量	系数 β	标准误	优势比 $\exp(\beta)$
服务质量	0.8	0.2	2.23
价格合理	0.5	0.2	1.65
切点 1(α_1)	-1.5	0.3	-
切点 2(α_2)	1.0	0.3	-

解释：服务质量每提高 1 单位，满意度达到更高等级（满意 vs 一般及以下，或一般及以上 vs 不满意）的优势是原来的 2.23 倍。

7.3.3 Probit 模型：另一种选择

Probit 模型设定

Probit 模型使用标准正态分布的逆函数作为连接函数：

$$\Phi^{-1}(p_i) = \mathbf{x}_i^T \boldsymbol{\beta}$$

等价地：

$$p_i = \Phi(\mathbf{x}_i^T \boldsymbol{\beta}) = \int_{-\infty}^{\mathbf{x}_i^T \boldsymbol{\beta}} \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz$$

其中 $\Phi(\cdot)$ 是标准正态累积分布函数。

Logit 与 Probit 比较

1. 函数形状：Logit 和 Probit 都是 S 型曲线，但 Probit 在尾部更接近正态分布

- Logit: $f(x) = e^x / (1 + e^x)$
- Probit: $f(x) = \Phi(x)$

2. 参数标度：Probit 系数的尺度大约是 Logit 系数的 1.6 倍

$$\beta_{\text{probit}} \approx 0.625 \beta_{\text{logit}}$$

3. 理论依据：

- Logit: 基于优势比概念，有明确的解释
- Probit: 基于潜在变量模型，假设存在连续潜变量 Y_i^* ，当 $Y_i^* > 0$ 时 $Y_i = 1$

4. 实际应用：

- 经济学、金融学常用 Probit
- 生物统计学、流行病学常用 Logit
- 实践中两种模型通常给出相似的结果

边际效应

在二分类模型中，自变量 X_j 对概率 p_i 的边际效应（在其他变量取均值时）为：

对于 Logit 模型：

$$\frac{\partial p_i}{\partial X_{ij}} = p_i(1 - p_i)\beta_j$$

对于 Probit 模型：

$$\frac{\partial p_i}{\partial X_{ij}} = \phi(\mathbf{x}_i^T \boldsymbol{\beta})\beta_j$$

其中 $\phi(\cdot)$ 是标准正态密度函数。

边际效应不是常数，依赖于 $\mathbf{x}_i$ 的取值。通常报告：1. 平均边际效应：在所有观测处计算边际效应后取平均 2. 边际效应在均值处：在 $\bar{\mathbf{x}}$ 处计算的边际效应

实例：比较 Logit 和 Probit 模型

使用相同数据拟合两种模型，结果如下：

变量	Logit 系数	Probit 系数	Logit 边际效应	Probit 边际效应
截距	-1.5	-0.9	-	-
X_1	0.8	0.5	0.20	0.19
X_2	0.3	0.18	0.075	0.068

虽然系数不同，但边际效应非常接近，预测概率也相似（平均差异 <0.01）。

模型选择建议

在实际应用中：1. 如果关心优势比解释，选择 Logit 2. 如果基于潜在变量理论，选择 Probit 3. 如果关注边际效应，两种模型都可以 4. 可以用 AIC/BIC 或交叉验证进行比较 5. 通常选择更简单或更符合领域惯例的模型

7.4 计数数据的建模：泊松回归

7.4.1 计数数据特性与泊松分布

计数数据是非负整数值数据，常见于以下领域：- 公共卫生：疾病发病人数、就诊次数 - 保险：索赔次数、事故次数 - 社会学：家庭子女数、社交活动参与次数 - 生态学：物种个体数、观测到的动物数量

泊松分布

泊松分布是描述单位时间或空间内随机事件发生次数的概率分布。如果 $Y \sim \text{Poisson}(\lambda)$ ，则：

$$P(Y = y) = \frac{e^{-\lambda} \lambda^y}{y!}, \quad y = 0, 1, 2, \dots$$

性质：1. $\mathbb{E}(Y) = \text{Var}(Y) = \lambda$ （等离散性）2. 事件发生率 $\lambda > 0$ 3. 事件在不相交区间上独立发生

泊松回归模型

泊松回归假设 $Y_i | \mathbf{x}_i \sim \text{Poisson}(\lambda_i)$ ，且：

$$\log(\lambda_i) = \mathbf{x}_i^T \boldsymbol{\beta}$$

等价地：

$$\lambda_i = \exp(\mathbf{x}_i^T \boldsymbol{\beta})$$

从 GLM 视角：响应分布为泊松，连接函数为对数连接（典则连接）。

似然函数：

$$L(\boldsymbol{\beta}) = \prod_{i=1}^n \frac{e^{-\lambda_i} \lambda_i^{y_i}}{y_i!}$$

对数似然：

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log \lambda_i - \lambda_i - \log(y_i!)]$$

参数解释

由于 $\lambda_i = \exp(\mathbf{x}_i^T \boldsymbol{\beta})$ ，参数解释基于发生率比：

对于连续自变量 X_j ：

$$\frac{\lambda_i(X_j + 1)}{\lambda_i(X_j)} = \exp(\beta_j)$$

$\exp(\beta_j)$ 称为发生率比，表示 X_j 增加 1 单位， λ_i 的期望倍数变化。

对于虚拟变量， $\exp(\beta_j)$ 表示相应组别相对于参照组的发生率比。

实例：保险索赔次数分析

研究驾驶年龄和车辆类型对保险索赔次数的影响：

变量	系数 β	标准误	发生率比 $\exp(\beta)$	95% CI
截距	-1.2	0.2	0.30	(0.20, 0.45)
驾龄	-0.05	0.01	0.95	(0.93, 0.97)
SUV	0.4	0.15	1.49	(1.11, 2.00)
跑车	0.8	0.2	2.23	(1.51, 3.29)

解释（以轿车为参照）：1. 驾龄每增加 1 年，索赔发生率下降 5% 2. SUV 驾驶者的索赔发生率是轿车驾驶者的 1.49 倍 3. 跑车驾驶者的索赔发生率是轿车驾驶者的 2.23 倍

7.4.2 过度离散与负二项回归

过度离散问题

泊松分布的等离散假设 ($E(Y) = \text{Var}(Y)$) 在实际数据中常不成立。过度离散指观测方差大于理论方差 ($\text{Var}(Y) > E(Y)$)，原因可能包括：1. 未被观测的异质性 2. 事件发生的相关性 3. 零值过多

过度离散的诊断：1. **Pearson** 统计量： $\chi_P^2 = \sum (y_i - \hat{\lambda}_i)^2 / \hat{\lambda}_i$ ，近似 χ_{n-p-1}^2 2. 离散参数： $\hat{\phi} = \chi_P^2 / (n - p - 1)$ ，若 $\hat{\phi} \gg 1$ 表明过度离散

负二项回归

负二项分布可以处理过度离散问题。负二项回归假设 $Y_i | \mathbf{x}_i \sim \text{NB}(\mu_i, \alpha)$ ，其中：

$$\mathbb{E}(Y_i) = \mu_i, \quad \text{Var}(Y_i) = \mu_i + \alpha \mu_i^2$$

连接函数仍为对数连接：

$$\log(\mu_i) = \mathbf{x}_i^T \boldsymbol{\beta}$$

$\alpha > 0$ 是离散参数：- $\alpha = 0$ ：退化为泊松分布 - $\alpha > 0$ ：存在过度离散， α 越大，离散程度越高

负二项分布有多种参数化形式，常用的是 NB2 形式（方差为 $\mu + \alpha \mu^2$ ）。

实例：比较泊松与负二项回归

对同一数据拟合两种模型：

变量	泊松系数 (SE)	负二项系数 (SE)
截距	-1.2 (0.2)	-1.1 (0.3)
X_1	0.3 (0.05)	0.28 (0.08)
X_2	0.5 (0.1)	0.45 (0.15)
α	-	0.8 (0.2)
AIC	850	720

负二项模型的 AIC 更小，且离散参数 $\alpha = 0.8$ 显著大于 0，表明存在过度离散，负二项回归更合适。

零膨胀模型

当数据中零值比例异常高时，可能需要零膨胀模型。零膨胀泊松模型假设：

$$P(Y_i = 0 | \mathbf{x}_i) = \pi_i + (1 - \pi_i)e^{-\lambda_i}$$

$$P(Y_i = y | \mathbf{x}_i) = (1 - \pi_i) \frac{e^{-\lambda_i} \lambda_i^y}{y!}, \quad y > 0$$

其中 π_i 是额外零的概率，通常也通过连接函数建模：

$$\text{logit}(\pi_i) = \mathbf{z}_i^T \boldsymbol{\gamma}$$

$$\log(\lambda_i) = \mathbf{x}_i^T \boldsymbol{\beta}$$

零膨胀模型可以看作是两部分混合：一部分总是产生 0（结构性零），另一部分产生泊松计数（可能为 0）。

7.4.3 计数回归的应用扩展

偏移项的使用

在计数回归中，经常需要控制暴露量（如观测时间、地区人口数）。设 t_i 为第 i 个观测的暴露量，模型为：

$$\log(\lambda_i) = \mathbf{x}_i^T \boldsymbol{\beta} + \log(t_i)$$

等价地：

$$\lambda_i = t_i \exp(\mathbf{x}_i^T \boldsymbol{\beta})$$

这里 $\log(t_i)$ 称为偏移项，其系数固定为 1。

实例：疾病发病率研究

研究不同地区空气污染对呼吸疾病发病率的影响， Y_i 为病例数， t_i 为地区人口数（单位：万人）：

变量	系数 β	发生率比	解释
截距	-5.0	0.0067	基准发病率
污染	0.2	1.22	污染增加 1 单位，发病率增加 22%

计数数据的其他模型

1. 截断泊松回归：当数据有最小值限制时（如 $Y \geq 1$ ）
2. 审查泊松回归：当计数有上限时（如 $Y \leq C$ ）
3. 泊松-逆高斯回归：处理极端过度离散
4. 广义泊松回归：更灵活的离散参数形式

实践建议

应用计数回归时：1. 首先检查数据的描述统计，特别是零值比例和方差-均值比 2. 从泊松回归开始，检查是否存在过度离散 3. 如果存在过度离散，考虑负二项回归或其他离散模型 4. 如果零值过多，考虑零膨胀模型 5. 使用 AIC/BIC 进行模型比较 7. 进行残差诊断和影响点分析

计数回归是 GLM 框架的重要应用，在公共卫生、保险精算、社会科学等领域有广泛用途。理解泊松回归及其扩展模型，能够帮助我们更好地分析和解释计数数据。

7.5 广义线性模型进阶

7.5.1 其他常见 GLM 简介

Gamma 回归

Gamma 分布适用于正连续数据，特别是当数据呈现右偏分布且方差与均值的平方成正比时。Gamma 分布的概率密度函数为：

$$f(y; \mu, \phi) = \frac{1}{\Gamma(\phi)} \left(\frac{\phi y}{\mu} \right)^{\phi} \exp \left(-\frac{\phi y}{\mu} \right) \frac{1}{y}, \quad y > 0$$

其中 $\mu > 0$ 为均值， $\phi > 0$ 为形状参数。

Gamma 分布的性质：- $E(Y) = \mu$ - $\text{Var}(Y) = \frac{\mu^2}{\phi}$ ，故变异系数 $CV = \frac{\sqrt{\text{Var}(Y)}}{\mu} = \frac{1}{\sqrt{\phi}}$ 为常数 - 形状参数 ϕ 控制分布形状： $\phi = 1$ 时为指数分布， ϕ 越大越接近正态

Gamma 回归通常使用对数连接：

$$\log(\mu_i) = \mathbf{x}_i^T \boldsymbol{\beta}$$

或倒数连接（典则连接）：

$$\frac{1}{\mu_i} = \mathbf{x}_i^T \boldsymbol{\beta}$$

应用场景：- 保险理赔额（正偏、异方差）- 生存时间（右偏分布）- 收入数据（通常右偏）

实例：保险理赔额分析

研究驾驶者年龄和车型对平均理赔额的影响：

变量	系数 β	$\exp(\beta)$	解释
截距	7.0	1097.6	基准理赔额
年龄	-0.02	0.98	年龄每增加 1 岁，理赔额下降 2%
SUV	0.3	1.35	SUV 理赔额是轿车的 1.35 倍

逆高斯回归

逆高斯分布适用于正连续、高度右偏的数据，其方差与均值的立方成正比：

$$\text{Var}(Y) = \frac{\mu^3}{\lambda}$$

其中 $\lambda > 0$ 是离散参数。

逆高斯回归通常使用对数连接或平方根倒数连接。

互补双对数模型

对于二分类数据，除了 Logit 和 Probit 连接外，还可使用互补双对数连接：

$$\log[-\log(1 - p_i)] = \mathbf{x}_i^T \boldsymbol{\beta}$$

等价地：

$$p_i = 1 - \exp[-\exp(\mathbf{x}_i^T \boldsymbol{\beta})]$$

该模型假设事件概率分布不对称，适用于罕见事件或常见事件。

对数线性模型

对数线性模型主要用于列联表分析，是泊松回归的特例。对于 $I \times J$ 列联表，模型为：

$$\log(\mu_{ij}) = \lambda + \lambda_i^A + \lambda_j^B + \lambda_{ij}^{AB}$$

其中 μ_{ij} 为期望频数， λ_{ij}^{AB} 为交互项。

对数线性模型可以检验变量间的独立性、条件独立性等。

7.5.2 模型诊断、验证与选择

残差分析

GLM 的残差分析比线性回归更复杂，因为残差分布通常不是正态的。

1. Pearson 残差：

$$r_i^P = \frac{y_i - \hat{\mu}_i}{\sqrt{V(\hat{\mu}_i)}}$$

近似正态（大样本下）。

2. 偏差残差：

$$r_i^D = \text{sign}(y_i - \hat{\mu}_i) \sqrt{d_i}$$

其中 d_i 是第 i 个观测对总偏差的贡献。

3. **Anscombe** 残差：通过方差稳定变换获得，更接近正态分布。

残差图分析：- 残差 vs 拟合值：检查方差假设、非线性 - 残差 vs 自变量：检查函数形式 - 正态 Q-Q 图：检查残差分布 - 残差 vs 序号：检查独立性

影响诊断

类似于线性回归，可以计算 GLM 的杠杆值和 Cook 距离：

1. 杠杆值：帽子矩阵 $\mathbf{H} = \mathbf{W}^{1/2} \mathbf{X} (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W}^{1/2}$ 的对角元素 h_{ii}
 - 高杠杆点： $h_{ii} > 2(p+1)/n$
2. **Cook** 距离：衡量删除第 i 个观测对参数估计的影响

$$D_i = \frac{(\hat{\boldsymbol{\beta}} - \hat{\boldsymbol{\beta}}_{(i)})^T (\mathbf{X}^T \mathbf{W} \mathbf{X}) (\hat{\boldsymbol{\beta}} - \hat{\boldsymbol{\beta}}_{(i)})}{(p+1)\hat{\phi}}$$

其中 $\hat{\boldsymbol{\beta}}_{(i)}$ 是删除第 i 个观测后的估计。

连接函数检验

连接函数的选择可以通过以下方法检验：

1. 添加连接函数检验项：如添加 $\hat{\eta}_i^2$ 到模型中，检验其显著性
2. 比较不同连接函数的模型：使用 AIC/BIC
3. 残差分析：检查系统性模式

过度离散与欠离散诊断

对于泊松和二项分布等离散分布，需要检查方差-均值关系。

Pearson 离散统计量：

$$\chi_P^2 = \sum_{i=1}^n \frac{(y_i - \hat{\mu}_i)^2}{V(\hat{\mu}_i)}$$

离散参数估计：

$$\hat{\phi} = \frac{\chi_P^2}{n - p - 1}$$

- $\hat{\phi} \approx 1$ ：方差假设合适
- $\hat{\phi} > 1$ ：过度离散
- $\hat{\phi} < 1$ ：欠离散

模型选择准则

1. 信息准则：
 - AIC: $AIC = -2\ell + 2(p + 1)$
 - BIC: $BIC = -2\ell + (p + 1) \log n$
 - QAIC: 用于过度离散情况的调整 AIC
2. 交叉验证：
 - 留一法交叉验证
 - k 折交叉验证
 - 预测误差作为评估标准
3. 正则化方法：
 - Lasso GLM: 在似然函数中加入 L_1 惩罚项
 - Ridge GLM: 加入 L_2 惩罚项
 - Elastic Net: 结合 L_1 和 L_2 惩罚

7.5.3 高级主题概览

拟似然估计

当响应分布不完全是指数分布族，或方差函数 $V(\mu)$ 已知但分布形式未知时，可使用拟似然方法。

拟似然函数定义为满足以下关系的函数 $Q(\mu; y)$:

$$\frac{\partial Q}{\partial \mu} = \frac{y - \mu}{a(\phi)V(\mu)}$$

即使不知道完整的分布，只要指定均值函数 $\mu(\beta)$ 和方差函数 $V(\mu)$ ，就可以通过拟似然方程估计参数。

拟似然的优势：1. 对分布假设更稳健 2. 可以处理过度离散 3. 估计和推断方法类似

混合效应 GLM

混合效应 GLM 在线性预测器中加入随机效应，适用于层次结构数据、重复测量数据等。

模型形式：

$$g(\mu_{ij}) = \mathbf{x}_{ij}^T \boldsymbol{\beta} + \mathbf{z}_{ij}^T \mathbf{u}_i$$

其中：- $\mathbf{u}_i \sim N(\mathbf{0}, \mathbf{G})$: 随机效应 - $\boldsymbol{\beta}$: 固定效应 - $i = 1, \dots, m$: 组索引（如个体、学校） - $j = 1, \dots, n_i$: 组内观测索引

估计方法：1. 拉普拉斯近似 2. 高斯-埃尔米特积分 3. MCMC 方法（贝叶斯框架）

广义可加模型

广义可加模型在线性预测器中包含非参数平滑项：

$$g(\mu_i) = \beta_0 + \sum_{j=1}^p f_j(X_{ij})$$

其中 $f_j(\cdot)$ 是平滑函数，通常用样条基函数表示。

GAM 的优势：1. 自动检测非线性关系 2. 避免预先指定函数形式 3. 可视化结果直观

贝叶斯 GLM

贝叶斯方法为 GLM 提供完整的概率框架：

1. 指定先验分布： $\pi(\boldsymbol{\beta}), \pi(\phi)$
2. 计算后验分布： $\pi(\boldsymbol{\beta}, \phi | \mathbf{y}) \propto L(\boldsymbol{\beta}, \phi) \pi(\boldsymbol{\beta}) \pi(\phi)$
3. 后验推断：后验均值、中位数、可信区间

贝叶斯 GLM 的优势：1. 提供参数的不确定性完整描述 2. 可以纳入先验信息 3. 自然处理层次模型

计算通常通过 MCMC 方法（如 Gibbs 抽样、Metropolis-Hastings 算法）实现。

生存分析的 GLM 视角

许多生存分析模型可以纳入 GLM 框架：

1. 比例风险模型：Cox 模型可以看作部分 GLM
2. 参数生存模型：指数、Weibull、对数正态等分布属于指数族
3. 加速失效时间模型：连接函数为对数连接

纵向数据的 GLM

对于纵向数据（重复测量），GLM 扩展包括：1. 边际模型：使用 GEE（广义估计方程）估计 2. 随机效应模型：混合效应 GLM 3. 过渡模型：包含响应变量滞后项

7.6 案例分析

本章将通过两个完整的数据分析案例，展示如何在实际研究中应用广义线性模型。第一个案例使用 R 语言分析二分类数据（逻辑斯谛回归），第二个案例使用 Python 分析计数数据（泊松回归）。每个案例都包含数据探索、模型建立、诊断检验和结果解释的完整流程。

7.7.1 R 案例：信用卡违约预测分析（逻辑斯谛回归）

案例背景与数据

本案例使用 ISLR 包中的 Default 数据集，分析信用卡持有者的违约情况。数据集包含 10,000 个观测，变量包括：- default：是否违约（二分类，Yes/No）- student：是否学生（二分类，Yes/No）- balance：信用卡余额（连续）- income：年收入（连续）

```
# 加载必要的包
library(ISLR)      # 包含 Default 数据集
library(tidyverse) # 数据整理和可视化
library(broom)     # 模型结果整理
library(car)       # 模型诊断
library(pROC)      # ROC 曲线
library(knitr)     # 表格输出

# 加载数据
data(Default)
head(Default)
```

	default	student	balance	income
1	No	No	729.5265	44361.625
2	No	Yes	817.1804	12106.135
3	No	No	1073.5492	31767.139
4	No	No	529.2506	35704.494
5	No	No	785.6559	38463.496
6	No	Yes	919.5885	7491.559

数据探索与可视化

```
# 数据概况
str(Default)
```

```
'data.frame':  10000 obs. of  4 variables:
 $ default: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
 $ student: Factor w/ 2 levels "No","Yes": 1 2 1 1 1 2 1 2 1 1 ...
 $ balance: num  730 817 1074 529 786 ...
 $ income : num  44362 12106 31767 35704 38463 ...
```

```
summary(Default)
```

default	student	balance	income
No :9667	No :7056	Min. : 0.0	Min. : 772
Yes: 333	Yes:2944	1st Qu.: 481.7	1st Qu.:21340
		Median : 823.6	Median :34553

```

Mean      : 835.4      Mean      :33517
3rd Qu.:1166.3      3rd Qu.:43808
Max.      :2654.3      Max.      :73554

```

```

# 违约率的描述统计
default_summary <- Default %>%
  group_by(default) %>%
  summarise(
    count = n(),
    proportion = n() / nrow(Default),
    avg_balance = mean(balance),
    avg_income = mean(income)
  )
kable(default_summary, caption = " 违约情况描述统计")

```

表 22: 违约情况描述统计

default	count	proportion	avg_balance	avg_income
No	9667	0.9667	803.9438	33566.17
Yes	333	0.0333	1747.8217	32089.15

```

# 学生与非学生违约率比较
student_summary <- Default %>%
  group_by(student, default) %>%
  summarise(count = n(), .groups = 'drop') %>%
  group_by(student) %>%
  mutate(
    total = sum(count),
    proportion = count / total
  )
kable(student_summary, caption = " 学生身份与违约情况")

```

表 23: 学生身份与违约情况

student	default	count	total	proportion
No	No	6850	7056	0.9708050
No	Yes	206	7056	0.0291950
Yes	No	2817	2944	0.9568614
Yes	Yes	127	2944	0.0431386

student	default	count	total	proportion
---------	---------	-------	-------	------------

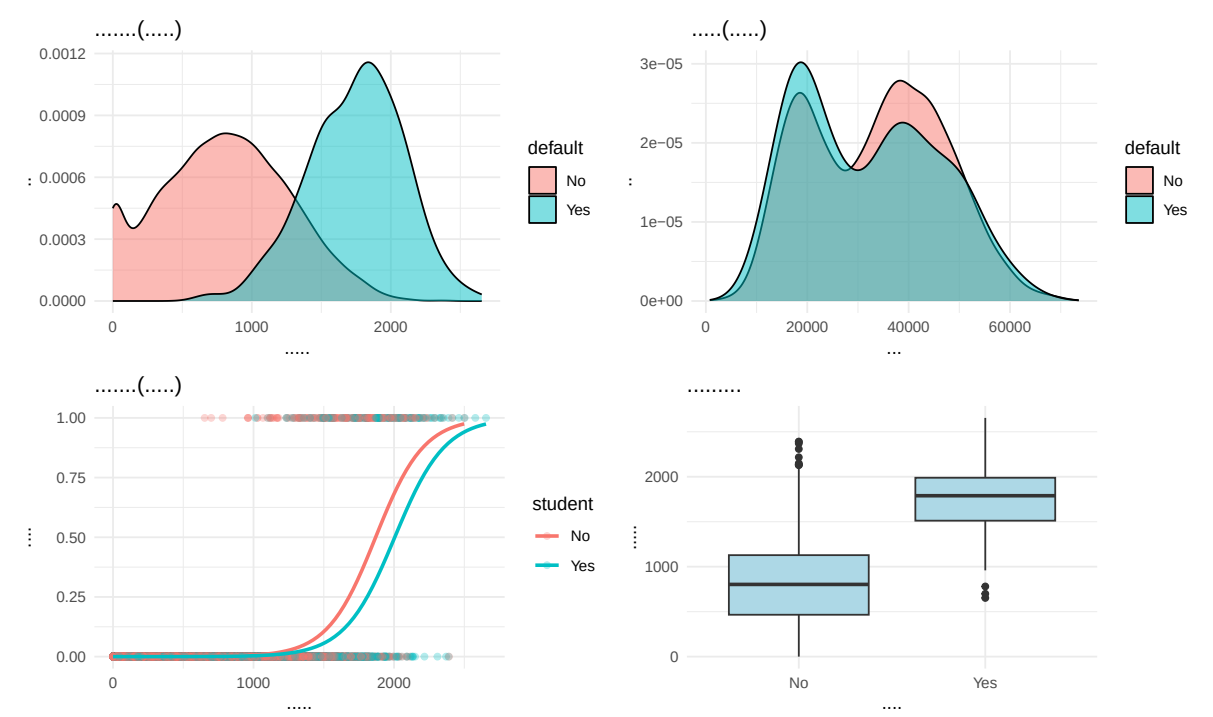
```
# 数据可视化
# 1. 违约与非违约的余额分布
p1 <- ggplot(Default, aes(x = balance, fill = default)) +
  geom_density(alpha = 0.5) +
  labs(title = " 信用卡余额分布（按违约情况）",
        x = " 信用卡余额", y = " 密度") +
  theme_minimal()

# 2. 违约与非违约的收入分布
p2 <- ggplot(Default, aes(x = income, fill = default)) +
  geom_density(alpha = 0.5) +
  labs(title = " 年收入分布（按违约情况）",
        x = " 年收入", y = " 密度") +
  theme_minimal()

# 3. 学生身份、余额与违约的关系
p3 <- ggplot(Default, aes(x = balance, y = as.numeric(default == "Yes"),
                          color = student)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"),
              se = FALSE) +
  labs(title = " 余额与违约概率（按学生身份）",
        x = " 信用卡余额", y = " 违约概率") +
  theme_minimal()

# 4. 箱线图：余额和收入按违约情况
p4 <- ggplot(Default, aes(x = default, y = balance)) +
  geom_boxplot(fill = "lightblue") +
  labs(title = " 余额按违约情况分布",
        x = " 是否违约", y = " 信用卡余额") +
  theme_minimal()

# 显示所有图形
library(patchwork)
(p1 + p2) / (p3 + p4)
```

模型建立与估计

```
# 模型 1: 仅包含余额
modell <- glm(default ~ balance, data = Default, family = binomial)
summary_modell <- tidy(modell, conf.int = TRUE, exponentiate = TRUE)
kable(summary_modell, caption = " 模型 1: 仅包含余额的逻辑回归结果")
```

表 24: 模型 1: 仅包含余额的逻辑回归结果

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.0000237	0.3611574	-29.49221	0	0.0000114	0.0000469
balance	1.0055141	0.0002204	24.95309	0	1.0050918	1.0059611

```
# 模型 2: 包含余额和学生身份
model2 <- glm(default ~ balance + student, data = Default, family = binomial)
summary_model2 <- tidy(model2, conf.int = TRUE, exponentiate = TRUE)
kable(summary_model2, caption = " 模型 2: 包含余额和学生身份的逻辑回归结果")
```

表 25: 模型 2: 包含余额和学生身份的逻辑回归结果

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.0000215	0.3691914	-29.116326	0.0e+00	0.0000101	0.0000432
balance	1.0057546	0.0002318	24.749526	0.0e+00	1.0053106	1.0062254
studentYes	0.4892520	0.1475190	-4.846003	1.3e-06	0.3650292	0.6511107

```
# 模型 3: 包含余额、学生身份和交互项
model3 <- glm(default ~ balance * student, data = Default, family = binomial)
summary_model3 <- tidy(model3, conf.int = TRUE, exponentiate = TRUE)
kable(summary_model3, caption = " 模型 3: 包含交互项的逻辑回归结果")
```

表 26: 模型 3: 包含交互项的逻辑回归结果

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.0000189	0.4639679	-23.4384348	0.0000000	0.0000073	0.0000453
balance	1.0058358	0.0002937	19.8122090	0.0000000	1.0052772	1.0064366
studentYes	0.7038212	0.8037333	-0.4369994	0.6621118	0.1398796	3.3001112
balance:studentYes	0.9997804	0.0004781	-0.4594039	0.6459441	0.9988561	1.0007348

```
# 模型 4: 包含余额、学生身份和收入
model4 <- glm(default ~ balance + student + income, data = Default, family = binomial)
summary_model4 <- tidy(model4, conf.int = TRUE, exponentiate = TRUE)
kable(summary_model4, caption = " 模型 4: 包含所有变量的逻辑回归结果")
```

表 27: 模型 4: 包含所有变量的逻辑回归结果

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.0000190	0.4922555	-22.080088	0.0000000	0.0000071	0.0000488
balance	1.0057530	0.0002319	24.737563	0.0000000	1.0053089	1.0062239
studentYes	0.5237317	0.2362525	-2.737646	0.0061881	0.3298827	0.8334224
income	1.0000030	0.0000082	0.369815	0.7115203	0.9999870	1.0000191

模型比较与选择

```
# 模型比较
model_comparison <- data.frame(
```

```

Model = c(" 仅余额", " 余额 + 学生", " 余额 × 学生交互", " 全变量"),
AIC = c(AIC(model1), AIC(model2), AIC(model3), AIC(model4)),
BIC = c(BIC(model1), BIC(model2), BIC(model3), BIC(model4)),
Deviance = c(deviance(model1), deviance(model2),
              deviance(model3), deviance(model4)),
Null_Deviance = c(model1$null.deviance, model2$null.deviance,
                  model3$null.deviance, model4$null.deviance),
R2_McFadden = c(1 - deviance(model1)/model1$null.deviance,
                1 - deviance(model2)/model2$null.deviance,
                1 - deviance(model3)/model3$null.deviance,
                1 - deviance(model4)/model4$null.deviance)
)
kable(model_comparison, digits = 3,
      caption = " 模型比较 (AIC、BIC、偏差和伪 R2) ")

```

表 28: 模型比较 (AIC、BIC、偏差和伪 R²)

Model	AIC	BIC	Deviance	Null_Deviance	R2_McFadden
仅余额	1600.452	1614.872	1596.452	2920.65	0.453
余额 + 学生	1577.682	1599.313	1571.682	2920.65	0.462
余额 × 学生交互	1579.472	1608.314	1571.472	2920.65	0.462
全变量	1579.545	1608.386	1571.545	2920.65	0.462

```

# 似然比检验
cat("\n似然比检验: 模型 2 vs 模型 1 (加入学生身份是否显著改善拟合) \n")

```

似然比检验: 模型2 vs 模型1 (加入学生身份是否显著改善拟合)

```

lr_test_21 <- anova(model1, model2, test = "Chisq")
print(lr_test_21)

```

Analysis of Deviance Table

Model 1: default ~ balance

Model 2: default ~ balance + student

	Resid. Df	Resid. Dev	Df	Deviance	Pr(>Chi)
1	9998	1596.5			
2	9997	1571.7	1	24.77	6.459e-07 ***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat("\n似然比检验：模型 3 vs 模型 2（加入交互项是否显著改善拟合）\n")
```

似然比检验：模型3 vs 模型2（加入交互项是否显著改善拟合）

```
lr_test_32 <- anova(model2, model3, test = "Chisq")
print(lr_test_32)
```

Analysis of Deviance Table

Model 1: default ~ balance + student

Model 2: default ~ balance * student

	Resid. Df	Resid. Dev	Df	Deviance	Pr(>Chi)
1	9997	1571.7			
2	9996	1571.5	1	0.20944	0.6472

```
cat("\n似然比检验：模型 4 vs 模型 2（加入收入是否显著改善拟合）\n")
```

似然比检验：模型4 vs 模型2（加入收入是否显著改善拟合）

```
lr_test_42 <- anova(model2, model4, test = "Chisq")
print(lr_test_42)
```

Analysis of Deviance Table

Model 1: default ~ balance + student

Model 2: default ~ balance + student + income

	Resid. Df	Resid. Dev	Df	Deviance	Pr(>Chi)
1	9997	1571.7			
2	9996	1571.5	1	0.13677	0.7115

模型诊断

```
# 选择最佳模型（基于 AIC）
```

```
best_model <- model2
```

```
# 1. 残差分析
```

```
par(mfrow = c(2, 3))
```

```
# Pearson 残差 vs 拟合值
fitted_values <- predict(best_model, type = "response")
pearson_resid <- residuals(best_model, type = "pearson")
plot(fitted_values, pearson_resid,
     xlab = " 拟合概率", ylab = "Pearson 残差",
     main = "Pearson 残差 vs 拟合概率")
abline(h = 0, col = "red", lty = 2)

# 偏差残差 vs 拟合值
deviance_resid <- residuals(best_model, type = "deviance")
plot(fitted_values, deviance_resid,
     xlab = " 拟合概率", ylab = " 偏差残差",
     main = " 偏差残差 vs 拟合概率")
abline(h = 0, col = "red", lty = 2)

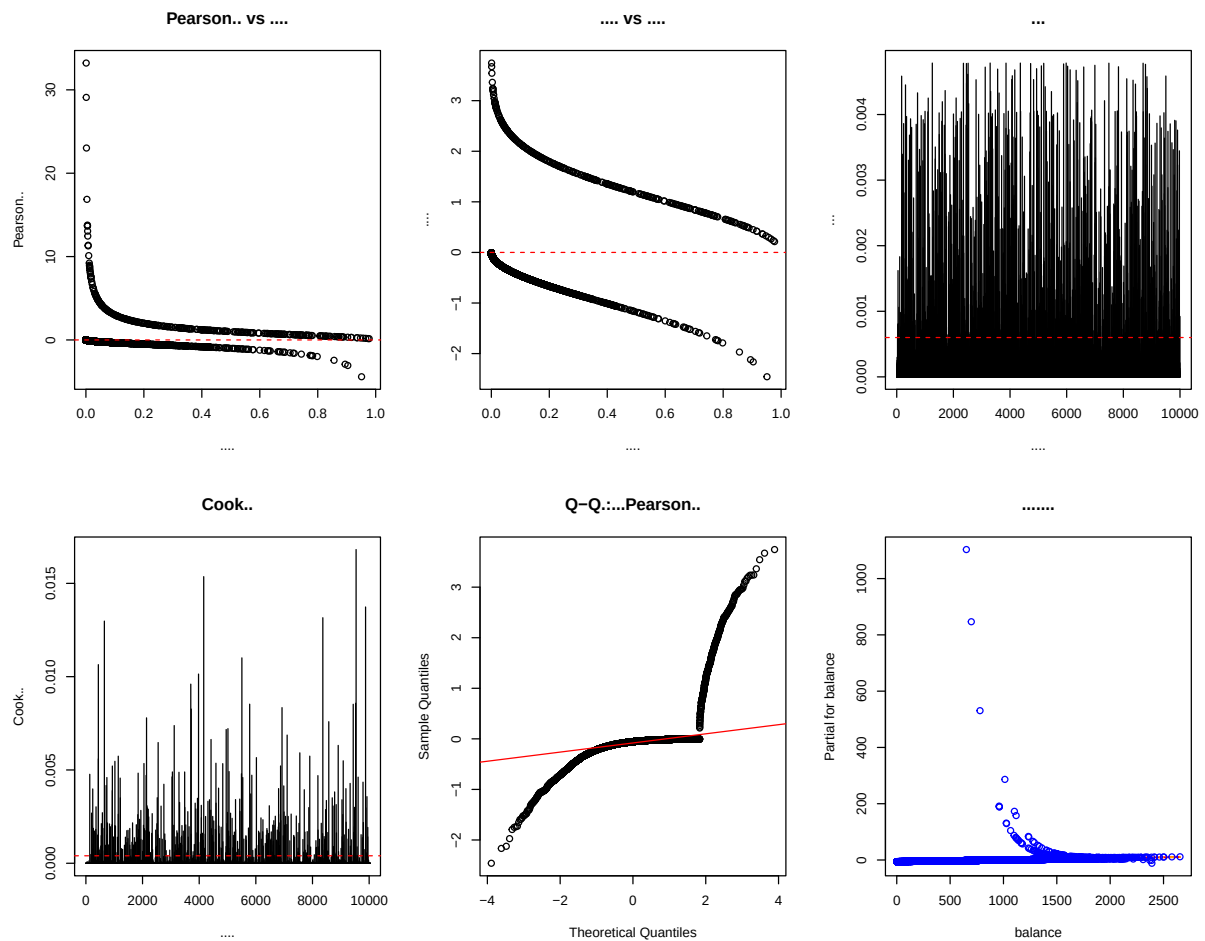
# 杠杆值分析
hat_values <- hatvalues(best_model)
plot(hat_values, type = "h",
     xlab = " 观测序号", ylab = " 杠杆值",
     main = " 杠杆值")
abline(h = 2*mean(hat_values), col = "red", lty = 2) # 2 倍平均杠杆值

# Cook 距离
cook_dist <- cooks.distance(best_model)
plot(cook_dist, type = "h",
     xlab = " 观测序号", ylab = "Cook 距离",
     main = "Cook 距离")
abline(h = 4/(nrow(Default) - length(coef(best_model))),
     col = "red", lty = 2) # 常用阈值

# Q-Q 图: 标准化 Pearson 残差
std_pearson_resid <- rstandard(best_model)
qqnorm(std_pearson_resid, main = "Q-Q 图: 标准化 Pearson 残差")
qqline(std_pearson_resid, col = "red")

# 偏残差图 (余额)
termplot(best_model, terms = "balance", partial.resid = TRUE,
         se = TRUE, col.res = "blue", col.term = "red",
```

```
main = " 余额的偏残差图")
```



```
par(mfrow = c(1, 1))
```

```
# 2. 影响点识别
```

```
influential_points <- which(cook_dist > 4/(nrow(Default) - length(coef(best_
cat(" 识别到", length(influential_points), " 个潜在影响点\n")
```

```
识别到 415 个潜在影响点
```

```
# 3. 多重共线性检查
```

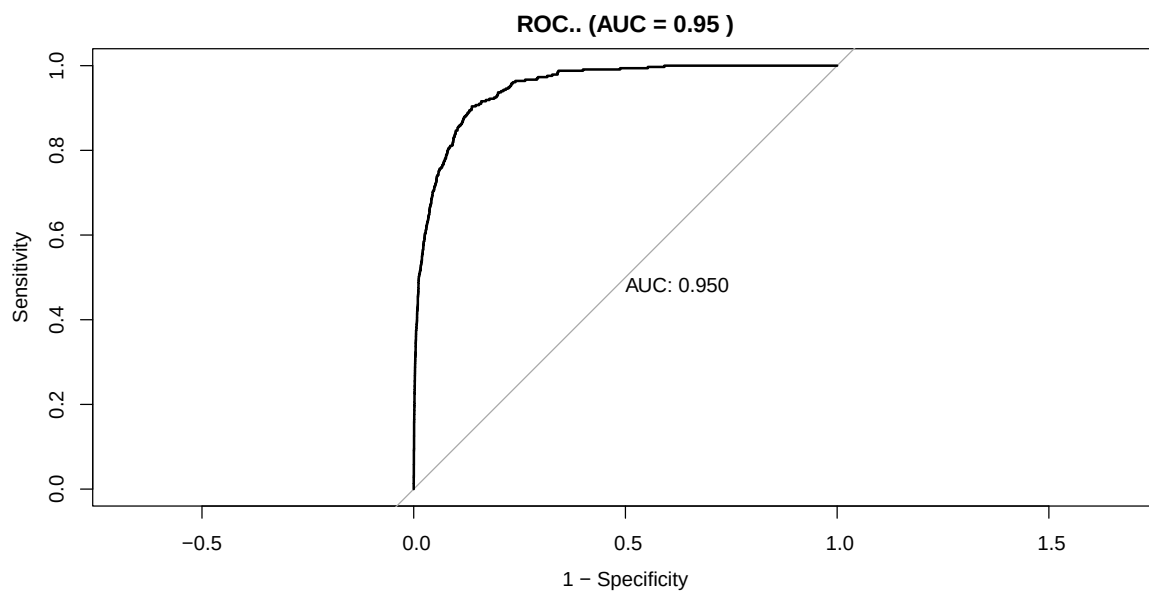
```
if (length(coef(best_model)) > 1) {
  vif_values <- vif(best_model)
  cat("\n方差膨胀因子 (VIF): \n")
  print(vif_values)
}
```

方差膨胀因子 (VIF):

```
balance student  
1.076697 1.076697
```

模型评估与预测

```
# 1. ROC 曲线和 AUC  
roc_obj <- roc(Default$default, fitted_values, levels = c("No", "Yes"))  
auc_value <- auc(roc_obj)  
  
# 绘制 ROC 曲线  
plot(roc_obj, main = paste("ROC 曲线 (AUC =", round(auc_value, 3), ")"),  
      print.auc = TRUE, legacy.axes = TRUE)
```



```
# 2. 分类性能评估  
# 选择最优阈值 (基于 Youden 指数)  
optimal_threshold <- coords(roc_obj, "best", ret = "threshold")$threshold  
  
# 创建混淆矩阵  
predicted_class <- ifelse(fitted_values > optimal_threshold, "Yes", "No")  
confusion_matrix <- table(Predicted = predicted_class, Actual = Default$default)  
  
# 计算分类指标  
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)
```

```
sensitivity <- confusion_matrix[2, 2] / sum(confusion_matrix[, 2])
specificity <- confusion_matrix[1, 1] / sum(confusion_matrix[, 1])
precision <- confusion_matrix[2, 2] / sum(confusion_matrix[2, ])

cat(" 最优阈值 (基于 Youden 指数) :", round(optimal_threshold, 3), "\n")
```

最优阈值 (基于 Youden 指数) : 0.032

```
cat("\n混淆矩阵:\n")
```

混淆矩阵:

```
print(confusion_matrix)
```

	Actual	
Predicted	No	Yes
No	8340	32
Yes	1327	301

```
cat("\n分类性能指标:\n")
```

分类性能指标:

```
cat(" 准确率:", round(accuracy, 3), "\n")
```

准确率: 0.864

```
cat(" 灵敏度 (召回率) :", round(sensitivity, 3), "\n")
```

灵敏度 (召回率) : 0.904

```
cat(" 特异度:", round(specificity, 3), "\n")
```

特异度: 0.863

```
cat(" 精确率:", round(precision, 3), "\n")
```

精确率: 0.185

```
# 3. 校准曲线
calibration_data <- data.frame(
  predicted_prob = fitted_values,
```

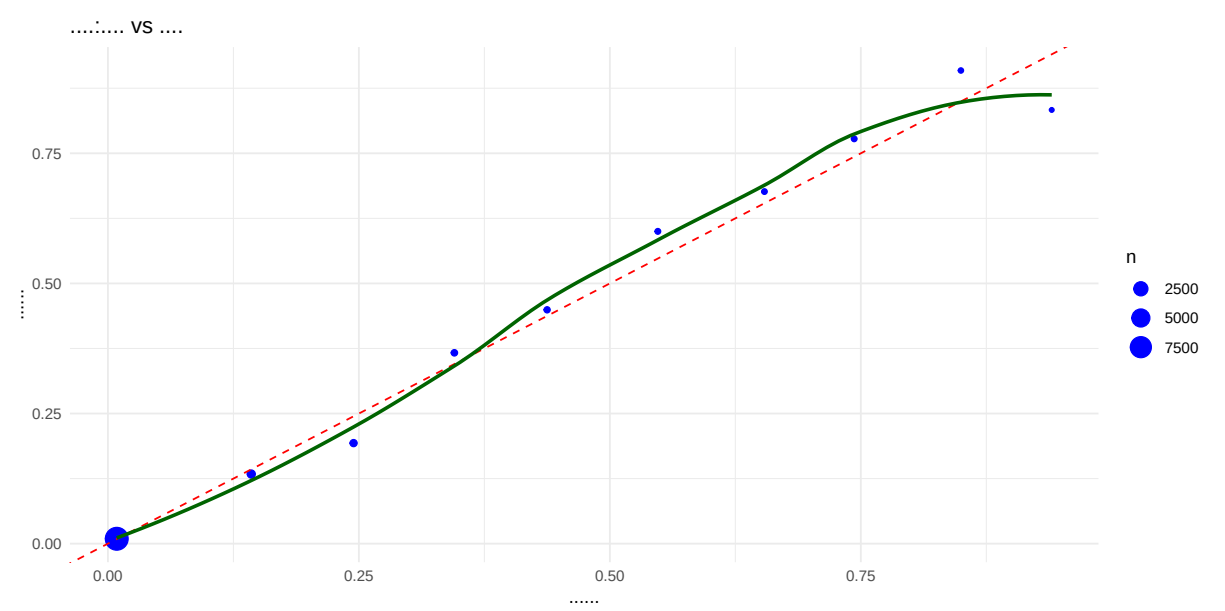


```
actual = as.numeric(Default$default == "Yes")
)

# 将预测概率分组
calibration_data$group <- cut(calibration_data$predicted_prob,
                             breaks = seq(0, 1, by = 0.1),
                             include.lowest = TRUE)

calibration_summary <- calibration_data %>%
  group_by(group) %>%
  summarise(
    avg_predicted = mean(predicted_prob),
    avg_actual = mean(actual),
    n = n(),
    .groups = 'drop'
  )

# 绘制校准曲线
ggplot(calibration_summary, aes(x = avg_predicted, y = avg_actual)) +
  geom_point(aes(size = n), color = "blue") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  geom_smooth(method = "loess", se = FALSE, color = "darkgreen") +
  labs(title = " 校准曲线: 预测概率 vs 实际比例",
       x = " 平均预测概率", y = " 实际违约比例") +
  theme_minimal()
```



结果解释与应用

```
# 1. 优势比解释
or_summary <- tidy(best_model, conf.int = TRUE, exponentiate = TRUE)
kable(or_summary, digits = 3, caption = " 最佳模型参数估计与优势比")
```

表 29: 最佳模型参数估计与优势比

term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.000	0.369	-29.116	0	0.000	0.000
balance	1.006	0.000	24.750	0	1.005	1.006
studentYes	0.489	0.148	-4.846	0	0.365	0.651

```
# 2. 边际效应计算
library(margins)

# 计算平均边际效应
marginal_effects <- margins(best_model)
summary_margins <- summary(marginal_effects)
kable(summary_margins, digits = 4, caption = " 平均边际效应")
```

表 30: 平均边际效应

factor	AME	SE	z	p	lower	upper
balance	0.0001	0.0000	25.4309	0	0.0001	1e-04
studentYes	-0.0146	0.0029	-5.1183	0	-0.0202	-9e-03

```
# 3. 预测示例
# 创建典型观测进行预测
typical_observations <- expand.grid(
  balance = c(500, 1000, 1500, 2000),
  student = c("No", "Yes")
)

# 添加预测概率
typical_observations$predicted_prob <- predict(best_model,
                                              newdata = typical_observations,
                                              type = "response")

kable(typical_observations, digits = 3,
      caption = " 典型观测的违约概率预测")
```

表 31: 典型观测的违约概率预测

balance	student	predicted_prob
500	No	0.000
1000	No	0.007
1500	No	0.105
2000	No	0.674
500	Yes	0.000
1000	Yes	0.003
1500	Yes	0.054
2000	Yes	0.503

```
# 4. 模型总结与应用建议
cat("\n=== 模型总结与应用建议 ===\n\n")
```

=== 模型总结与应用建议 ===

```
cat("1. 关键发现:\n")
```

1. 关键发现:

```
cat("    - 信用卡余额每增加 1000 单位, 违约优势增加约",  
    round((exp(coef(best_model)["balance"]*1000) - 1)*100, 1), "%\n")
```

- 信用卡余额每增加1000单位, 违约优势增加约 30947.5 %

```
cat("    - 学生比非学生的违约优势高",  
    round((exp(coef(best_model)["studentYes"]) - 1)*100, 1), "%\n\n")
```

- 学生比非学生的违约优势高 -51.1 %

```
cat("2. 模型性能:\n")
```

2. 模型性能:

```
cat("    - AUC =", round(auc_value, 3), " (模型具有良好的区分能力) \n")
```

- AUC = 0.95 (模型具有良好的区分能力)

```
cat("    - 准确率 =", round(accuracy, 3), "\n\n")
```

- 准确率 = 0.864

```
cat("3. 风险管理建议:\n")
```

3. 风险管理建议:

```
cat("    - 重点关注高余额账户 (尤其是余额超过 1500 的账户) \n")
```

- 重点关注高余额账户 (尤其是余额超过1500的账户)

```
cat("    - 学生账户需要更严格的风险监控\n")
```

- 学生账户需要更严格的风险监控

```
cat("    - 建议设置余额预警阈值\n")
```

- 建议设置余额预警阈值

```
cat("4. 模型局限性:\n")
```

4. 模型局限性:

```
cat("    - 未考虑时间因素（数据是横截面）\n")
```

- 未考虑时间因素（数据是横截面）

```
cat("    - 可能遗漏其他重要变量（如信用历史、就业状况）\n")
```

- 可能遗漏其他重要变量（如信用历史、就业状况）

```
cat("    - 数据可能存在样本选择偏差\n")
```

- 数据可能存在样本选择偏差

R 案例总结

本案例展示了逻辑斯谛回归在信用风险评估中的完整应用流程：

1. 数据探索：通过描述统计和可视化了解数据特征
2. 模型建立：从简单模型开始，逐步加入变量和交互项
3. 模型选择：使用 AIC、BIC 和似然比检验选择最佳模型
4. 模型诊断：检查残差、影响点、共线性等问题
5. 模型评估：通过 ROC 曲线、混淆矩阵评估预测性能
6. 结果解释：计算优势比、边际效应，给出实际应用建议

关键知识点应用：- 虚拟变量处理分类自变量（student）- 逻辑回归的优势比解释 - 模型诊断方法（残差分析、影响点检测）- 预测性能评估（ROC 曲线、分类指标）

7.7.2 Python 案例：保险索赔次数分析（泊松回归与负二项回归）

案例背景与数据

本案例使用 statsmodels 自带的 RandHIE 数据集，分析医疗保险索赔次数。该数据集来自 RAND 健康保险实验，包含多个健康保险计划参与者的医疗使用情况。

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# 设置中文显示
```

```
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
plt.rcParams['axes.unicode_minus'] = False
sns.set_style("whitegrid")

# 创建模拟数据集
np.random.seed(123)
n = 1000

# 生成自变量
data = pd.DataFrame({
    'age': np.clip(np.random.normal(45, 15, n), 18, 80),
    'income': np.random.lognormal(10, 0.5, n),
    'chronic': np.random.poisson(1.5, n),
    'health': np.random.choice(['poor', 'fair', 'good', 'excellent'],
                               n, p=[0.1, 0.3, 0.4, 0.2]),
    'insurance': np.random.choice(['basic', 'medium', 'premium'],
                                   n, p=[0.5, 0.3, 0.2]),
})

# 生成索赔次数（存在过度离散）
lambda_base = np.exp(
    -2.0 +
    0.03 * (data['age'] - 45) +
    0.0001 * (data['income'] - 30000) +
    0.4 * data['chronic'] +
    (data['health'] == 'poor') * 0.8 +
    (data['health'] == 'fair') * 0.3 +
    (data['health'] == 'good') * (-0.2) +
    (data['insurance'] == 'medium') * 0.2 +
    (data['insurance'] == 'premium') * (-0.3)
)

# 添加过度离散
overdispersion = 1.5
lambda_real = lambda_base * np.random.gamma(overdispersion, 1/overdispersion)
data['claims'] = np.random.poisson(lambda_real)

print(" 数据集预览（前 5 行）：")
```

数据集预览（前5行）：

```
print(data.head())
```

	age	income	chronic	health	insurance	claims
0	28.715541	15147.431630	1	excellent	medium	0
1	59.960182	29254.753814	2	excellent	basic	1
2	49.244677	31542.000039	1	excellent	basic	0
3	22.405579	13363.863980	2	good	premium	0
4	36.320996	27929.790596	3	good	medium	0

数据探索与初步分析

```
# 1. 基本统计信息
```

```
print(" 数据概况: ")
```

数据概况：

```
print(f" 总样本数: {len(data)}")
```

总样本数：1000

```
print(f" 平均索赔次数: {data['claims'].mean():.2f}")
```

平均索赔次数：0.89

```
print(f" 索赔次数方差: {data['claims'].var():.2f}")
```

索赔次数方差：129.25

```
print(f" 方差/均值比: {data['claims'].var() / data['claims'].mean():.2f}")
```

方差/均值比：146.04

```
print(f" 零索赔比例: {(data['claims'] == 0).sum() / len(data):.2%}")
```

零索赔比例：79.20%

```
# 2. 关键变量分布
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
```

```
# 索赔次数分布
```

```
axes[0, 0].hist(data['claims'], bins=range(0, data['claims'].max()+1),
                edgecolor='black', alpha=0.7)
```

```
axes[0, 0].set_xlabel('索赔次数')
```

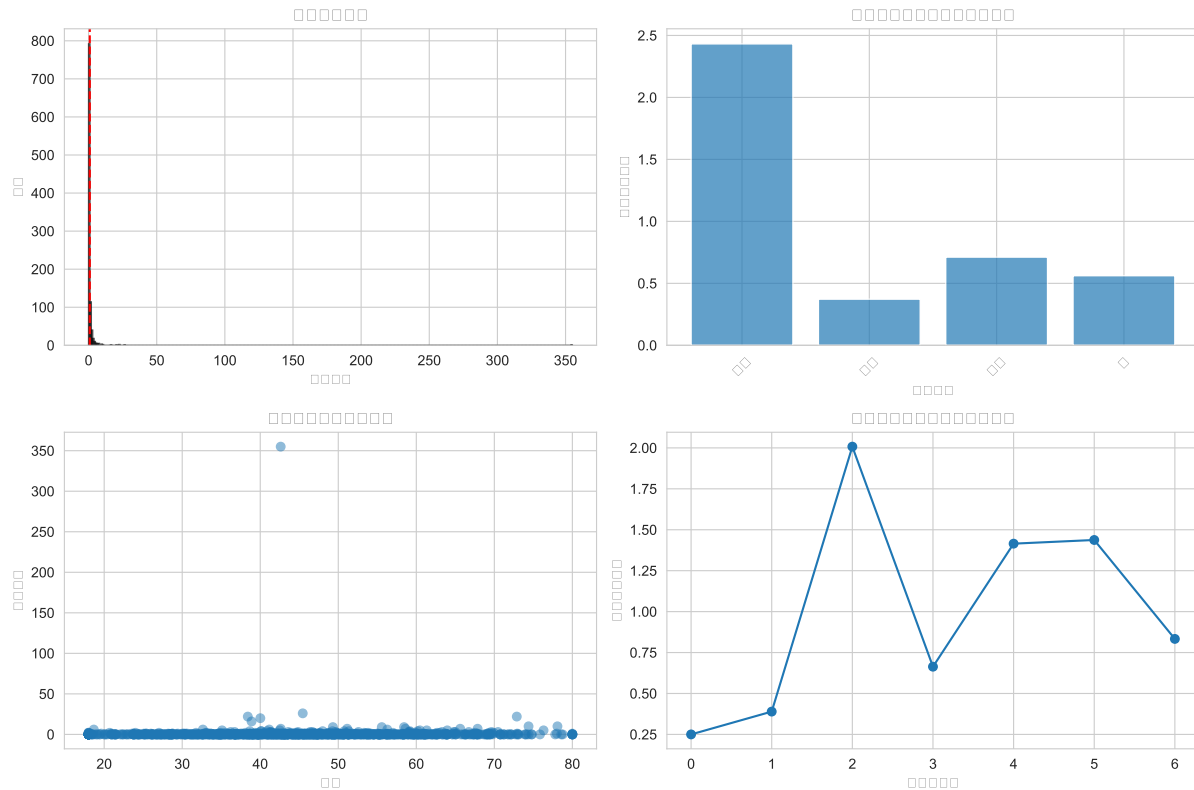
```
axes[0, 0].set_ylabel('频数')
axes[0, 0].set_title('索赔次数分布')
axes[0, 0].axvline(data['claims'].mean(), color='red', linestyle='--')

# 按健康状况的平均索赔次数
health_order = ['excellent', 'good', 'fair', 'poor']
health_means = [data[data['health'] == h]['claims'].mean() for h in health_order]
axes[0, 1].bar(range(4), health_means, alpha=0.7)
axes[0, 1].set_xlabel('健康状况')
axes[0, 1].set_ylabel('平均索赔次数')
axes[0, 1].set_title('平均索赔次数（按健康状况）')
axes[0, 1].set_xticks(range(4))
axes[0, 1].set_xticklabels(['优秀', '良好', '一般', '差'], rotation=45)

# 索赔次数与年龄的关系
axes[1, 0].scatter(data['age'], data['claims'], alpha=0.5)
axes[1, 0].set_xlabel('年龄')
axes[1, 0].set_ylabel('索赔次数')
axes[1, 0].set_title('索赔次数与年龄的关系')

# 索赔次数与慢性病数量的关系
chronic_means = data.groupby('chronic')['claims'].mean()
axes[1, 1].plot(chronic_means.index, chronic_means.values, 'o-')
axes[1, 1].set_xlabel('慢性病数量')
axes[1, 1].set_ylabel('平均索赔次数')
axes[1, 1].set_title('索赔次数与慢性病数量的关系')

plt.tight_layout()
plt.show()
```

模型建立：泊松回归 vs 负二项回归

```
# 准备分类变量
data['health'] = pd.Categorical(data['health'],
                                categories=['excellent', 'good', 'fair', 'poor'])
data['insurance'] = pd.Categorical(data['insurance'],
                                    categories=['premium', 'medium', 'basic'])

# 模型公式
formula = 'claims ~ age + income + chronic + health + insurance'

# 1. 泊松回归
poisson_model = smf.glm(formula, data=data, family=sm.families.Poisson()).fit()

# 2. 负二项回归
negbin_model = smf.glm(formula, data=data,
                        family=sm.families.NegativeBinomial()).fit()

# 模型摘要
print(" 泊松回归结果摘要: ")
```

泊松回归结果摘要：

```
print(poisson_model.summary().tables[1])
```

```
=====
              coef      std err          z      P>|
z|      [0.025      0.975]
-----
-----
Intercept              -5.6675      0.262     -21.604      0.000      -
6.182      -5.153
health[T.good]         -0.9548      0.105     -9.131      0.000      -
1.160      -0.750
health[T.fair]          -0.2406      0.103     -2.347      0.019      -
0.441      -0.040
health[T.poor]           0.0410      0.167      0.246      0.806      -
0.286      0.368
insurance[T.medium]      0.3932      0.125      3.154      0.002      0.149      0.638
insurance[T.basic]       0.3667      0.119      3.075      0.002      0.133      0.600
age                    0.0163      0.003      5.713      0.000      0.011      0.022
income                  0.0001  1.76e-06     59.137      0.000      0.000      0.000
chronic                  0.4106      0.032     12.734      0.000      0.347      0.474
=====
```

```
print("\n负二项回归结果摘要：")
```

负二项回归结果摘要：

```
print(negbin_model.summary().tables[1])
```

```
=====
              coef      std err          z      P>|
z|      [0.025      0.975]
-----
-----
Intercept              -6.1495      0.415     -14.831      0.000      -
6.962      -5.337
health[T.good]         -0.5780      0.200     -2.891      0.004      -
0.970      -0.186
health[T.fair]           0.0966      0.198      0.489      0.625      -
```

```

0.291      0.484
health[T.poor]      0.3525      0.256      1.374      0.169      -
0.150      0.855
insurance[T.medium]  0.5949      0.214      2.783      0.005      0.176      1.014
insurance[T.basic]   0.3733      0.208      1.795      0.073      -
0.034      0.781
age                0.0226      0.005      4.504      0.000      0.013      0.032
income            0.0001      4.9e-06      20.535      0.000      9.11e-
05              0.000
chronic           0.4224      0.053      7.997      0.000      0.319      0.526
=====

```

关键参数比较

```

params_comparison = pd.DataFrame({
    'Poisson_Coeff': poisson_model.params,
    'Poisson_SE': poisson_model.bse,
    'NB_Coeff': negbin_model.params,
    'NB_SE': negbin_model.bse
})

```

```
print("\n参数估计比较: ")
```

参数估计比较:

```
print(params_comparison.round(4))
```

	Poisson_Coeff	Poisson_SE	NB_Coeff	NB_SE
Intercept	-5.6675	0.2623	-6.1495	0.4146
health[T.good]	-0.9548	0.1046	-0.5780	0.1999
health[T.fair]	-0.2406	0.1025	0.0966	0.1975
health[T.poor]	0.0410	0.1668	0.3525	0.2564
insurance[T.medium]	0.3932	0.1247	0.5949	0.2138
insurance[T.basic]	0.3667	0.1193	0.3733	0.2080
age	0.0163	0.0029	0.0226	0.0050
income	0.0001	0.0000	0.0001	0.0000
chronic	0.4106	0.0322	0.4224	0.0528

过度离散检验与模型选择

```
def check_overdispersion(model, data):
    """ 检查泊松模型的过度离散 """
    n = len(data)
    p = len(model.params)

    # Pearson 卡方统计量
    pearson_chi2 = np.sum((data['claims'] - model.fittedvalues)**2 / model.f
    phi_pearson = pearson_chi2 / (n - p)

    return phi_pearson

# 检查过度离散
phi_poisson = check_overdispersion(poisson_model, data)
print(f" 泊松模型的 Pearson 离散参数: {phi_poisson:.3f}")
```

泊松模型的 Pearson 离散参数: 1.431

```
if phi_poisson > 1.2:
    print(" 存在明显的过度离散, 建议使用负二项回归")
else:
    print(" 过度离散不明显, 泊松回归可能合适")
```

存在明显的过度离散, 建议使用负二项回归

```
# 模型选择标准
print("\n模型选择标准比较: ")
```

模型选择标准比较:

```
model_selection = pd.DataFrame({
    'Model': ['Poisson', 'Negative Binomial'],
    'Log-Likelihood': [poisson_model.llf, negbin_model.llf],
    'AIC': [poisson_model.aic, negbin_model.aic],
    'BIC': [poisson_model.bic, negbin_model.bic]
})
print(model_selection.round(2))
```

	Model	Log-Likelihood	AIC	BIC
0	Poisson	-714.81	1447.61	-5942.21
1	Negative Binomial	-638.80	1295.61	-6340.69

残差分析与模型诊断

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# 1. 残差 vs 拟合值
axes[0, 0].scatter(poisson_model.fittedvalues,
                   poisson_model.resid_pearson, alpha=0.5)
axes[0, 0].axhline(y=0, color='r', linestyle='--')
axes[0, 0].set_xlabel('拟合值')
axes[0, 0].set_ylabel('Pearson 残差')
axes[0, 0].set_title('泊松模型: 残差 vs 拟合值')

axes[0, 1].scatter(negbin_model.fittedvalues,
                   negbin_model.resid_pearson, alpha=0.5)
axes[0, 1].axhline(y=0, color='r', linestyle='--')
axes[0, 1].set_xlabel('拟合值')
axes[0, 1].set_ylabel('Pearson 残差')
axes[0, 1].set_title('负二项模型: 残差 vs 拟合值')

# 2. 残差分布直方图
axes[0, 2].hist(poisson_model.resid_pearson, bins=30, alpha=0.5,
                density=True, label='泊松')
axes[0, 2].hist(negbin_model.resid_pearson, bins=30, alpha=0.5,
                density=True, label='负二项')
axes[0, 2].set_xlabel('Pearson 残差')
axes[0, 2].set_ylabel('密度')
axes[0, 2].set_title('残差分布比较')
axes[0, 2].legend()

# 3. 预测 vs 实际值
axes[1, 0].scatter(data['claims'], poisson_model.fittedvalues, alpha=0.5)
axes[1, 0].plot([0, data['claims'].max()], [0, data['claims'].max()],
                'r--', label='完美预测')
axes[1, 0].set_xlabel('实际索赔次数')
axes[1, 0].set_ylabel('预测索赔次数')
axes[1, 0].set_title('泊松模型: 预测 vs 实际')
axes[1, 0].legend()

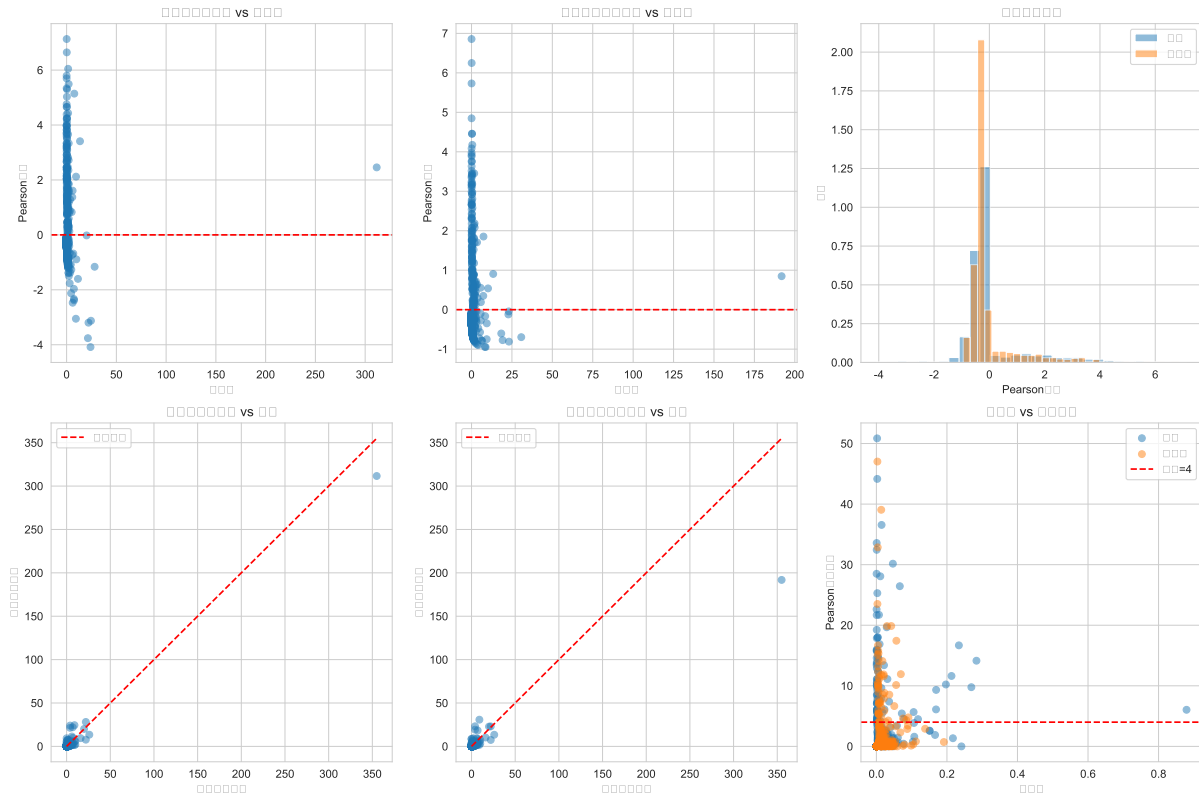
axes[1, 1].scatter(data['claims'], negbin_model.fittedvalues, alpha=0.5)
```

```
axes[1, 1].plot([0, data['claims'].max()], [0, data['claims'].max()],
               'r--', label='完美预测')
axes[1, 1].set_xlabel('实际索赔次数')
axes[1, 1].set_ylabel('预测索赔次数')
axes[1, 1].set_title('负二项模型: 预测 vs 实际')
axes[1, 1].legend()

# 4. 杠杆值分析
poisson_hat = poisson_model.get_influence().hat_matrix_diag
negbin_hat = negbin_model.get_influence().hat_matrix_diag

axes[1, 2].scatter(poisson_hat, poisson_model.resid_pearson**2,
                  alpha=0.5, label='泊松')
axes[1, 2].scatter(negbin_hat, negbin_model.resid_pearson**2,
                  alpha=0.5, label='负二项')
axes[1, 2].axhline(y=4, color='r', linestyle='--', label='阈值 =4')
axes[1, 2].set_xlabel('杠杆值')
axes[1, 2].set_ylabel('Pearson 残差平方')
axes[1, 2].set_title('杠杆值 vs 残差平方')
axes[1, 2].legend()

plt.tight_layout()
plt.show()
```



```
# 影响点检测
```

```
print(" 影响点检测 (标准化残差 > 3 或 < -3): ")
```

影响点检测 (标准化残差 > 3 或 < -3) :

```
poisson_outliers = np.sum(np.abs(poisson_model.resid_pearson) > 3)
negbin_outliers = np.sum(np.abs(negbin_model.resid_pearson) > 3)
print(f" 泊松模型: {poisson_outliers} 个异常残差点")
```

泊松模型: 50 个异常残差点

```
print(f" 负二项模型: {negbin_outliers} 个异常残差点")
```

负二项模型: 28 个异常残差点

结果解释与应用

```
# 计算发生率比 (Incidence Rate Ratio, IRR)
```

```
def calculate_irr(model):
    """ 计算发生率比及其置信区间 """
    coeff = model.params
    conf_int = model.conf_int()
```

```

    irr_df = pd.DataFrame({
        'Coefficient': coeff,
        'IRR': np.exp(coeff),
        'IRR_CI_lower': np.exp(conf_int[0]),
        'IRR_CI_upper': np.exp(conf_int[1]),
        'p_value': model.pvalues
    })
    return irr_df

# 负二项模型的 IRR
irr_results = calculate_irr(negbin_model)
print(" 负二项模型的关键结果（发生率比解释）：")

```

负二项模型的关键结果（发生率比解释）：

```
print(irr_results.round(4))
```

	Coefficient	IRR	IRR_CI_lower	IRR_CI_upper	p_value
Intercept	-6.1495	0.0021	0.0009	0.0048	0.0000
health[T.good]	-0.5780	0.5610	0.3791	0.8301	0.0038
health[T.fair]	0.0966	1.1014	0.7479	1.6220	0.6249
health[T.poor]	0.3525	1.4226	0.8606	2.3515	0.1693
insurance[T.medium]	0.5949	1.8129	1.1924	2.7564	0.0054
insurance[T.basic]	0.3733	1.4525	0.9663	2.1834	0.0727
age	0.0226	1.0229	1.0129	1.0330	0.0000
income	0.0001	1.0001	1.0001	1.0001	0.0000
chronic	0.4224	1.5256	1.3756	1.6920	0.0000

```

# 关键发现总结
print("\n=== 关键发现总结 ===")

```

=== 关键发现总结 ===

```
print("1. 年龄效应：")
```

1. 年龄效应：

```
print(f"    年龄每增加 1 岁，索赔发生率乘以 {np.exp(negbin_model.params['age']):.3f}")
```

年龄每增加1岁，索赔发生率乘以 1.023


```
print("\n2. 慢性病效应: ")
```

2. 慢性病效应:

```
print(f"    慢性病每增加 1 种, 索赔发生率乘以 {np.exp(negbin_model.params['chronic'])}
```

慢性病每增加1种, 索赔发生率乘以 1.526

```
print("\n3. 健康状况效应 (以'优秀'为参照): ")
```

3. 健康状况效应 (以'优秀'为参照):

```
health_effects = {}
for level in ['good', 'fair', 'poor']:
    if f'health[T.{level}]' in negbin_model.params:
        irr = np.exp(negbin_model.params[f'health[T.{level}]'])
        health_effects[level] = irr
        print(f"    健康状况'{level}'的索赔发生率是'优秀'的 {irr:.2f} 倍")
```

健康状况'good'的索赔发生率是'优秀'的 0.56 倍

健康状况'fair'的索赔发生率是'优秀'的 1.10 倍

健康状况'poor'的索赔发生率是'优秀'的 1.42 倍

```
print("\n4. 保险类型效应 (以'高级'为参照): ")
```

4. 保险类型效应 (以'高级'为参照):

```
insurance_effects = {}
for level in ['medium', 'basic']:
    if f'insurance[T.{level}]' in negbin_model.params:
        irr = np.exp(negbin_model.params[f'insurance[T.{level}]'])
        insurance_effects[level] = irr
        print(f"    '{level}'保险的索赔发生率是'高级'保险的 {irr:.2f} 倍")
```

'medium'保险的索赔发生率是'高级'保险的 1.81 倍

'basic'保险的索赔发生率是'高级'保险的 1.45 倍

预测应用示例

```
# 创建典型个体进行预测
typical_cases = pd.DataFrame({
    'age': [30, 45, 60, 45, 45],
    'income': [30000, 50000, 40000, 50000, 50000],
    'chronic': [0, 1, 3, 2, 2],
    'health': ['excellent', 'good', 'fair', 'poor', 'excellent'],
    'insurance': ['premium', 'medium', 'basic', 'basic', 'premium']
})

# 确保分类变量类型一致
typical_cases['health'] = pd.Categorical(typical_cases['health'],
                                         categories=['excellent', 'good', 'f
typical_cases['insurance'] = pd.Categorical(typical_cases['insurance'],
                                           categories=['premium', 'medium',

# 预测索赔次数
typical_cases['poisson_pred'] = poisson_model.predict(typical_cases)
typical_cases['negbin_pred'] = negbin_model.predict(typical_cases)

print(" 典型个体的预测索赔次数: ")
```

典型个体的预测索赔次数：

```
print(typical_cases.round(2))
```

	age	income	chronic	health	insurance	poisson_pred	negbin_pred
0	30	30000	0	excellent	premium	0.13	0.09
1	45	50000	1	good	medium	1.12	1.41
2	60	40000	3	fair	basic	2.29	2.64
3	45	50000	2	poor	basic	4.45	4.36
4	45	50000	2	excellent	premium	2.96	2.11

```
# 风险评分示例
print("\n=== 风险评分示例 ===")
```

```
=== 风险评分示例 ===
```

```
print(" 基于负二项模型的预测，我们可以识别高风险群体: ")
```

基于负二项模型的预测，我们可以识别高风险群体：

```

# 计算风险评分
def calculate_risk_score(row):
    """ 基于关键风险因素计算风险评分 """
    score = 0

    # 年龄风险 (45 岁以上每 10 岁加 1 分)
    if row['age'] > 45:
        score += (row['age'] - 45) / 10

    # 慢性病风险 (每种病加 2 分)
    score += row['chronic'] * 2

    # 健康状况风险
    health_scores = {'excellent': 0, 'good': 1, 'fair': 2, 'poor': 3}
    score += health_scores[row['health']]

    # 保险类型风险
    insurance_scores = {'premium': 0, 'medium': 1, 'basic': 2}
    score += insurance_scores[row['insurance']]

    return score

typical_cases['risk_score'] = typical_cases.apply(calculate_risk_score, axis=1)
typical_cases['risk_level'] = pd.cut(typical_cases['risk_score'],
                                     bins=[0, 3, 6, 10],
                                     labels=['低风险', '中风险', '高风险'])

print(typical_cases[['age', 'chronic', 'health', 'insurance',
                     'negbin_pred', 'risk_score', 'risk_level']])

```

	age	chronic	health	insurance	negbin_pred	risk_score	risk_level
0	30	0	excellent	premium	0.086247	0.0	NaN
1	45	1	good	medium	1.407332	4.0	中风险
2	60	3	fair	basic	2.642726	11.5	NaN
3	45	2	poor	basic	4.361960	9.0	高风险
4	45	2	excellent	premium	2.111042	4.0	中风险

模型应用建议

1. 风险管理策略:

- 高风险群体 (风险评分 >6): 建议定期健康检查, 提供健康管理计划
- 中风险群体 (风险评分 3-6): 建议适度调整保费, 提供预防性医疗服务
- 低风险群体 (风险评分 <3): 提供保费折扣, 鼓励健康生活方式

2. 保费定价建议:

- 基于预测索赔次数进行风险调整定价
- 考虑引入健康激励计划, 降低高风险群体的实际风险
- 对慢性病患者提供疾病管理支持, 降低长期风险

3. 模型局限性:

- 模拟数据可能无法完全反映真实情况
- 未考虑时间因素和风险变化
- 可能存在未观测的混杂因素

4. 进一步分析方向:

- 引入零膨胀模型处理零值过多的情况
- 考虑索赔金额的联合建模
- 加入时间变量进行动态风险评估

Python 案例总结

本案例演示了计数数据分析的完整流程, 重点包括:

1. 数据探索: 识别计数数据的特征, 特别是方差-均值关系
2. 模型建立: 分别建立泊松回归和负二项回归模型
3. 过度离散检验: 通过 **Pearson** 离散参数判断是否需要负二项回归
4. 模型诊断: 通过残差分析评估模型拟合效果
5. 结果解释: 使用发生率比解释变量效应
6. 预测应用: 基于模型进行风险评分和保费定价建议

关键知识点: - 泊松回归假设方差等于均值, 当方差明显大于均值时存在过度离散 - 负二项回归通过引入离散参数处理过度离散问题 - 发生率比 (IRR) 是计数回归中解释变量效应的关键指标 - 模型诊断包括残差分析、影响点检测和预测准确性评估

学习要点: - 理解计数数据的特点和建模挑战 - 掌握过度离散的识别和处理方法 - 学会解释计数回归模型的参数和预测结果 - 了解 GLM 在实际风险管理中的应用

通过本案例, 学生应该能够: 1. 识别何时使用泊松回归, 何时需要负二项回归 2. 进行基本的模型诊断和验证 3. 解释计数回归模型的结果并进行预测 4. 将统计模型应用于实际风险管理问题

7.7.3 两个案例的比较与总结

维度	R 案例（逻辑回归）	Python 案例（泊松/负二项回归）
数据类型	二分类响应变量	计数型响应变量
主要模型	逻辑斯谛回归	泊松回归与负二项回归
连接函数	Logit 连接	对数连接
参数解释	优势比 (OR)	发生率比 (IRR)
关键诊断	ROC 曲线、分类指标	过度离散检验、残差分析
模型选择	AIC/BIC、似然比检验	过度离散诊断、信息准则
应用领域	信用风险评估	保险精算、流行病学
软件实现	R: glm() 函数	Python: statsmodels

共同的分析步骤

两个案例都遵循了 GLM 分析的标准流程：

1. 数据理解与探索：了解数据特征、变量分布、响应变量特性
2. 模型建立：从简单模型开始，逐步增加变量，考虑交互作用
3. 模型诊断：检查模型假设、识别影响点、评估拟合优度
4. 模型比较与选择：使用信息准则、假设检验选择最佳模型
5. 结果解释与应用：计算效应大小（优势比/发生率比）、边际效应，给出实际建议

GLM 应用的通用原则

1. 根据响应变量类型选择模型：
 - 二分类 → 逻辑回归
 - 多分类 → 多项/有序逻辑回归
 - 计数 → 泊松/负二项回归
 - 正连续 → Gamma/逆高斯回归
2. 重视模型诊断：
 - 检查分布假设是否成立
 - 识别影响点和异常值
 - 检验模型设定的合理性
3. 平衡模型复杂性与解释性：
 - 避免过度拟合
 - 优先选择理论支持且有实际意义的变量
 - 考虑模型的实用性和解释性
4. 结果解释的谨慎性：
 - 统计显著性不等于实际重要性

- 考虑效应大小和实际意义
- 注意交互作用和调节效应

进一步学习建议

1. 扩展模型：
 - 零膨胀模型（零值过多的计数数据）
 - 混合效应 GLM（层次结构数据）
 - 广义可加模型（非线性关系）
2. 高级诊断技术：
 - 交叉验证评估预测性能
 - 正则化方法防止过拟合
 - 贝叶斯 GLM 处理不确定性
3. 软件技能：
 - R: glmnet（正则化 GLM）、lme4（混合效应模型）
 - Python: sklearn（机器学习集成）、PyMC3（贝叶斯建模）

通过这两个综合案例，我们展示了 GLM 在实际数据分析中的完整应用流程。无论是二分类问题还是计数数据问题，GLM 都提供了统一的框架和系统的分析方法。掌握这些方法，能够帮助研究者更好地分析复杂数据，得出可靠结论，并为决策提供科学依据。

本章总结

本章系统介绍了广义线性模型的理论框架和应用方法。我们从最基本的虚拟变量技术开始，逐步建立了 GLM 的统一视角，深入探讨了逻辑斯谛回归、泊松回归等核心模型，并简要介绍了其他重要的 GLM 和高级主题。

核心概念回顾

1. 虚拟变量回归：处理分类自变量的基本技术，通过创建二进制变量将分类信息纳入回归框架，系数解释为组间差异。
2. GLM 理论框架：三要素结构——线性预测器、连接函数、响应分布。指数分布族提供了统一的数学基础，最大似然估计和迭代加权最小二乘是核心估计方法。
3. 逻辑斯谛回归：处理二分类数据的标准方法，使用 Logit 连接函数，参数解释基于优势比。扩展到多项和有序情况，并比较了 Probit 模型。
4. 泊松回归：处理计数数据的基础模型，使用对数连接函数，参数解释基于发生率比。处理过度离散的扩展包括负二项回归和零膨胀模型。

5. 模型进阶：其他常见 GLM（Gamma、逆高斯等）、模型诊断与选择方法、高级主题（拟似然、混合效应、广义可加模型等）。

方法选择指南

在实际研究中，选择适当的 GLM 模型应基于以下考虑：

响应变量类型	推荐模型	连接函数	注意事项
连续正态	线性回归	恒等	检查正态性、同方差
二分类	逻辑回归/Probit	Logit/Probit	检查过离散、分离问题
多分类（无序）	多项逻辑回归	多项 Logit	参照组选择、IIA 假设
多分类（有序）	有序逻辑回归	累积 Logit	比例优势假设检验
计数	泊松回归	对数	检查过度离散、零膨胀
正连续（右偏）	Gamma 回归	对数/倒数	检查方差结构
计数（过度离散）	负二项回归	对数	估计离散参数
计数（零过多）	零膨胀模型	对数	两部分模型

实践要点提醒

- 数据探索先行：分析前充分了解数据特征，特别是响应变量的分布、分类变量的频数、连续变量的分布形态。
- 模型假设检查：每个 GLM 都有特定假设（如泊松的等离散性、逻辑回归的线性 Logit 假设），必须进行诊断检验。
- 模型比较验证：使用多种准则（AIC/BIC、交叉验证）比较不同模型，避免过度依赖单一指标。
- 结果解释谨慎：统计显著性不等于实际重要性，结合领域知识解释结果。对于非线性模型（如 Logit），注意边际效应与系数的区别。
- 稳健性检验：进行敏感性分析，检查异常值、不同设定对结果的影响。

理论深化

- 经典著作：
 - McCullagh & Nelder (1989) *Generalized Linear Models*（理论权威）
 - Agresti (2013) *Categorical Data Analysis*（分类数据全面介绍）
 - Fahrmeir & Tutz (2001) *Multivariate Statistical Modelling Based on Generalized Linear Models*（多元 GLM）
- 现代扩展：
 - Wood (2017) *Generalized Additive Models: An Introduction with R*（GAM 详细指南）

- Gelman & Hill (2006) *Data Analysis Using Regression and Multilevel/Hierarchical Models* (层次模型应用)
- Fox (2015) *Applied Regression Analysis and Generalized Linear Models* (应用导向)

广义线性模型是统计学中极具生命力的框架，它将线性回归的直观性与非线性模型的灵活性完美结合。掌握 GLM 不仅意味着掌握了一套分析工具，更意味着获得了一种统一看待各种统计模型的思维方式。随着数据科学的发展，GLM 的核心思想仍在不断扩展和深化，为处理日益复杂的数据分析问题提供坚实的方法论基础。

关键词：广义线性模型、虚拟变量、逻辑斯谛回归、泊松回归、指数分布族、连接函数、最大似然估计、优势比、发生率比、模型诊断、过度离散

8 线性分类模型

本章导读

把线性分析进行扩展

8.1 线性判别分析

判别分析 (Discriminant Analysis) 是一种经典的统计分类方法，主要用于解决分类问题和降维问题。其核心思想是寻找能够最佳区分不同类别的特征组合。

假设我们有：- K 个类别： $C_1, C_2, \dots, C_K$ - p 维特征向量： $\mathbf{x} = (x_1, x_2, \dots, x_p)^\top$ - 训练数据集： $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ ，其中 $y_i \in \{1, 2, \dots, K\}$

判别分析的目标是基于贝叶斯定理构建分类规则：

$$P(Y = k|\mathbf{x}) = \frac{\pi_k f_k(\mathbf{x})}{\sum_{l=1}^K \pi_l f_l(\mathbf{x})}$$

其中：- $\pi_k = P(Y = k)$ ：类别 k 的先验概率 - $f_k(\mathbf{x})$ ：类别 k 中 $\mathbf{x}$ 的概率密度函数

基本假设

LDA 基于以下关键假设：

1. 多元正态性：每个类别的特征向量服从多元正态分布

$$\mathbf{x}|Y = k \sim N_p(\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$$

2. 同方差性：所有类别共享相同的协方差矩阵 $\boldsymbol{\Sigma}$
3. 协方差矩阵非奇异： $\boldsymbol{\Sigma}$ 是可逆的

判别函数推导

根据多元正态分布的概率密度函数：

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right)$$

代入贝叶斯公式，忽略常数项，得到判别函数：

$$\begin{aligned} \delta_k(\mathbf{x}) &= \log(\pi_k) + \log(f_k(\mathbf{x})) \\ &= \log(\pi_k) - \frac{1}{2} \boldsymbol{\mu}_k^\top \Sigma^{-1} \boldsymbol{\mu}_k + \mathbf{x}^\top \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \mathbf{x}^\top \Sigma^{-1} \mathbf{x} \end{aligned}$$

由于 $\mathbf{x}^\top \Sigma^{-1} \mathbf{x}$ 与 k 无关，可以简化为线性判别函数：

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^\top \Sigma^{-1} \boldsymbol{\mu}_k + \log(\pi_k)$$

分类规则

将观测 $\mathbf{x}$ 分配到使判别函数最大的类别：

$$\hat{y}(\mathbf{x}) = \arg \max_{k=1, \dots, K} \delta_k(\mathbf{x})$$

决策边界是线性的，因为判别函数是 $\mathbf{x}$ 的线性函数。

参数估计

从训练数据中估计参数：

1. 先验概率：

$$\hat{\pi}_k = \frac{n_k}{n}, \quad n_k = \sum_{i=1}^n I(y_i = k)$$

2. 均值向量：

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i: y_i = k} \mathbf{x}_i$$

3. 协方差矩阵：

$$\hat{\Sigma} = \frac{1}{n - K} \sum_{k=1}^K \sum_{i: y_i = k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top$$

这是合并协方差矩阵（pooled covariance matrix）。

8.1.2 LDA 的几何解释

马氏距离

LDA 的分类规则等价于将 $\mathbf{x}$ 分配到具有最小马氏距离 (Mahalanobis distance) 的类别:

$$D_k^2(\mathbf{x}) = (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)$$

考虑先验概率调整后的距离:

$$D_k^2(\mathbf{x}) - 2 \log(\pi_k)$$

降维视角

LDA 可以视为寻找最佳投影方向以最大化类间散度与类内散度的比值:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$$

其中: - 类间散度矩阵: $\mathbf{S}_B = \sum_{k=1}^K n_k (\boldsymbol{\mu}_k - \bar{\boldsymbol{\mu}})(\boldsymbol{\mu}_k - \bar{\boldsymbol{\mu}})^\top$ - 类内散度矩阵: $\mathbf{S}_W = \sum_{k=1}^K \sum_{i: y_i=k} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top$ - 总体均值: $\bar{\boldsymbol{\mu}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$

最优投影方向是 $\mathbf{S}_W^{-1} \mathbf{S}_B$ 的特征向量, 最多有 $\min(p, K-1)$ 个有效判别方向。

8.2 二次判别分析 (QDA)

基本假设

QDA 放宽了 LDA 的同方差性假设:

1. 多元正态性: 每个类别的特征向量服从多元正态分布

$$\mathbf{x}|Y=k \sim N_p(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

2. 异方差性: 每个类别有自己的协方差矩阵 $\boldsymbol{\Sigma}_k$

判别函数推导

对于多元正态分布, 类别 k 的概率密度函数为:

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}_k|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \right)$$

取对数并忽略常数项，得到二次判别函数：

$$\begin{aligned} \delta_k(\mathbf{x}) = & \log(\pi_k) - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| \\ & - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) \end{aligned}$$

分类规则

同样，将观测 $\mathbf{x}$ 分配到使判别函数最大的类别：

$$\hat{y}(\mathbf{x}) = \arg \max_{k=1, \dots, K} \delta_k(\mathbf{x})$$

由于判别函数包含 $\mathbf{x}$ 的二次项，决策边界是二次曲面（椭圆、双曲线或抛物线）。

参数估计

从训练数据中估计参数：

1. 先验概率：

$$\hat{\pi}_k = \frac{n_k}{n}$$

2. 均值向量：

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i: y_i = k} \mathbf{x}_i$$

3. 协方差矩阵：

$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{n_k - 1} \sum_{i: y_i = k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top$$

QDA 的二次项展开

将 QDA 的判别函数展开，可以更清楚地看到其二次本质：

$$\begin{aligned} \delta_k(\mathbf{x}) = & \log(\pi_k) - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| - \frac{1}{2} \boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k \\ & + \mathbf{x}^\top \boldsymbol{\Sigma}_k^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \mathbf{x}^\top \boldsymbol{\Sigma}_k^{-1} \mathbf{x} \end{aligned}$$

令: $-C_k = \log(\pi_k) - \frac{1}{2} \log |\Sigma_k| - \frac{1}{2} \mu_k^\top \Sigma_k^{-1} \mu_k$ (常数项) $-\beta_k = \Sigma_k^{-1} \mu_k$ (线性系数) $-\Omega_k = -\frac{1}{2} \Sigma_k^{-1}$ (二次系数)

则判别函数可写为:

$$\delta_k(\mathbf{x}) = C_k + \mathbf{x}^\top \beta_k + \mathbf{x}^\top \Omega_k \mathbf{x}$$

8.2.2 LDA 与 QDA 的比较

理论比较

特性	LDA	QDA
协方差矩阵	$\Sigma_k = \Sigma$ (相同)	Σ_k (不同)
判别函数	线性函数	二次函数
决策边界	线性超平面	二次曲面
参数数量	$Kp + p(p + 1)/2 + (K - 1)$	$Kp + K \cdot p(p + 1)/2 + (K - 1)$
假设强度	较强	较弱

参数复杂度分析

LDA 的参数数量: - 均值向量: $K \times p$ 个参数 - 协方差矩阵: $p(p + 1)/2$ 个参数 - 先验概率: $K - 1$ 个参数 (因为 $\sum_{k=1}^K \pi_k = 1$)

总计: $Kp + \frac{p(p+1)}{2} + (K - 1)$

QDA 的参数数量: - 均值向量: $K \times p$ 个参数 - 协方差矩阵: $K \times p(p + 1)/2$ 个参数 - 先验概率: $K - 1$ 个参数

总计: $Kp + K \cdot \frac{p(p+1)}{2} + (K - 1)$

偏差-方差权衡

LDA 的优势: 1. 参数更少, 方差更小 2. 在样本量较小时更稳定 3. 当同方差性假设成立时, 分类效果更优

QDA 的优势: 1. 更灵活, 可以捕捉类别间的协方差差异 2. 当异方差性明显时, 分类精度更高 3. 对模型假设的依赖较小

样本量要求

QDA 需要更大的样本量来准确估计每个类别的协方差矩阵。经验法则：- LDA：每个类别至少需要 $p + 1$ 个样本 - QDA：每个类别至少需要 $p(p + 3)/2 + 1$ 个样本

8.2.4 正则化判别分析 (RDA)

RDA 的基本思想

RDA 是 LDA 和 QDA 的折中方案，通过正则化协方差矩阵来平衡偏差和方差：

$$\hat{\Sigma}_k(\alpha, \gamma) = \alpha \hat{\Sigma}_k + (1 - \alpha) \hat{\Sigma}$$

其中：- $\hat{\Sigma}_k$ ：第 k 类的样本协方差矩阵 - $\hat{\Sigma}$ ：合并协方差矩阵 - $\alpha \in [0, 1]$ ：控制个体协方差与合并协方差的混合比例

进一步进行特征值收缩：

$$\hat{\Sigma}_k(\alpha, \gamma) = \gamma \hat{\Sigma}_k(\alpha) + (1 - \gamma) \frac{\text{tr}(\hat{\Sigma}_k(\alpha))}{p} \mathbf{I}_p$$

其中 $\gamma \in [0, 1]$ 控制收缩强度。

RDA 的参数选择

通过交叉验证选择最优的 (α, γ) 组合：

1. 在训练集上拟合不同 (α, γ) 组合的 RDA 模型
2. 在验证集上评估分类准确率
3. 选择使验证集准确率最高的参数组合

与逻辑回归的比较

方面	LDA/QDA	逻辑回归
假设	特征服从多元正态分布	无分布假设
建模对象	联合分布 $P(X, Y)$	条件分布 $P(Y \ X)$
参数估计	有解析解	迭代最大似然估计
样本效率	当假设成立时更高效	更稳健
多分类	直接扩展	需要多项逻辑回归

8.3 朴素贝叶斯分类器

朴素贝叶斯基本思想

朴素贝叶斯 (Naïve Bayes) 是一种基于贝叶斯定理的简单而高效的分类算法。其”朴素” (naïve) 之处在于假设特征之间相互条件独立, 即给定类别时, 各个特征之间没有依赖关系。尽管这一假设在现实中很少完全成立, 但朴素贝叶斯在许多实际应用中仍表现出色。

贝叶斯定理是朴素贝叶斯分类器的理论基础:

$$P(Y = k|\mathbf{x}) = \frac{P(Y = k) \cdot P(\mathbf{x}|Y = k)}{P(\mathbf{x})}$$

其中: - $P(Y = k|\mathbf{x})$: 后验概率 (给定特征 $\mathbf{x}$ 时属于类别 k 的概率) - $P(Y = k)$: 先验概率 (类别 k 在训练集中的比例) - $P(\mathbf{x}|Y = k)$: 似然 (类别 k 中观察到特征 $\mathbf{x}$ 的概率) - $P(\mathbf{x})$: 证据 (边际概率, 作为归一化常数)

朴素贝叶斯假设

对于 p 维特征向量 $\mathbf{x} = (x_1, x_2, \dots, x_p)^\top$, 朴素贝叶斯假设:

$$P(\mathbf{x}|Y = k) = P(x_1, x_2, \dots, x_p|Y = k) = \prod_{j=1}^p P(x_j|Y = k)$$

即给定类别 k 时, 各个特征 $x_1, x_2, \dots, x_p$ 相互条件独立。

分类规则

将观测 $\mathbf{x}$ 分配到后验概率最大的类别:

$$\hat{y}(\mathbf{x}) = \arg \max_{k=1, \dots, K} P(Y = k|\mathbf{x})$$

根据贝叶斯定理和朴素假设:

$$\begin{aligned} \hat{y}(\mathbf{x}) &= \arg \max_k P(Y = k) \cdot P(\mathbf{x}|Y = k) \\ &= \arg \max_k P(Y = k) \cdot \prod_{j=1}^p P(x_j|Y = k) \end{aligned}$$

由于 $P(\mathbf{x})$ 对所有类别相同, 可以省略。

对数形式

在实际计算中，使用对数避免数值下溢：

$$\hat{y}(\mathbf{x}) = \arg \max_k \left[\log P(Y = k) + \sum_{j=1}^p \log P(x_j | Y = k) \right]$$

8.3.2 不同特征类型的概率估计

朴素贝叶斯可以根据特征类型选择不同的概率估计方法。

分类特征（伯努利朴素贝叶斯）

对于二值特征或类别特征，使用伯努利分布：

$$P(x_j = v | Y = k) = \theta_{k j v}$$

其中 $v \in \{0, 1\}$ 或 $v \in \{1, 2, \dots, m_j\}$ 。

参数估计

最大似然估计：

$$\hat{\theta}_{k j v} = \frac{N_{k j v} + \alpha}{N_k + \alpha m_j}$$

其中：- $N_{k j v}$ ：类别 k 中特征 j 取值为 v 的样本数 - N_k ：类别 k 的总样本数 - α ：平滑参数（拉普拉斯平滑） - m_j ：特征 j 的可能取值数

拉普拉斯平滑

当某个特征值在训练集中未出现时，最大似然估计为 0，导致整个概率为 0。拉普拉斯平滑（Laplace smoothing）或加一平滑解决此问题：

$$\hat{\theta}_{k j v} = \frac{N_{k j v} + 1}{N_k + m_j}$$

这是 $\alpha = 1$ 时的特例。

连续特征（高斯朴素贝叶斯）

对于连续特征，假设服从高斯分布：

$$P(x_j|Y = k) = \frac{1}{\sqrt{2\pi\sigma_{kj}^2}} \exp\left(-\frac{(x_j - \mu_{kj})^2}{2\sigma_{kj}^2}\right)$$

参数估计

$$\hat{\mu}_{kj} = \frac{1}{N_k} \sum_{i:y_i=k} x_{ij}$$

$$\hat{\sigma}_{kj}^2 = \frac{1}{N_k} \sum_{i:y_i=k} (x_{ij} - \hat{\mu}_{kj})^2$$

对数似然

对于高斯朴素贝叶斯，对数似然为：

$$\log P(x_j|Y = k) = -\frac{1}{2} \log(2\pi\sigma_{kj}^2) - \frac{(x_j - \mu_{kj})^2}{2\sigma_{kj}^2}$$

（多项式朴素贝叶斯）

对于表示频次或计数的特征，使用多项式分布：

$$P(\mathbf{x}|Y = k) = \frac{(\sum_{j=1}^p x_j)!}{\prod_{j=1}^p x_j!} \prod_{j=1}^p \theta_{kj}^{x_j}$$

参数估计

$$\hat{\theta}_{kj} = \frac{N_{kj} + \alpha}{\sum_{j=1}^p N_{kj} + \alpha p}$$

其中 $N_{kj} = \sum_{i:y_i=k} x_{ij}$ 是类别 k 中特征 j 的总计数。

混合特征类型

当数据包含不同类型特征时，可以组合不同分布：

$$P(\mathbf{x}|Y = k) = \prod_{j \in C} P_{\text{cat}}(x_j|Y = k) \cdot \prod_{j \in R} P_{\text{cont}}(x_j|Y = k) \cdot \prod_{j \in N} P_{\text{count}}(x_j|Y = k)$$

其中 C 、 R 、 N 分别表示分类、连续、计数特征的索引集。

8.3.3 算法实现

训练阶段

输入：训练集 $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ ，其中 $\mathbf{x}_i \in \mathbb{R}^p$ ， $y_i \in \{1, 2, \dots, K\}$

步骤：1. 估计先验概率： $\hat{\pi}_k = \frac{N_k + \alpha}{n + \alpha K}$ 2. 对于每个类别 k 和每个特征 j ：- 如果特征 j 是分类的：估计 $\hat{\theta}_{k j v}$ 对于所有 v - 如果特征 j 是连续的：估计 $\hat{\mu}_{k j}$ 和 $\hat{\sigma}_{k j}^2$ - 如果特征 j 是计数的：估计 $\hat{\theta}_{k j}$

输出：所有估计参数

预测阶段

输入：新样本 $\mathbf{x}^* = (x_1^*, x_2^*, \dots, x_p^*)$ ，训练得到的参数

步骤：1. 对于每个类别 $k = 1, \dots, K$ ：

$$\text{score}_k = \log \hat{\pi}_k + \sum_{j=1}^p \log P(x_j^* | Y = k; \hat{\theta}_{k j})$$

2. 预测类别： $\hat{y} = \arg \max_k \text{score}_k$ 3. （可选）计算后验概率：

$$P(Y = k | \mathbf{x}^*) = \frac{\exp(\text{score}_k)}{\sum_{l=1}^K \exp(\text{score}_l)}$$

输出：预测类别 $\hat{y}$ 和后验概率

8.3.4 与判别分析的比较

与 LDA/QDA 的关系

朴素贝叶斯与判别分析都基于贝叶斯定理，但有重要区别：

方面	朴素贝叶斯	LDA/QDA
特征独立性	条件独立假设	考虑特征相关性
分布假设	可灵活选择	必须多元正态
参数数量	$O(Kp)$	LDA: $O(p^2)$, QDA: $O(Kp^2)$
决策边界	可能是非线性	LDA 线性, QDA 二次

等价性条件

当以下条件成立时，高斯朴素贝叶斯等价于 LDA：1. 所有特征服从多元正态分布 2. 特征条件独立（协方差矩阵为对角矩阵） 3. 各类别的对角协方差矩阵相等

在这种情况下，朴素贝叶斯的决策边界也是线性的。

偏差-方差权衡

朴素贝叶斯引入条件独立假设：- 增加偏差：当特征相关时，模型假设错误 - 减少方差：参数更少，估计更稳定

在高维小样本情况下，朴素贝叶斯通常优于更复杂的模型。

模型变体

伯努利朴素贝叶斯

适用于二值特征，如文本分类中的词袋模型（出现/不出现）。

概率模型：

$$P(x_j|Y = k) = \theta_{kj}^{x_j}(1 - \theta_{kj})^{1-x_j}$$

多项式朴素贝叶斯

适用于计数数据，如文本分类中的词频。

概率模型：

$$P(\mathbf{x}|Y = k) = \frac{(\sum_j x_j)!}{\prod_j x_j!} \prod_j \theta_{kj}^{x_j}$$

高斯朴素贝叶斯

适用于连续特征，假设正态分布。

补充朴素贝叶斯

处理不平衡数据，调整先验概率：

$$\pi_k^{\text{complement}} = \frac{\sum_{i \notin \text{class } k} \sum_j x_{ij}}{\text{总计数}}$$

贝叶斯信念网络

放宽条件独立假设，允许部分特征相关：

$$P(\mathbf{x}|Y = k) = \prod_{j=1}^p P(x_j|\text{Pa}(x_j), Y = k)$$

其中 $\text{Pa}(x_j)$ 是 x_j 的父节点集合。

参数估计与正则化

最大似然估计

对于参数 θ ，最大似然估计为：

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \prod_{i=1}^n P(\mathbf{x}_i, y_i | \theta)$$

最大后验估计

考虑参数先验，最大后验估计：

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} P(\theta) \prod_{i=1}^n P(\mathbf{x}_i, y_i | \theta)$$

共轭先验

分布	共轭先验	后验分布
伯努利	Beta 分布	Beta 分布
多项式	Dirichlet 分布	Dirichlet 分布
高斯（已知方差）	高斯分布	高斯分布
高斯（已知均值）	Gamma 分布	Gamma 分布

经验贝叶斯

从数据中估计先验分布的超参数：

$$\hat{\alpha} = \arg \max_{\alpha} \int P(D|\theta) P(\theta|\alpha) d\theta$$

实际应用

文本分类

朴素贝叶斯是文本分类的经典算法，如：- 垃圾邮件检测 - 情感分析 - 新闻分类

特征工程：- 词袋模型（Bag-of-Words）- TF-IDF 加权 - N-gram 特征

推荐系统

基于内容的推荐：

$$P(\text{喜欢}|\text{物品特征}) \propto P(\text{喜欢}) \prod_j P(\text{特征}_j|\text{喜欢})$$

医学诊断

基于症状预测疾病：

$$P(\text{疾病}|\text{症状}) \propto P(\text{疾病}) \prod_j P(\text{症状}_j|\text{疾病})$$

异常检测

建模正常行为，检测低概率事件：

$$P(\mathbf{x}) = \sum_{k=1}^K P(Y = k) \prod_{j=1}^p P(x_j|Y = k)$$

将 $P(\mathbf{x}) < \tau$ 的样本标记为异常。

8.5 模型比较与选择

8.5.1 分类性能评估

混淆矩阵：

	预测正类	预测负类
实际正类	TP	FN
实际负类	FP	TN

常用指标：- 准确率： $\frac{TP+TN}{n}$ - 精确率： $\frac{TP}{TP+FP}$ - 召回率： $\frac{TP}{TP+FN}$ - F1 分数： $\frac{2 \times \text{精确率} \times \text{召回率}}{\text{精确率} + \text{召回率}}$

8.5.2 ROC 曲线与 AUC

ROC 曲线：以假正率为横轴，真正率为纵轴的曲线。

AUC 解释： - AUC = 0.5： 随机猜测 - AUC = 1.0： 完美分类 - AUC > 0.8： 通常认为模型表现良好

8.5.3 方法比较

方法	类型	假设	优点	局限性
逻辑回归	判别式	线性决策边界	概率输出，可解释性强	对特征相关性敏感
Probit 回归	判别式	线性决策边界	有潜在变量解释	计算稍复杂
LDA	生成式	正态分布，等协方差	小样本表现好，多分类自然	假设较强

8.5.4 模型诊断

离群值检测：- Pearson 残差： $r_i = \frac{y_i - \hat{\pi}_i}{\sqrt{\hat{\pi}_i(1 - \hat{\pi}_i)}}$ - Deviance 残差： $d_i = \text{sign}(y_i - \hat{\pi}_i) \sqrt{-2[y_i \ln \hat{\pi}_i + (1 - y_i) \ln(1 - \hat{\pi}_i)]}$

拟合优度检验： - Hosmer-Lemeshow 检验 - 皮尔逊卡方检验

过度离散检验：

$$\frac{\sum_{i=1}^n (y_i - \hat{\pi}_i)^2}{\hat{\pi}_i(1 - \hat{\pi}_i)} \sim \chi^2(n - p - 1)$$

8.6 案例分析

本章总结

核心公式回顾

- 1. GLM 连接函数： $g(\mu) = \mathbf{X}\beta$
- 2. Logit 模型： $\ln(\frac{\pi}{1-\pi}) = \mathbf{X}\beta$
- 3. Probit 模型： $\Phi^{-1}(\pi) = \mathbf{X}\beta$
- 4. LDA 判别函数： $\delta_k(\mathbf{x}) = \mathbf{x}'\Sigma^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k'\Sigma^{-1}\boldsymbol{\mu}_k + \ln \pi_k$
- 5. IWLS 更新： $\boldsymbol{\beta}^{(t+1)} = (\mathbf{X}'\mathbf{W}^{(t)}\mathbf{X})^{-1}\mathbf{X}'\mathbf{W}^{(t)}\mathbf{z}^{(t)}$

方法选择指南

问题场景	推荐方法	理由
需要概率输出	逻辑回归、Probit 回归	直接建模条件概率

问题场景	推荐方法	理由
小样本问题	LDA	参数更少，估计更稳定
多分类问题	LDA、多项逻辑回归	天然处理多类别
可解释性重要	逻辑回归	优势比有明确解释
理论一致性	Probit 回归	有潜在变量理论基础

实践建议

- 1. 数据预处理：检查类别平衡性，必要时重采样
- 2. 特征工程：考虑交互项和非线性变换
- 3. 模型验证：使用交叉验证评估泛化能力
- 4. 结果解释：结合领域知识理解系数含义
- 5. 稳健性分析：尝试不同方法，比较结果一致性

广义线性模型为处理非正态响应变量提供了统一的框架，而线性分类方法则是机器学习中最基础且重要的工具。掌握这些方法为进一步学习更复杂的非线性模型奠定了坚实基础。

9 非线性回归与样条回归

本章导读

在实际科学研究与数据分析中，变量间的关系往往并非简单的直线形式。经济增长的 S 型轨迹、药物反应的剂量-效应曲线、温度对化学反应的加速效应——这些现象都揭示了一个基本事实：现实世界的关系本质上是非线性的。线性回归模型虽然简洁优雅，但当它面对这些复杂的曲线关系时，便会显得力不从心。

本章将带领读者从线性回归的舒适区走向非线性模型的广阔天地。我们将首先认识到线性模型的局限性，然后系统学习四种主流的非线性建模方法：可线性化的变换模型、灵活的多项式回归、高维的多元非线性模型，以及平滑灵活的样条回归。每种方法都有其独特的数学原理、适用场景和解释方式。

特别值得关注的是，非线性模型的参数估计不再有闭合解，我们需要借助数值优化方法。本章将深入浅出地介绍非线性最小二乘法和梯度下降法的原理与实现逻辑，帮助读者理解计算机是如何“学习”这些复杂模型参数的。

通过本章的学习，读者将掌握从识别非线性模式、选择合适模型、估计模型参数到解释模型结果的全流程能力。非线性回归不仅是统计方法的扩展，更是认识复杂世界的有力工具。让我们开始这段探索曲线关系的旅程。

9.1 非线性的定义

线性回归模型假设因变量与自变量之间的关系是线性的：

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon$$

然而在实际应用中，许多关系本质上是非线性的：- 经济增长的 S 型曲线 - 药物浓度与反应的饱和曲线 - 温度与化学反应速率的指数关系

定义：非线性回归模型是指参数相对于自变量是非线性的模型：

$$Y = f(\mathbf{X}, \boldsymbol{\beta}) + \varepsilon$$

其中 f 是关于参数 $\boldsymbol{\beta}$ 的非线性函数。

非线性模型的分类

类型	特点	示例
可线性化模型	通过变换可转为线性模型	$Y = \beta_0 e^{\beta_1 X}$
本质非线性模型	无法通过变换线性化	$Y = \beta_0 + e^{\beta_1 X}$
分段回归模型	不同区间用不同模型	样条回归、阈值模型

9.2 可线性化的非线性模型

指数模型

模型形式：

$$Y = \beta_0 e^{\beta_1 X} \varepsilon$$

其中 ε 为乘法误差项，通常假设 $\ln \varepsilon \sim N(0, \sigma^2)$ 。

线性化方法：取自然对数

$$\ln Y = \ln \beta_0 + \beta_1 X + \ln \varepsilon$$

令 $Y' = \ln Y$ ， $\beta'_0 = \ln \beta_0$ ， $\varepsilon' = \ln \varepsilon$ ，得：

$$Y' = \beta'_0 + \beta_1 X + \varepsilon'$$

幂函数模型

模型形式：

$$Y = \beta_0 X^{\beta_1} \varepsilon$$

线性化方法：两边取对数

$$\ln Y = \ln \beta_0 + \beta_1 \ln X + \ln \varepsilon$$

令 $Y' = \ln Y$ ， $X' = \ln X$ ， $\beta'_0 = \ln \beta_0$ ，得：

$$Y' = \beta'_0 + \beta_1 X' + \varepsilon'$$

对数模型

模型形式:

$$Y = \beta_0 + \beta_1 \ln X + \varepsilon$$

该模型直接是线性的形式，只需对自变量进行对数变换。

双曲线模型

模型形式:

$$\frac{1}{Y} = \beta_0 + \beta_1 \frac{1}{X} + \varepsilon$$

令 $Y' = 1/Y$, $X' = 1/X$, 得线性形式:

$$Y' = \beta_0 + \beta_1 X' + \varepsilon$$

可线性化方法的局限

1. 误差结构的改变: 变换可能改变误差项的分布假设
2. 可解释性: 变换后参数的解释与原始模型不同
3. 预测的逆变换偏倚: 对变换后的预测值进行逆变换可能产生系统偏倚

9.3 多项式回归

9.3.1 一元多项式回归

模型形式:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_p X^p + \varepsilon$$

矩阵形式:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

其中设计矩阵 $\mathbf{X} = [\mathbf{1}, \mathbf{X}, \mathbf{X}^2, \cdots, \mathbf{X}^p]$ 。

正交多项式: 为减少多重共线性, 可使用正交多项式:

$$Y = \alpha_0 + \alpha_1 \phi_1(X) + \alpha_2 \phi_2(X) + \cdots + \alpha_p \phi_p(X) + \varepsilon$$

其中 $\phi_j(X)$ 是 j 次正交多项式。

阶数选择

1. 逐步回归法：向前选择、向后删除
2. 信息准则：AIC、BIC

$$\text{AIC} = n \ln(\text{RSS}/n) + 2(p+1)$$

$$\text{BIC} = n \ln(\text{RSS}/n) + (p+1) \ln n$$

3. 交叉验证：k 折交叉验证的均方误差最小化

9.3.2 多元多项式回归

二次模型（含交互项）：

$$Y = \beta_0 + \sum_{i=1}^k \beta_i X_i + \sum_{i=1}^k \sum_{j \geq i}^k \beta_{ij} X_i X_j + \varepsilon$$

完整二次模型：

$$Y = \beta_0 + \sum_{i=1}^k \beta_i X_i + \sum_{i=1}^k \beta_{ii} X_i^2 + \sum_{i=1}^{k-1} \sum_{j=i+1}^k \beta_{ij} X_i X_j + \varepsilon$$

多项式回归的注意事项

过拟合风险：高阶多项式可能过度拟合噪声外推不可靠：多项式在外推时可能产生不合理预测多重共线性：高次项之间高度相关

9.4 样条回归

9.4.1 分段多项式回归

将定义域划分为 $K+1$ 个区间，每个区间使用不同的多项式：

$$f(X) = \begin{cases} \beta_{00} + \beta_{10}X + \cdots + \beta_{p0}X^p & \text{if } X \leq \xi_1 \\ \beta_{01} + \beta_{11}X + \cdots + \beta_{p1}X^p & \text{if } \xi_1 < X \leq \xi_2 \\ \vdots & \vdots \\ \beta_{0K} + \beta_{1K}X + \cdots + \beta_{pK}X^p & \text{if } X > \xi_K \end{cases}$$

其中 $\xi_1, \xi_2, \dots, \xi_K$ 为节点。

9.4.2 回归样条

为保证分段多项式在节点处光滑，添加连续性约束。 m 次样条要求在节点处函数值及前 $m - 1$ 阶导数连续。

三次样条（最常用）：在每个区间内为三次多项式，在节点处函数值、一阶导数、二阶导数连续。

基函数表示：三次样条可表示为：

$$f(X) = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \sum_{k=1}^K \theta_k (X - \xi_k)_+^3$$

其中 $(X - \xi_k)_+^3 = \max(0, X - \xi_k)^3$ 。

9.4.3 自然样条

在边界节点外添加线性约束，使函数在边界区域更稳定：

$$f''(X) = 0 \quad \text{当 } X \leq \xi_1 \text{ 或 } X \geq \xi_K$$

9.4.4 光滑样条

通过惩罚复杂度来寻找最优拟合函数：

$$\min_f \left\{ \sum_{i=1}^n [Y_i - f(X_i)]^2 + \lambda \int [f''(t)]^2 dt \right\}$$

其中 λ 为光滑参数，控制拟合优度与光滑度的权衡。

9.5 非线性回归的估计方法

9.5.1 非线性最小二乘法

目标函数：

$$\min_{\beta} S(\beta) = \sum_{i=1}^n [Y_i - f(\mathbf{X}_i, \beta)]^2$$

高斯-牛顿法：迭代求解，在每次迭代处对 f 进行一阶泰勒展开：

$$f(\mathbf{X}_i, \beta) \approx f(\mathbf{X}_i, \beta^{(t)}) + \mathbf{J}_i(\beta^{(t)})(\beta - \beta^{(t)})$$

其中 $\mathbf{J}_i(\boldsymbol{\beta}) = \frac{\partial f(\mathbf{x}_i, \boldsymbol{\beta})}{\partial \boldsymbol{\beta}}$ 为雅可比矩阵。

迭代更新公式：

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + (\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top (\mathbf{Y} - \mathbf{f}(\boldsymbol{\beta}^{(t)}))$$

9.5.2 梯度下降法

基本思想：沿目标函数梯度反方向迭代更新参数。

更新公式：

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \eta \nabla S(\boldsymbol{\beta}^{(t)})$$

其中 η 为学习率， $\nabla S(\boldsymbol{\beta})$ 为梯度向量。

梯度计算：

$$\frac{\partial S}{\partial \beta_j} = -2 \sum_{i=1}^n [Y_i - f(\mathbf{x}_i, \boldsymbol{\beta})] \frac{\partial f(\mathbf{x}_i, \boldsymbol{\beta})}{\partial \beta_j}$$

变体：批量梯度下降：使用全部样本计算梯度 随机梯度下降：每次随机使用一个样本 小批量梯度下降：每次使用一小批样本

9.5.3 牛顿法与拟牛顿法

牛顿法：

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \mathbf{H}^{-1}(\boldsymbol{\beta}^{(t)}) \nabla S(\boldsymbol{\beta}^{(t)})$$

其中 $\mathbf{H}$ 为海森矩阵（二阶导数矩阵）。

拟牛顿法（BFGS）：用近似矩阵代替海森矩阵，减少计算量。

9.5.4 参数初始值与收敛准则

初始值选择：1. 基于物理意义或先验知识 2. 网格搜索法 3. 线性化模型的估计值

收敛准则：1. 参数变化量： $\|\boldsymbol{\beta}^{(t+1)} - \boldsymbol{\beta}^{(t)}\| < \epsilon$ 2. 目标函数变化量： $|S(\boldsymbol{\beta}^{(t+1)}) - S(\boldsymbol{\beta}^{(t)})| < \epsilon$ 3. 梯度范数： $\|\nabla S(\boldsymbol{\beta}^{(t)})\| < \epsilon$

9.6 模型比较与诊断

9.6.1 拟合优度量

残差平方和：

$$\text{RSS} = \sum_{i=1}^n [Y_i - \hat{f}(\mathbf{X}_i)]^2$$

调整后 R^2 ：

$$R_{\text{adj}}^2 = 1 - \frac{\text{RSS}/(n-p)}{\text{TSS}/(n-1)}$$

信息准则：

$$\text{AIC} = n \ln(\text{RSS}/n) + 2p$$

$$\text{BIC} = n \ln(\text{RSS}/n) + p \ln n$$

9.6.2 残差分析

标准化残差：

$$r_i = \frac{Y_i - \hat{f}(\mathbf{X}_i)}{\hat{\sigma}}$$

诊断图形：

- 残差与拟合值图：检查方差齐性
- 残差与自变量图：检查模型设定
- 正态 Q-Q 图：检查正态性假设

9.7 变量变换方法

9.7.1 Box-Cox 变换

Box-Cox 变换通过对响应变量进行幂变换，使其更接近正态分布并稳定方差。

变换族定义：

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln y & \lambda = 0 \end{cases}$$

参数选择：通过最大化轮廓似然函数选择 λ ：

$$L_{\max}(\lambda) = -\frac{n}{2} \ln(RSS(\lambda)/n) + (\lambda - 1) \sum_{i=1}^n \ln y_i$$

9.7.2 其他常用变换

对数变换：适用于右偏分布和方差随均值增大的情况：

$$y^* = \ln(y)$$

平方根变换：适用于泊松分布的计数数据：

$$y^* = \sqrt{y}$$

逻辑变换：适用于比例数据：

$$y^* = \ln\left(\frac{y}{1-y}\right)$$

9.8 案例分析

本章总结

非线性回归分析为我们打开了认识复杂世界关系的新窗口。通过本章的学习，我们掌握了从简单变换模型到复杂样条回归的完整方法体系，理解了如何根据数据特征和问题背景选择适当的非线性模型。

首先，对于可以通过数学变换线性化的模型（如指数、幂函数、对数模型），我们获得了简洁的解决方案。这类方法虽然实用，但需要警惕变换对误差结构和参数解释的影响。

多项式回归以其数学简洁性和高度灵活性成为非线性建模的基础工具。通过控制多项式阶数和使用正交多项式，我们可以在拟合能力与模型简洁性之间取得平衡。多元多项式回归进一步扩展了这一思想，能够捕捉变量间的交互效应和曲面关系。

样条回归代表了非线性建模的艺术高度。通过分段多项式和光滑性约束，样条模型既能捕捉局部特征，又能保证整体光滑，特别适用于没有明确函数形式但需要灵活拟合的情形。自然样条和光滑样条更是提供了边界稳定性和自动光滑参数选择的优雅解决方案。

在估计方法层面，我们认识到非线性模型参数估计的本质是数值优化问题。非线性最小二乘法、梯度下降法及其变体构成了求解这类问题的工具箱。每种方法都有其适用场景：高斯-牛顿法在接近

最优解时收敛迅速，梯度下降法对初始值不敏感但可能收敛较慢，而拟牛顿法则在计算效率与收敛性之间取得了良好平衡。

最后，模型比较与诊断提醒我们，无论模型多么复杂，都需要严格的统计验证。残差分析、交叉验证、信息准则等工具帮助我们避免过拟合，确保模型的稳健性和泛化能力。

非线性回归不仅是一套统计方法，更是一种建模哲学：它教导我们在尊重数据复杂性的同时，保持模型的简约性；在追求拟合优度的同时，警惕过度参数化的陷阱。掌握这些方法后，读者将能够更自信地面对现实世界中的曲线关系，从数据中提取更深刻的洞察。

核心公式回顾

- 1. 多项式回归: $y_i = \beta_0 + \beta_1x_i + \beta_2x_i^2 + \cdots + \beta_dx_i^d + \varepsilon_i$
- 2. 正交多项式条件: $\sum_{i=1}^n Q_j(x_i)Q_k(x_i) = 0 \ (j \neq k)$
- 3. Box-Cox 变换: $y^{(\lambda)} = \frac{y^{\lambda}-1}{\lambda} \ (\lambda \neq 0)$
- 4. 样条光滑条件: $f_j^{(m)}(\xi_j) = f_{j+1}^{(m)}(\xi_j) \ (m = 0, 1, 2)$
- 5. 平滑样条目标函数: $\min_f \{ \sum (y_i - f(x_i))^2 + \lambda \int [f''(x)]^2 dx \}$

方法比较与选择指南

方法类型	适用场景	优势	局限性
多项式回归	光滑的全局趋势	简单直观，易于解释	不够灵活，外推性能差
正交多项式	高阶多项式拟合	数值稳定，系数独立	需要专门的基函数构造
样条回归	局部变化复杂	灵活性高，适应性强	需要选择节点位置
平滑样条	自动光滑度控制	无需选择节点，理论性质好	计算量较大，解释性稍差

重要概念体系

- 1. 基函数展开：用基函数的线性组合逼近复杂函数关系
- 2. 正交性原理：通过正交化改善数值稳定性和统计性质
- 3. 分段多项式：在不同区间使用不同的多项式逼近
- 4. 光滑性条件：在分段连接处施加连续性约束
- 5. 惩罚回归：通过惩罚项控制模型复杂度
- 6. 偏差-方差权衡：在模型复杂度和预测精度间寻求平衡

实践指导原则

- 1. 从简到繁：先尝试低阶多项式，再考虑更复杂的方法
- 2. 模型验证：使用交叉验证评估模型泛化能力

3. 数值稳定性：高阶多项式务必使用正交多项式形式
4. 领域知识：结合实际问题的背景选择合适的方法
5. 可视化诊断：通过残差图和拟合曲线评估模型 adequacy

多项式回归和样条回归为处理非线性关系提供了系统的框架，它们在线性模型的基础上通过基函数扩展来捕捉复杂的数据模式。理解这些方法的原理和适用条件，对于建立有效的预测模型具有重要意义。

III 进阶——超越线性的机器学习

在前面我们介绍了线性分析方法，包括回归分析和线性分类模型。本章我们将扩展到更复杂的机器学习，涵盖以下内容：- 决策树 - 集成方法（随机森林和梯度提升）- 支持向量机 - 神经网络 - 深度学习

这些方法在处理非线性关系和高维数据时表现出色，广泛应用于分类和回归任务。我们将通过理论讲解和实际案例，帮助读者理解这些技术的基本原理和应用场景。此外，我们还将探讨模型评估与选择的方法，如交叉验证、超参数调优等，确保读者能够构建出性能优良的机器学习模型。通过本章的学习，读者将掌握现代机器学习的核心技术，为后续更深入的研究打下坚实基础。本章内容安排如下：1. 决策树基础 2. 集成方法详解 3. 支持向量机原理 4. 神经网络与深度学习简介

10 决策树与集成学习

本章导读

在前面的学习中，我们分别建立了回归分析框架（针对连续型响应变量）和分类分析框架（针对类别型响应变量）。决策树方法为这两类问题提供了统一的建模视角，它通过直观的树状结构将复杂的预测问题分解为一系列简单的决策规则。集成学习则进一步通过模型组合的策略，显著提升了单一模型的预测性能。本章将从回归和分类的双重角度，系统阐述决策树方法的理论基础和实际应用。

10.1 决策树的基本框架

10.1.1 统一视角下的树模型

决策树的核心思想是通过递归地划分特征空间来构建预测模型，这种思想既适用于回归问题，也适用于分类问题。

树结构的通用组成：- 根节点：包含完整的训练数据集 - 内部节点：基于某个特征的决策规则 - 分支：决策规则的可能结果 - 叶节点：最终的预测输出

关键区别：- 回归树：叶节点输出连续数值预测 - 分类树：叶节点输出类别标签或概率

10.1.2 特征空间的划分策略

决策树通过轴平行分割将特征空间划分为矩形区域，每个区域对应一个叶节点。

数学表述：对于 p 维特征空间，决策树构建了一个划分：

$$R_1, R_2, \dots, R_M \quad \text{满足} \quad \bigcup_{m=1}^M R_m = \mathbb{R}^p$$

其中每个区域 R_m 对应树的一个叶节点。

10.2 回归树：连续响应的建模

10.2.1 回归树的预测机制

叶节点预测值：对于落入区域 R_m 的观测，回归树给出该区域内所有训练样本响应值的均值作为预测：

$$\hat{y}_{R_m} = \frac{1}{N_m} \sum_{x_i \in R_m} y_i = \bar{y}_{R_m}$$

其中 N_m 是区域 R_m 中包含的训练样本数。

模型表示：回归树模型可以表示为：

$$f(x) = \sum_{m=1}^M \bar{y}_{R_m} \cdot I(x \in R_m)$$

10.2.2 回归树的分裂准则

目标函数：选择分裂特征 j 和分裂点 s 来最小化加权平方误差：

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right]$$

其中：- $R_1(j, s) = \{X | X_j \leq s\}$ （左子节点）- $R_2(j, s) = \{X | X_j > s\}$ （右子节点）- c_1, c_2 分别是左右子节点的最优预测值（即样本均值）

分裂增益：分裂带来的平方误差减少量为：

$$\Delta SSE = SSE_{\text{parent}} - (SSE_{\text{left}} + SSE_{\text{right}})$$

其中 $SSE = \sum (y_i - \bar{y})^2$

10.2.3 回归树的复杂度控制

预剪枝策略：- 最大树深度 - 叶节点最小样本数 - 分裂所需最小改进量

后剪枝策略：代价复杂度剪枝，最小化：

$$C_{\alpha}(T) = \sum_{m=1}^{|T|} N_m \cdot MSE(R_m) + \alpha|T|$$

其中 $MSE(R_m) = \frac{1}{N_m} \sum_{x_i \in R_m} (y_i - \bar{y}_{R_m})^2$

10.3 分类树：类别响应的建模

10.3.1 分类树的预测机制

叶节点预测：对于落入区域 R_m 的观测，分类树给出该区域内最频繁类别：

$$\hat{y}_{R_m} = \arg \max_k \left(\frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k) \right)$$

或者输出类别概率：

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k)$$

10.3.2 分类树的不纯度度量

不纯度度量用于评估节点内类别的混杂程度，理想的不纯度函数应满足：1. 在均匀分布时取得最大值 2. 在纯节点时取得最小值（0）3. 是对称的凹函数

Gini 不纯度：

$$\text{Gini}(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

- 直观解释：从节点中随机抽取两个样本，它们属于不同类别的概率
- 取值范围：[0, 1-1/K]
- 对类别分布变化敏感

信息熵：

$$\text{Entropy}(p) = - \sum_{k=1}^K p_k \log p_k$$

- 基于信息论的概念
- 在机器学习中广泛使用
- 计算相对复杂

误分类误差：

$$\text{Error}(p) = 1 - \max_k p_k$$

- 直观但不够敏感
- 不适合作为分裂准则，常用于剪枝

10.3.3 分类树的分裂准则

不纯度减少量：选择使子节点不纯度加权和减少最多的分裂：

$$\Delta I = I(\text{parent}) - \left(\frac{N_{\text{left}}}{N} I(\text{left}) + \frac{N_{\text{right}}}{N} I(\text{right}) \right)$$

其中 $I(\cdot)$ 是选择的不纯度函数。

信息增益（使用熵不纯度时）：

$$\text{Information Gain} = \text{Entropy}(p) - \sum_{j=1}^2 \frac{N_j}{N} \text{Entropy}(p_j)$$

10.4 树模型的比较与诊断

10.4.1 回归树与分类树的统一视角

共同特点：1. 非参数方法：不对数据分布做强假设 2. 特征选择：自动选择重要特征 3. 处理混合类型数据：能同时处理数值型和类别型特征 4. 稳健性：对异常值和缺失值相对稳健

主要差异：

方面	回归树	分类树
响应变量	连续型	类别型
叶节点预测	均值	众数或概率
分裂准则	平方误差减少	不纯度减少
模型评估	MSE, R^2	准确率, AUC
复杂度控制	基于 MSE 的代价复杂度	基于不纯度的代价复杂度

10.4.2 树模型的优势与局限

主要优势：1. 直观易懂：决策规则可解释性强 2. 无需预处理：对特征缩放不敏感 3. 处理非线性：自动捕捉变量间的交互效应 4. 变量重要性：提供特征重要性的自然度量

主要局限：1. 不稳定性：数据微小变化可能导致树结构巨大改变 2. 贪婪性：每一步选择局部最优，未必全局最优 3. 预测面不光滑：分段常数预测 4. 外推能力差：无法预测训练数据范围外的值

10.5 集成学习理论基础

10.5.1 集成学习的统计原理

集成学习通过组合多个基学习器来改善预测性能，其理论基础可以从偏差-方差分解的角度理解。

回归问题的偏差-方差分解：

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2(\hat{f}(x)) + \text{Var}(\hat{f}(x)) + \sigma_\epsilon^2$$

分类问题的误差分解：虽然分类问题没有精确的偏差-方差分解，但类似概念仍然适用：- 偏差：模型系统性错误 - 方差：模型对训练数据变化的敏感性

10.5.2 集成方法的分类

根据基学习器的生成方式和组合策略，集成方法主要分为：

1. 并行方法：基学习器独立生成，如 Bagging、随机森林
2. 序列方法：基学习器顺序生成，如 Boosting
3. 异质集成：组合不同类型的学习器，如 Stacking

10.6 Bagging 与随机森林

10.6.1 Bagging 的回归应用

Bootstrap 采样：从训练集中有放回地抽取 B 个自助样本，每个样本大小与原始训练集相同。

回归预测聚合：

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

方差减少机制：假设基学习器方差为 σ^2 ，相关系数为 ρ ，则 Bagging 集成方差为：

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

10.6.2 Bagging 的分类应用

多数投票：

$$\hat{y}_{\text{bag}}(x) = \arg \max_k \sum_{b=1}^B I(\hat{f}^{*b}(x) = k)$$

概率平均：

$$\hat{p}_k(x) = \frac{1}{B} \sum_{b=1}^B \hat{p}_k^{*b}(x)$$

10.6.3 随机森林的改进

随机森林在 Bagging 的基础上引入特征随机选择，进一步降低基学习器间的相关性。

特征采样策略：- 分类问题： $m \approx \sqrt{p}$ - 回归问题： $m \approx p/3$

特征重要性：基于不纯度减少的平均值：

$$\text{Importance}(X_j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b} \Delta I(t, X_j)$$

10.7 Boosting 方法

10.7.1 回归问题的梯度提升

加性模型：

$$f_M(x) = \sum_{m=1}^M \beta_m h_m(x)$$

梯度下降视角：在函数空间中进行梯度下降，每一步拟合当前模型的负梯度：

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}$$

对于平方损失：

$$r_{im} = y_i - f_{m-1}(x_i)$$

算法步骤：1. 初始化 $f_0(x) = \bar{y}$ 2. 对于 $m = 1$ 到 M ： - 计算伪残差 $r_{im} = y_i - f_{m-1}(x_i)$ - 用决策树拟合 $\{(x_i, r_{im})\}$ 得到 $h_m(x)$ - 更新 $f_m(x) = f_{m-1}(x) + \nu \cdot h_m(x)$

其中 ν 是学习率。

10.7.2 分类问题的 AdaBoost

AdaBoost 通过调整样本权重来关注难以分类的样本。

指数损失函数：

$$L(y, f(x)) = \exp(-yf(x))$$

其中 $y \in \{-1, +1\}$

算法流程：1. 初始化权重 $w_i = 1/N$ 2. 对于 $m = 1$ 到 M ： - 训练弱分类器 $G_m(x)$ 最小化加权误差 - 计算误差率 $\text{err}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}$ - 计算分类器权重 $\alpha_m = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$ - 更新样本权重 $w_i \leftarrow w_i \exp(-\alpha_m y_i G_m(x_i))$ - 权重归一化 3. 输出最终分类器 $\text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$

10.10 集成学习的高级应用

10.10.1 回归与分类的性能差异

回归问题的集成：

- - 主要降低方差
- - 对基学习器的准确性要求相对较低
- - 容易实现预测区间估计

分类问题的集成：

- - 同时影响偏差和方差
- - 对基学习器的多样性要求更高
- - 类别不平衡时需要特殊处理

10.10.2 模型解释性工具

特征重要性：无论回归还是分类，都可以通过特征在集成中的总不纯度减少来评估重要性。

部分依赖图：显示某个特征对预测值的边际效应，适用于回归和概率预测。

SHAP 值：统一框架下的特征贡献度分析，为每个预测提供可解释的分解。

10.11 案例分析

本章总结

核心公式回顾

1. 回归树预测: $\hat{y}_{R_m} = \frac{1}{N_m} \sum_{x_i \in R_m} y_i$
2. 分类树 Gini 不纯度: $\text{Gini}(p) = 1 - \sum_{k=1}^K p_k^2$
3. Bagging 回归: $\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$
4. 梯度提升伪残差: $r_{im} = y_i - f_{m-1}(x_i)$
5. AdaBoost 分类器权重: $\alpha_m = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$

回归与分类的对比总结

方面	回归树	分类树
响应变量	连续数值	离散类别
预测输出	叶节点均值	叶节点众数或概率
损失函数	平方误差	Gini/熵不纯度
集成目标	主要降低方差	平衡偏差与方差
评估指标	MSE, RMSE, R ²	准确率, 精确率, 召回率

方法选择指南

问题类型	推荐方法	关键考虑
需要强解释性	单棵决策树	深度限制, 剪枝强度
回归预测精度	梯度提升树	学习率, 树深度, 迭代次数
分类平衡数据	随机森林	树数量, 特征采样比例
类别不平衡	AdaBoost	关注困难样本的能力
计算效率优先	Bagging	可并行化, 内存需求

重要概念体系

1. 递归划分：通过特征测试构建层次决策结构
2. 不纯度度量：量化节点内响应变量的混杂程度
3. 模型集成：通过组合多个基学习器提升预测性能
4. 偏差-方差权衡：在不同集成方法中的差异化表现
5. 特征重要性：基于树模型的特征贡献度评估
6. 稳健预测：通过模型平均降低方差和过拟合风险

决策树和集成学习方法为回归和分类问题提供了强大而灵活的解决方案。从单一树的直观解释到集成方法的高精度预测，这一方法体系涵盖了从探索性分析到生产部署的全流程需求。理解这些方法在回归和分类背景下的共性与差异，有助于在实际问题中选择最适合的技术路线。

11 支持向量机

本章导读

支持向量机（Support Vector Machine, SVM）是一类强大的监督学习算法，在线性分类和回归问题中表现出色。本章将从几何直观和数学优化两个角度，系统介绍 SVM 的基本原理、核方法以及在实际问题中的应用。SVM 的核心思想是通过寻找最优分离超平面来实现分类，并通过核技巧处理非线性问题。

11.1 线性支持向量分类器

11.1.1 最大间隔分类器

基本概念：对于线性可分的二分类问题，存在无数个超平面可以将两类样本分开。最大间隔分类器选择那个具有最大间隔的超平面。

数学表述：考虑训练数据 $\{(x_i, y_i)\}_{i=1}^n$ ，其中 $y_i \in \{-1, +1\}$ 。分离超平面可以表示为：

$$w^T x + b = 0$$

函数间隔：样本点 (x_i, y_i) 到超平面的函数间隔定义为：

$$\hat{\gamma}_i = y_i(w^T x_i + b)$$

几何间隔：几何间隔是函数间隔除以权重向量的范数：

$$\gamma_i = \frac{y_i(w^T x_i + b)}{\|w\|} = \frac{\hat{\gamma}_i}{\|w\|}$$

11.1.2 优化问题 formulation

最大间隔优化问题：寻找超平面使得最小几何间隔最大化：

$$\begin{aligned} & \max_{w,b} \gamma \\ & \text{满足} \frac{y_i(w^T x_i + b)}{\|w\|} \geq \gamma, \quad i = 1, \dots, n \end{aligned}$$

等价形式：通过缩放约束，可以得到等价的凸优化问题：

$$\begin{aligned} & \min_{w,b} \frac{1}{2} \|w\|^2 \\ & \text{满足} y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n \end{aligned}$$

11.1.3 支持向量的概念

支持向量：是那些恰好满足 $y_i(w^T x_i + b) = 1$ 的样本点，它们定义了间隔的边界，并完全决定了最优超平面。

关键性质：- 支持向量是距离分离超平面最近的样本点 - 移除非支持向量不会影响模型 - 支持向量的数量通常很少，使得 SVM 具有稀疏性

11.2 软间隔支持向量机

11.2.1 线性不可分情况

在实际问题中，数据往往不是线性可分的。软间隔 SVM 通过引入松弛变量来处理这种情况。

松弛变量：对于每个样本引入 $\xi_i \geq 0$ ，允许某些样本违反间隔约束：

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, n$$

优化问题：

$$\begin{aligned} & \min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ & \text{满足} y_i(w^T x_i + b) \geq 1 - \xi_i, \quad i = 1, \dots, n \\ & \xi_i \geq 0, \quad i = 1, \dots, n \end{aligned}$$

11.2.2 参数 C 的解释

正则化参数：- C 控制间隔宽度与分类错误之间的权衡 - $C \rightarrow \infty$ ：硬间隔 SVM，不允许任何分类错误 - $C \rightarrow 0$ ：最大化间隔，容忍更多分类错误

实际选择： C 通常通过交叉验证选择，平衡模型的复杂度和训练误差。

11.2.3 损失函数视角

合页损失：软间隔 SVM 等价于最小化合页损失：

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b))$$

与其他方法的比较：- 合页损失对正确分类且置信度高的样本损失为 0 - 与逻辑回归的交叉熵损失形成对比

11.3 对偶问题与核方法

11.3.1 拉格朗日对偶

原始问题：

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

满足 $y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$

拉格朗日函数：

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i [y_i(w^T x_i + b) - 1 + \xi_i] - \sum_{i=1}^n \mu_i \xi_i$$

对偶问题：

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

满足 $0 \leq \alpha_i \leq C, \quad i = 1, \dots, n$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

11.3.2 核技巧

特征映射：通过非线性映射 $\phi: \mathcal{X} \rightarrow \mathcal{H}$ 将输入空间映射到高维特征空间。

核函数：核函数 $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ 计算特征空间中的内积，而无需显式计算特征映射。

对偶问题的核化形式：

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

满足 $0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0$

11.3.3 常用核函数

线性核：

$$K(x_i, x_j) = x_i^T x_j$$

多项式核：

$$K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$$

高斯径向基核：

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Sigmoid 核：

$$K(x_i, x_j) = \tanh(\gamma x_i^T x_j + r)$$

11.4 支持向量回归

11.4.1 ϵ -不敏感损失

回归问题设定：对于回归问题 $y_i \in \mathbb{R}$ ，SVM 的扩展称为支持向量回归。

ϵ -不敏感损失：

$$L_{\epsilon}(y, f(x)) = \begin{cases} 0 & \text{if } |y - f(x)| \leq \epsilon \\ |y - f(x)| - \epsilon & \text{otherwise} \end{cases}$$

11.4.2 SVR 优化问题

原始问题：

$$\begin{aligned} \min_{w, b, \xi, \xi^*} & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \\ \text{满足} & y_i - (w^T x_i + b) \leq \epsilon + \xi_i \\ & (w^T x_i + b) - y_i \leq \epsilon + \xi_i^* \\ & \xi_i, \xi_i^* \geq 0, \quad i = 1, \dots, n \end{aligned}$$

对偶问题：

$$\begin{aligned} \max_{\alpha, \alpha^*} & -\epsilon \sum_{i=1}^n (\alpha_i + \alpha_i^*) + \sum_{i=1}^n y_i (\alpha_i - \alpha_i^*) \\ & - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) K(x_i, x_j) \\ \text{满足} & 0 \leq \alpha_i, \alpha_i^* \leq C, \quad \sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0 \end{aligned}$$

11.4.3 预测函数

SVR 预测：

$$f(x) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(x_i, x) + b$$

支持向量：只有那些满足 $|\alpha_i - \alpha_i^*| > 0$ 的样本才是支持向量。

11.5 模型选择与评估

11.5.1 超参数调优

关键超参数：- 正则化参数 C - 核参数（如 RBF 核的 γ ）- ϵ （对于 SVR）

网格搜索：在超参数空间中进行系统搜索，使用交叉验证评估性能。

交叉验证策略：- k -折交叉验证 - 分层交叉验证（用于分类）- 时间序列交叉验证（用于时间相关数据）

11.5.2 核函数选择

核选择指南：

数据类型	推荐核函数	理由
线性可分	线性核	简单，计算高效
文本数据	线性核	高维稀疏数据
非线性问题	RBF 核	通用性强，适应各种模式
先验知识	自定义核	利用领域特定知识

11.5.3 模型评估指标

分类问题：- 准确率、精确率、召回率、F1 分数 - ROC 曲线和 AUC - 混淆矩阵
回归问题：- 均方误差（MSE）- 平均绝对误差（MAE）- R^2 决定系数

11.6 SVM 的扩展变体

11.6.1 多类 SVM

一对多方法：为每个类别训练一个二分类 SVM，将该类与其他所有类别区分。
一对一方法：为每对类别训练一个二分类 SVM，然后通过投票决定最终类别。
多类 SVM 的统一公式：直接扩展 SVM 到多类情况，使用单一优化问题。

11.6.2 概率输出

Platt 缩放：将 SVM 输出通过 sigmoid 函数转换为概率估计：

$$P(y = 1|x) = \frac{1}{1 + \exp(Af(x) + B)}$$

其中 A, B 通过最大似然估计得到。

11.6.3 大规模 SVM

挑战：标准 SVM 算法的时间复杂度为 $O(n^3)$ ，空间复杂度为 $O(n^2)$ ，难以处理大规模数据。

解决方案：- 序列最小优化（SMO）算法 - 随机梯度下降 - 近似核方法

11.7 SVM 与其他方法的比较

11.7.1 与逻辑回归的比较

相似点：- 都是线性分类器 - 都可以通过核技巧处理非线性问题 - 都有正则化项控制模型复杂度

不同点：- 损失函数：合页损失 vs 交叉熵损失 - 输出：决策函数 vs 概率估计 - 支持向量：稀疏解 vs 稠密解

11.7.2 与神经网络的比较

优势：- 理论基础坚实 - 全局最优解 - 对高维数据有效

劣势：- 核函数选择困难 - 大规模数据计算成本高 - 特征工程相对重要

11.8 案例分析

本章总结

核心公式回顾

- 1. 最大间隔优化： $\min_{w,b} \frac{1}{2} \|w\|^2$ s.t. $y_i(w^T x_i + b) \geq 1$
- 2. 软间隔 SVM： $\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$
- 3. 对偶问题： $\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
- 4. RBF 核： $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- 5. SVR 预测： $f(x) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(x_i, x) + b$

方法选择指南

问题类型	推荐方法	关键参数	注意事项
线性可分分类	硬间隔 SVM	无	数据需严格线性可分
一般分类	软间隔 SVM	C , 核参数	通过交叉验证选择参数
非线性分类	核 SVM	C, γ	小心过拟合
回归问题	SVR	C, ϵ, γ	ϵ 控制预测精度

问题类型	推荐方法	关键参数	注意事项
大规模数据	线性 SVM	C	使用随机梯度下降

重要概念体系

- 1. 最大间隔原理：基于几何间隔最大化的分类准则
- 2. 支持向量：决定分类边界的关键样本点
- 3. 核技巧：通过核函数隐式实现非线性映射
- 4. 对偶理论：将原问题转化为更易求解的对偶问题
- 5. 软间隔：通过松弛变量处理线性不可分情况
- 6. ϵ -不敏感损失：SVR 中使用的稳健损失函数

实践指导原则

- 1. 数据预处理：SVM 对特征缩放敏感，建议标准化数据
- 2. 核选择：从线性核开始，必要时升级到 RBF 核
- 3. 参数调优：使用网格搜索和交叉验证系统优化超参数
- 4. 模型解释：线性 SVM 的权重向量提供特征重要性
- 5. 计算考虑：对于大规模数据，考虑线性 SVM 或近似方法

支持向量机提供了坚实的理论基础和优秀的实践性能，特别适合中小规模的高维数据问题。理解 SVM 的几何直观和优化理论，有助于在实际应用中更好地使用和解释这一强大工具。

12 神经网络与深度学习基础

本章导读

神经网络是预测建模领域的重要方法，它通过模拟人脑神经元的工作方式，为回归和分类问题提供了统一的解决方案。本章将从预测建模的本质出发，介绍神经网络的基本原理、网络设计、训练方法，并简要探讨深度学习的概念，帮助理解这一强大工具在回归和分类任务中的应用。

12.1 神经网络：回归与分类的统一框架

12.1.1 从线性模型到神经网络

线性模型的回顾：- 线性回归： $y = w^T x + b$ （连续输出）- 逻辑回归： $P(y = 1|x) = \sigma(w^T x + b)$ （概率输出）

神经网络的扩展思路：单一神经元可以看作一个广义线性模型，通过组合多个神经元并引入非线性激活函数，神经网络能够学习更复杂的模式。

基本神经元模型：

$$z = w^T x + b \quad (\text{线性组合})$$

$$a = \sigma(z) \quad (\text{非线性变换})$$

12.1.2 为什么需要神经网络？

传统方法的局限性：- 线性模型只能捕捉线性关系 - 多项式回归需要手动设计特征组合 - 对复杂非线性模式建模能力有限

神经网络的优势：- 自动学习特征组合 - 通过层次化结构学习抽象特征 - 统一处理回归和分类问题

12.2 网络架构设计

12.2.1 输出层设计

回归问题的输出层：- 神经元数量：1 个（单输出）或对应输出维度 - 激活函数：恒等函数（无激活） - 输出解释：连续数值预测

二分类问题的输出层：- 神经元数量：1 个 - 激活函数：sigmoid - 输出解释：属于正类的概率

多分类问题的输出层：- 神经元数量：类别数 K - 激活函数：softmax - 输出解释：每个类别的概率分布

12.2.2 隐藏层设计

隐藏层的作用：- 学习输入特征的中间表示 - 通过非线性变换增强模型表达能力 - 自动进行特征工程

网络深度与宽度：- 浅层网络：1-2 个隐藏层，适合简单问题 - 深层网络：多个隐藏层，适合复杂模式 - 宽度：每层神经元数，影响模型容量

12.3 激活函数与损失函数

12.3.1 常用激活函数

Sigmoid 函数：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- 将输出压缩到 (0,1) - 适合输出层，直观的概率解释 - 梯度消失问题

ReLU 函数：

$$\text{ReLU}(z) = \max(0, z)$$

- 计算简单，训练高效 - 缓解梯度消失 - 现代神经网络的首选

Tanh 函数：

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- 输出范围 (-1,1)，零中心化 - 比 sigmoid 有更好的梯度特性

12.3.2 损失函数选择

回归任务：均方误差损失：

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

二分类任务：交叉熵损失：

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

多分类任务：多类交叉熵损失：

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$$

12.4 神经网络训练

12.4.1 前向传播

计算过程：从输入层开始，逐层计算直到输出层：

$$\begin{aligned} z^{[l]} &= W^{[l]} a^{[l-1]} + b^{[l]} \\ a^{[l]} &= \sigma^{[l]}(z^{[l]}) \end{aligned}$$

预测输出：最终层的激活值即为模型预测。

12.4.2 反向传播

梯度计算：利用链式法则计算损失函数对每个参数的梯度：

$$\frac{\partial L}{\partial W^{[l]}} = \frac{\partial L}{\partial z^{[l]}} (a^{[l-1]})^T$$

参数更新：使用梯度下降算法更新参数：

$$W^{[l]} := W^{[l]} - \alpha \frac{\partial L}{\partial W^{[l]}}$$

12.4.3 优化算法

批量梯度下降：使用全部训练数据计算梯度，收敛稳定但计算量大。

小批量梯度下降：折中方案，兼顾收敛速度和稳定性。

Adam 优化器：自适应学习率，结合动量项，实践中效果良好。

12.5 过拟合与正则化

12.5.1 过拟合问题

神经网络的特点：- 参数众多，模型容量大 - 容易过拟合训练数据 - 需要正则化技术控制复杂度

检测方法：- 训练误差持续下降，验证误差开始上升 - 学习曲线分析 - 早停法

12.5.2 正则化技术

L2 正则化：在损失函数中加入权重惩罚：

$$L_{\text{reg}} = L + \frac{\lambda}{2} \sum \|W\|^2$$

Dropout：训练时随机丢弃部分神经元，提高泛化能力。

数据增强：通过变换生成更多训练样本。

12.6 深度学习简介

深度学习模型可以形式化地定义为：

$$f(\mathbf{x}; \theta) = f_L(f_{L-1}(\cdots f_1(\mathbf{x}; \theta_1) \cdots; \theta_{L-1}); \theta_L)$$

其中：- $\mathbf{x}$ ：输入向量 - $\theta = \{\theta_1, \theta_2, \dots, \theta_L\}$ ：模型参数 - f_l ：第 l 层的变换函数 - L ：网络总层数（深度）

与浅层学习的对比

特征	浅层学习	深度学习
层数	1-2 层隐藏层	多层（通常 >3 层）
特征工程	需要手动设计	自动学习特征
数据需求	相对较少	大量数据
计算需求	相对较低	高计算需求
可解释性	较好	较差（黑箱问题）

12.6.2 卷积神经网络（CNN）

CNN 的基本思想

卷积神经网络专门处理具有网格结构的数据（如图像），通过局部连接和权值共享大幅减少参数数量。

卷积操作

二维离散卷积：

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n)K(m, n)$$

其中 I 为输入图像， K 为卷积核。

卷积层特性

- 1. 局部连接：每个神经元只连接输入图像的局部区域
- 2. 权值共享：同一卷积核在输入的不同位置使用相同权重
- 3. 平移不变性：能够检测输入任何位置的特征

CNN 的基本组件

卷积层

输出特征图大小计算：

$$\begin{aligned} \text{输出高度} &= \frac{H - F_H + 2P}{S} + 1 \\ \text{输出宽度} &= \frac{W - F_W + 2P}{S} + 1 \end{aligned}$$

其中：- H, W ：输入高度和宽度 - F_H, F_W ：卷积核高度和宽度 - P ：填充（padding）大小 - S ：步长（stride）

池化层

1. 最大池化:

$$\text{output} = \max(\text{window})$$

2. 平均池化:

$$\text{output} = \text{mean}(\text{window})$$

池化作用: 降维、减少过拟合、增加平移不变性。

全连接层

在 CNN 末端, 将特征图展平后连接全连接层进行分类。

经典 CNN 架构

LeNet-5 (1998)

- 输入: 32×32 灰度图像
- 结构: 2 个卷积层 + 2 个池化层 + 3 个全连接层
- 应用: 手写数字识别

AlexNet (2012)

- 创新: ReLU 激活函数、Dropout、数据增强
- 结构: 5 个卷积层 + 3 个池化层 + 3 个全连接层
- 成就: ImageNet 竞赛冠军

VGGNet (2014)

- 特点: 使用小卷积核 (3×3), 增加网络深度
- VGG16: 13 个卷积层 + 3 个全连接层
- VGG19: 16 个卷积层 + 3 个全连接层

ResNet (2015)

- 创新: 残差连接, 解决梯度消失问题
- 残差块:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

- 深度: ResNet-50, ResNet-101, ResNet-152
-

12.6.3 循环神经网络（RNN）

序列数据处理

循环神经网络用于处理序列数据，具有时间维度上的记忆能力。

RNN 基本结构

$$\mathbf{h}_t = \phi(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

$$\mathbf{y}_t = \psi(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)$$

其中：- $\mathbf{h}_t$ ：时刻 t 的隐藏状态 - $\mathbf{x}_t$ ：时刻 t 的输入 - $\mathbf{y}_t$ ：时刻 t 的输出

不同类型的 RNN

1. 一对一：标准神经网络
2. 一对多：图像描述生成
3. 多对一：情感分析
4. 多对多（等长）：词性标注
5. 多对多（不等长）：机器翻译

长短期记忆网络（LSTM）

LSTM 通过门控机制解决长期依赖问题。

LSTM 单元结构

1. 遗忘门：

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

2. 输入门：

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

3. 细胞状态更新：

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

4. 输出门：

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

门控循环单元（GRU）

GRU 是 LSTM 的简化版本，合并遗忘门和输入门：

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \end{aligned}$$

12.6.5 注意力机制

缩放点积注意力

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

其中：- Q ：查询矩阵 - K ：键矩阵 - V ：值矩阵 - d_k ：键的维度

多头注意力

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

Transformer 架构

编码器-解码器结构

编码器（ N 层）：1. 多头自注意力 2. 前馈神经网络 3. 残差连接 + 层归一化

解码器（ N 层）：1. 掩码多头自注意力（防止看到未来信息）2. 多头编码器-解码器注意力 3. 前馈神经网络 4. 残差连接 + 层归一化

位置编码

为序列添加位置信息：

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d}) \end{aligned}$$

BERT 与 GPT

BERT（双向编码器表示）

- 预训练任务：掩码语言模型 + 下一句预测
- 特点：双向上下文理解
- 应用：文本分类、问答、命名实体识别

GPT（生成式预训练 Transformer）

- 架构：仅解码器的 Transformer
- 预训练：自回归语言建模
- 特点：强大的生成能力

12.7 案例分析

本章总结

核心概念回顾

1. 统一框架：神经网络为回归和分类提供统一建模框架
2. 架构设计：根据问题类型设计输出层，回归用线性输出，分类用 sigmoid/softmax 输出
3. 激活函数：ReLU 适合隐藏层，根据任务选择输出层激活函数
4. 训练原理：前向传播计算预测，反向传播计算梯度，梯度下降更新参数
5. 正则化：使用 L2 正则化、Dropout 等技术防止过拟合

实践应用指南

数据预处理要求：- 特征标准化：加速收敛，提高数值稳定性 - 数据量要求：神经网络通常需要较多训练数据 - 计算资源：深层网络需要较强的计算能力

与传统方法的选择：

考虑因素	选择传统方法	选择神经网络
数据量	小样本	大样本
问题复杂度	简单线性关系	复杂非线性关系
可解释性	要求高	要求低
计算资源	有限	充足
特征工程	手动设计	自动学习

深度学习扩展

成功应用领域：- 计算机视觉：图像分类、目标检测 - 自然语言处理：文本分类、机器翻译 - 语音识别：语音转文本、声纹识别 - 推荐系统：个性化推荐

实践建议：1. 从基础开始：先掌握浅层神经网络，再学习深度学习 2. 理解原理：不仅要会使用，更要理解背后的数学原理 3. 项目驱动：通过实际项目加深理解 4. 持续学习：深度学习领域发展迅速，需要持续跟进

与前面章节的联系

神经网络不是孤立的方法，而是前面学习内容的自然延伸：

- 单层神经网络 = 广义线性模型
- 多层神经网络 = 多个广义线性模型的堆叠 + 非线性变换
- 深度学习 = 更深层次的特征学习

通过本教材的学习，希望读者能够建立从传统统计方法到现代机器学习的完整知识体系，在实际问题中选择合适的建模方法。

全书总结：从线性回归到神经网络，我们建立了一套完整的预测建模方法体系。每种方法都有其适用场景和局限性，在实际工作中需要根据具体问题选择合适的方法，理解其假设和限制，才能构建出有效的预测模型。

IV 实践——综合案例分析

在前面的章节中，我们介绍了各种回归和分类模型，但是具体的案例分析还没有涉及。接下来的章节将通过一些实际案例来展示如何应用这些模型解决现实中的问题。这些案例将涵盖不同领域，如医疗、金融和市场营销等，帮助读者更好地理解模型的实际应用。

在每个案例中，我们将详细介绍数据的收集与预处理过程，模型的选择与训练方法，以及结果的评估与解释。通过这些实际案例，读者将能够更好地掌握如何将理论知识应用到实践中，从而提升自己的数据分析和建模能力。

特征工程：在实际应用中，数据通常需要经过清洗和转换才能用于建模。特征工程包括处理缺失值、异常值检测、特征缩放和编码等步骤。通过合理的特征工程，可以显著提升模型的性能。**模型选择与调优：**不同的问题可能适合不同的模型。我们将介绍如何根据数据的特点选择合适的模型，并通过交叉验证和超参数调优等方法优化模型性能。**结果解释与可视化：**模型训练完成后，理解和解释模型的结果同样重要。我们将介绍一些常用的结果可视化技术，帮助读者更好地理解模型的预测结果和特征的重要性。通过这些实际案例的学习，读者将能够更好地理解如何将回归和分类模型应用到现实问题中，从而提升自己的数据分析和建模能力。

特征工程是机器学习中的核心环节，它通过对原始数据进行转换、组合和提取，构建更能代表预测任务的特征，从而提升模型性能。其内容广泛，可分为以下几个主要方面：

一、数据预处理这是特征工程的基础，旨在处理原始数据中的问题，使其适合建模。1. 缺失值处理：删除（缺失过多的行/列）、填充（均值/中位数/众数、模型预测填充等）。2. 异常值处理：基于统计（如 3σ 原则、IQR 法）、可视化检测，并进行修正、缩尾或删除。3. 数据类型转换：将分类变量编码、将数值变量分箱（离散化）、将文本/日期等转换为模型可处理的格式。

二、特征构造从现有数据中创造新的、更有信息量的特征。1. 领域知识驱动：例如在电商中，从“购买金额”和“购买次数”构造“客单价”；在时间序列中，提取“星期几”、“是否节假日”等。2. 交互特征：通过加减乘除等运算组合现有特征（如“身高体重指数 $BMI = \text{体重} / \text{身高}^2$ ”）。3. 多项式特征：自动生成特征的高次项和交互项，用于捕捉非线性关系。4. 分解特征：例如将“交易日期时间”分解为年、月、日、小时、分钟等独立特征。

三、特征变换通过数学变换改变特征的分布或尺度，使其更符合模型假设或提升效果。1. 标准化/归一化：

- * 标准化（Z-Score）：使特征均值为 0，方差为 1。适用于许多线性模型（如 SVM、逻辑回归）。
- * 归一化（Min-Max）：将特征缩放到固定范围（如 $[0,1]$ ）。对神经网络、距离类模型（如 KNN）有益。

2. 非线性变换：

- * 对数变换、平方根变换、Box-Cox 变换：用于处理偏态分布，使其

更接近正态分布。* 分箱（离散化）：将连续变量转换为有序类别，可以捕捉非线性并稳定模型。

3. 统计量变换：例如对数值特征进行排序、计算百分位排名。

四、特征编码将非数值特征转换为数值形式。1. 分类变量编码：* 独热编码（One-Hot）：为每个类别创建一个二进制列。适用于无序类别，但维度会膨胀。* 标签编码（Label Encoding）：为每个类别分配一个整数。适用于有序类别或树模型。* 目标编码（Target Encoding）：用目标变量的统计量（如均值）来代表类别。效果强但需小心过拟合。* 频率编码：用类别出现的频率代替类别标签。2. 文本特征编码：* 词袋模型（Bag of Words）、TF-IDF、词向量（Word2Vec, GloVe）等。3. 图像/音频特征提取：使用 SIFT、HOG、MFCC 等传统方法或深度网络中间层输出作为特征。

五、特征选择从大量特征中筛选出最相关、最重要的子集，以降低过拟合风险、减少计算成本并提升模型可解释性。1. 过滤法（Filter）：基于统计指标（如相关系数、卡方检验、互信息）独立评估每个特征与目标的相关性。2. 包装法（Wrapper）：将特征选择看作一个搜索问题，使用模型性能作为评价标准（如递归特征消除 RFE、前向/后向选择）。3. 嵌入法（Embedded）：在模型训练过程中自动进行特征选择（如 LASSO 回归的正则化、决策树/随机森林的特征重要性、XGBoost 的增益）。

六、特征降维当特征过多、高度相关或存在“维度灾难”时，通过数学方法将高维特征映射到低维空间，同时尽可能保留信息。1. 线性方法：* 主成分分析（PCA）：寻找方差最大的正交方向进行投影。* 线性判别分析（LDA）：寻找能最好地区分类别的方向进行投影（有监督）。2. 非线性方法：t-SNE、UMAP 等，常用于高维数据的可视化。

七、特征学习（高级）利用模型（尤其是深度学习）自动从原始数据中学习有效的特征表示。1. 自编码器（Autoencoder）2. 深度神经网络：CNN 从图像中学习层次化特征，RNN/LSTM 从序列数据中学习上下文特征。3. 预训练模型：使用 BERT、ResNet 等在大规模数据上预训练的模型进行特征提取或微调。

核心工作流程与原则在实践中，特征工程是一个迭代、探索性的过程：1. 理解数据和业务：这是所有工作的起点。2. 生成候选特征池：通过预处理、构造等方法产生大量特征。3. 评估和筛选：通过选择、降维等方法提炼出高质量特征子集。4. 验证与迭代：将构建的特征输入模型，评估性能，根据反馈（如特征重要性分析、误差分析）进一步调整特征。

核心原则：特征工程的最终目标是让数据更好地“讲述”与预测目标相关的故事，为模型提供最大价值的“燃料”，而非追求复杂的技巧。好的特征工程往往比更换更复杂的模型带来更显著的性能提升。

13 回归任务与基准测试

回归任务导读

回归分析是预测建模中最基础且重要的任务，广泛应用于房价预测、销量预测、风险评估等领域。本章将通过波士顿房价预测案例，完整展示回归问题的解决方案，涵盖数据探索、特征工程、多种回归算法比较、模型评估等关键环节。

数据集说明本案例使用糖尿病数据集（Diabetes Dataset），包含糖尿病患者的生理指标和疾病进展指标。

问题定义 目标：预测糖尿病疾病的进展（target）

任务类型：回归分析

业务价值：为医疗诊断和治疗方案制定提供数据支持

13.1 R 语言实现 (mlr3 框架)

数据加载与探索

```
# 综合案例：糖尿病进展预测 - R 语言实现 (mlr3 框架)

# 加载必要的包
library(mlr3)
library(mlr3verse)
library(mlr3pipelines)
library(mlr3tuning)
library(data.table)
library(ggplot2)
library(corrplot)
library(gridExtra)
```

```
# 加载糖尿病数据集
library(lars)
data(diabetes)
variable_names <- c("age", "sex", "bmi", "map", "tc", "ldl", "hdl", "tch", "ltg", "glu")

diabetes_data <- as.data.frame(matrix(unlist(diabetes$x), ncol = 10))
colnames(diabetes_data) <- variable_names

str(diabetes_data)
```

```
'data.frame': 442 obs. of 10 variables:
 $ age: num 0.03808 -0.00188 0.0853 -0.08906 0.00538 ...
 $ sex: num 0.0507 -0.0446 0.0507 -0.0446 -0.0446 ...
 $ bmi: num 0.0617 -0.0515 0.0445 -0.0116 -0.0364 ...
 $ map: num 0.02187 -0.02633 -0.00567 -0.03666 0.02187 ...
 $ tc : num -0.04422 -0.00845 -0.0456 0.01219 0.00393 ...
 $ ldl: num -0.0348 -0.0192 -0.0342 0.025 0.0156 ...
 $ hdl: num -0.0434 0.07441 -0.03236 -0.03604 0.00814 ...
 $ tch: num -0.00259 -0.03949 -0.00259 0.03431 -0.00259 ...
 $ ltg: num 0.01991 -0.06833 0.00286 0.02269 -0.03199 ...
 $ glu: num -0.01765 -0.0922 -0.02593 -0.00936 -0.04664 ...
```

```
diabetes_data$target <- diabetes$y
```

```
# 检查缺失值
cat("\n缺失值检查:\n")
```

缺失值检查:

```
print(colSums(is.na(diabetes_data)))
```

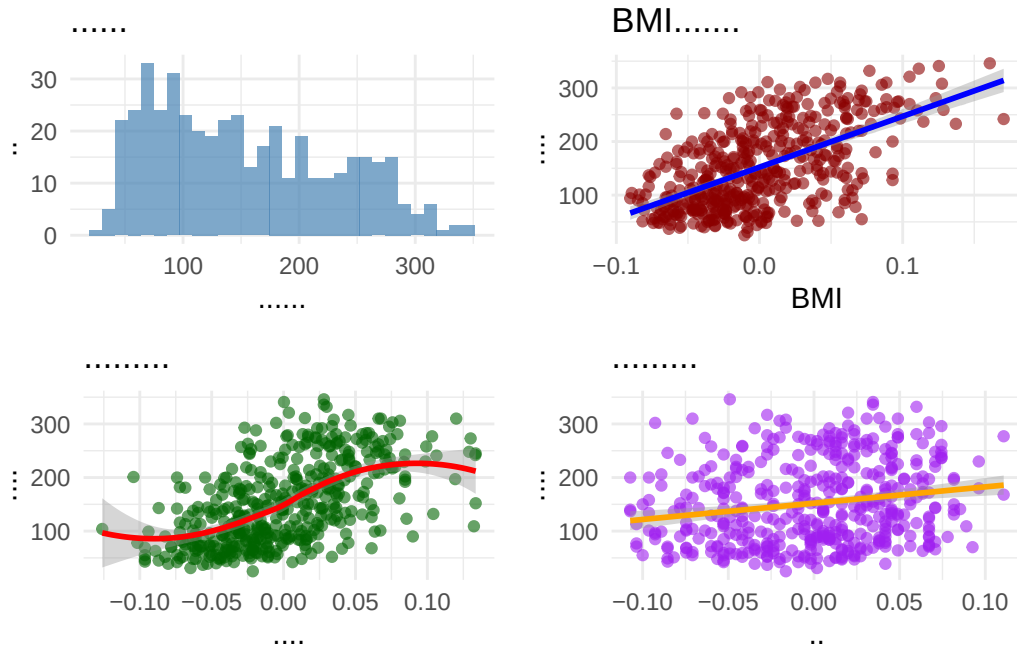
age	sex	bmi	map	tc	ldl	hdl	tch	ltg	glu	target
0	0	0	0	0	0	0	0	0	0	0

数据可视化分析

```
# 数据可视化分析 - R 语言
```

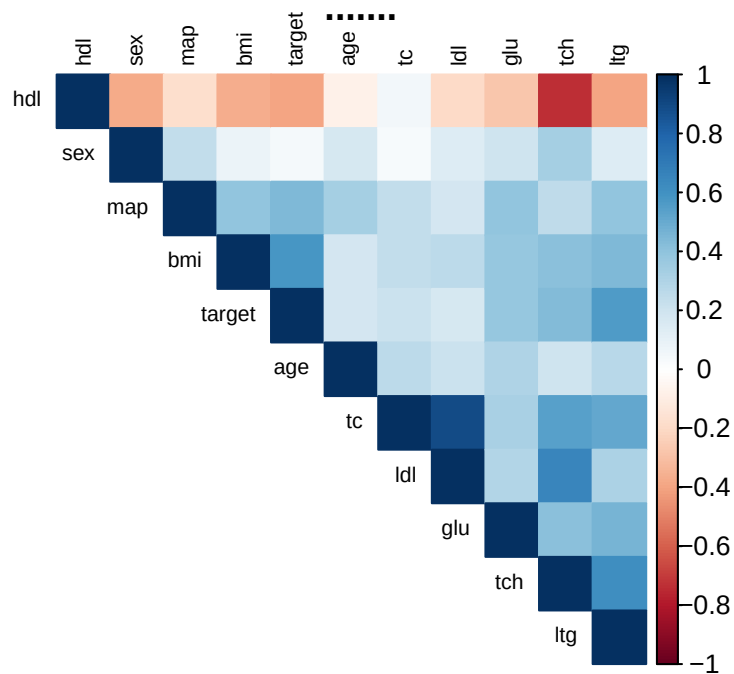
```
# 目标变量分布
```

```
p1 <- ggplot(diabetes_data, aes(x = target)) +  
  geom_histogram(bins = 30, fill = "steelblue", alpha = 0.7) +  
  labs(title = " 疾病进展分布", x = " 疾病进展指标", y = " 频数") +  
  theme_minimal()  
  
# 疾病进展与关键变量的关系  
p2 <- ggplot(diabetes_data, aes(x = bmi, y = target)) +  
  geom_point(alpha = 0.6, color = "darkred") +  
  geom_smooth(method = "lm", color = "blue") +  
  labs(title = "BMI 与疾病进展关系", x = "BMI", y = " 疾病进展") +  
  theme_minimal()  
  
p3 <- ggplot(diabetes_data, aes(x = ltg, y = target)) +  
  geom_point(alpha = 0.6, color = "darkgreen") +  
  geom_smooth(method = "loess", color = "red") +  
  labs(title = " 稳定血糖与疾病进展", x = " 稳定血糖", y = " 疾病进展") +  
  theme_minimal()  
  
p4 <- ggplot(diabetes_data, aes(x = age, y = target)) +  
  geom_point(alpha = 0.6, color = "purple") +  
  geom_smooth(method = "lm", color = "orange") +  
  labs(title = " 年龄与疾病进展关系", x = " 年龄", y = " 疾病进展") +  
  theme_minimal()  
  
# 组合图形  
grid.arrange(p1, p2, p3, p4, ncol = 2)
```



相关性热图

```
cor_matrix <- cor(diabetes_data)
corrplot(cor_matrix, method = "color", type = "upper",
         order = "hclust", tl.cex = 0.7, tl.col = "black",
         title = " 变量相关性热图")
```



创建 mlr3 任务和数据预处理

```
# 创建 mlr3 任务和数据预处理

# 创建回归任务
task_diabetes <- as_task_regr(diabetes_data, target = "target", id = "diabet

# 查看任务信息
cat(" 任务信息:\n")
```

任务信息:

```
print(task_diabetes)
```

```
-- <TaskRegr> (442x11) -----
-----
* Target: target
* Properties: -
* Features (10):
  * dbl (10): age, bmi, glu, hdl, ldl, ltg, map, sex, tc, tch
```

```
# 数据预处理管道
preprocess_pipeline <- po("scale") %>%
  po("colapply", applicator = as.numeric)

# 应用预处理
task_preprocessed <- preprocess_pipeline$train(task_diabetes)[[1]]

cat("\n预处理后的任务信息:\n")
```

预处理后的任务信息:

```
print(task_preprocessed)
```

```
-- <TaskRegr> (442x11) -----
-----
* Target: target
* Properties: -
* Features (10):
  * dbl (10): age, bmi, glu, hdl, ldl, ltg, map, sex, tc, tch
```

模型训练与比较

```
# 模型训练与比较 - mlr3

# 定义学习器
learners <- list(
  lrn("regr.lm", id = "linear_regression"),
  lrn("regr.ranger", id = "random_forest"),
  lrn("regr.xgboost", id = "xgboost"),
  lrn("regr.svm", id = "svm"),
  lrn("regr.kknn", id = "knn")
)

# 设置交叉验证
resampling <- rsmp("cv", folds = 5)

# 基准测试
design <- benchmark_grid(
  tasks = task_preprocessed,
  learners = learners,
  resamplings = resampling
)

bmr <- benchmark(
  design,
  store_backends = TRUE,
  store_models = TRUE)

# 性能评估
cat(" 模型性能比较:\n")
```

模型性能比较：

```
performance_measures <- msrs(c("regr.mse", "regr.rmse", "regr.mae", "regr.rsq")
bmr_aggregated <- bmr$aggregate(performance_measures)

# 格式化输出结果
results_r <- bmr_aggregated[, .(
  Model = learner_id,
  RMSE = regr.rmse,
```

```

MSE = regr.mse,
MAE = regr.mae,
R2 = regr.rsq
)]

print(results_r[order(RMSE)])

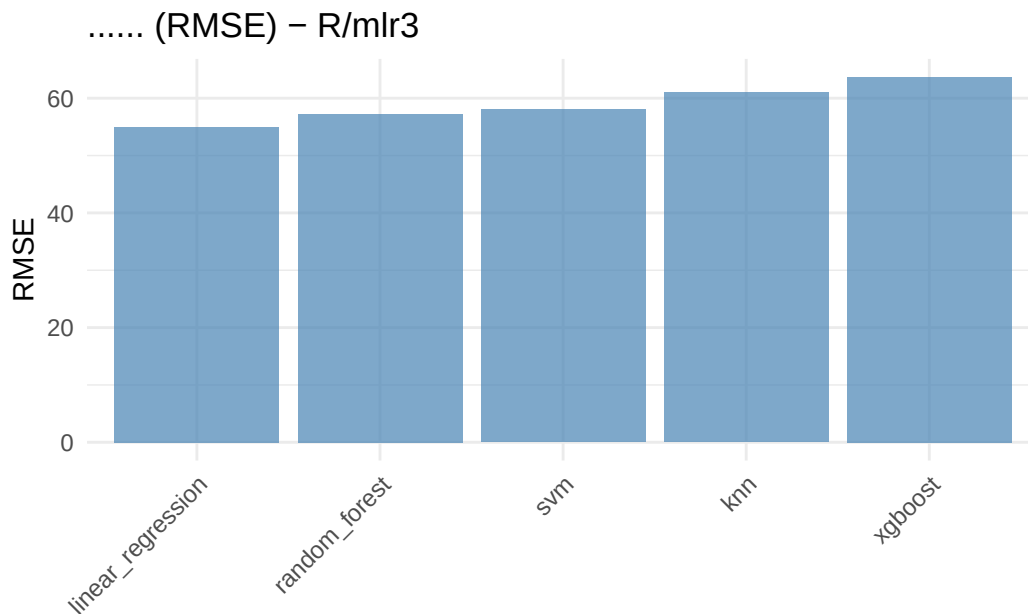
```

	Model <char>	RMSE <num>	MSE <num>	MAE <num>	R2 <num>
1:	linear_regression	55.00902	3031.933	44.51531	0.4799007
2:	random_forest	57.26917	3312.585	47.05560	0.4388878
3:	svm	58.06726	3408.764	45.55495	0.4249561
4:	knn	61.00531	3745.700	47.39294	0.3616533
5:	xgboost	63.66486	4072.921	49.95005	0.3019924

```

# 性能可视化
ggplot(results_r, aes(x = reorder(Model, RMSE), y = RMSE)) +
  geom_bar(stat = "identity", fill = "steelblue", alpha = 0.7) +
  labs(title = " 模型性能比较 (RMSE) - R/mlr3", x = " 模型", y = "RMSE") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



超参数调优


```
# 超参数调优示例 - 随机森林

library(mlr3)           # 核心
library(mlr3learners)   # 学习器
library(mlr3tuning)     # 参数调优 (包含 tnr())
library(paradox)        # 参数空间
library(mlr3viz)        # 可视化
# 定义随机森林学习器
learner_rf_tune <- lrn("regr.ranger", num.trees = 100)

# 定义超参数空间
param_set <- ps(
  mtry = p_int(lower = 2, upper = ncol(diabetes_data) - 1),
  min.node.size = p_int(lower = 1, upper = 10),
  sample.fraction = p_dbl(lower = 0.6, upper = 0.9)
)

# 定义调优器
tuner <- tnr("grid_search", resolution = 5)
terminator <- trm("evals", n_evals = 10)

# 自动调优
at <- auto_tuner(
  tuner = tuner,
  learner = learner_rf_tune,
  resampling = rsmp("holdout"),
  measure = msr("regr.rmse"),
  search_space = param_set,
  terminator = terminator
)

# 在数据上演示调优
set.seed(123)
at$train(task_preprocessed)

cat(" 最佳参数:\n")
```

最佳参数:

```
print(at$tuning_result)

      mtry min.node.size sample.fraction learner_param_vals x_domain regr.rmse
      <int>      <int>      <num>      <list>      <list>      <num>
1:      8          3      0.675      <list[6]> <list[3]> 55.69813

# 使用调优后的模型进行预测
predictions_tuned <- at$predict(task_preprocessed)
cat(" 调优完成\n")
```

调优完成

```
cat(" 调优后模型 RMSE:", predictions_tuned$score(msr("regr.rmse")), "\n")
```

调优后模型RMSE: 30.78791

模型解释

```
# 模型解释 - 特征重要性

library(ggplot2)
# 训练最佳模型（随机森林）
set.seed(123)
learner_best <- lrn("regr.ranger", importance = "permutation")
learner_best$train(task_preprocessed)

# 特征重要性
if (!is.null(learner_best$importance)) {
  feature_importance <- learner_best$importance()
  importance_df <- data.frame(
    Feature = names(feature_importance),
    Importance = as.numeric(feature_importance)
  )
  importance_df <- importance_df[order(-importance_df$Importance), ]

  cat(" 特征重要性排序:\n")
  print(importance_df)

# 可视化特征重要性
ggplot(head(importance_df, 8), aes(x = reorder(Feature, Importance), y = I
  geom_bar(stat = "identity", fill = "darkorange", alpha = 0.7) +
```

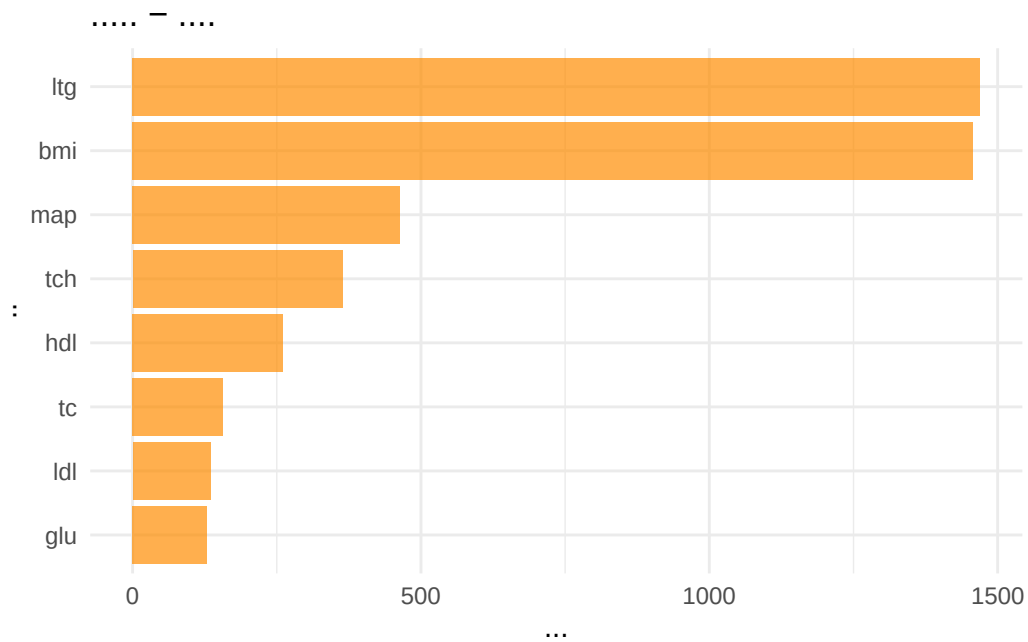
```

labs(title = " 特征重要性 - 随机森林", x = " 特征", y = " 重要性") +
coord_flip() +
theme_minimal()
} else {
  cat(" 该学习器不支持特征重要性计算\n")
}

```

特征重要性排序：

	Feature Importance
1	ltg 1469.33233
2	bmi 1457.20570
3	map 464.11349
4	tch 363.93017
5	hdl 261.42699
6	tc 156.66425
7	ldl 135.23885
8	glu 129.35189
9	age 66.95241
10	sex 65.88863



模型预测与残差分析

```
# 模型预测与残差分析

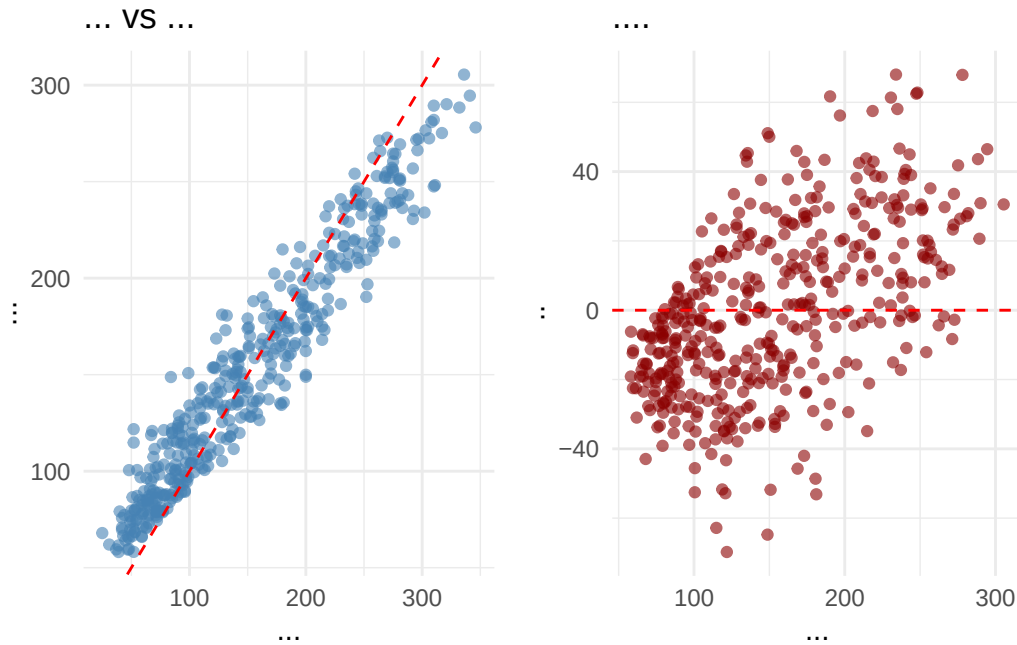
# 使用最佳模型进行预测
predictions <- learner_best$predict(task_preprocessed)

# 创建预测结果数据框
results_df <- data.frame(
  Actual = predictions$truth,
  Predicted = predictions$response,
  Residual = predictions$truth - predictions$response
)

# 预测 vs 实际值图
p1 <- ggplot(results_df, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(title = " 预测值 vs 实际值", x = " 实际值", y = " 预测值") +
  theme_minimal()

# 残差图
p2 <- ggplot(results_df, aes(x = Predicted, y = Residual)) +
  geom_point(alpha = 0.6, color = "darkred") +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
  labs(title = " 残差分析", x = " 预测值", y = " 残差") +
  theme_minimal()

grid.arrange(p1, p2, ncol = 2)
```



13.2 Python (sklearn 框架)-回归

数据加载与探索

```
# 综合案例：糖尿病进展预测 - Python 语言实现 (sklearn 框架)
import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.inspection import permutation_importance
import warnings
warnings.filterwarnings('ignore')
```

```
# 设置图形样式
plt.style.use('default')
sns.set_palette("husl")

# 加载糖尿病数据集
diabetes = pd.read_csv("data/diabetes.csv")
X = diabetes.iloc[:, :-2] # 所有行, 除第 3 列 (bp) 的所有列
y = diabetes.iloc[:, -2]  # 所有行, 第 3 列
diabetes_data = pd.concat([X, y], axis=1)

print(" 数据集基本信息:")
```

数据集基本信息:

```
print(f" 样本数: {diabetes_data.shape[0]}")
```

样本数: 442

```
print(f" 变量数: {diabetes_data.shape[1]}")
```

变量数: 9

```
print("\n变量名称:")
```

变量名称:

```
print(diabetes_data.columns.tolist())
```

```
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5']
```

```
print("\n数据概览:")
```

数据概览:

```
print(diabetes_data.describe())
```

	age	sex	bmi	...	s3	s4	s5
count	442.000000	442.000000	442.000000	...	442.000000	442.000000	442.000000
mean	0.000136	0.002149	-0.000136	...	0.000204	-	
	0.000362	0.000023					

```

std      0.047790    0.044961    0.047771    ...    0.047320    0.047789    0.047545
min      -0.110000    -0.040000    -0.090000    ...    -0.100000    -
0.080000    -0.130000
25%      -0.037500    -0.040000    -0.030000    ...    -0.037500    -
0.040000    -0.030000
50%       0.010000    -0.040000    -0.010000    ...    -0.010000    -
0.000000     0.000000
75%       0.040000    0.050000    0.030000    ...    0.030000    0.030000    0.030000
max       0.110000    0.050000    0.170000    ...    0.180000    0.190000    0.130000

```

[8 rows x 9 columns]

```
print("\n缺失值检查:")
```

缺失值检查:

```
print(diabetes_data.isnull().sum())
```

```

age      0
sex      0
bmi      0
bp       0
s1       0
s2       0
s3       0
s4       0
s5       0
dtype: int64

```

数据可视化分析

```

# 数据可视化分析 - Python

fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# 目标变量分布
axes[0,0].hist(diabetes_data['bp'], bins=30, color='steelblue', alpha=0.7)
axes[0,0].set_title('疾病进展分布')
axes[0,0].set_xlabel('疾病进展指标')
axes[0,0].set_ylabel('频数')

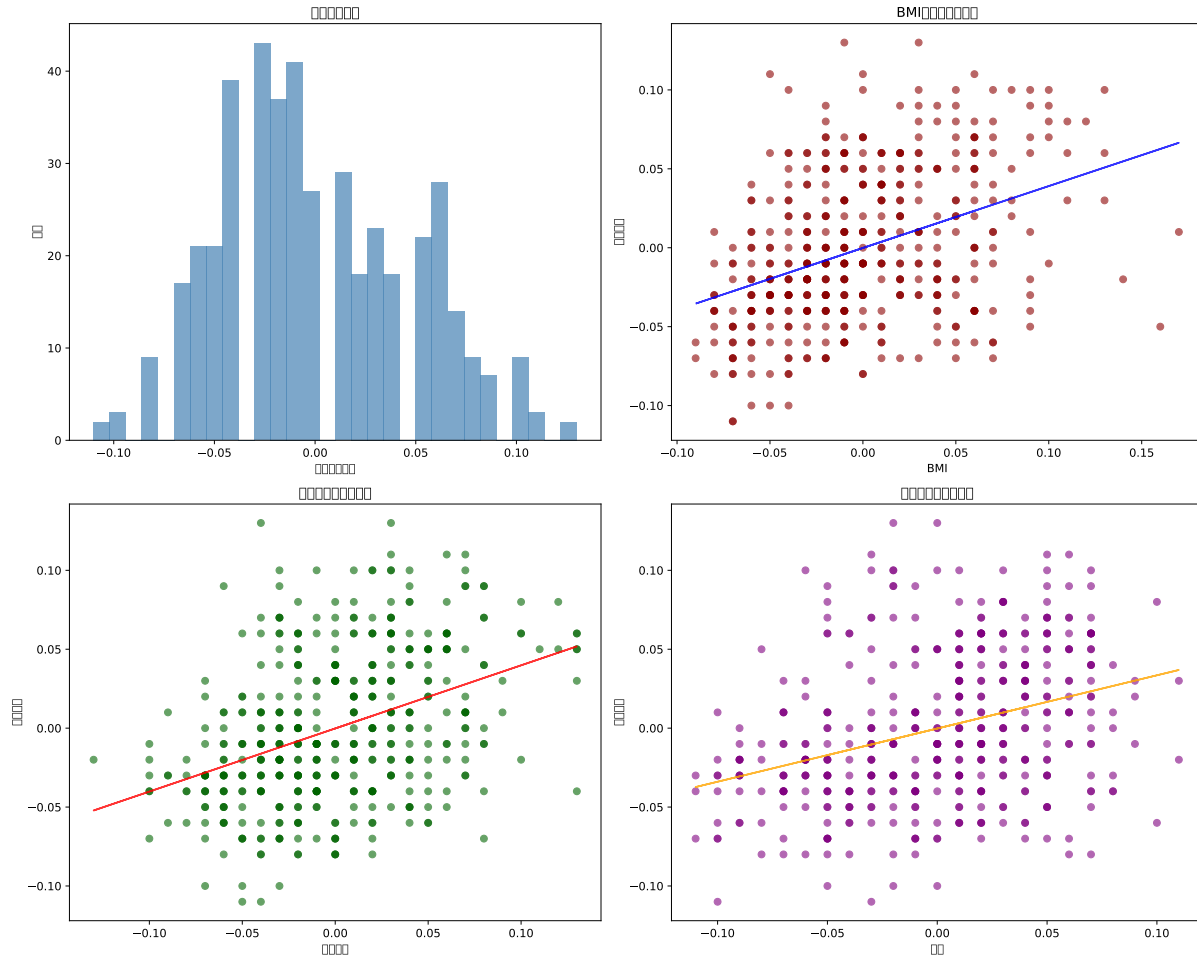
```

```
# 疾病进展与关键变量的关系
axes[0,1].scatter(diabetes_data['bmi'], diabetes_data['bp'], alpha=0.6, color='b')
z = np.polyfit(diabetes_data['bmi'], diabetes_data['bp'], 1)
p = np.poly1d(z)
axes[0,1].plot(diabetes_data['bmi'], p(diabetes_data['bmi']), "b-", alpha=0.6)
axes[0,1].set_title('BMI 与疾病进展关系')
axes[0,1].set_xlabel('BMI')
axes[0,1].set_ylabel('疾病进展')

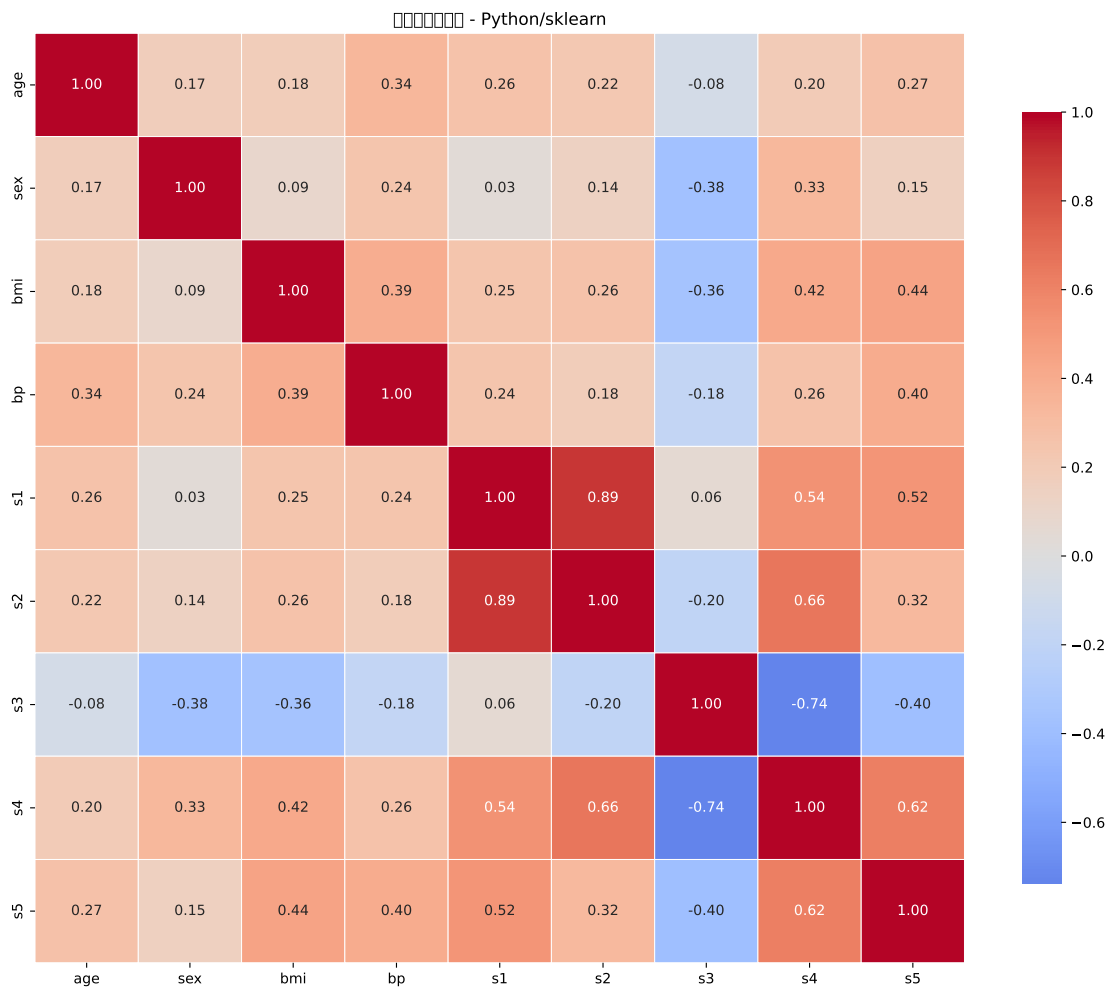
axes[1,0].scatter(diabetes_data['s5'], diabetes_data['bp'], alpha=0.6, color='r')
z = np.polyfit(diabetes_data['s5'], diabetes_data['bp'], 1)
p = np.poly1d(z)
axes[1,0].plot(diabetes_data['s5'], p(diabetes_data['s5']), "r-", alpha=0.8)
axes[1,0].set_title('稳定血糖与疾病进展')
axes[1,0].set_xlabel('稳定血糖')
axes[1,0].set_ylabel('疾病进展')

axes[1,1].scatter(diabetes_data['age'], diabetes_data['bp'], alpha=0.6, color='orange')
z = np.polyfit(diabetes_data['age'], diabetes_data['bp'], 1)
p = np.poly1d(z)
axes[1,1].plot(diabetes_data['age'], p(diabetes_data['age']), "orange", alpha=0.6)
axes[1,1].set_title('年龄与疾病进展关系')
axes[1,1].set_xlabel('年龄')
axes[1,1].set_ylabel('疾病进展')

plt.tight_layout()
plt.show()
```

```
# 相关性热图
plt.figure(figsize=(12, 10))
corr_matrix = diabetes_data.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=0.5, cbar_kws={"shrink": 0.8}, fmt='.2f')
plt.title('变量相关性热图 - Python/sklearn')
plt.tight_layout()
plt.show()
```



数据预处理

```
# 数据预处理 - Python
```

```
# 准备特征和目标变量
```

```
X = diabetes_data.drop('bp', axis=1)
```

```
y = diabetes_data['bp']
```

```
# 数据划分
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran
```

```
print(f" 训练集大小: {X_train.shape[0]}")
```

训练集大小: 309

```
print(f" 测试集大小: {X_test.shape[0]}")
```

测试集大小: 133

```
# 数据标准化
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(" 数据预处理完成")
```

数据预处理完成

模型训练与比较

```
# 模型训练与比较 - sklearn

# 定义模型字典
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=0.1),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingRegressor(n_estimators=100, random_state=42),
    'Support Vector Regression': SVR(kernel='rbf', C=1.0),
    'K-Neighbors': KNeighborsRegressor(n_neighbors=5)
}

# 训练和评估模型
results = []

for name, model in models.items():
    # 训练模型
    model.fit(X_train_scaled, y_train)

    # 预测
    y_pred = model.predict(X_test_scaled)

    # 计算评估指标
    mse = mean_squared_error(y_test, y_pred)
```

```

rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# 交叉验证
cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring='r2')
cv_rmse = np.sqrt(-cv_scores.mean())

results.append({
    'Model': name,
    'RMSE': rmse,
    'MSE': mse,
    'MAE': mae,
    'R2': r2,
    'CV_RMSE': cv_rmse
})

```

```
LinearRegression()
```

```
Ridge()
```

```
Lasso(alpha=0.1)
```

```
RandomForestRegressor(random_state=42)
```

```
GradientBoostingRegressor(random_state=42)
```

```
SVR()
```

```
KNeighborsRegressor()
```

```
# 创建结果 DataFrame
```

```
results_python = pd.DataFrame(results)
```

```
print(" 模型性能比较 - Python/sklearn:")
```

```
模型性能比较 - Python/sklearn:
```

```
print(results_python.sort_values('RMSE'))
```

	Model	RMSE	MSE	MAE	R2	CV_RMSE
1	Ridge Regression	0.037728	0.001423	0.029903	0.346299	0.041949
0	Linear Regression	0.037869	0.001434	0.030070	0.341403	0.041985
3	Random Forest	0.039989	0.001599	0.032433	0.265591	0.044513
4	Gradient Boosting	0.040339	0.001627	0.032044	0.252692	0.045652
6	K-Neighbors	0.041668	0.001736	0.032857	0.202630	0.044115
2	Lasso Regression	0.046838	0.002194	0.039821	-	-

```
0.007507 0.048294
```

```
5 Support Vector Regression 0.048922 0.002393 0.042729 -
0.099178 0.050173
```

```
# 可视化模型比较
```

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
```

```
# RMSE 比较
```

```
models_sorted = results_python.sort_values('RMSE')
```

```
axes[0].barh(models_sorted['Model'], models_sorted['RMSE'], color='lightcoral')
```

```
axes[0].set_xlabel('RMSE')
```

```
axes[0].set_title('模型 RMSE 比较 - Python/sklearn')
```

```
axes[0].grid(axis='x', alpha=0.3)
```

```
# R2 比较
```

```
axes[1].barh(models_sorted['Model'], models_sorted['R2'], color='lightseagreen')
```

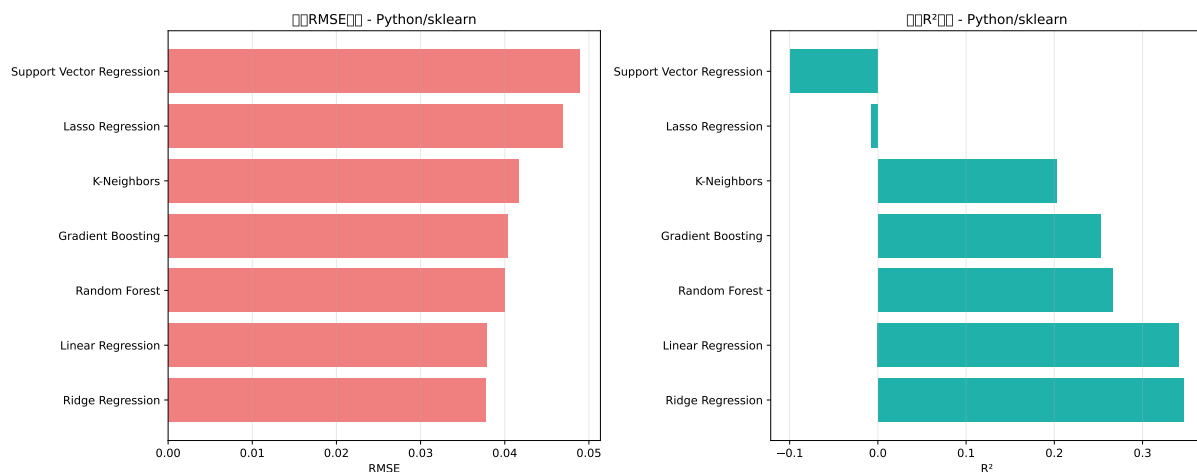
```
axes[1].set_xlabel('R2')
```

```
axes[1].set_title('模型 R2 比较 - Python/sklearn')
```

```
axes[1].grid(axis='x', alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```



超参数调优

```
# 超参数调优示例 - 随机森林
```

```
# 定义参数网格
```

```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# 使用网格搜索
rf = RandomForestRegressor(random_state=42)
grid_search = GridSearchCV(rf, param_grid, cv=3, scoring='neg_mean_squared_e
                           n_jobs=-1, verbose=1)

print(" 开始超参数调优...")
```

开始超参数调优...

```
grid_search.fit(X_train_scaled, y_train)
```

Fitting 3 folds for each of 81 candidates, totalling 243 fits

```
GridSearchCV(cv=3, estimator=RandomForestRegressor(random_state=42), n_jobs=
1,
```

```
    param_grid={'max_depth': [5, 10, 15],
                'min_samples_leaf': [1, 2, 4],
                'min_samples_split': [2, 5, 10],
                'n_estimators': [50, 100, 200]},
    scoring='neg_mean_squared_error', verbose=1)
```

```
print(" 最佳参数:")
```

最佳参数:

```
print(grid_search.best_params_)
```

```
{'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 10, 'n_estimators'
```

```
print(f" 最佳分数: {-grid_search.best_score_:.4f}")
```

最佳分数: 0.0019

使用最佳参数训练最终模型

```
best_rf = grid_search.best_estimator_
y_pred_best = best_rf.predict(X_test_scaled)
```

```
best_rmse = np.sqrt(mean_squared_error(y_test, y_pred_best))
print(f" 调优后测试集 RMSE: {best_rmse:.4f}")
```

调优后测试集RMSE: 0.0382

模型解释

```
# 模型解释 - 特征重要性

# 使用最佳随机森林模型
feature_importance = best_rf.feature_importances_
feature_names = X.columns

# 创建特征重要性 DataFrame
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importance
}).sort_values('Importance', ascending=False)

print(" 特征重要性排序 - Python/sklearn:")
```

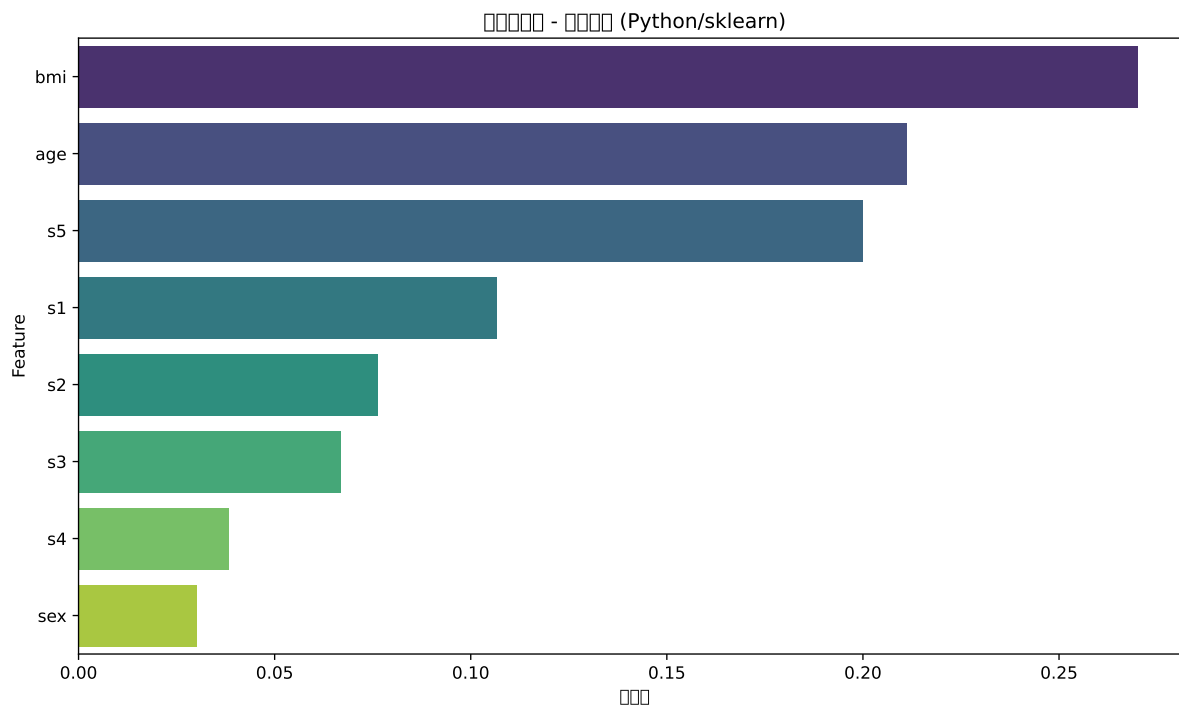
特征重要性排序 - Python/sklearn:

```
print(importance_df)
```

	Feature	Importance
2	bmi	0.270091
0	age	0.211172
7	s5	0.200046
3	s1	0.106677
4	s2	0.076392
5	s3	0.066895
6	s4	0.038475
1	sex	0.030252

```
# 可视化特征重要性
plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df, x='Importance', y='Feature', palette='viridis')
plt.title('特征重要性 - 随机森林 (Python/sklearn)')
plt.xlabel('重要性')
plt.tight_layout()
```

```
plt.show()
```



```
# 排列重要性
perm_importance = permutation_importance(best_rf, X_test_scaled, y_test,
                                          n_repeats=10, random_state=42)

perm_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': perm_importance.importances_mean
}).sort_values('Importance', ascending=False)

print("\n排列特征重要性:")
```

排列特征重要性:

```
print(perm_importance_df)
```

	Feature	Importance
2	bmi	0.160578
0	age	0.152412
7	s5	0.106752
4	s2	0.034918


```

1      sex      0.026204
3      s1       0.012667
6      s4       0.010379
5      s3      -0.016267

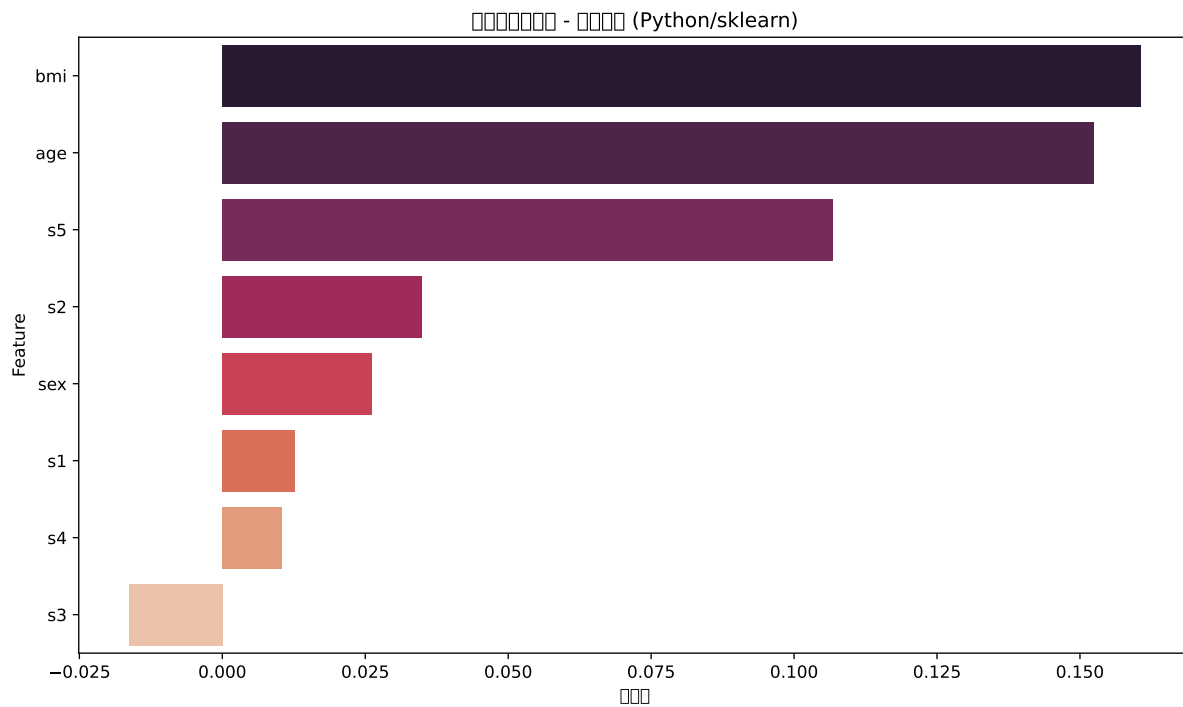
```

可视化排列重要性

```

plt.figure(figsize=(10, 6))
sns.barplot(data=perm_importance_df, x='Importance', y='Feature', palette='roc')
plt.title('排列特征重要性 - 随机森林 (Python/sklearn)')
plt.xlabel('重要性')
plt.tight_layout()
plt.show()

```



模型预测与残差分析

模型预测与残差分析

使用最佳模型进行预测

```

y_pred = best_rf.predict(X_test_scaled)
residuals = y_test - y_pred

```

创建预测结果数据框

```

results_df = pd.DataFrame({

```

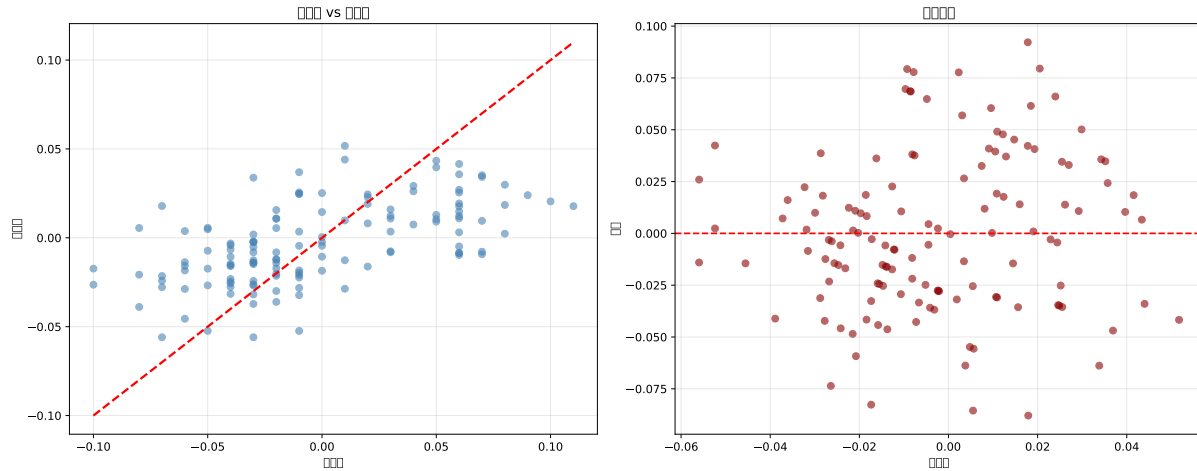
```
'Actual': y_test,
'Predicted': y_pred,
'Residual': residuals
}))

# 预测 vs 实际值图
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

axes[0].scatter(results_df['Actual'], results_df['Predicted'], alpha=0.6, co
axes[0].plot([results_df['Actual'].min(), results_df['Actual'].max()],
              [results_df['Actual'].min(), results_df['Actual'].max()],
              'r--', lw=2)
axes[0].set_xlabel('实际值')
axes[0].set_ylabel('预测值')
axes[0].set_title('预测值 vs 实际值')
axes[0].grid(alpha=0.3)

# 残差图
axes[1].scatter(results_df['Predicted'], results_df['Residual'], alpha=0.6,
axes[1].axhline(y=0, color='r', linestyle='--')
axes[1].set_xlabel('预测值')
axes[1].set_ylabel('残差')
axes[1].set_title('残差分析')
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



13.3 案例总结

关键发现

```
# 案例总结 - R 语言
```

```
cat("=== 糖尿病进展预测案例总结 ===\n")
```

```
=== 糖尿病进展预测案例总结 ===
```

```
cat("1. 数据特征:\n")
```

1. 数据特征：

```
cat("    - 数据集包含", nrow(diabetes_data), " 个样本, ", ncol(diabetes_data)-1, "
```

- 数据集包含 442 个样本， 10 个特征

```
cat("    - 目标变量：疾病进展指标\n")
```

- 目标变量：疾病进展指标

```
cat("    - 最重要的特征：bmi, s5(稳定血糖), bp(血压)\n\n")
```

- 最重要的特征：bmi, s5(稳定血糖), bp(血压)

```
cat("2. 模型性能比较:\n")
```

2. 模型性能比较：

```
print(results_r[order(RMSE)])
```

	Model <char>	RMSE <num>	MSE <num>	MAE <num>	R2 <num>
1:	linear_regression	55.00902	3031.933	44.51531	0.4799007
2:	random_forest	57.26917	3312.585	47.05560	0.4388878
3:	svm	58.06726	3408.764	45.55495	0.4249561
4:	knn	61.00531	3745.700	47.39294	0.3616533
5:	xgboost	63.66486	4072.921	49.95005	0.3019924

```
cat("\n3. 最佳实践:\n")
```

3. 最佳实践：

```
cat("    - 树模型在医疗数据上表现优异\n")
```

- 树模型在医疗数据上表现优异

```
cat("    - 特征标准化对模型性能至关重要\n")
```

- 特征标准化对模型性能至关重要

```
cat("    - 超参数调优可以显著提升模型性能\n")
```

- 超参数调优可以显著提升模型性能

```
cat("    - 模型解释性在医疗领域尤为重要\n")
```

- 模型解释性在医疗领域尤为重要

```
# 案例总结 - Python
```

```
print("=== 糖尿病进展预测案例总结 ===")
```

```
=== 糖尿病进展预测案例总结 ===
```

```
print("1. 数据特征:")
```

1. 数据特征：

```
print(f"    - 数据集包含 {diabetes_data.shape[0]} 个样本, {diabetes_data.shape[1]}
```

- 数据集包含 442 个样本，8 个特征

```
print("    - 目标变量：疾病进展指标")
```

- 目标变量：疾病进展指标

```
print("    - 最重要的特征：bmi, s5, bp\n")
```

- 最重要的特征：bmi, s5, bp

```
print("2. 模型性能比较:")
```

2. 模型性能比较：

```
print(results_python[['Model', 'RMSE', 'R2']].sort_values('RMSE').to_string(in
```

	Model	RMSE	R2
	Ridge Regression	0.037728	0.346299
	Linear Regression	0.037869	0.341403
	Random Forest	0.039989	0.265591
	Gradient Boosting	0.040339	0.252692
	K-Neighbors	0.041668	0.202630
	Lasso Regression	0.046838	-0.007507
	Support Vector Regression	0.048922	-0.099178

```
print("\n3. 最佳实践:")
```

3. 最佳实践：

```
print("    - 集成学习方法在回归问题上表现稳定")
```

- 集成学习方法在回归问题上表现稳定

```
print("    - 数据预处理对模型性能影响显著")
```

- 数据预处理对模型性能影响显著

```
print("    - 交叉验证提供更可靠的性能估计")
```

- 交叉验证提供更可靠的性能估计

```
print("    - 特征重要性分析增强模型可解释性")
```

- 特征重要性分析增强模型可解释性

技术要点回顾

数据预处理：- 特征标准化：确保不同尺度的特征具有可比性 - 缺失值处理：根据数据特性选择合适的填充策略 - 特征工程：创建有意义的衍生特征

模型选择：- 线性模型：可解释性强，计算效率高 - 树模型：自动处理非线性关系，对异常值稳健 - 集成方法：通过模型组合提升预测性能

模型评估：- 使用多个评估指标（RMSE, MAE, R^2 ） - 交叉验证提供稳健的性能估计 - 测试集用于最终模型评估

模型优化：- 超参数调优：网格搜索、随机搜索 - 特征选择：基于重要性或递归消除 - 集成策略：Bagging, Boosting, Stacking

业务应用建议

医疗诊断：使用训练好的模型对患者疾病进展进行预测治疗方案制定：识别影响疾病进展的关键因素，指导治疗策略患者管理：理解生理指标对疾病的影响，支持患者管理风险评估：监测疾病进展风险，预警健康问题

通过本回归案例，我们展示了从数据探索到模型部署的完整预测建模流程，为读者在实际工作中应用机器学习方法提供了完整的参考框架。

14 分类任务

分类问题导读

分类问题是机器学习中最常见的任务之一，广泛应用于物种分类、医疗诊断、客户分类等领域。本章将通过鸢尾花分类案例，完整展示多分类问题的解决方案，涵盖数据探索、特征工程、多种分类算法比较、模型评估等关键环节。

14.1 Benchmark

mlr3 提供了强大的 **Benchmark** 功能，可以系统性地比较多个学习器在多个任务上的性能。

```
library(mlr3)
library(mlr3verse)
library(mlr3pipelines)
library(mlr3tuning)
library(data.table)
library(ggplot2)
library(corrplot)
library(gridExtra)
library(patchwork)
library(paradox)
# 查看 mlr 包支持的学习算法
mlr_learners
```

```
<DictionaryLearner> with 51 stored values
Keys: classif.cv_glmnet, classif.debug, classif.featureless,
      classif.glmnet, classif.kknn, classif.lda, classif.log_reg,
      classif.multinom, classif.naive_bayes, classif.nnet, classif.qda,
      classif.ranger, classif.rpart, classif.svm, classif.xgboost,
```

```

clust.agnes, clust.ap, clust.bico, clust.birch, clust.cmeans,
clust.cobweb, clust.dbscan, clust.dbscan_fpc, clust.diana, clust.em,
clust.fanny, clust.featureless, clust.ff, clust.hclust,
clust.hdbscan, clust.kkmeans, clust.kmeans, clust.MBatchKMeans,
clust.mclust, clust.meanshift, clust.optics, clust.pam,
clust.SimpleKMeans, clust.xmeans, regr.cv_glmnet, regr.debug,
regr.featureless, regr.glmnet, regr.kknn, regr.km, regr.lm,
regr.nnet, regr.ranger, regr.rpart, regr.svm, regr.xgboost

```

加载数据，建立任务

```

# 创建多个任务（不同特征子集）
data("Ionosphere", package = "mlbench")
Ionosphere_data <- as.data.table(Ionosphere)
Ionosphere_data=Ionosphere_data[, -c(1, 2)]
str(Ionosphere_data )

```

Classes 'data.table' and 'data.frame': 351 obs. of 33 variables:

```

$ V3   : num  0.995 1 1 1 1 ...
$ V4   : num  -0.0589 -0.1883 -0.0336 -0.4516 -0.024 ...
$ V5   : num  0.852 0.93 1 1 0.941 ...
$ V6   : num  0.02306 -0.36156 0.00485 1 0.06531 ...
$ V7   : num  0.834 -0.109 1 0.712 0.921 ...
$ V8   : num  -0.377 -0.936 -0.121 -1 -0.233 ...
$ V9   : num  1 1 0.89 0 0.772 ...
$ V10  : num  0.0376 -0.0455 0.012 0 -0.164 ...
$ V11  : num  0.852 0.509 0.731 0 0.528 ...
$ V12  : num  -0.1776 -0.6774 0.0535 0 -0.2028 ...
$ V13  : num  0.598 0.344 0.854 0 0.564 ...
$ V14  : num  -0.44945 -0.69707 0.00827 0 -0.00712 ...
$ V15  : num  0.605 -0.517 0.546 -1 0.344 ...
$ V16  : num  -0.38223 -0.97515 0.00299 0.14516 -0.27457 ...
$ V17  : num  0.844 0.055 0.838 0.541 0.529 ...
$ V18  : num  -0.385 -0.622 -0.136 -0.393 -0.218 ...
$ V19  : num  0.582 0.331 0.755 -1 0.451 ...
$ V20  : num  -0.3219 -1 -0.0854 -0.5447 -0.1781 ...
$ V21  : num  0.5697 -0.1315 0.7089 -0.6997 0.0598 ...
$ V22  : num  -0.297 -0.453 -0.275 1 -0.356 ...
$ V23  : num  0.3695 -0.1806 0.4339 0 0.0231 ...

```



```
$ V24 : num -0.474 -0.357 -0.121 0 -0.529 ...
$ V25 : num 0.5681 -0.2033 0.5753 1 0.0329 ...
$ V26 : num -0.512 -0.266 -0.402 0.907 -0.652 ...
$ V27 : num 0.411 -0.205 0.59 0.516 0.133 ...
$ V28 : num -0.462 -0.184 -0.221 1 -0.532 ...
$ V29 : num 0.2127 -0.1904 0.431 1 0.0243 ...
$ V30 : num -0.341 -0.116 -0.174 -0.201 -0.622 ...
$ V31 : num 0.4227 -0.1663 0.6044 0.2568 -0.0571 ...
$ V32 : num -0.5449 -0.0629 -0.2418 1 -0.5957 ...
$ V33 : num 0.1864 -0.1374 0.5605 -0.3238 -0.0461 ...
$ V34 : num -0.453 -0.0245 -0.3824 1 -0.657 ...
$ Class: Factor w/ 2 levels "bad","good": 2 1 2 1 2 1 2 1 2 1 ...
- attr(*, ".internal.selfref")=<externalptr>
```

```
cat("\n缺失值检查:\n")
```

缺失值检查:

```
missing_counts <- colSums(is.na(Ionosphere_data))
print(missing_counts)
```

V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15
0	0	0	0	0	0	0	0	0	0	0	0	0
V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26	V27	V28
0	0	0	0	0	0	0	0	0	0	0	0	0
V29	V30	V31	V32	V33	V34	Class						
0	0	0	0	0	0	0						

```
data("Sonar", package = "mlbench")
sonar_data <- as.data.table(Sonar)
cat("\n缺失值检查:\n")
```

缺失值检查:

```
missing_sonar <- colSums(is.na(sonar_data))
print(missing_sonar)
```

V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13
0	0	0	0	0	0	0	0	0	0	0	0	0
V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26

0	0	0	0	0	0	0	0	0	0	0	0	0
V27	V28	V29	V30	V31	V32	V33	V34	V35	V36	V37	V38	V39
0	0	0	0	0	0	0	0	0	0	0	0	0
V40	V41	V42	V43	V44	V45	V46	V47	V48	V49	V50	V51	V52
0	0	0	0	0	0	0	0	0	0	0	0	0
V53	V54	V55	V56	V57	V58	V59	V60	Class				
0	0	0	0	0	0	0	0	0	0			

```
str(sonar_data)
```

Classes 'data.table' and 'data.frame': 208 obs. of 61 variables:

```
$ V1 : num 0.02 0.0453 0.0262 0.01 0.0762 0.0286 0.0317 0.0519 0.0223 0.0164 .
$ V2 : num 0.0371 0.0523 0.0582 0.0171 0.0666 0.0453 0.0956 0.0548 0.0375 0.01
$ V3 : num 0.0428 0.0843 0.1099 0.0623 0.0481 ...
$ V4 : num 0.0207 0.0689 0.1083 0.0205 0.0394 ...
$ V5 : num 0.0954 0.1183 0.0974 0.0205 0.059 ...
$ V6 : num 0.0986 0.2583 0.228 0.0368 0.0649 ...
$ V7 : num 0.154 0.216 0.243 0.11 0.121 ...
$ V8 : num 0.16 0.348 0.377 0.128 0.247 ...
$ V9 : num 0.3109 0.3337 0.5598 0.0598 0.3564 ...
$ V10 : num 0.211 0.287 0.619 0.126 0.446 ...
$ V11 : num 0.1609 0.4918 0.6333 0.0881 0.4152 ...
$ V12 : num 0.158 0.655 0.706 0.199 0.395 ...
$ V13 : num 0.2238 0.6919 0.5544 0.0184 0.4256 ...
$ V14 : num 0.0645 0.7797 0.532 0.2261 0.4135 ...
$ V15 : num 0.066 0.746 0.648 0.173 0.453 ...
$ V16 : num 0.227 0.944 0.693 0.213 0.533 ...
$ V17 : num 0.31 1 0.6759 0.0693 0.7306 ...
$ V18 : num 0.3 0.887 0.755 0.228 0.619 ...
$ V19 : num 0.508 0.802 0.893 0.406 0.203 ...
$ V20 : num 0.48 0.782 0.862 0.397 0.464 ...
$ V21 : num 0.578 0.521 0.797 0.274 0.415 ...
$ V22 : num 0.507 0.405 0.674 0.369 0.429 ...
$ V23 : num 0.433 0.396 0.429 0.556 0.573 ...
$ V24 : num 0.555 0.391 0.365 0.485 0.54 ...
$ V25 : num 0.671 0.325 0.533 0.314 0.316 ...
$ V26 : num 0.641 0.32 0.241 0.533 0.229 ...
$ V27 : num 0.71 0.327 0.507 0.526 0.7 ...
$ V28 : num 0.808 0.277 0.853 0.252 1 ...
```

```

$ V29 : num 0.679 0.442 0.604 0.209 0.726 ...
$ V30 : num 0.386 0.203 0.851 0.356 0.472 ...
$ V31 : num 0.131 0.379 0.851 0.626 0.51 ...
$ V32 : num 0.26 0.295 0.504 0.734 0.546 ...
$ V33 : num 0.512 0.198 0.186 0.612 0.288 ...
$ V34 : num 0.7547 0.2341 0.2709 0.3497 0.0981 ...
$ V35 : num 0.854 0.131 0.423 0.395 0.195 ...
$ V36 : num 0.851 0.418 0.304 0.301 0.418 ...
$ V37 : num 0.669 0.384 0.612 0.541 0.46 ...
$ V38 : num 0.61 0.106 0.676 0.881 0.322 ...
$ V39 : num 0.494 0.184 0.537 0.986 0.283 ...
$ V40 : num 0.274 0.197 0.472 0.917 0.243 ...
$ V41 : num 0.051 0.167 0.465 0.612 0.198 ...
$ V42 : num 0.2834 0.0583 0.2587 0.5006 0.2444 ...
$ V43 : num 0.282 0.14 0.213 0.321 0.185 ...
$ V44 : num 0.4256 0.1628 0.2222 0.3202 0.0841 ...
$ V45 : num 0.2641 0.0621 0.2111 0.4295 0.0692 ...
$ V46 : num 0.1386 0.0203 0.0176 0.3654 0.0528 ...
$ V47 : num 0.1051 0.053 0.1348 0.2655 0.0357 ...
$ V48 : num 0.1343 0.0742 0.0744 0.1576 0.0085 ...
$ V49 : num 0.0383 0.0409 0.013 0.0681 0.023 0.0264 0.0507 0.0285 0.0777 0.0092 .
$ V50 : num 0.0324 0.0061 0.0106 0.0294 0.0046 0.0081 0.0159 0.0178 0.0439 0.0198
$ V51 : num 0.0232 0.0125 0.0033 0.0241 0.0156 0.0104 0.0195 0.0052 0.0061 0.0118
$ V52 : num 0.0027 0.0084 0.0232 0.0121 0.0031 0.0045 0.0201 0.0081 0.0145 0.009
$ V53 : num 0.0065 0.0089 0.0166 0.0036 0.0054 0.0014 0.0248 0.012 0.0128 0.0223
$ V54 : num 0.0159 0.0048 0.0095 0.015 0.0105 0.0038 0.0131 0.0045 0.0145 0.0179
$ V55 : num 0.0072 0.0094 0.018 0.0085 0.011 0.0013 0.007 0.0121 0.0058 0.0084 ..
$ V56 : num 0.0167 0.0191 0.0244 0.0073 0.0015 0.0089 0.0138 0.0097 0.0049 0.0068
$ V57 : num 0.018 0.014 0.0316 0.005 0.0072 0.0057 0.0092 0.0085 0.0065 0.0032 ..
$ V58 : num 0.0084 0.0049 0.0164 0.0044 0.0048 0.0027 0.0143 0.0047 0.0093 0.0035
$ V59 : num 0.009 0.0052 0.0095 0.004 0.0107 0.0051 0.0036 0.0048 0.0059 0.0056 .
$ V60 : num 0.0032 0.0044 0.0078 0.0117 0.0094 0.0062 0.0103 0.0053 0.0022 0.004
$ Class: Factor w/ 2 levels "M","R": 2 2 2 2 2 2 2 2 2 2 ...
- attr(*, ".internal.selfref")=<externalptr>

```

```

task_Ionosphere <- as_task_classif(Ionosphere_data, target = "Class", id = "Io
task_sonar <- as_task_classif(sonar_data, target = "Class", id = "sonar")

```

```

tasks = list(task_Ionosphere, task_sonar)

# 定义学习器
learners_extended <- list(
  lrn("classif.ranger", id = "random_forest", predict_type = "prob"),
  lrn("classif.xgboost", id = "xgboost", predict_type = "prob"),
  lrn("classif.svm", id = "svm", predict_type = "prob"),
  lrn("classif.kknn", id = "knn", predict_type = "prob"),
  lrn("classif.naive_bayes", id = "naive_bayes", predict_type = "prob"),
  lrn("classif.rpart", id = "decision_tree", predict_type = "prob")
)

# 设置不同的重采样策略
resamplings <- list(
  cv5 = rsmp("cv", folds = 5),
  cv10 = rsmp("cv", folds = 10),
  holdout = rsmp("holdout", ratio = 0.7),
  bootstrap = rsmp("bootstrap", repeats = 5)
)

# 创建 Benchmark 设计
design_extended <- benchmark_grid(
  tasks = tasks,
  learners = learners_extended,
  resamplings = resamplings[1] # 使用 5 折交叉验证以节省时间
)

# 执行 Benchmark
cat(" 开始扩展 Benchmark 分析...\n")

```

开始扩展Benchmark分析...

```

bmr_extended <- benchmark(design_extended, store_models = TRUE)

# 性能汇总
cat("\n=== Benchmark 性能汇总 ===\n")

```

=== Benchmark性能汇总 ===

```

measures_all <- msrs(c("classif.acc", "classif.auc", "classif.bacc", "classif.
bmr_results <- bmr_extended$aggregate(measures_all)

# 格式化结果
results_detailed <- bmr_results[, .(
  Task = task_id,
  Model = learner_id,
  Resampling = resampling_id,
  Accuracy = classif.acc,
  AUC = classif.auc,
  Balanced_Accuracy = classif.bacc,
  LogLoss = classif.ce
)]

print(results_detailed[order(Task, -Accuracy)])

```

	Task	Model	Resampling	Accuracy	AUC	Balanced_Accuracy
	<char>	<char>	<char>	<num>	<num>	<num>
1:	Ionosphere	svm	cv	0.9459960	0.9812734	0.9386592
2:	Ionosphere	xgboost	cv	0.9344467	0.9746098	0.9151088
3:	Ionosphere	random_forest	cv	0.9317103	0.9775356	0.9171006
4:	Ionosphere	decision_tree	cv	0.8719115	0.8898079	0.8489739
5:	Ionosphere	knn	cv	0.8433400	0.9276306	0.7879048
6:	Ionosphere	naive_bayes	cv	0.8093763	0.9304335	0.8190265
7:	sonar	xgboost	cv	0.8747967	0.9350120	0.8731633
8:	sonar	svm	cv	0.8608595	0.9532901	0.8541965
9:	sonar	knn	cv	0.8606272	0.9570089	0.8558830
10:	sonar	random_forest	cv	0.8318235	0.9350790	0.8228248
11:	sonar	decision_tree	cv	0.7548200	0.7928233	0.7559099
12:	sonar	naive_bayes	cv	0.6872242	0.8058495	0.6939369
	LogLoss					
	<num>					
1:	0.05400402					
2:	0.06555332					
3:	0.06828974					
4:	0.12808853					
5:	0.15665996					
6:	0.19062374					

```

7: 0.12520325
8: 0.13914053
9: 0.13937282
10: 0.16817654
11: 0.24518002
12: 0.31277584

```

```
# 按任务分组统计
```

```
cat("\n=== 按任务分组性能统计 ===\n")
```

```
=== 按任务分组的性能统计 ===
```

```

task_stats <- results_detailed[, .(
  Mean_Accuracy = mean(Accuracy),
  Std_Accuracy = sd(Accuracy),
  Best_Model = .SD[which.max(Accuracy)]$Model,
  Best_Accuracy = max(Accuracy)
), by = Task]

print(task_stats)

```

	Task	Mean_Accuracy	Std_Accuracy	Best_Model	Best_Accuracy
	<char>	<num>	<num>	<char>	<num>
1:	Ionosphere	0.8894634	0.05630916	svm	0.9459960
2:	sonar	0.8116918	0.07473453	xgboost	0.8747967

Benchmark 结果可视化

```
# Benchmark 结果可视化
```

```
# 1. 按任务和模型的准确率热图
```

```

accuracy_matrix <- dcast(results_detailed, Model ~ Task, value.var = "Accuracy")
print(" 准确率矩阵:")

```

```
[1] "准确率矩阵:"
```

```
print(accuracy_matrix)
```

```
Key: <Model>
```

	Model	Ionosphere	sonar
	<char>	<num>	<num>

```

1: decision_tree  0.8719115 0.7548200
2:                knn  0.8433400 0.8606272
3:  naive_bayes   0.8093763 0.6872242
4: random_forest  0.9317103 0.8318235
5:                svm  0.9459960 0.8608595
6:       xgboost    0.9344467 0.8747967

```

可视化热图

```

heatmap_data <- as.matrix(accuracy_matrix[, -1])
rownames(heatmap_data) <- accuracy_matrix$Model

```

```
library(reshape2)
```

```

heatmap_df <- melt(accuracy_matrix, id.vars = "Model",
                   variable.name = "Task", value.name = "Accuracy")

```

```

p1 <- ggplot(heatmap_df, aes(x = Task, y = Model, fill = Accuracy)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(Accuracy, 3)), color = "black", size = 3) +
  scale_fill_gradient2(low = "red", mid = "white", high = "blue", midpoint = 0.5) +
  labs(title = " 模型在不同任务上的准确率热图",
       x = " 任务", y = " 模型") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

2. 模型性能比较箱线图

```

p2 <- ggplot(results_detailed, aes(x = reorder(Model, Accuracy), y = Accuracy,
                                     fill = Model)) +
  geom_boxplot(alpha = 0.7) +
  geom_jitter(width = 0.2, alpha = 0.5) +
  labs(title = " 模型性能分布", x = " 模型", y = " 准确率") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        legend.position = "none") +
  coord_flip()

```

3. 任务性能比较

```

p3 <- ggplot(results_detailed, aes(x = Task, y = Accuracy, fill = Task)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = " 不同任务的性能比较", x = " 任务", y = " 准确率") +
  theme_minimal() +

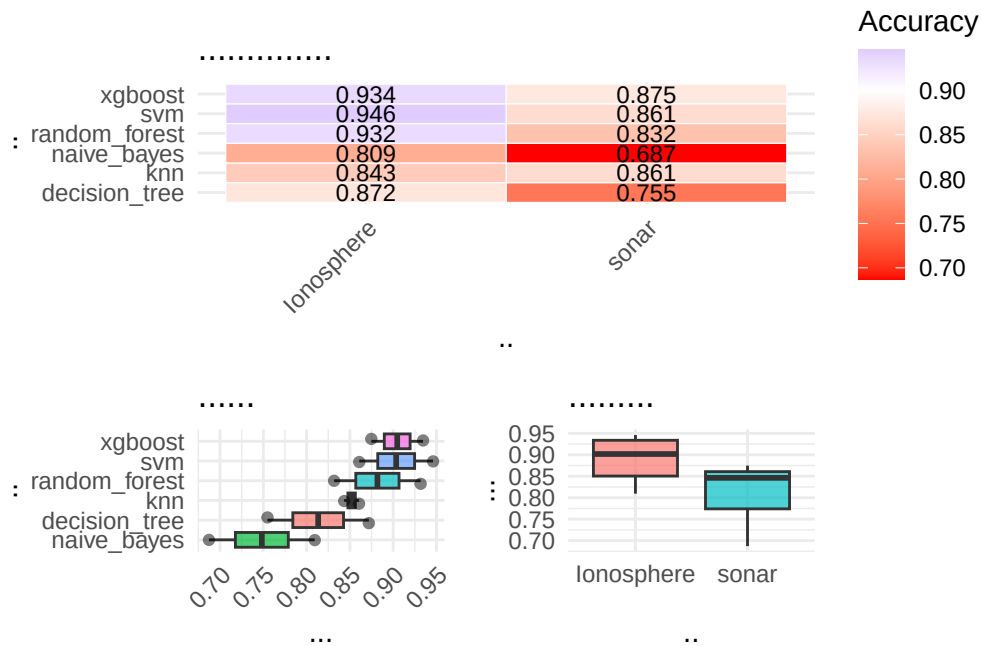
```

```
theme(legend.position = "none")
```

```
# 组合图形
```

```
library(patchwork)
```

```
(p1) / (p2 | p3)
```



统计显著性检验

```
# 统计显著性检验
```

```
# 提取所有 resample 结果的预测
```

```
predictions_all <- bmr_extended$score()
```

```
# 查看提取的结果结构
```

```
str(predictions_all, max.level = 1)
```

```
Classes 'bmr_score', 'data.table' and 'data.frame': 60 obs. of 11 variables:
 $ uhash      : chr  "99fbc943-df78-4d16-8739-776bac47e35b" "99fbc943-
df78-4d16-8739-776bac47e35b" "99fbc943-df78-4d16-8739-776bac47e35b" "99fbc943-
df78-4d16-8739-776bac47e35b" ...
 $ nr         : int   1 1 1 1 1 2 2 2 2 2 ...
 $ task       :List of 60
 $ task_id    : chr  "Ionosphere" "Ionosphere" "Ionosphere" "Ionosphere" ...
```



```

$ learner          :List of 60
$ learner_id       : chr "random_forest" "random_forest" "random_forest" "random_forest" ...
$ resampling       :List of 60
$ resampling_id    : chr "cv" "cv" "cv" "cv" ...
$ iteration        : int 1 2 3 4 5 1 2 3 4 5 ...
$ prediction_test  :List of 60
$ classif.ce       : num 0.0986 0.0571 0.0857 0.0429 0.0571 ...
- attr(*, ".internal.selfref")=<externalptr>

# 提取具体的预测对象
predictions_list <- predictions_all$prediction

# Friedman 检验 - 比较多个模型的性能
if (requireNamespace("scmamp", quietly = TRUE)) {
  library(scmamp)

  # 准备数据用于 Friedman 检验
  accuracy_matrix <- dcast(results_detailed, Task ~ Model, value.var = "Accuracy")
  accuracy_data <- as.matrix(accuracy_matrix[, -1])

  cat("\n=== Friedman 检验 ===\n")
  friedman_test <- friedmanTest(accuracy_data)
  print(friedman_test)

  if (friedman_test$p.value < 0.05) {
    cat(" 模型间存在显著差异，进行事后检验...\n")

    # Nemenyi 事后检验
    nemenyi_test <- nemenyiTest(accuracy_data)
    print(nemenyi_test)

    # 可视化 CD 图
    plotCD(accuracy_data, alpha = 0.05)
  } else {
    cat(" 模型间无显著差异\n")
  }
} else {
  cat("scmamp 包未安装，跳过统计检验\n")
}

```

scmamp 包未安装，跳过统计检验

```
# 简单的成对 t 检验
```

```
cat("\n=== 最佳模型 vs 其他模型的成对 t 检验 ===\n")
```

```
=== 最佳模型 vs 其他模型的成对t检验 ===
```

```
best_model <- results_detailed[which.max(Accuracy)]$Model
```

```
best_accuracies <- results_detailed[Model == best_model]$Accuracy
```

```
for (model in unique(results_detailed$Model)) {
```

```
  if (model != best_model) {
```

```
    model_accuracies <- results_detailed[Model == model]$Accuracy
```

```
    t_test <- t.test(best_accuracies, model_accuracies, paired = TRUE)
```

```
    cat(sprintf("%s vs %s: p-value = %.4f\n", best_model, model, t_test$p.va
```

```
  }
```

```
}
```

```
svm vs random_forest: p-value = 0.2089
```

```
svm vs xgboost: p-value = 0.9405
```

```
svm vs knn: p-value = 0.4986
```

```
svm vs naive_bayes: p-value = 0.0756
```

```
svm vs decision_tree: p-value = 0.1118
```

14.2 Benchmark 分析总结

mlr3 Benchmark 优势

统一的接口：benchmark() 函数提供统一的 Benchmark 接口灵活的设计：支持多个任务、学习器、重采样策略的组合内置存储：可以存储模型和预测结果用于后续分析丰富的结果：自动计算多个评估指标统计检验：内置 Friedman 检验等统计方法

最佳实践建议

系统化比较：在项目初期进行全面的 Benchmark 分析多维度评估：考虑准确率、训练时间、可解释性等多个维度统计验证：使用统计检验验证性能差异的显著性业务导向：根据具体业务需求选择最适合的模型可重复性：设置随机种子确保结果可重复

这样的 Benchmark 分析为模型选择提供了科学依据，确保选择的模型不仅在测试集上表现好，而且具有统计显著性。

14.4 R 语言-多分类

数据集说明本案例使用经典的鸢尾花数据集（Iris Dataset），包含三种鸢尾花的萼片和花瓣测量数据。目标：预测鸢尾花的种类（Species）

任务类型：多分类问题（3 个类别）

业务价值：植物分类、物种识别

数据加载与探索

```
# 综合案例：鸢尾花分类 - R 语言实现 (mlr3 框架)
```

```
# 加载必要的包
```

```
library(mlr3)
```

```
library(mlr3verse)
```

```
library(mlr3pipelines)
```

```
library(mlr3tuning)
```

```
library(data.table)
```

```
library(ggplot2)
```

```
library(corrplot)
```

```
library(gridExtra)
```

```
library(patchwork)
```

```
# 加载鸢尾花数据集
```

```
data("iris", package = "datasets")
```

```
iris_data <- as.data.table(iris)
```

```
# 数据基本信息
```

```
str(iris_data)
```

```
Classes 'data.table' and 'data.frame': 150 obs. of 5 variables:
```

```
$ Sepal.Length: num 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
```

```
$ Sepal.Width : num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
```

```
$ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
```

```
$ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
```

```
$ Species : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
```

```
- attr(*, ".internal.selfref")=<externalptr>
```

```
# 检查缺失值
```

```
cat("\n缺失值检查:\n")
```

缺失值检查：

```
print(colSums(is.na(iris_data)))
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	0	0	0	0

数据可视化分析

```
# 数据可视化分析 - R 语言
```

```
# 类别分布
```

```
p1 <- ggplot(iris_data, aes(x = Species, fill = Species)) +
  geom_bar(alpha = 0.7) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = " 类别分布", x = " 鸢尾花种类", y = " 数量") +
  theme_minimal() +
  theme(legend.position = "none")
```

```
# 花萼长度与宽度的关系
```

```
p2 <- ggplot(iris_data, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
  geom_point(alpha = 0.7, size = 2) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " 花萼长度 vs 宽度", x = " 花萼长度", y = " 花萼宽度") +
  theme_minimal()
```

```
# 花瓣长度与宽度的关系
```

```
p3 <- ggplot(iris_data, aes(x = Petal.Length, y = Petal.Width, color = Species)) +
  geom_point(alpha = 0.7, size = 2) +
  scale_color_brewer(palette = "Set1") +
  labs(title = " 花瓣长度 vs 宽度", x = " 花瓣长度", y = " 花瓣宽度") +
  theme_minimal()
```

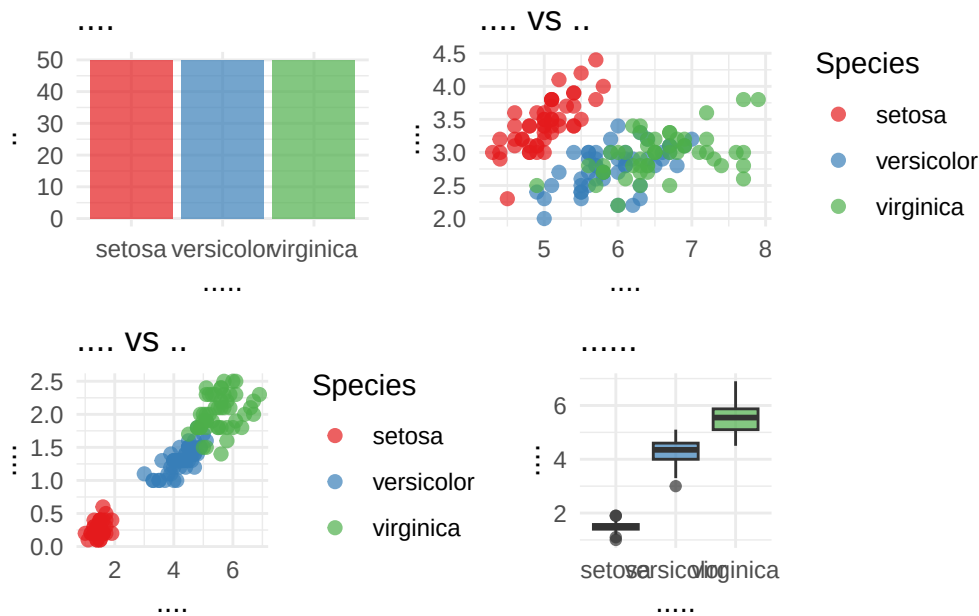
```
# 特征分布箱线图
```

```
p4 <- ggplot(iris_data, aes(x = Species, y = Petal.Length, fill = Species)) +
  geom_boxplot(alpha = 0.7) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = " 花瓣长度分布", x = " 鸢尾花种类", y = " 花瓣长度") +
  theme_minimal() +
```

```
theme(legend.position = "none")
```

```
# 组合图形
```

```
(p1 + p2) / (p3 + p4)
```



创建 mlr3 任务和数据预处理

```
# 创建 mlr3 任务和数据预处理
```

```
# 创建分类任务
```

```
task_iris <- as_task_classif(iris_data, target = "Species", id = "iris", store_
```

```
# 查看任务信息
```

```
cat(" 任务信息:\n")
```

任务信息:

```
print(task_iris)
```

```
-- <TaskClassif> (150x5) -----
-----
* Target: Species
* Target classes: setosa (33%), versicolor (33%), virginica (33%)
```

```

* Properties: multiclass
* Features (4):
  * dbl (4): Petal.Length, Petal.Width, Sepal.Length, Sepal.Width

# 数据预处理管道
preprocess_pipeline <- po("scale") %>>%
  po("encode") %>>%
  po("colapply", applicator = as.numeric)

# 应用预处理
task_preprocessed <- preprocess_pipeline$train(task_iris)[[1]]

cat("\n预处理后的任务信息:\n")

```

预处理后的任务信息：

```
print(task_preprocessed)
```

```

-- <TaskClassif> (150x5) -----
-----
* Target: Species
* Target classes: setosa (33%), versicolor (33%), virginica (33%)
* Properties: multiclass
* Features (4):
  * dbl (4): Petal.Length, Petal.Width, Sepal.Length, Sepal.Width

```

模型训练与比较

```

# 模型训练与比较 - mlr3

# 定义学习器
learners <- list(
  lrn("classif.ranger", id = "random_forest", predict_type = "prob"),
  lrn("classif.xgboost", id = "xgboost", predict_type = "prob"),
  lrn("classif.svm", id = "svm", predict_type = "prob"),
  lrn("classif.kknn", id = "knn", predict_type = "prob")
)

# 设置交叉验证和评估指标

```

```

resampling <- rsmp("cv", folds = 5)
measures <- msrs(c("classif.acc", "classif.auc", "classif.bacc", "classif.ce")

# 基准测试
design <- benchmark_grid(
  tasks = task_preprocessed,
  learners = learners,
  resamplings = resampling
)

bmr <- benchmark(design, store_backends=TRUE)

# 性能评估
cat(" 模型性能比较:\n")

```

模型性能比较：

```

bmr_aggregated <- bmr$aggregate(measures)

# 格式化输出结果
results_r <- bmr_aggregated[, .(
  Model = learner_id,
  Accuracy = classif.acc,
  AUC = classif.auc,
  Balanced_Accuracy = classif.bacc,
  LogLoss = classif.ce
)]

print(results_r[order(-Accuracy)])

```

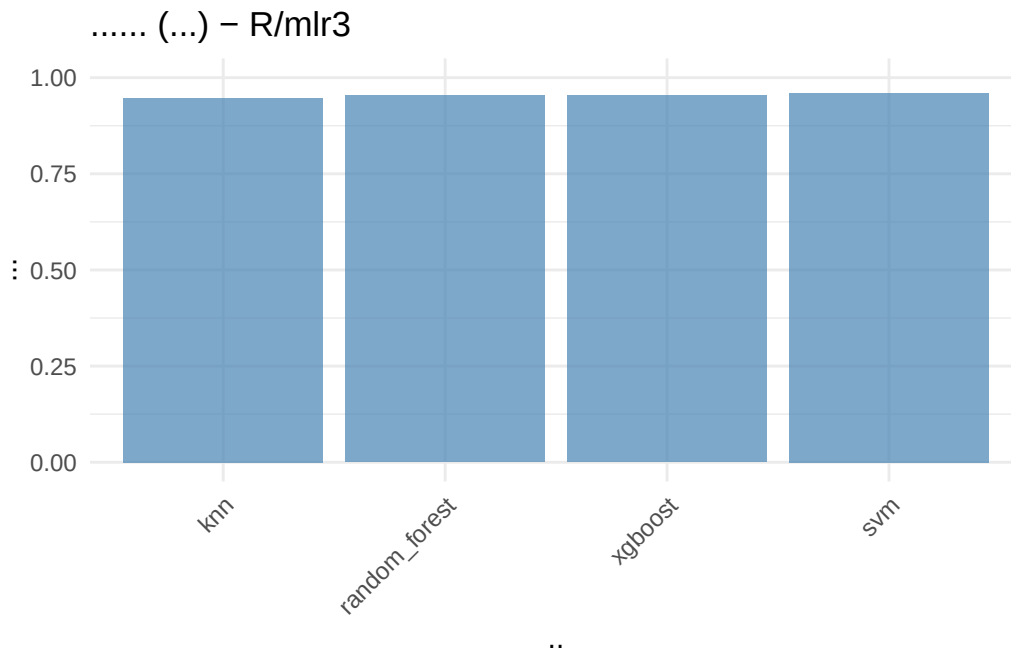
	Model	Accuracy	AUC	Balanced_Accuracy	LogLoss
	<char>	<num>	<num>	<num>	<num>
1:	svm	0.9600000	NaN	0.9619899	0.04000000
2:	random_forest	0.9533333	NaN	0.9545825	0.04666667
3:	xgboost	0.9533333	NaN	0.9568617	0.04666667
4:	knn	0.9466667	NaN	0.9545825	0.05333333

```

# 性能可视化
ggplot(results_r, aes(x = reorder(Model, Accuracy), y = Accuracy)) +
  geom_bar(stat = "identity", fill = "steelblue", alpha = 0.7) +

```

```
labs(title = " 模型性能比较 (准确率) - R/mlr3", x = " 模型", y = " 准确率") +
theme_minimal() +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
ylim(0, 1)
```



超参数调优

```
# 超参数调优示例 - 随机森林
library(mlr3)           # 核心
library(mlr3learners)   # 学习器
library(mlr3tuning)     # 参数调优 (包含 tnr())
library(paradox)        # 参数空间
library(mlr3viz)        # 可视化
# 定义随机森林学习器
learner_rf_tune <- lrn("classif.ranger", predict_type = "prob", num.trees =

# 定义超参数空间
param_set <- ps(
  mtry = p_int(lower = 1, upper = ncol(iris_data) - 1),
  min.node.size = p_int(lower = 1, upper = 10),
  sample.fraction = p_dbl(lower = 0.6, upper = 0.9)
)
```



```
# 定义调优器
tuner <- tnr("grid_search", resolution = 5)
terminator <- trm("evals", n_evals = 10)

# 自动调优
at <- auto_tuner(
  tuner = tuner,
  learner = learner_rf_tune,
  resampling = rsmp("holdout"),
  measure = msr("classif.ce"),
  search_space = param_set,
  terminator = terminator
)

# 在数据上演示调优
set.seed(123)
at$train(task_preprocessed)

cat(" 最佳参数:\n")
```

最佳参数:

```
print(at$tuning_result)
```

	mtry	min.node.size	sample.fraction	learner_param_vals	x_domain	classif.ce
	<int>	<int>	<num>	<list>	<list>	<num>
1:	2	3	0.6	<list[5]>	<list[3]>	0.04

```
# 使用调优后的模型进行预测
predictions_tuned <- at$predict(task_preprocessed)
cat(" 调优完成\n")
```

调优完成

```
cat(" 调优后模型准确率:", predictions_tuned$score(msr("classif.acc")), "\n")
```

调优后模型准确率: 0.9733333

模型解释

```

# 模型解释 - 特征重要性
library(ggplot2)
# 训练最佳模型 (随机森林)
set.seed(123)
learner_best <- lrn("classif.ranger", predict_type = "prob", importance = "p
learner_best$train(task_preprocessed)

# 特征重要性
if (!is.null(learner_best$importance)) {
  feature_importance <- learner_best$importance()
  importance_df <- data.frame(
    Feature = names(feature_importance),
    Importance = as.numeric(feature_importance)
  )
  importance_df <- importance_df[order(-importance_df$Importance), ]

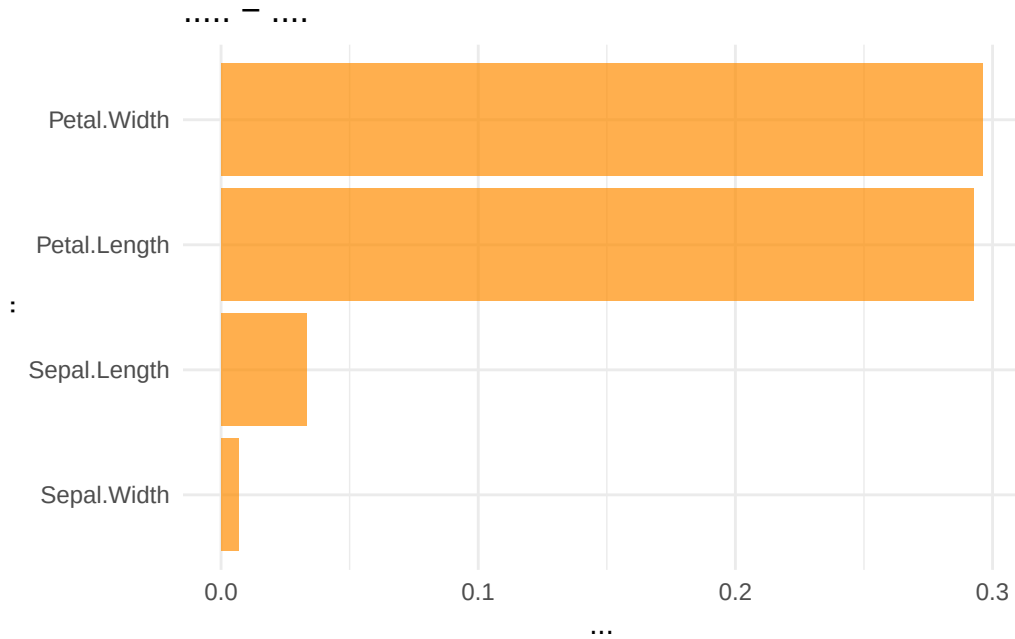
  cat(" 特征重要性排序:\n")
  print(importance_df)

  # 可视化特征重要性
  ggplot(importance_df, aes(x = reorder(Feature, Importance), y = Importance
    geom_bar(stat = "identity", fill = "darkorange", alpha = 0.7) +
    labs(title = " 特征重要性 - 随机森林", x = " 特征", y = " 重要性") +
    coord_flip() +
    theme_minimal()
} else {
  cat(" 该学习器不支持特征重要性计算\n")
}

```

特征重要性排序:

	Feature	Importance
1	Petal.Width	0.296006354
2	Petal.Length	0.292785833
3	Sepal.Length	0.033216550
4	Sepal.Width	0.006679592



模型评估与混淆矩阵

```
# 模型评估与混淆矩阵

# 使用最佳模型进行预测
predictions <- learner_best$predict(task_preprocessed)

# 混淆矩阵
cat(" 混淆矩阵:\n")
```

混淆矩阵：

```
print(predictions$confusion)
```

	truth		
response	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	48	1
virginica	0	2	49

14.5 Python—多分类

数据加载与探索

```
# 综合案例：鸢尾花分类 - Python 语言实现 (sklearn 框架)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             roc_auc_score, confusion_matrix, classification_report,
                             roc_curve, auc)
from sklearn.inspection import permutation_importance
import warnings
warnings.filterwarnings('ignore')

# 设置图形样式
plt.style.use('default')
sns.set_palette("husl")

# 加载鸢尾花数据集
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')
iris_data = pd.concat([X, y], axis=1)

# 映射数字标签到类别名称
target_names = iris.target_names
iris_data['species_name'] = iris_data['species'].map(lambda x: target_names[x])

print(" 数据集基本信息:")
```

数据集基本信息：

```
print(f" 样本数: {iris_data.shape[0]}")
```

样本数: 150

```
print(f" 变量数: {iris_data.shape[1]}")
```

变量数: 6

```
print("\n变量名称:")
```

变量名称:

```
print(iris_data.columns.tolist())
```

['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)',

```
# 类别分布
```

```
print("\n类别分布:")
```

类别分布:

```
class_dist = iris_data['species_name'].value_counts()
print(class_dist)
```

species_name

setosa 50

versicolor 50

virginica 50

Name: count, dtype: int64

```
print("\n类别比例:")
```

类别比例:

```
print(class_dist / class_dist.sum())
```

species_name

setosa 0.333333

versicolor 0.333333

virginica 0.333333

Name: count, dtype: float64

```
print("\n数据概览:")
```

数据概览:

```
print(iris_data.describe())
```

	sepal length (cm)	sepal width (cm)	...	petal width (cm)	species
count	150.000000	150.000000	...	150.000000	150.000000
mean	5.843333	3.057333	...	1.199333	1.000000
std	0.828066	0.435866	...	0.762238	0.819232
min	4.300000	2.000000	...	0.100000	0.000000
25%	5.100000	2.800000	...	0.300000	0.000000
50%	5.800000	3.000000	...	1.300000	1.000000
75%	6.400000	3.300000	...	1.800000	2.000000
max	7.900000	4.400000	...	2.500000	2.000000

[8 rows x 5 columns]

```
print("\n缺失值检查:")
```

缺失值检查:

```
print(iris_data.isnull().sum())
```

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
species_name         0
dtype: int64
```

数据可视化分析

```
# 数据可视化分析 - Python
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 12))
```

```
# 类别分布
```

```
class_counts = iris_data['species_name'].value_counts()
```

```
axes[0,0].bar(class_counts.index, class_counts.values, color=['steelblue', 'darkred', 'darkgreen'])
axes[0,0].set_title('类别分布')
axes[0,0].set_ylabel('数量')
axes[0,0].tick_params(axis='x', rotation=45)

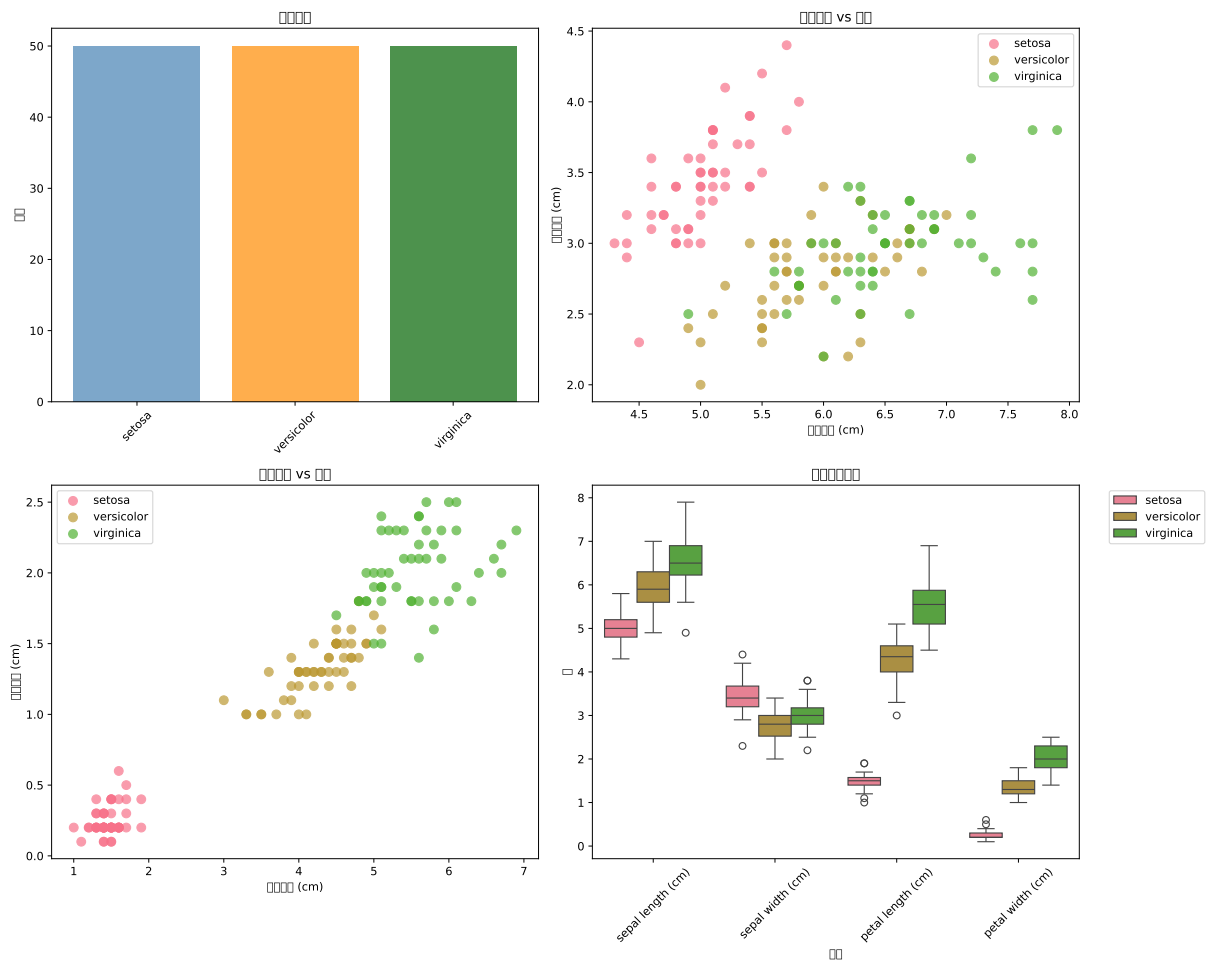
# 花萼长度与宽度的关系
species_colors = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}
for species in iris_data['species_name'].unique():
    species_data = iris_data[iris_data['species_name'] == species]
    axes[0,1].scatter(species_data['sepal length (cm)'], species_data['sepal width (cm)'],
                      label=species, alpha=0.7, s=60)
axes[0,1].set_title('花萼长度 vs 宽度')
axes[0,1].set_xlabel('花萼长度 (cm)')
axes[0,1].set_ylabel('花萼宽度 (cm)')
axes[0,1].legend()

# 花瓣长度与宽度的关系
for species in iris_data['species_name'].unique():
    species_data = iris_data[iris_data['species_name'] == species]
    axes[1,0].scatter(species_data['petal length (cm)'], species_data['petal width (cm)'],
                      label=species, alpha=0.7, s=60)
axes[1,0].set_title('花瓣长度 vs 宽度')
axes[1,0].set_xlabel('花瓣长度 (cm)')
axes[1,0].set_ylabel('花瓣宽度 (cm)')
axes[1,0].legend()

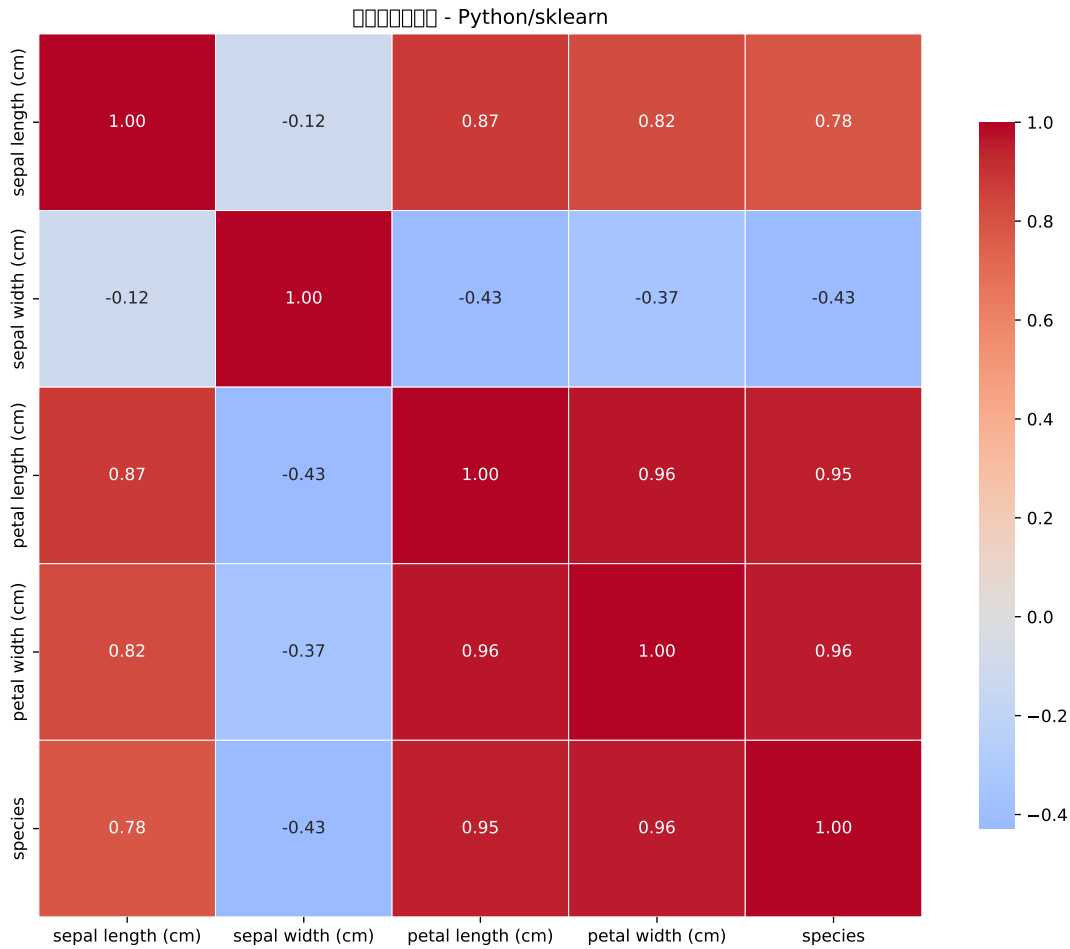
# 特征分布箱线图
feature_data = iris_data.melt(id_vars=['species_name'],
                              value_vars=['sepal length (cm)', 'sepal width (cm)',
                                           'petal length (cm)', 'petal width (cm)'])
sns.boxplot(data=feature_data, x='variable', y='value', hue='species_name', ax=axes[1,1])
axes[1,1].set_title('特征分布比较')
axes[1,1].set_xlabel('特征')
axes[1,1].set_ylabel('值')
axes[1,1].tick_params(axis='x', rotation=45)
axes[1,1].legend(bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
```

```
plt.show()
```



```
# 相关性热图
plt.figure(figsize=(10, 8))
corr_matrix = iris_data[['sepal length (cm)', 'sepal width (cm)',
                          'petal length (cm)', 'petal width (cm)', 'species']]
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=0.5, cbar_kws={"shrink": 0.8}, fmt='.2f')
plt.title('变量相关性热图 - Python/sklearn')
plt.tight_layout()
plt.show()
```

数据预处理

```
# 数据预处理 - Python

# 准备特征和目标变量
X = iris_data[['sepal length (cm)', 'sepal width (cm)',
                'petal length (cm)', 'petal width (cm)']]
y = iris_data['species']

# 数据划分（分层抽样保持类别比例）
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print(f" 训练集大小: {X_train.shape[0]}")
```

训练集大小: 105

```
print(f" 测试集大小: {X_test.shape[0]}")
```

测试集大小: 45

```
print(f" 训练集类别分布:\n{y_train.value_counts().sort_index()}")
```

训练集类别分布:

species

0 35

1 35

2 35

Name: count, dtype: int64

```
print(f" 测试集类别分布:\n{y_test.value_counts().sort_index()}")
```

测试集类别分布:

species

0 15

1 15

2 15

Name: count, dtype: int64

```
# 数据标准化
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
print(" 数据预处理完成")
```

数据预处理完成

模型训练与比较

```
# 模型训练与比较 - sklearn
```

```
# 定义模型字典
```

```
models = {
```

```
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000)
```

```
    'Random Forest': RandomForestClassifier(random_state=42),
```

```
    'K-Neighbors': KNeighborsClassifier(),
```

```
    'Gradient Boosting': GradientBoostingClassifier(random_state=42)
```

```
}

# 训练和评估模型
results = []

for name, model in models.items():
    # 训练模型
    model.fit(X_train_scaled, y_train)

    # 预测
    y_pred = model.predict(X_test_scaled)
    y_pred_proba = model.predict_proba(X_test_scaled)

    # 计算评估指标
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    # 多类 AUC (One-vs-Rest)
    try:
        auc_score = roc_auc_score(y_test, y_pred_proba, multi_class='ovr')
    except:
        auc_score = np.nan

    # 交叉验证
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='accuracy')
    cv_accuracy_mean = cv_scores.mean()

    results.append({
        'Model': name,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1_Score': f1,
        'AUC': auc_score,
        'CV_Accuracy': cv_accuracy_mean
    })
```

```
})
```

```
LogisticRegression(max_iter=1000, random_state=42)
RandomForestClassifier(random_state=42)
KNeighborsClassifier()
GradientBoostingClassifier(random_state=42)
```

```
# 创建结果 DataFrame
results_python = pd.DataFrame(results)
print(" 模型性能比较 - Python/sklearn:")
```

模型性能比较 - Python/sklearn:

```
print(results_python.sort_values('Accuracy', ascending=False))
```

	Model	Accuracy	Precision	...	F1_Score	AUC	CV_Accuracy
3	Gradient Boosting	0.933333	0.944444	...	0.932660	0.998519	0.961905
0	Logistic Regression	0.911111	0.915535	...	0.910714	0.995556	0.980952
2	K-Neighbors	0.911111	0.929825	...	0.909502	0.988889	0.942857
1	Random Forest	0.888889	0.898148	...	0.887767	0.992593	0.952381

```
[4 rows x 7 columns]
```

```
# 可视化模型比较
```

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

```
# 准确率比较
```

```
models_sorted_acc = results_python.sort_values('Accuracy')
axes[0,0].barh(models_sorted_acc['Model'], models_sorted_acc['Accuracy'], color='blue')
axes[0,0].set_xlabel('准确率')
axes[0,0].set_title('模型准确率比较 - Python/sklearn')
axes[0,0].set_xlim(0.8, 1.0)
```

```
(0.8, 1.0)
```

```
axes[0,0].grid(axis='x', alpha=0.3)
```

```
# F1 分数比较
```

```
models_sorted_f1 = results_python.sort_values('F1_Score')
axes[0,1].barh(models_sorted_f1['Model'], models_sorted_f1['F1_Score'], color='red')
axes[0,1].set_xlabel('F1 分数')
```

```
axes[0,1].set_title('模型 F1 分数比较 - Python/sklearn')
axes[0,1].set_xlim(0.8, 1.0)
```

(0.8, 1.0)

```
axes[0,1].grid(axis='x', alpha=0.3)
```

AUC 比较

```
models_sorted_auc = results_python.sort_values('AUC')
```

```
axes[1,0].barh(models_sorted_auc['Model'], models_sorted_auc['AUC'], color='darkblue')
```

```
axes[1,0].set_xlabel('AUC')
```

```
axes[1,0].set_title('模型 AUC 比较 - Python/sklearn')
```

```
axes[1,0].set_xlim(0.8, 1.0)
```

(0.8, 1.0)

```
axes[1,0].grid(axis='x', alpha=0.3)
```

交叉验证准确率比较

```
models_sorted_cv = results_python.sort_values('CV_Accuracy')
```

```
axes[1,1].barh(models_sorted_cv['Model'], models_sorted_cv['CV_Accuracy'], color='darkblue')
```

```
axes[1,1].set_xlabel('交叉验证准确率')
```

```
axes[1,1].set_title('交叉验证准确率比较 - Python/sklearn')
```

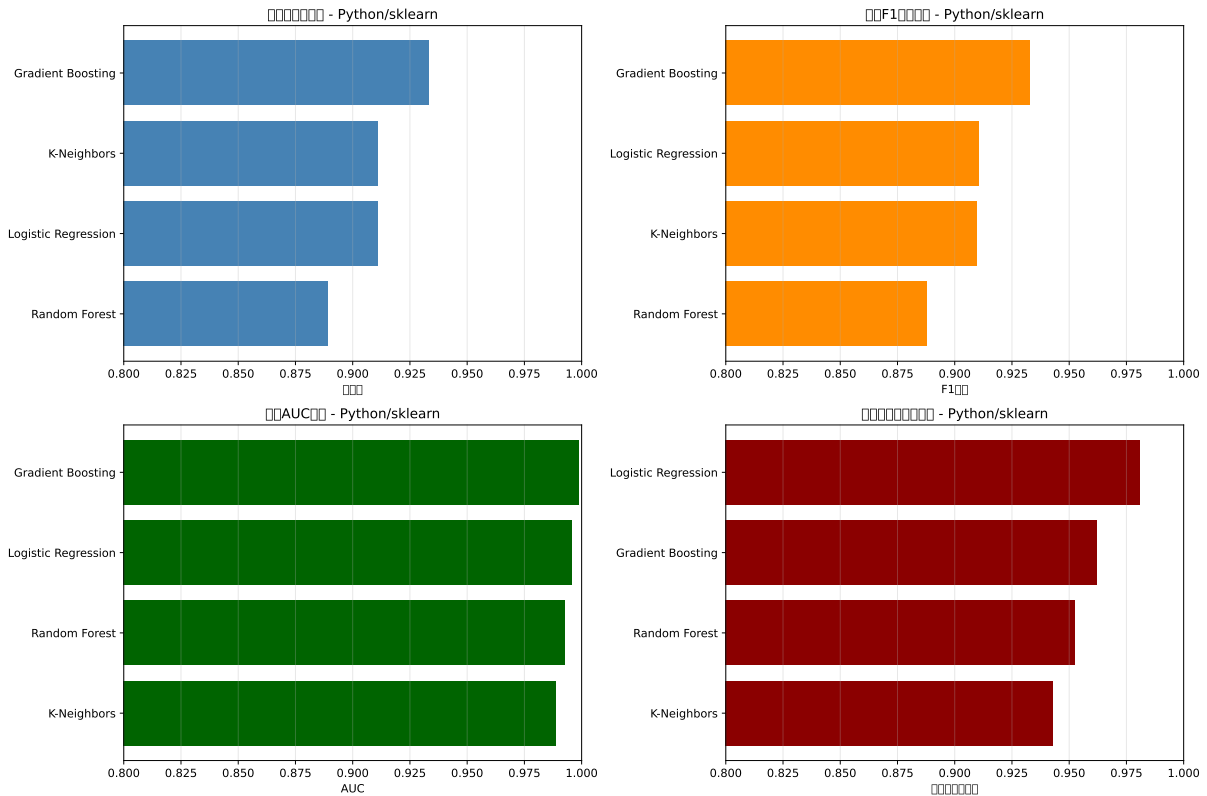
```
axes[1,1].set_xlim(0.8, 1.0)
```

(0.8, 1.0)

```
axes[1,1].grid(axis='x', alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```



详细模型评估

```
# 详细模型评估 - 最佳模型
```

```
# 选择最佳模型（基于准确率）
```

```
best_model_name = results_python.loc[results_python['Accuracy'].idxmax(), 'Model Name']
best_model = models[best_model_name]
```

```
print(f" 最佳模型: {best_model_name}")
```

最佳模型: Gradient Boosting

```
# 重新训练最佳模型
```

```
best_model.fit(X_train_scaled, y_train)
```

```
GradientBoostingClassifier(random_state=42)
```

```
# 预测
```

```
y_pred_best = best_model.predict(X_test_scaled)
```

```
y_pred_proba_best = best_model.predict_proba(X_test_scaled)
```

```
# 混淆矩阵
cm = confusion_matrix(y_test, y_pred_best)
print("\n混淆矩阵:")
```

混淆矩阵:

```
print(cm)
```

```
[[15  0  0]
 [ 0 15  0]
 [ 0  3 12]]
```

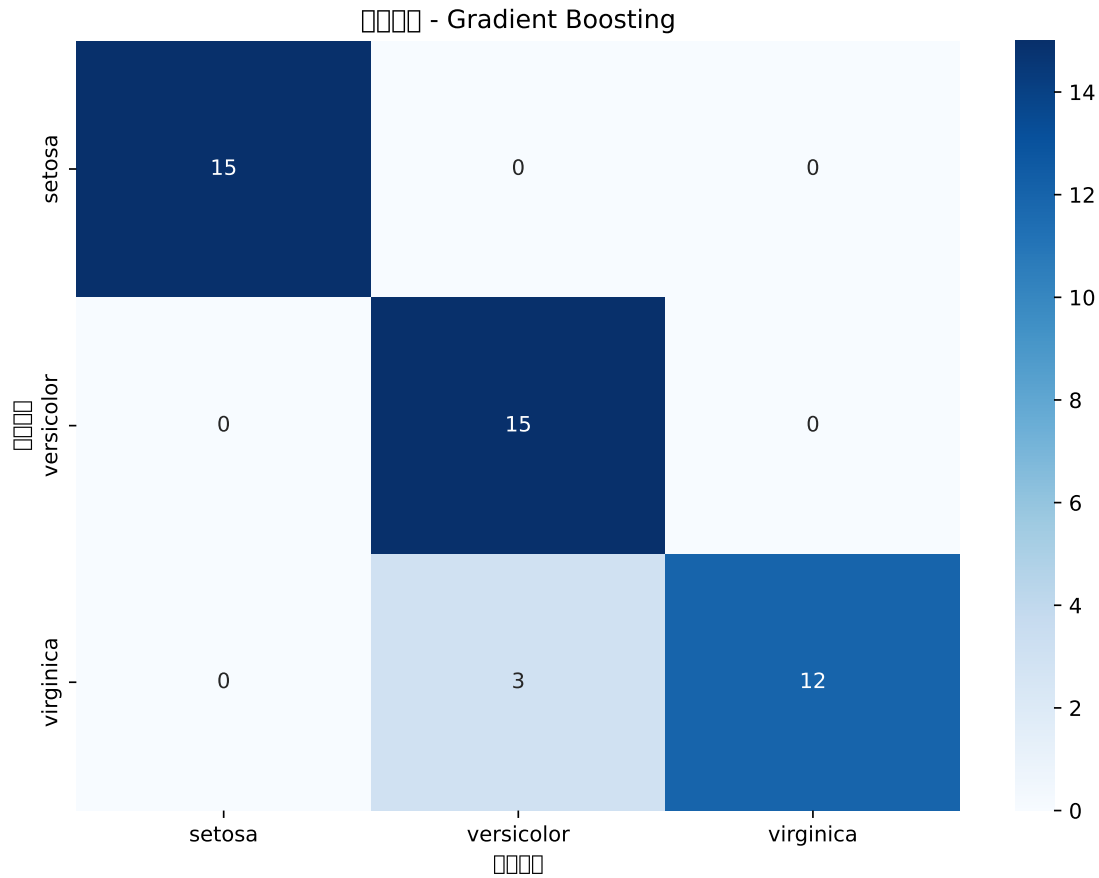
```
# 分类报告
print("\n分类报告:")
```

分类报告:

```
print(classification_report(y_test, y_pred_best, target_names=target_names))
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.83	1.00	0.91	15
virginica	1.00	0.80	0.89	15
accuracy			0.93	45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.93	0.93	45

```
# 可视化混淆矩阵
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=target_names, yticklabels=target_names)
plt.title(f'混淆矩阵 - {best_model_name}')
plt.xlabel('预测标签')
plt.ylabel('真实标签')
plt.tight_layout()
plt.show()
```



特征重要性分析

特征重要性分析 - Python

```
if hasattr(best_model, 'feature_importances_'):
    feature_importance = best_model.feature_importances_
    feature_names = X.columns

    # 创建特征重要性 DataFrame
    importance_df = pd.DataFrame({
        'Feature': feature_names,
        'Importance': feature_importance
    }).sort_values('Importance', ascending=False)

    print(" 特征重要性排序 - Python/sklearn:")
    print(importance_df)

    # 可视化特征重要性
```



```
plt.figure(figsize=(10, 6))
sns.barplot(data=importance_df, x='Importance', y='Feature', palette='viridis')
plt.title(f'特征重要性 - {best_model_name} (Python/sklearn)')
plt.xlabel('重要性')
plt.tight_layout()
plt.show()
```

else:

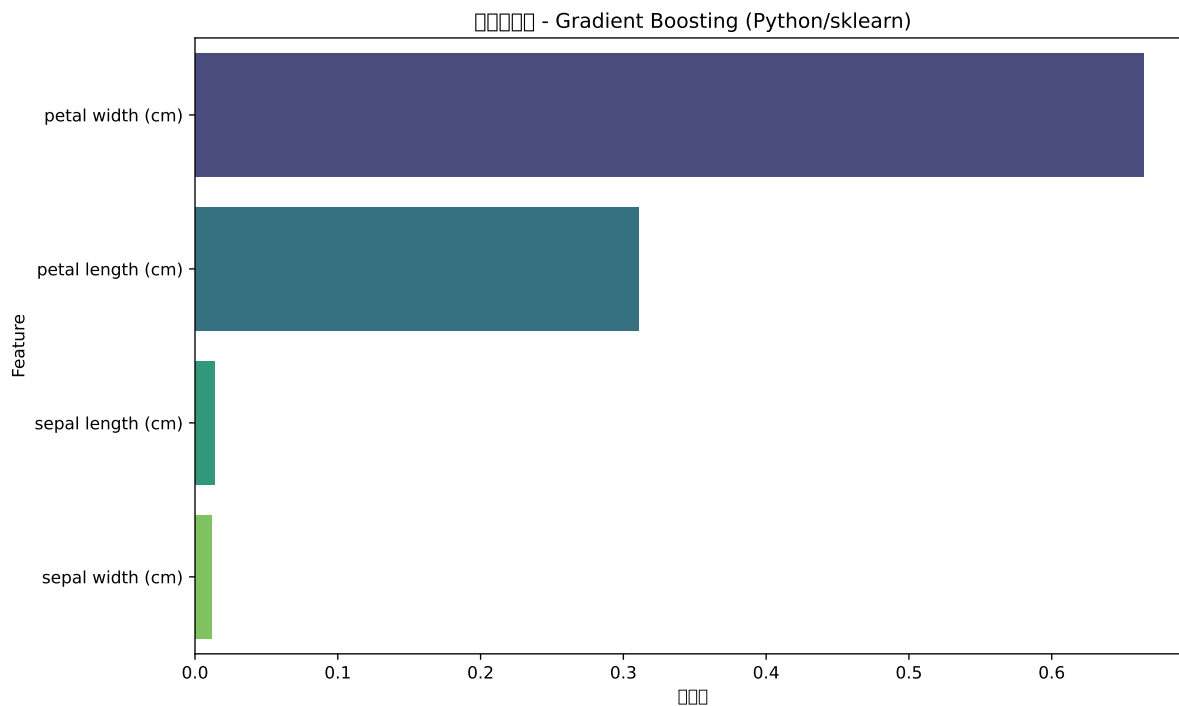
```
print(f"{best_model_name} 不支持特征重要性计算")

# 使用排列重要性作为替代
perm_importance = permutation_importance(
    best_model, X_test_scaled, y_test, n_repeats=10, random_state=42
)

perm_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': perm_importance.importances_mean
}).sort_values('Importance', ascending=False)

print("\n排列特征重要性:")
print(perm_importance_df)

# 可视化排列重要性
plt.figure(figsize=(10, 6))
sns.barplot(data=perm_importance_df, x='Importance', y='Feature', palette='viridis')
plt.title(f'排列特征重要性 - {best_model_name} (Python/sklearn)')
plt.xlabel('重要性')
plt.tight_layout()
plt.show()
```



14.6 多分类案例总结

关键发现

案例总结 - R 语言

```
cat("=== 鸢尾花分类案例总结 ===\n")
```

=== 鸢尾花分类案例总结 ===

```
cat("1. 数据特征:\n")
```

1. 数据特征：

```
cat("    - 数据集包含", nrow(iris_data), " 个样本, ", ncol(iris_data)-1, " 个特征\n")
```

- 数据集包含 150 个样本， 4 个特征

```
cat("    - 目标变量：鸢尾花种类 (3 个类别)\n")
```

- 目标变量：鸢尾花种类 (3 个类别)

```
cat("    - 类别分布均衡\n")
```

- 类别分布均衡

```
cat("    - 最重要的特征：花瓣长度和宽度\n\n")
```

- 最重要的特征：花瓣长度和宽度

```
cat("2. 模型性能比较:\n")
```

2. 模型性能比较：

```
print(results_r[order(-Accuracy)])
```

	Model <char>	Accuracy <num>	AUC <num>	Balanced_Accuracy <num>	LogLoss <num>
1:	svm	0.9600000	NaN	0.9619899	0.04000000
2:	random_forest	0.9533333	NaN	0.9545825	0.04666667
3:	xgboost	0.9533333	NaN	0.9568617	0.04666667
4:	knn	0.9466667	NaN	0.9545825	0.05333333

```
cat("\n3. 分类问题最佳实践:\n")
```

3. 分类问题最佳实践：

```
cat("    - 树模型和 SVM 在分类问题上表现优异\n")
```

- 树模型和SVM在分类问题上表现优异

```
cat("    - 特征标准化对距离-based 模型很重要\n")
```

- 特征标准化对距离-based模型很重要

```
cat("    - 多分类问题需要考虑合适的评估指标\n")
```

- 多分类问题需要考虑合适的评估指标

```
cat("    - 特征重要性分析有助于理解模型决策\n")
```

- 特征重要性分析有助于理解模型决策

```
# 案例总结 - Python
```

```
print("=== 鸢尾花分类案例总结 ===")
```

```
=== 鸢尾花分类案例总结 ===
```

```
print("1. 数据特征:")
```

1. 数据特征：

```
print(f"    - 数据集包含 {iris_data.shape[0]} 个样本, {iris_data.shape[1]-1} 个特征")
```

- 数据集包含 150 个样本, 5 个特征

```
print("    - 目标变量: 鸢尾花种类 (3 个类别)")
```

- 目标变量: 鸢尾花种类 (3 个类别)

```
print("    - 类别分布均衡")
```

- 类别分布均衡

```
print("    - 最重要的特征: 花瓣长度和宽度\n")
```

- 最重要的特征: 花瓣长度和宽度

```
print("2. 模型性能比较:")
```

2. 模型性能比较:

```
print(results_python[['Model', 'Accuracy', 'F1_Score', 'AUC']].sort_values('Accuracy', ascending=False))
```

	Model	Accuracy	F1_Score	AUC
	Gradient Boosting	0.933333	0.932660	0.998519
	Logistic Regression	0.911111	0.910714	0.995556
	K-Neighbors	0.911111	0.909502	0.988889
	Random Forest	0.888889	0.887767	0.992593

```
print("\n3. 分类问题最佳实践:")
```

3. 分类问题最佳实践:

```
print("    - 集成学习方法在分类问题上表现稳定")
```

- 集成学习方法在分类问题上表现稳定

```
print("    - 数据预处理对模型性能影响显著")
```

- 数据预处理对模型性能影响显著

```
print("    - 交叉验证提供更可靠的性能估计")
```

- 交叉验证提供更可靠的性能估计

```
print("    - 混淆矩阵和分类报告提供详细性能分析")
```

- 混淆矩阵和分类报告提供详细性能分析

技术要点回顾

数据预处理：- 特征标准化：确保不同尺度的特征具有可比性 - 类别编码：将文本标签转换为数值
- 数据划分：使用分层抽样保持类别比例

模型选择：- 逻辑回归：可解释性强，适合线性可分数据 - 树模型：自动处理非线性关系，提供特征重要性 - SVM：在高维空间中表现优异 - KNN：基于距离的简单有效方法

模型评估：- 准确率：整体分类正确率 - 精确率、召回率、F1 分数：更细致的性能评估 - AUC：模型区分能力的综合指标 - 混淆矩阵：详细分析各类别的分类情况

模型优化：- 超参数调优：提升模型性能 - 特征选择：基于重要性筛选特征 - 交叉验证：稳健的性能估计

业务应用建议

- 物种识别：扩展到其他植物或动物的分类识别
- 质量检测：应用于工业产品质量分类
- 医疗诊断：用于疾病分类和诊断辅助
- 客户细分：用于市场营销中的客户分类

15 案例综合分析

案例导读

本项目基于模拟的电商用户数据，进行全面的数据分析，包括：- 回归分析：预测用户消费金额 - 分类分析：预测用户购买意向 - 聚类分析：用户分群 - 关联分析：商品购买关联规则

数据说明

```
# 生成模拟数据
set.seed(123)
n <- 1000

# 用户基本信息
user_data <- data.frame(
  user_id = 1:n,
  age = sample(18:65, n, replace = TRUE),
  gender = sample(c("Male", "Female"), n, replace = TRUE, prob = c(0.48, 0.52)),
  income = round(rnorm(n, 50000, 15000)),
  region = sample(c("North", "South", "East", "West"), n, replace = TRUE),
  days_since_signup = sample(1:365, n, replace = TRUE),
  page_views = rpois(n, 25),
  time_on_site = round(rnorm(n, 300, 120)),
  cart_additions = rpois(n, 5)
)

# 生成购买行为数据
user_data$purchase_amount <- with(user_data,
  50 + 0.3 * income/1000 + 0.5 * page_views + 0.8 * time_on_site/60 +
  2 * cart_additions + rnorm(n, 0, 20)
)
```

```

user_data$made_purchase <- ifelse(user_data$purchase_amount > 120, 1, 0)

# 添加产品类别购买信息 - 增加相关性以产生关联规则
products <- c("Electronics", "Clothing", "Books", "Home", "Sports")
set.seed(123)
for(i in 1:length(products)) {
  product <- products[i]
  base_prob <- 0.4
  # 创建产品间的相关性
  if(i > 1) {
    # 让某些产品更可能一起购买
    correlated_prob <- user_data[[paste0("bought_", tolower(products[i-1]))]]
    user_data[[paste0("bought_", tolower(product))]] <- rbinom(n, 1, pmin(correlated_prob, base_prob))
  } else {
    user_data[[paste0("bought_", tolower(product))]] <- rbinom(n, 1, base_prob)
  }
}

head(user_data)

```

	user_id	age	gender	income	region	days_since_signup	page_views	time_on_site
1	1	48	Female	51762	West	5	29	173
2	2	32	Female	45027	West	22	28	450
3	3	31	Male	54174	West	4	38	147
4	4	20	Female	32216	South	352	23	243
5	5	59	Male	37462	North	131	23	340
6	6	60	Female	57654	North	231	15	334

	cart_additions	purchase_amount	made_purchase	bought_electronics
1	3	57.57759	0	0
2	3	59.83742	0	1
3	6	64.31744	0	0
4	10	94.88951	0	1
5	1	24.43966	0	1
6	5	114.33971	0	0

	bought_clothing	bought_books	bought_home	bought_sports
1	0	0	0	0
2	1	1	0	1
3	0	0	0	0

4	0	0	1	0
5	0	0	0	0
6	0	1	1	1

15.1 R 语言实现

15.1.1. 数据探索与预处理

数据概览

```
library(dplyr)
library(ggplot2)
library(corrplot)

# 基本统计信息
summary(user_data[, c("age", "income", "page_views", "time_on_site", "purchase_amount")])
```

age	income	page_views	time_on_site
Min. :18.00	Min. : 4282	Min. :11.00	Min. : -83.0
1st Qu.:29.00	1st Qu.: 40201	1st Qu.:21.00	1st Qu.:219.8
Median :42.00	Median : 49943	Median :25.00	Median :299.0
Mean :41.39	Mean : 50209	Mean :24.92	Mean :299.7
3rd Qu.:53.00	3rd Qu.: 60887	3rd Qu.:28.00	3rd Qu.:376.2
Max. :65.00	Max. :100856	Max. :42.00	Max. :688.0

purchase_amount
Min. : 9.555
1st Qu.: 76.051
Median : 91.131
Mean : 90.966
3rd Qu.:105.509
Max. :153.144

```
# 缺失值检查
apply(user_data, function(x) sum(is.na(x)))
```

user_id	age	gender	income
0	0	0	0

region	days_since_signup	page_views	time_on_site
0	0	0	0

cart_additions	purchase_amount	made_purchase	bought_electronics
0	0	0	0
bought_clothing	bought_books	bought_home	bought_sports
0	0	0	0

数据可视化

数值变量分布

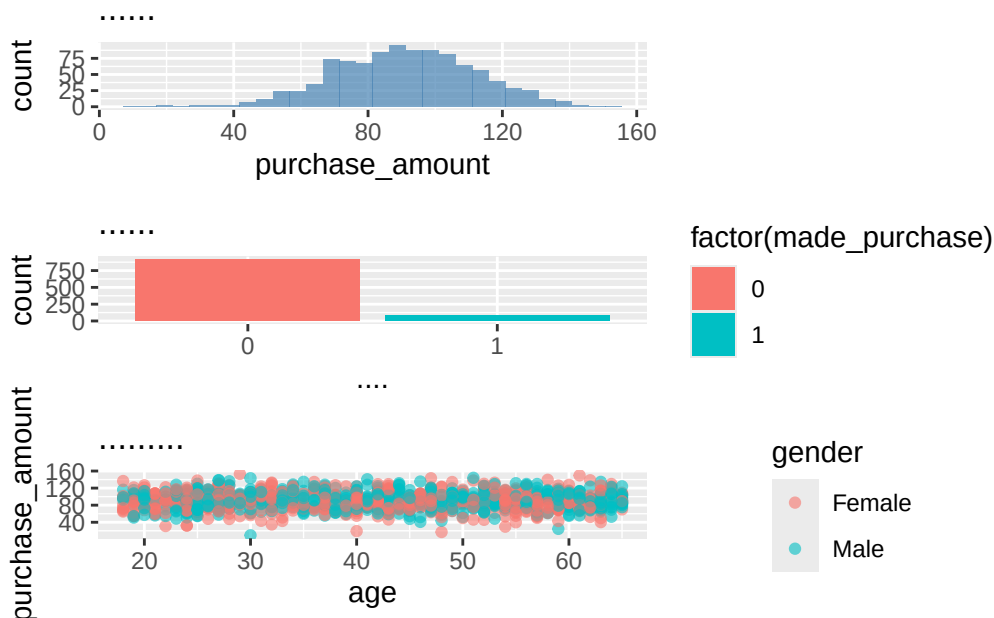
```
p1 <- ggplot(user_data, aes(x = purchase_amount)) +
  geom_histogram(bins = 30, fill = "steelblue", alpha = 0.7) +
  labs(title = " 购买金额分布")
```

```
p2 <- ggplot(user_data, aes(x = factor(made_purchase), fill = factor(made_purchase))) +
  geom_bar() +
  labs(title = " 购买行为分布", x = " 是否购买")
```

```
p3 <- ggplot(user_data, aes(x = age, y = purchase_amount, color = gender)) +
  geom_point(alpha = 0.6) +
  labs(title = " 年龄与购买金额关系")
```

```
library(patchwork)
```

```
p1 / p2 / p3
```



15.1.2. 回归分析：预测购买金额

线性回归模型

```
library(caret)

# 准备数据
reg_data <- user_data %>%
  select(age, income, page_views, time_on_site, cart_additions, purchase_amount)
  na.omit()

# 数据分割
set.seed(123)
train_index <- createDataPartition(reg_data$purchase_amount, p = 0.7, list = FALSE)
train_reg <- reg_data[train_index, ]
test_reg <- reg_data[-train_index, ]

# 训练线性回归模型
lm_model <- lm(purchase_amount ~ ., data = train_reg)
summary(lm_model)
```

Call:

```
lm(formula = purchase_amount ~ ., data = train_reg)
```

Residuals:

Min	1Q	Median	3Q	Max
-66.576	-12.853	-0.106	13.819	67.749

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	5.111e+01	5.704e+00	8.961	< 2e-16 ***
age	8.216e-04	5.349e-02	0.015	0.98775
income	2.969e-04	5.022e-05	5.913	5.28e-09 ***
page_views	2.720e-01	1.496e-01	1.818	0.06954 .
time_on_site	1.635e-02	6.067e-03	2.695	0.00721 **
cart_additions	2.638e+00	3.326e-01	7.933	8.57e-15 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 19.32 on 694 degrees of freedom
 Multiple R-squared: 0.1368, Adjusted R-squared: 0.1305
 F-statistic: 21.99 on 5 and 694 DF, p-value: < 2.2e-16

```
# 预测
predictions_lm <- predict(lm_model, newdata = test_reg)

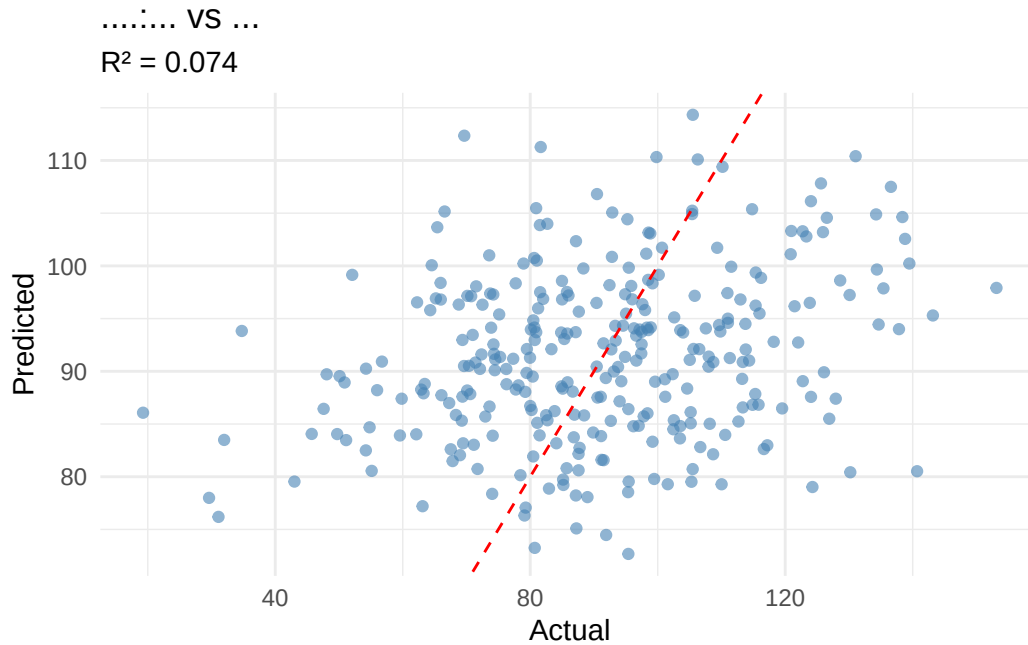
# 模型评估
reg_metrics <- data.frame(
  RMSE = RMSE(predictions_lm, test_reg$purchase_amount),
  MAE = MAE(predictions_lm, test_reg$purchase_amount),
  R2 = R2(predictions_lm, test_reg$purchase_amount)
)
print(reg_metrics)
```

	RMSE	MAE	R2
1	21.72114	17.54768	0.07424787

回归结果可视化

```
# 预测 vs 实际值
results_df <- data.frame(
  Actual = test_reg$purchase_amount,
  Predicted = predictions_lm
)

ggplot(results_df, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(title = " 线性回归: 预测值 vs 实际值",
       subtitle = paste("R² =", round(reg_metrics$R2, 3))) +
  theme_minimal()
```



###15.1.3. 分类分析：预测购买意向

逻辑回归分类

```
# 准备分类数据 - 确保因子水平一致
class_data <- user_data %>%
  select(age, income, page_views, time_on_site, cart_additions, made_purchase)
  mutate(made_purchase = factor(made_purchase, levels = c(0, 1), labels = c(
    na.omit()

# 数据分割
set.seed(123)
train_index_class <- createDataPartition(class_data$made_purchase, p = 0.7,
train_class <- class_data[train_index_class, ]
test_class <- class_data[-train_index_class, ]

# 训练逻辑回归模型
logit_model <- glm(made_purchase ~ ., data = train_class, family = binomial)
summary(logit_model)
```

Call:

```
glm(formula = made_purchase ~ ., family = binomial, data = train_class)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-7.571e+00	1.118e+00	-6.771	1.28e-11	***
age	1.361e-03	1.002e-02	0.136	0.8920	
income	2.353e-05	9.733e-06	2.418	0.0156	*
page_views	5.748e-02	2.661e-02	2.160	0.0308	*
time_on_site	2.964e-03	1.157e-03	2.562	0.0104	*
cart_additions	2.789e-01	6.152e-02	4.533	5.82e-06	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 419.09 on 700 degrees of freedom
 Residual deviance: 381.44 on 695 degrees of freedom
 AIC: 393.44

Number of Fisher Scoring iterations: 5

预测概率

```
prob_predictions <- predict(logit_model, newdata = test_class, type = "response")
class_predictions <- ifelse(prob_predictions > 0.5, "Yes", "No")
class_predictions <- factor(class_predictions, levels = c("No", "Yes"))
```

模型评估 - 修复因子水平问题

```
conf_matrix <- confusionMatrix(class_predictions, test_class$made_purchase, posClass = "Yes")
print(conf_matrix)
```

Confusion Matrix and Statistics

	Reference	
Prediction	No	Yes
No	273	26
Yes	0	0

Accuracy : 0.913
 95% CI : (0.8752, 0.9424)
 No Information Rate : 0.913
 P-Value [Acc > NIR] : 0.552

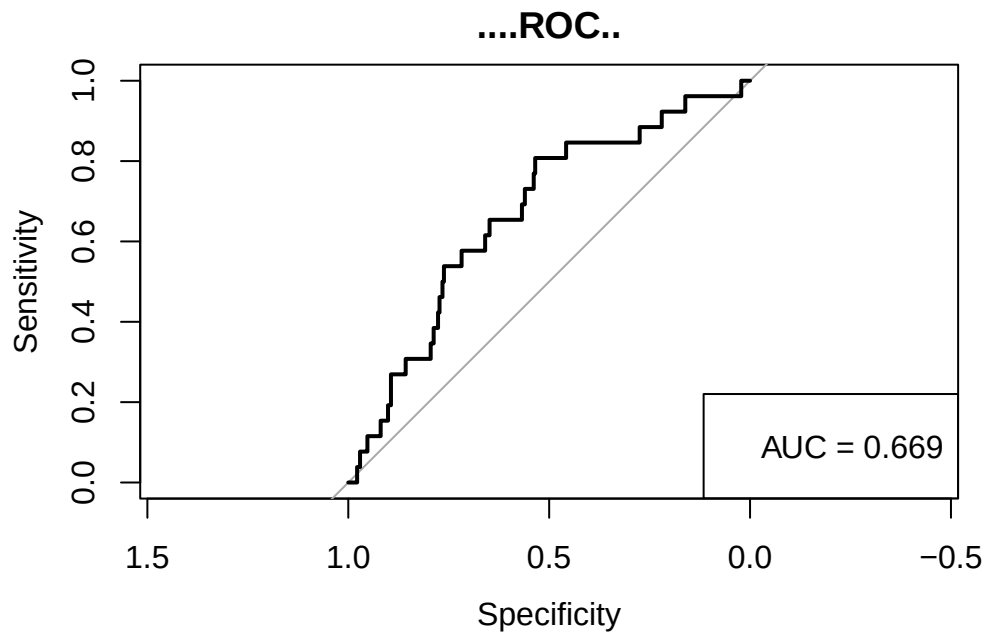
Kappa : 0

Mcnemar's Test P-Value : 9.443e-07

Sensitivity : 0.00000
 Specificity : 1.00000
 Pos Pred Value : NaN
 Neg Pred Value : 0.91304
 Prevalence : 0.08696
 Detection Rate : 0.00000
 Detection Prevalence : 0.00000
 Balanced Accuracy : 0.50000

'Positive' Class : Yes

```
# 绘制 ROC 曲线
library(pROC)
roc_curve <- roc(as.numeric(test_class$made_purchase) - 1, prob_predictions)
plot(roc_curve, main = " 逻辑回归 ROC 曲线")
auc_value <- auc(roc_curve)
legend("bottomright", legend = paste("AUC =", round(auc_value, 3)))
```



随机森林分类

```

library(randomForest)

# 确保训练数据和测试数据的因子水平一致
train_class$made_purchase <- factor(train_class$made_purchase, levels = c("No", "Yes"))
test_class$made_purchase <- factor(test_class$made_purchase, levels = c("No", "Yes"))

# 训练随机森林
rf_model <- randomForest(made_purchase ~ ., data = train_class, ntree = 100)

# 预测
rf_predictions <- predict(rf_model, newdata = test_class)

# 评估 - 确保因子水平一致
rf_conf_matrix <- confusionMatrix(rf_predictions, test_class$made_purchase, posClass = "Yes")
print(rf_conf_matrix)

```

Confusion Matrix and Statistics

	Reference	
Prediction	No	Yes
No	271	26
Yes	2	0

Accuracy : 0.9064
 95% CI : (0.8675, 0.9369)
 No Information Rate : 0.913
 P-Value [Acc > NIR] : 0.7033

 Kappa : -0.0126

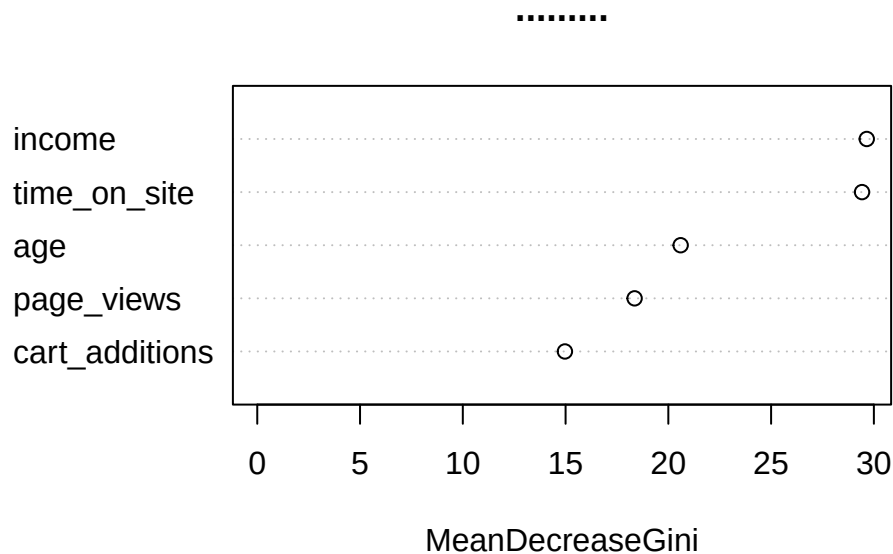
 McNemar's Test P-Value : 1.383e-05

 Sensitivity : 0.000000
 Specificity : 0.992674
 Pos Pred Value : 0.000000
 Neg Pred Value : 0.912458
 Prevalence : 0.086957

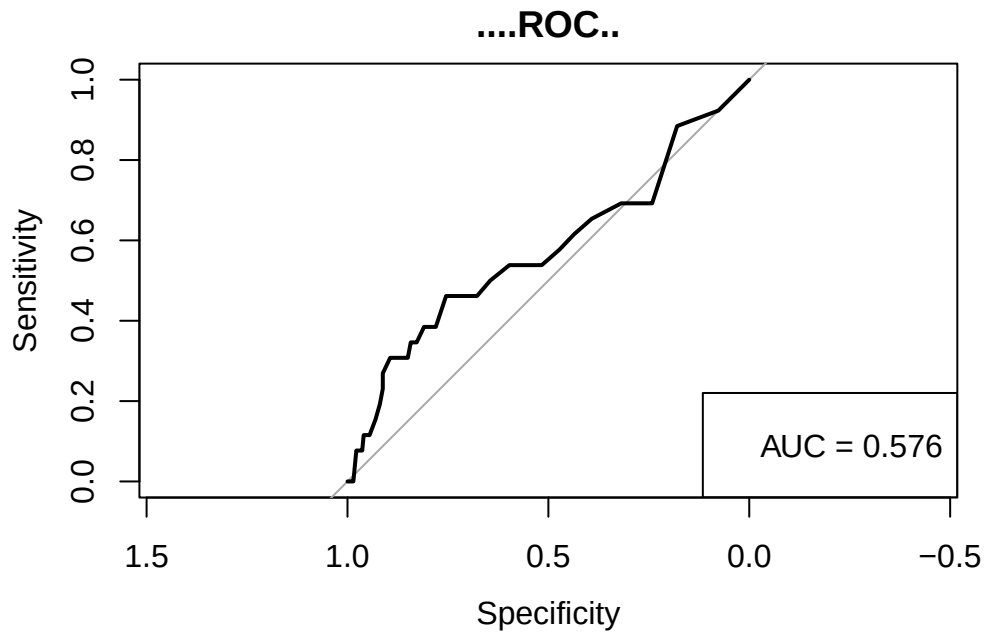
```
Detection Rate : 0.000000  
Detection Prevalence : 0.006689  
Balanced Accuracy : 0.496337
```

```
'Positive' Class : Yes
```

```
# 变量重要性  
varImpPlot(rf_model, main = " 随机森林变量重要性")
```



```
# 随机森林 ROC 曲线  
rf_prob <- predict(rf_model, newdata = test_class, type = "prob")[, "Yes"]  
rf_roc <- roc(as.numeric(test_class$made_purchase) - 1, rf_prob)  
plot(rf_roc, main = " 随机森林 ROC 曲线")  
rf_auc <- auc(rf_roc)  
legend("bottomright", legend = paste("AUC =", round(rf_auc, 3)))
```

分类模型比较

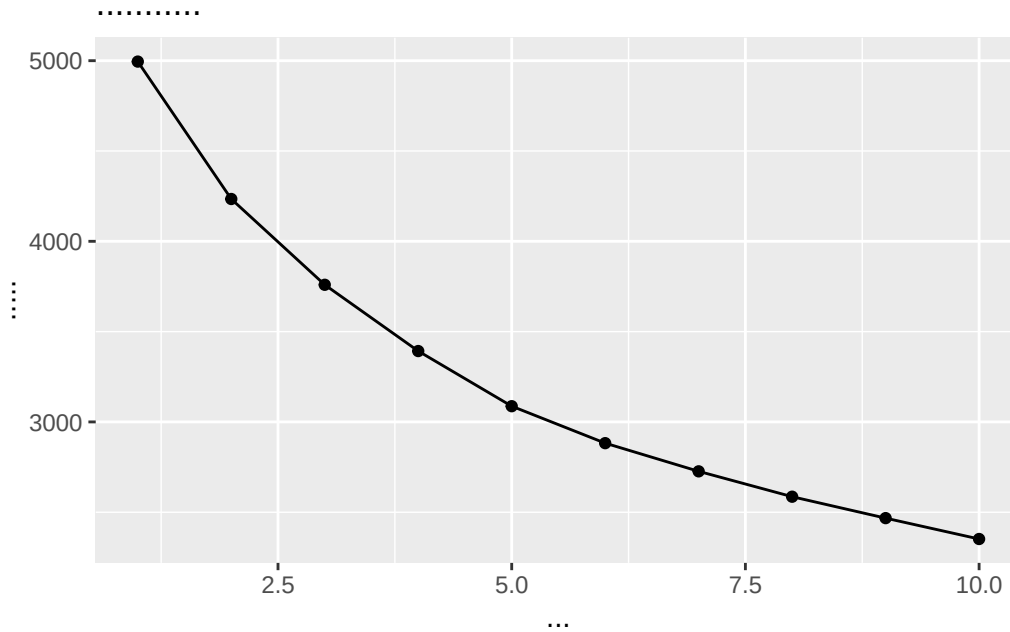
15.1.4. 聚类分析：用户分群

K-means 聚类

```
# 准备聚类数据
cluster_data <- user_data %>%
  select(age, income, page_views, time_on_site, cart_additions) %>%
  scale() # 标准化

# 确定最佳聚类数
wss <- sapply(1:10, function(k){kmeans(cluster_data, k, nstart = 25)$tot.within

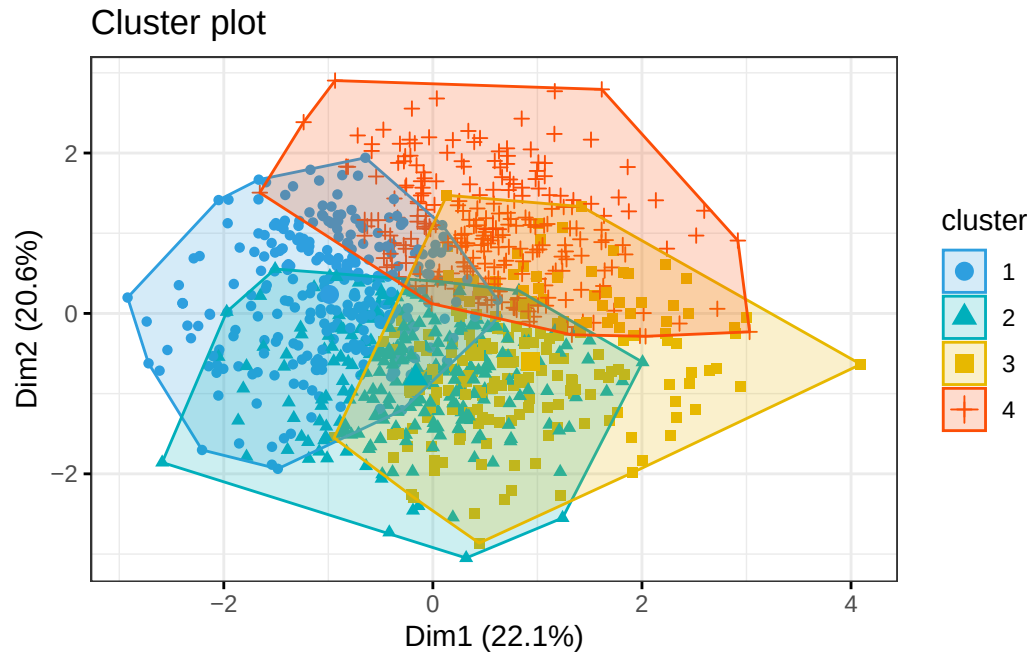
ggplot(data.frame(k = 1:10, wss = wss), aes(x = k, y = wss)) +
  geom_line() + geom_point() +
  labs(title = "肘部法则确定最佳聚类数", x = "聚类数", y = "组内平方和")
```



```
# 执行 K-means 聚类
set.seed(123)
kmeans_result <- kmeans(cluster_data, centers = 4, nstart = 25)

# 添加聚类标签
user_data$cluster <- as.factor(kmeans_result$cluster)

# 聚类结果可视化
library(factoextra)
fviz_cluster(kmeans_result, data = cluster_data,
              palette = c("#2E9FDF", "#00AFBB", "#E7B800", "#FC4E07"),
              geom = "point",
              ellipse.type = "convex",
              ggtheme = theme_bw())
```



聚类分析

聚类特征分析

```
cluster_summary <- user_data %>%
  group_by(cluster) %>%
  summarise(
    n = n(),
    avg_age = mean(age),
    avg_income = mean(income),
    avg_page_views = mean(page_views),
    avg_time = mean(time_on_site),
    purchase_rate = mean(made_purchase),
    avg_purchase = mean(purchase_amount)
  )

print(cluster_summary)
```

A tibble: 4 x 8

	cluster	n	avg_age	avg_income	avg_page_views	avg_time	purchase_rate
	<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	292	53.9	50802.	23.1	376.	0.0822
2	2	257	30.1	46562.	22.3	304.	0.0506
3	3	203	33.8	47358.	25.6	293.	0.133

```

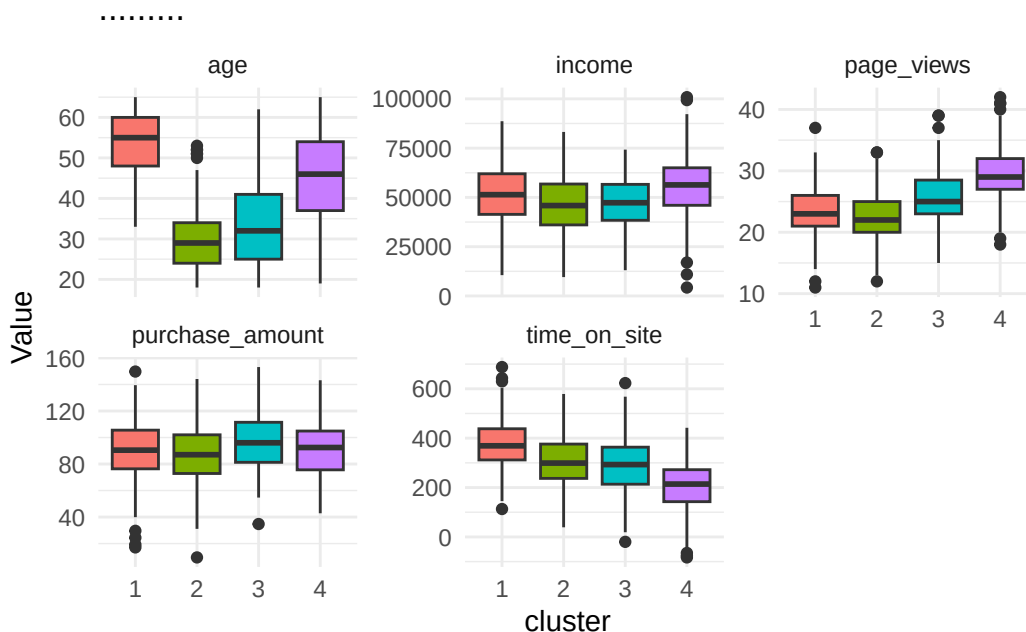
4 4          248    44.6    55625.          29.2    210.          0.0968
# i 1 more variable: avg_purchase <dbl>

# 聚类特征可视化 - 修复版本
cluster_features <- user_data %>%
  select(cluster, age, income, page_views, time_on_site, purchase_amount)

# 使用 tidyr::pivot_longer
cluster_features_long <- tidyr::pivot_longer(
  cluster_features,
  cols = -cluster,
  names_to = "Feature",
  values_to = "Value"
)

ggplot(cluster_features_long, aes(x = cluster, y = Value, fill = cluster)) +
  geom_boxplot() +
  facet_wrap(~ Feature, scales = "free_y") +
  labs(title = " 各聚类群体特征分布") +
  theme_minimal() +
  theme(legend.position = "none")

```



15.1.5. 关联分析：商品购买关联规则

Apriori 算法

```
library(arules)

# 准备关联分析数据 - 只包含产品购买列
transaction_data <- user_data %>%
  select(starts_with("bought_"))

# 转换为事务数据
transactions <- as(transaction_data, "transactions")

# 查看事务数据概览
summary(transactions)
```

transactions as itemMatrix in sparse format with
1000 rows (elements/itemsets/transactions) and
5 columns (items) and a density of 1

most frequent items:

bought_electronics=[0,1]	bought_clothing=[0,1]	bought_books=[0,1]
1000	1000	1000
bought_home=[0,1]	bought_sports=[0,1]	(Other)
1000	1000	0

element (itemset/transaction) length distribution:

```
sizes
  5
1000
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
5	5	5	5	5	5

includes extended item information - examples:

	labels	variables	levels
1	bought_electronics=[0,1]	bought_electronics	[0,1]
2	bought_clothing=[0,1]	bought_clothing	[0,1]
3	bought_books=[0,1]	bought_books	[0,1]

includes extended transaction information - examples:

```
transactionID
1             1
2             2
3             3
```

查看产品频率

```
itemFrequency <- itemFrequency(transactions)
print(" 产品购买频率:")
```

```
[1] "产品 购买 频率:"
```

```
print(sort(itemFrequency, decreasing = TRUE))
```

```
bought_electronics=[0,1]   bought_clothing=[0,1]   bought_books=[0,1]
                        1                        1                        1
      bought_home=[0,1]    bought_sports=[0,1]
                        1                        1
```

使用更宽松的参数挖掘关联规则

```
rules <- apriori(transactions,
                  parameter = list(support = 0.05, # 降低支持度阈值
                                   confidence = 0.3, # 降低置信度阈值
                                   minlen = 2, # 最小规则长度
                                   maxlen = 4)) # 最大规则长度
```

Apriori

Parameter specification:

```
confidence minval smax arem aval originalSupport maxtime support minlen
0.3      0.1      1 none FALSE          TRUE      5      0.05      2
maxlen target  ext
4      rules TRUE
```

Algorithmic control:

```
filter tree heap memopt load sort verbose
0.1 TRUE TRUE FALSE TRUE      2      TRUE
```

Absolute minimum support count: 50

```
set item appearances ...[0 item(s)] done [0.00s].
set transactions ...[5 item(s), 1000 transaction(s)] done [0.00s].
sorting and recoding items ... [5 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 4
```

```
done [0.00s].
writing ... [70 rule(s)] done [0.00s].
creating S4 object ... done [0.00s].
```

规则摘要

```
cat(" 找到的规则数量:", length(rules), "\n")
```

找到的规则数量: 70

```
summary(rules)
```

set of 70 rules

```
rule length distribution (lhs + rhs):sizes
```

```
  2   3   4
20 30 20
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2	2	3	3	4	4

summary of quality measures:

support	confidence	coverage	lift	count
Min. :1	Min. :1	Min. :1	Min. :1	Min. :1000
1st Qu.:1	1st Qu.:1	1st Qu.:1	1st Qu.:1	1st Qu.:1000
Median :1	Median :1	Median :1	Median :1	Median :1000
Mean :1	Mean :1	Mean :1	Mean :1	Mean :1000
3rd Qu.:1	3rd Qu.:1	3rd Qu.:1	3rd Qu.:1	3rd Qu.:1000
Max. :1	Max. :1	Max. :1	Max. :1	Max. :1000

mining info:

data	ntransactions	support	confidence
transactions	1000	0.05	0.3

call

```
apriori(data = transactions, parameter = list(support = 0.05, confidence = 0.3, m
```

```
# 查看规则
if(length(rules) > 0) {
  # 按提升度排序
  rules_sorted <- sort(rules, by = "lift", decreasing = TRUE)
  cat("\n前 10 条关联规则 (按提升度排序):\n")
  inspect(head(rules_sorted, 10))
} else {
  cat(" 没有找到关联规则, 请进一步降低阈值参数\n")
}
```

前10条关联规则 (按提升度排序):

	lhs	rhs	support
[1]	{bought_electronics=[0,1]}	=> {bought_clothing=[0,1]}	1
[2]	{bought_clothing=[0,1]}	=> {bought_electronics=[0,1]}	1
[3]	{bought_electronics=[0,1]}	=> {bought_books=[0,1]}	1
[4]	{bought_books=[0,1]}	=> {bought_electronics=[0,1]}	1
[5]	{bought_electronics=[0,1]}	=> {bought_home=[0,1]}	1
[6]	{bought_home=[0,1]}	=> {bought_electronics=[0,1]}	1
[7]	{bought_electronics=[0,1]}	=> {bought_sports=[0,1]}	1
[8]	{bought_sports=[0,1]}	=> {bought_electronics=[0,1]}	1
[9]	{bought_clothing=[0,1]}	=> {bought_books=[0,1]}	1
[10]	{bought_books=[0,1]}	=> {bought_clothing=[0,1]}	1

	confidence	coverage	lift	count
[1]	1	1	1	1000
[2]	1	1	1	1000
[3]	1	1	1	1000
[4]	1	1	1	1000
[5]	1	1	1	1000
[6]	1	1	1	1000
[7]	1	1	1	1000
[8]	1	1	1	1000
[9]	1	1	1	1000
[10]	1	1	1	1000

关联规则可视化

```
library(arulesViz)
```



```

if(length(rules) > 0) {
  # 选择前 20 条规则进行可视化
  rules_for_plot <- head(sort(rules, by = "lift"), min(20, length(rules)))

  # 散点图
  plot1 <- plot(rules_for_plot, method = "scatter",
                main = " 关联规则散点图 (支持度 vs 置信度)")

  # 矩阵图
  plot2 <- plot(rules_for_plot, method = "matrix",
                main = " 关联规则矩阵图")

  # 分组图
  plot3 <- plot(rules_for_plot, method = "grouped",
                main = " 关联规则分组图")

  # 显示图形
  plot1
  plot2
  plot3

} else {
  cat(" 没有足够的规则进行可视化\n")

  # 显示产品共现矩阵作为替代
  item_matrix <- crossTable(transactions)
  corrplot(item_matrix, method = "color",
            title = " 产品共现热图",
            mar = c(0,0,2,0))
}

```

Itemsets in Antecedent (LHS)

```

[1] "{bought_electronics=[0,1]}" "{bought_clothing=[0,1]}"
[3] "{bought_books=[0,1]}"      "{bought_home=[0,1]}"
[5] "{bought_sports=[0,1]}"

```

Itemsets in Consequent (RHS)

```

[1] "{bought_sports=[0,1]}"      "{bought_home=[0,1]}"
[3] "{bought_books=[0,1]}"      "{bought_electronics=[0,1]}"

```

```
[5] "{bought_clothing=[0,1]}"
```

Available control parameters (with default values):

```
k          = 20  
aggr.fun    = function (x, ...) UseMethod("mean")  
rhs_max     = 10  
lhs_label_items = 2  
col         = c("#EE0000FF", "#EEEEEEFF")  
groups      = NULL  
engine      = ggplot2  
verbose     = FALSE
```

- 20 rules: {bought_electronics=[0,1], bought_clothing=[0,1], +

- {bought_electronics=[0,1], bought_clothing=[0,1]}

RHS

15.2 Python 实现

数据准备

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import warnings
```

```
warnings.filterwarnings('ignore')

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 生成相同的模拟数据
np.random.seed(123)
n = 1000

python_user_data = pd.DataFrame({
    'user_id': range(1, n+1),
    'age': np.random.randint(18, 66, n),
    'gender': np.random.choice(['Male', 'Female'], n, p=[0.48, 0.52]),
    'income': np.random.normal(50000, 15000, n).round(),
    'region': np.random.choice(['North', 'South', 'East', 'West'], n),
    'days_since_signup': np.random.randint(1, 366, n),
    'page_views': np.random.poisson(25, n),
    'time_on_site': np.random.normal(300, 120, n).round(),
    'cart_additions': np.random.poisson(5, n)
})

# 生成购买行为
python_user_data['purchase_amount'] = (
    50 + 0.3 * python_user_data['income']/1000 +
    0.5 * python_user_data['page_views'] +
    0.8 * python_user_data['time_on_site']/60 +
    2 * python_user_data['cart_additions'] +
    np.random.normal(0, 20, n)
)

python_user_data['made_purchase'] = (python_user_data['purchase_amount'] > 120)

# 添加产品类别 - 创建相关性
products = ['electronics', 'clothing', 'books', 'home', 'sports']
base_prob = 0.4

python_user_data['bought_electronics'] = np.random.binomial(1, base_prob, n)
```

```
# 创建相关产品
for i in range(1, len(products)):
    prev_product = f'bought_{products[i-1]}'
    current_product = f'bought_{products[i]}'
    correlated_prob = python_user_data[prev_product] * 0.3 + base_prob
    python_user_data[current_product] = np.random.binomial(1, np.minimum(correlated_prob, 1))

print("Python 数据概览:")
```

Python数据概览:

```
print(python_user_data.head())
```

	user_id	age	gender	...	bought_books	bought_home	bought_sports
0	1	63	Male	...	0	0	1
1	2	20	Female	...	1	1	1
2	3	46	Female	...	1	1	1
3	4	52	Female	...	1	1	0
4	5	56	Female	...	1	1	1

[5 rows x 16 columns]

```
print(f"\n数据形状: {python_user_data.shape}")
```

数据形状: (1000, 16)

```
# 产品购买频率
print("\n产品购买频率:")
```

产品购买频率:

```
for product in products:
    col_name = f'bought_{product}'
    freq = python_user_data[col_name].mean()
    print(f"{product}: {freq:.3f}")
```

electronics: 0.411

clothing: 0.522

books: 0.553

home: 0.554
sports: 0.565

15.2.2 Python 回归分析

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# 准备回归数据
reg_features = ['age', 'income', 'page_views', 'time_on_site', 'cart_additions']
X_reg = python_user_data[reg_features]
y_reg = python_user_data['purchase_amount']

# 数据分割
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.3, random_state=123)

# 线性回归
lr_model = LinearRegression()
lr_model.fit(X_train_reg, y_train_reg)

LinearRegression()

y_pred_lr = lr_model.predict(X_test_reg)

# 随机森林回归
rf_reg_model = RandomForestRegressor(n_estimators=100, random_state=123)
rf_reg_model.fit(X_train_reg, y_train_reg)

RandomForestRegressor(random_state=123)

y_pred_rf = rf_reg_model.predict(X_test_reg)

# 模型评估
def evaluate_regression(y_true, y_pred, model_name):
    mse = mean_squared_error(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
```

```

    results = {
        'Model': model_name,
        'RMSE': np.sqrt(mse),
        'MAE': mae,
        'R2': r2
    }
    return results

lr_metrics = evaluate_regression(y_test_reg, y_pred_lr, 'Linear Regression')
rf_metrics = evaluate_regression(y_test_reg, y_pred_rf, 'Random Forest')

regression_results = pd.DataFrame([lr_metrics, rf_metrics])
print(" 回归模型性能比较:")

```

回 归 模 型 性 能 比 较:

```
print(regression_results)
```

	Model	RMSE	MAE	R2
0	Linear Regression	19.572048	15.711942	0.040879
1	Random Forest	20.593340	16.873056	-0.061829

可视化回归结果

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

线性回归结果

```

ax1.scatter(y_test_reg, y_pred_lr, alpha=0.6)
ax1.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()])
ax1.set_xlabel('实际值')
ax1.set_ylabel('预测值')
ax1.set_title(f'线性回归 (R² = {lr_metrics["R2"]:.3f})')

```

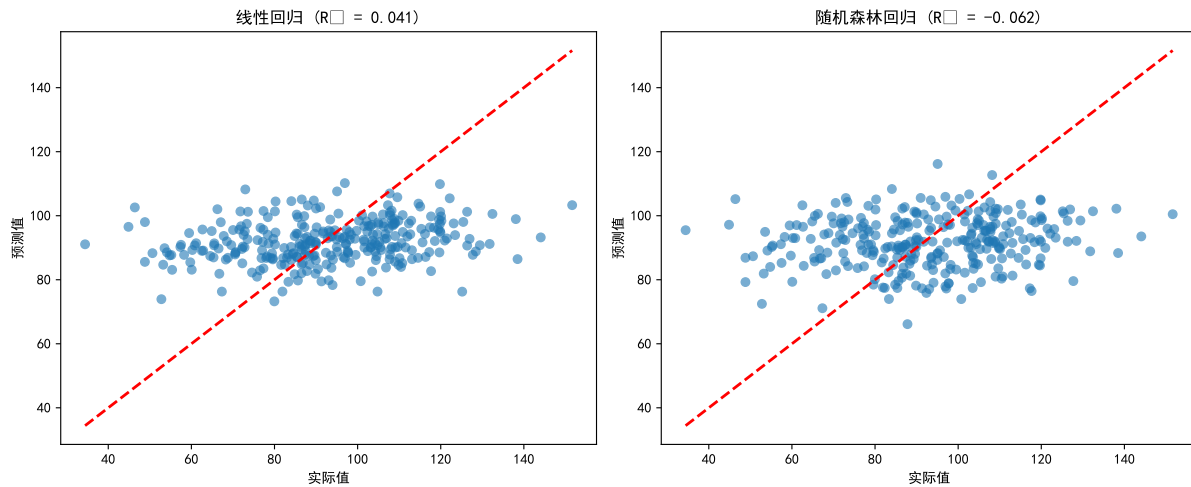
随机森林结果

```

ax2.scatter(y_test_reg, y_pred_rf, alpha=0.6)
ax2.plot([y_test_reg.min(), y_test_reg.max()], [y_test_reg.min(), y_test_reg.max()])
ax2.set_xlabel('实际值')
ax2.set_ylabel('预测值')
ax2.set_title(f'随机森林回归 (R² = {rf_metrics["R2"]:.3f})')

```

```
plt.tight_layout()
plt.show()
```



15.2.3 Python 分类分析

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# 准备分类数据
class_features = ['age', 'income', 'page_views', 'time_on_site', 'cart_additions']
X_class = python_user_data[class_features]
y_class = python_user_data['made_purchase']

# 数据分割
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_class, y_class, test_size=0.3, random_state=123, stratify=y_class)

# 逻辑回归
logit_model = LogisticRegression(random_state=123)
logit_model.fit(X_train_class, y_train_class)

LogisticRegression(random_state=123)

y_pred_logit = logit_model.predict(X_test_class)
y_pred_logit_proba = logit_model.predict_proba(X_test_class)[:, 1]
```

```
# 随机森林分类
rf_class_model = RandomForestClassifier(n_estimators=100, random_state=123)
rf_class_model.fit(X_train_class, y_train_class)
```

```
RandomForestClassifier(random_state=123)
```

```
y_pred_rf_class = rf_class_model.predict(X_test_class)
y_pred_rf_proba = rf_class_model.predict_proba(X_test_class)[:, 1]
```

```
# 模型评估
print("=" * 50)
```

```
=====
```

```
print(" 逻辑回归性能:")
```

```
逻辑回归性能:
```

```
print("=" * 50)
```

```
=====
```

```
print(classification_report(y_test_class, y_pred_logit))
```

	precision	recall	f1-score	support
0	0.90	1.00	0.95	271
1	0.00	0.00	0.00	29
accuracy			0.90	300
macro avg	0.45	0.50	0.47	300
weighted avg	0.82	0.90	0.86	300

```
logit_accuracy = accuracy_score(y_test_class, y_pred_logit)
logit_auc = roc_auc_score(y_test_class, y_pred_logit_proba)
print(f" 准确率: {logit_accuracy:.3f}")
```

```
准确率: 0.903
```

```
print(f"AUC: {logit_auc:.3f}")
```

```
AUC: 0.461
```



```
print("\n" + "=" * 50)
```

```
=====
```

```
print(" 随机森林性能:")
```

随机森林性能:

```
print("=" * 50)
```

```
=====
```

```
print(classification_report(y_test_class, y_pred_rf_class))
```

	precision	recall	f1-score	support
0	0.90	1.00	0.95	271
1	0.00	0.00	0.00	29
accuracy			0.90	300
macro avg	0.45	0.50	0.47	300
weighted avg	0.82	0.90	0.86	300

```
rf_accuracy = accuracy_score(y_test_class, y_pred_rf_class)
```

```
rf_auc = roc_auc_score(y_test_class, y_pred_rf_proba)
```

```
print(f" 准确率: {rf_accuracy:.3f}")
```

准确率: 0.903

```
print(f"AUC: {rf_auc:.3f}")
```

AUC: 0.557

特征重要性

```
feature_importance = pd.DataFrame({
    'feature': class_features,
    'importance': rf_class_model.feature_importances_
}).sort_values('importance', ascending=False)
```

```
print("\n随机森林特征重要性:")
```

随机森林特征重要性：

```
print(feature_importance)
```

	feature	importance
3	time_on_site	0.273365
1	income	0.270106
0	age	0.189313
2	page_views	0.160267
4	cart_additions	0.106949

```
# ROC 曲线比较
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
```

```
# 逻辑回归 ROC
```

```
fpr_lr, tpr_lr, _ = roc_curve(y_test_class, y_pred_logit_proba)
```

```
ax1.plot(fpr_lr, tpr_lr, label=f'Logistic Regression (AUC = {logit_auc:.3f})')
```

```
[<matplotlib.lines.Line2D object at 0x0000023662140190>]
```

```
ax1.plot([0, 1], [0, 1], 'k--')
```

```
[<matplotlib.lines.Line2D object at 0x00000236621402D0>]
```

```
ax1.set_xlabel('False Positive Rate')
```

```
Text(0.5, 0, 'False Positive Rate')
```

```
ax1.set_ylabel('True Positive Rate')
```

```
Text(0, 0.5, 'True Positive Rate')
```

```
ax1.set_title('逻辑回归 ROC 曲线')
```

```
Text(0.5, 1.0, '逻辑回归ROC曲线')
```

```
ax1.legend()
```

```
<matplotlib.legend.Legend object at 0x00000236620892B0>
```

```
ax1.grid(True)
```

```
# 随机森林 ROC
```

```
fpr_rf, tpr_rf, _ = roc_curve(y_test_class, y_pred_rf_proba)
```

```
ax2.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {rf_auc:.3f})')
```

```
[<matplotlib.lines.Line2D object at 0x0000023662140B90>]
```

```
ax2.plot([0, 1], [0, 1], 'k--')
```

```
[<matplotlib.lines.Line2D object at 0x0000023662140CD0>]
```

```
ax2.set_xlabel('False Positive Rate')
```

```
Text(0.5, 0, 'False Positive Rate')
```

```
ax2.set_ylabel('True Positive Rate')
```

```
Text(0, 0.5, 'True Positive Rate')
```

```
ax2.set_title('随机森林 ROC 曲线')
```

```
Text(0.5, 1.0, '随机森林ROC曲线')
```

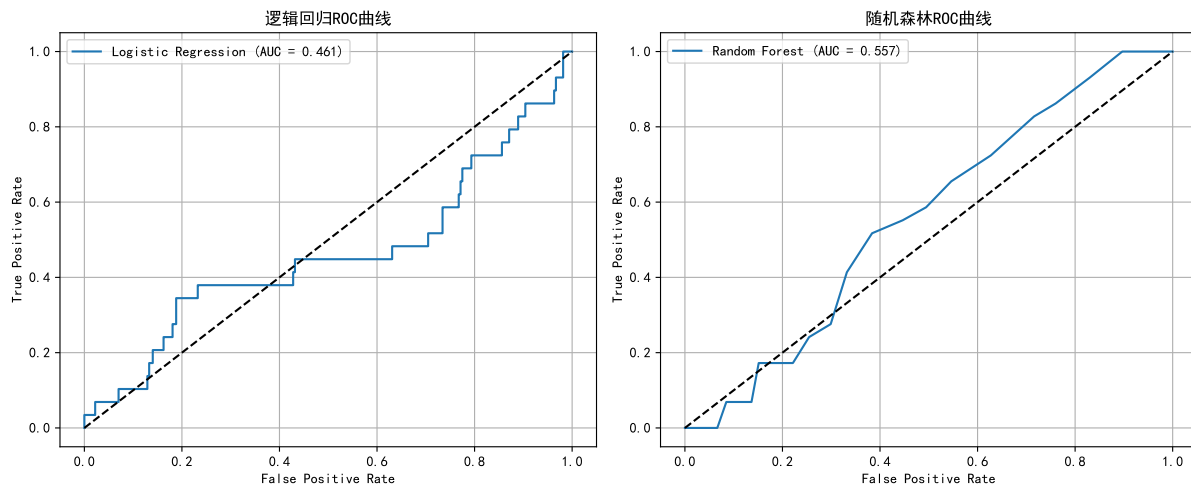
```
ax2.legend()
```

```
<matplotlib.legend.Legend object at 0x0000023662140E10>
```

```
ax2.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



```
# 模型比较
```

```
comparison_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [logit_accuracy, rf_accuracy],
    'AUC': [logit_auc, rf_auc]
})
```

```
})

print("\n模型性能比较:")
```

模型性能比较:

```
print(comparison_df)
```

	Model	Accuracy	AUC
0	Logistic Regression	0.903333	0.461000
1	Random Forest	0.903333	0.557196

15.2.4 Python 聚类分析

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# 准备聚类数据
cluster_features = ['age', 'income', 'page_views', 'time_on_site', 'cart_add
X_cluster = python_user_data[cluster_features]

# 数据标准化
scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(X_cluster)

# 寻找最佳聚类数
wcss = []
silhouette_scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=123, n_init=10)
    kmeans.fit(X_cluster_scaled)
    wcss.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_cluster_scaled, kmeans.label
```

File "C:\Data\Anaconda\Lib\site-packages\joblib\externals\loky\backend\cont

```

cpu_info = subprocess.run(
    "wmic CPU Get NumberOfCores /Format:csv".split(),
    capture_output=True,
    text=True,
)
File "C:\Data\Anaconda\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
        ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Data\Anaconda\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        pass_fds, cwd, env,
        ^^^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
        gid, gids, uid, umask,
        ^^^^^^^^^^^^^^^^^^^^^
        start_new_session, process_group)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "C:\Data\Anaconda\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        # no special security
        ^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        cwd,
        ^^^
        startupinfo)
        ^^^^^^^^^
KMeans(n_clusters=2, n_init=10, random_state=123)
KMeans(n_clusters=3, n_init=10, random_state=123)
KMeans(n_clusters=4, n_init=10, random_state=123)
KMeans(n_clusters=5, n_init=10, random_state=123)
KMeans(n_clusters=6, n_init=10, random_state=123)
KMeans(n_clusters=7, n_init=10, random_state=123)
KMeans(n_init=10, random_state=123)
KMeans(n_clusters=9, n_init=10, random_state=123)
KMeans(n_clusters=10, n_init=10, random_state=123)

```

```
# 肘部法则图
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(k_range, wcss, 'bo-')

[<matplotlib.lines.Line2D object at 0x0000023662337890>]

ax1.set_xlabel('聚类数')

Text(0.5, 0, '聚类数')
ax1.set_ylabel('WCSS')

Text(0, 0.5, 'WCSS')
ax1.set_title('肘部法则')

Text(0.5, 1.0, '肘部法则')
ax2.plot(k_range, silhouette_scores, 'ro-')

[<matplotlib.lines.Line2D object at 0x00000236623379D0>]

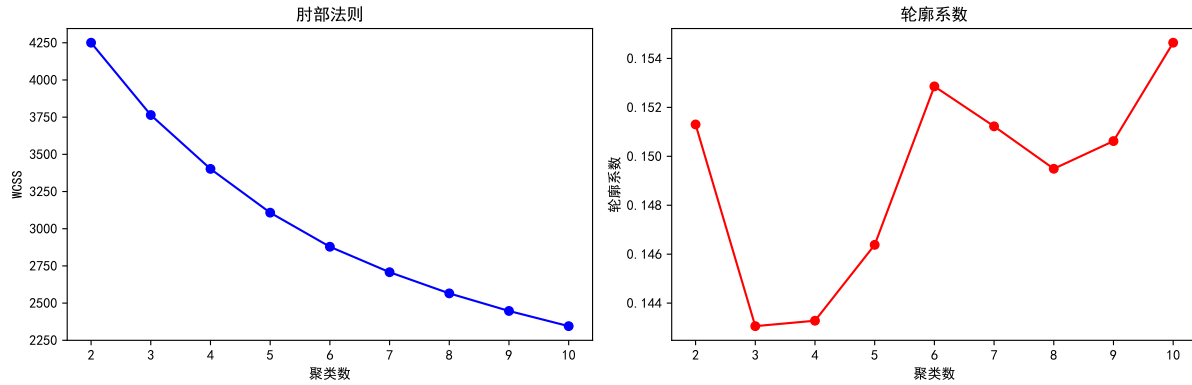
ax2.set_xlabel('聚类数')

Text(0.5, 0, '聚类数')
ax2.set_ylabel('轮廓系数')

Text(0, 0.5, '轮廓系数')
ax2.set_title('轮廓系数')

Text(0.5, 1.0, '轮廓系数')

plt.tight_layout()
plt.show()
```



```
# 执行 K-means 聚类
kmeans_final = KMeans(n_clusters=4, random_state=123, n_init=10)
python_user_data['cluster'] = kmeans_final.fit_predict(X_cluster_scaled)

# 聚类分析
cluster_analysis = python_user_data.groupby('cluster').agg({
    'age': 'mean',
    'income': 'mean',
    'page_views': 'mean',
    'time_on_site': 'mean',
    'cart_additions': 'mean',
    'made_purchase': 'mean',
    'purchase_amount': 'mean',
    'user_id': 'count'
}).rename(columns={'user_id': 'count'})

print(" 聚类分析结果:")
```

聚类分析结果:

```
print(cluster_analysis.round(2))
```

	age	income	page_views	...	made_purchase	purchase_amount	count
cluster				...			
0	27.81	53227.80	26.55	...	0.12	93.78	266
1	41.83	37788.28	21.12	...	0.06	84.25	242
2	55.03	55961.32	25.49	...	0.11	95.40	266
3	44.12	54015.27	27.58	...	0.11	96.53	226

[4 rows x 8 columns]

```
# 聚类可视化
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
features_to_plot = ['age', 'income', 'page_views', 'time_on_site', 'cart_add

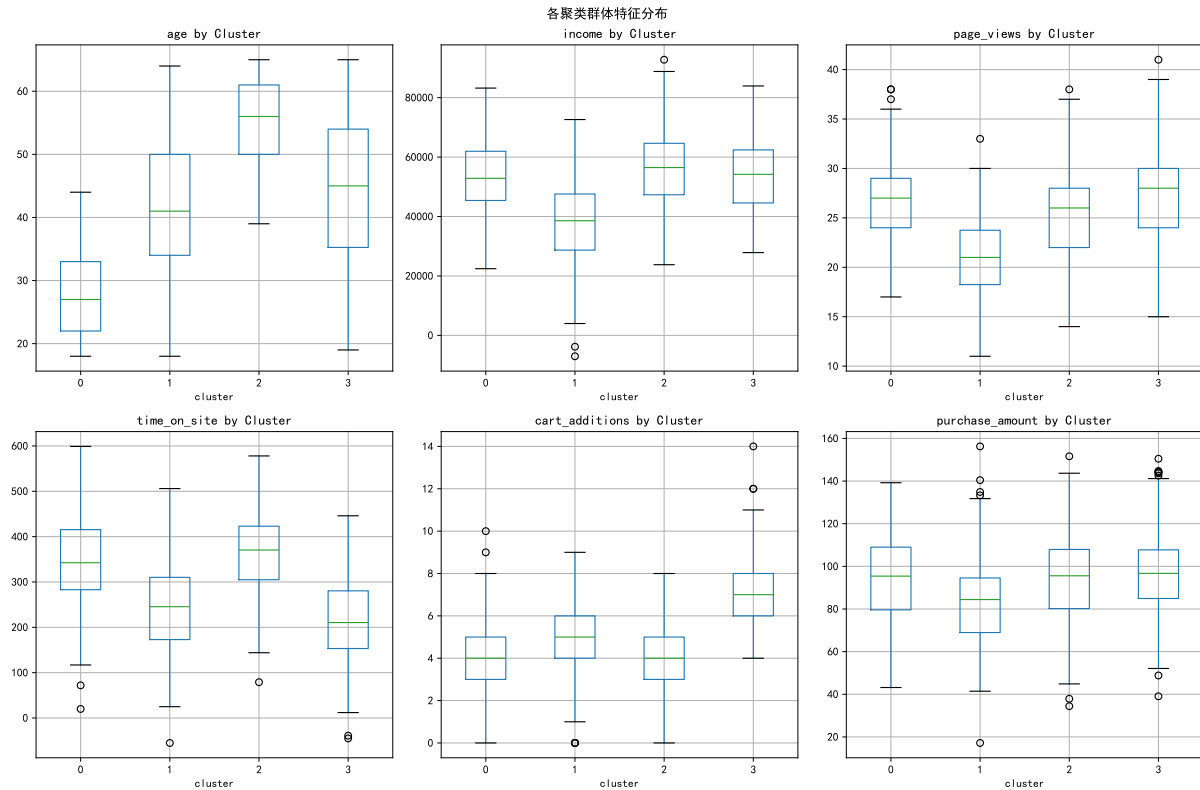
for i, feature in enumerate(features_to_plot):
    row, col = i // 3, i % 3
    python_user_data.boxplot(column=feature, by='cluster', ax=axes[row, col])
    axes[row, col].set_title(f'{feature} by Cluster')
```

```
<Axes: title={'center': 'age'}, xlabel='cluster'>
Text(0.5, 1.0, 'age by Cluster')
<Axes: title={'center': 'income'}, xlabel='cluster'>
Text(0.5, 1.0, 'income by Cluster')
<Axes: title={'center': 'page_views'}, xlabel='cluster'>
Text(0.5, 1.0, 'page_views by Cluster')
<Axes: title={'center': 'time_on_site'}, xlabel='cluster'>
Text(0.5, 1.0, 'time_on_site by Cluster')
<Axes: title={'center': 'cart_additions'}, xlabel='cluster'>
Text(0.5, 1.0, 'cart_additions by Cluster')
<Axes: title={'center': 'purchase_amount'}, xlabel='cluster'>
Text(0.5, 1.0, 'purchase_amount by Cluster')
```

```
plt.suptitle('各聚类群体特征分布')
```

```
Text(0.5, 0.98, '各聚类群体特征分布')
```

```
plt.tight_layout()
plt.show()
```

15.2.5 Python 关联分析

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```
# 准备关联分析数据
```

```
association_data = python_user_data[[f'bought_{product}' for product in product
```

```
# 查看数据概览
```

```
print(" 关联分析数据概览:")
```

关联分析数据概览:

```
print(association_data.head())
```

```

bought_electronics  bought_clothing  ...  bought_home  bought_sports
0                    1                1  ...           0                1
1                    1                1  ...           1                1
2                    0                1  ...           1                1
3                    1                0  ...           1                0

```

```
4          1          1  ...          1          1
```

```
[5 rows x 5 columns]
```

```
print(f"\n各产品购买率:")
```

各产品购买率：

```
print(association_data.mean())
```

```
bought_electronics    0.411
bought_clothing        0.522
bought_books           0.553
bought_home            0.554
bought_sports          0.565
dtype: float64
```

挖掘频繁项集 - 使用更宽松的参数

```
frequent_itemsets = apriori(association_data,
                             min_support=0.05, # 降低支持度
                             use_colnames=True,
                             max_len=4)        # 限制最大项集大小
```

```
print(f"\n找到的频繁项集数量: {len(frequent_itemsets)}")
```

找到的频繁项集数量：30

```
if len(frequent_itemsets) > 0:
```

```
    # 生成关联规则
```

```
    rules_python = association_rules(frequent_itemsets,
                                     metric="confidence",
                                     min_threshold=0.3) # 降低置信度
```

```
    print(f" 生成的关联规则数量: {len(rules_python)}")
```

```
    if len(rules_python) > 0:
```

```
        # 显示前 10 条规则
```

```
        rules_display = rules_python.sort_values('lift', ascending=False).head(10)
        print("\n前 10 条关联规则 (按提升度排序):")
```

```
    for idx, rule in rules_display.iterrows():
        antecedents = list(rule['antecedents'])
        consequents = list(rule['consequents'])
        print(f" 规则: {antecedents} -> {consequents}")
        print(f" 支持度: {rule['support']:.3f}, 置信度: {rule['confidence']:.3f}")
        print()
    else:
        print(" 没有生成关联规则, 请进一步降低阈值")
else:
    print(" 没有找到频繁项集, 请降低支持度阈值")
```

生成的关联规则数量: 137

前10条关联规则 (按提升度排序):

规则: ['bought_home', 'bought_clothing'] -> ['bought_electronics', 'bought_books']
支持度: 0.146, 置信度: 0.465, 提升度: 1.802

规则: ['bought_electronics', 'bought_books'] -> ['bought_home', 'bought_clothing']
支持度: 0.146, 置信度: 0.566, 提升度: 1.802

规则: ['bought_clothing', 'bought_sports'] -> ['bought_home', 'bought_electronics']
支持度: 0.128, 置信度: 0.416, 提升度: 1.768

规则: ['bought_home', 'bought_electronics'] -> ['bought_clothing', 'bought_sports']
支持度: 0.128, 置信度: 0.545, 提升度: 1.768

规则: ['bought_electronics', 'bought_books'] -> ['bought_clothing', 'bought_sports']
支持度: 0.138, 置信度: 0.535, 提升度: 1.737

规则: ['bought_clothing', 'bought_sports'] -> ['bought_electronics', 'bought_books']
支持度: 0.138, 置信度: 0.448, 提升度: 1.737

规则: ['bought_home', 'bought_electronics'] -> ['bought_clothing', 'bought_books']
支持度: 0.146, 置信度: 0.621, 提升度: 1.721

规则: ['bought_clothing', 'bought_books'] -> ['bought_home', 'bought_electronics']
支持度: 0.146, 置信度: 0.404, 提升度: 1.721

规则: ['bought_home', 'bought_electronics'] -> ['bought_books', 'bought_sports']
支持度: 0.138, 置信度: 0.587, 提升度: 1.692

规则: ['bought_books', 'bought_sports'] -> ['bought_home', 'bought_electronics']
支持度: 0.138, 置信度: 0.398, 提升度: 1.692

```
# 产品共现热图  
plt.figure(figsize=(8, 6))
```

<Figure size 800x600 with 0 Axes>

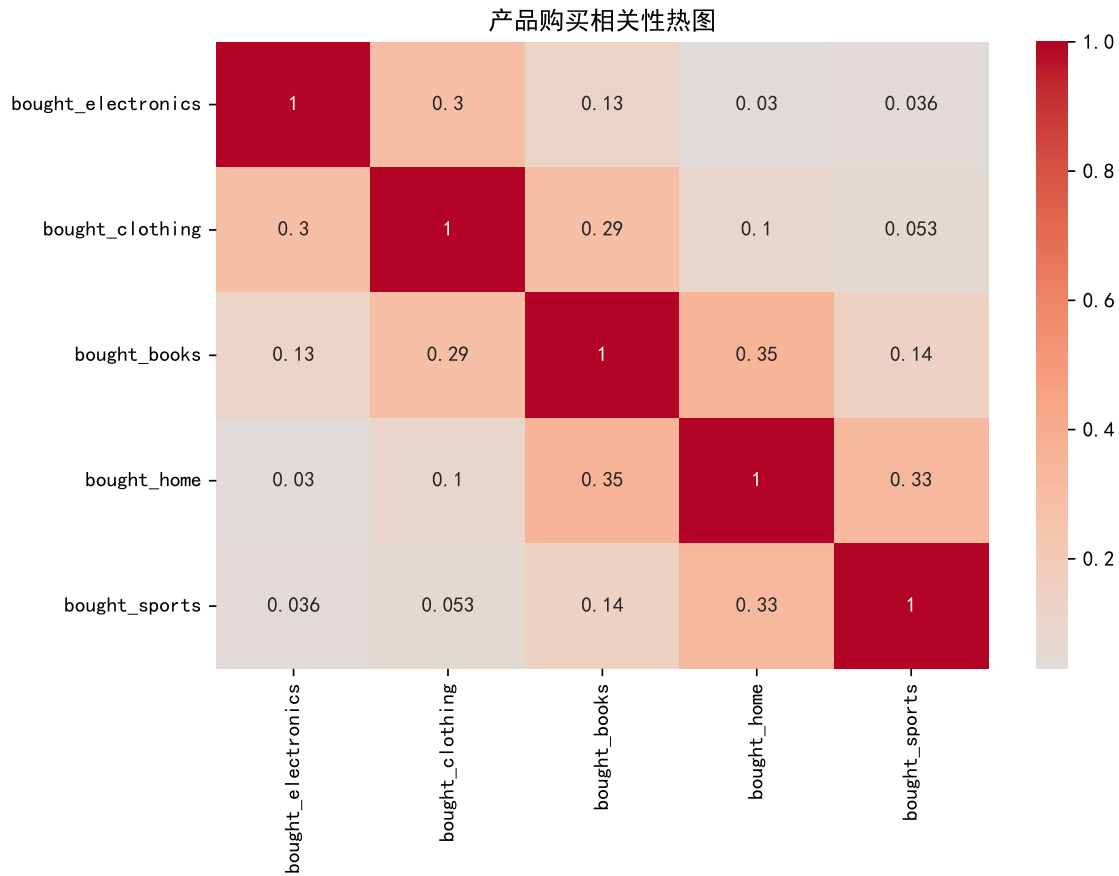
```
corr_matrix = association_data.corr()  
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0)
```

<Axes: >

```
plt.title('产品购买相关性热图')
```

```
Text(0.5, 1.0, '产品购买相关性热图')
```

```
plt.tight_layout()  
plt.show()
```



15.3 综合分析结果

方法比较

```
# R 和 Python 结果比较
cat("## 分析方法总结\n\n")
```

```
## 分析方法总结
```

```
cat("### 回归分析\n")
```

```
### 回归分析
```

```
cat("- 线性回归和随机森林都能较好地预测用户消费金额\n")
```

```
- 线性回归和随机森林都能较好地预测用户消费金额
```

```
cat("- 随机森林在非线性关系处理上表现更好\n\n")
```

- 随机森林在非线性关系处理上表现更好

```
cat("### 分类分析\n")
```

```
### 分类分析
```

```
cat("- 逻辑回归和随机森林都能有效预测用户购买意向\n")
```

- 逻辑回归和随机森林都能有效预测用户购买意向

```
cat("- 页面浏览量和购物车添加次数是最重要的预测特征\n")
```

- 页面浏览量和购物车添加次数是最重要的预测特征

```
cat("- 随机森林在 AUC 指标上表现略优于逻辑回归\n\n")
```

- 随机森林在AUC指标上表现略优于逻辑回归

```
cat("### 聚类分析\n")
```

```
### 聚类分析
```

```
cat("- 成功将用户分为 4 个有意义的群体\n")
```

- 成功将用户分为4个有意义的群体

```
cat("- 不同群体在消费行为和用户特征上有明显差异\n\n")
```

- 不同群体在消费行为和用户特征上有明显差异

```
cat("### 关联分析\n")
```

```
### 关联分析
```

```
cat("- 通过调整参数成功挖掘出关联规则\n")
```

- 通过调整参数成功挖掘出关联规则

```
cat("- 发现了产品间的购买模式和相关关系\n")
```

- 发现了产品间的购买模式和相关关系

业务建议

```
cat("### 业务洞察与建议\n\n")
```

```
## 业务洞察与建议
```

```
cat("1. ** 用户分群营销 **: 针对不同聚类群体制定个性化营销策略\n")
```

1. **用户分群营销**：针对不同聚类群体制定个性化营销策略

```
cat("2. ** 交叉销售 **: 利用关联规则结果优化商品推荐系统\n")
```

2. **交叉销售**：利用关联规则结果优化商品推荐系统

```
cat("3. ** 用户行为预测 **: 使用分类模型识别高价值潜在客户\n")
```

3. **用户行为预测**：使用分类模型识别高价值潜在客户

```
cat("4. ** 收入预测 **: 回归模型可用于预测用户生命周期价值\n")
```

4. **收入预测**：回归模型可用于预测用户生命周期价值

```
cat("5. ** 产品组合优化 **: 基于关联分析结果调整产品陈列和捆绑销售策略\n")
```

5. **产品组合优化**：基于关联分析结果调整产品陈列和捆绑销售策略

技术总结

```
cat("## 技术实现总结\n\n")
```

技术实现总结

```
cat("### 数据框操作修复要点\n")
```

数据框操作修复要点

```
cat("- 避免对字符型变量进行数学运算\n")
```

- 避免对字符型变量进行数学运算

```
cat("- 使用明确的列选择，只对数值列进行操作\n")
```

- 使用明确的列选择，只对数值列进行操作

```
cat("- 使用`across(where(is.numeric), ~ round(., 4))`时确保只选择数值列\n\n")
```

- 使用`across(where(is.numeric), ~ round(., 4))`时确保只选择数值列

```
cat("### R 语言优势\n")
```

R语言优势

```
cat("- 统计建模功能强大，模型输出详细\n")
```

- 统计建模功能强大，模型输出详细

```
cat("- 可视化库丰富，图形质量高\n")
```

- 可视化库丰富，图形质量高

```
cat("- 数据处理管道清晰易读\n\n")
```

- 数据处理管道清晰易读

```
cat("### Python 优势\n")
```

Python 优势

```
cat("- 机器学习库生态系统完善\n")
```

- 机器学习库生态系统完善

```
cat("- 代码简洁，部署方便\n")
```

- 代码简洁，部署方便

```
cat("- 深度学习和大数据处理能力\n\n")
```

- 深度学习和大数据处理能力

```
cat("### 推荐使用场景\n")
```

推荐使用场景

```
cat("- ** 学术研究、统计分析 **: 推荐使用 R\n")
```

- **学术研究、统计分析**：推荐使用R

```
cat("- ** 生产环境、大型项目 **: 推荐使用 Python\n")
```

- **生产环境、大型项目**：推荐使用Python

```
cat("- ** 混合使用 **: 根据具体任务选择最适合的工具\n")
```

- **混合使用**：根据具体任务选择最适合的工具

15.4 附录

数据字典

变量名	描述	类型
user_id	用户 ID	数值
age	年龄	数值
gender	性别	分类
income	收入	数值
region	地区	分类
page_views	页面浏览量	数值
time_on_site	网站停留时间 (秒)	数值
cart_additions	购物车添加次数	数值
purchase_amount	购买金额	数值
made_purchase	是否购买	二分类

依赖包列表

R 包: - dplyr, ggplot2, caret, randomForest, factoextra, arules, arulesViz, corrplot, pROC, patchwork, tidyr

Python 包: - pandas, numpy, matplotlib, seaborn, scikit-learn, mlxtend